Archivos de órdenes en Google Drive

Qué guarda la aplicación, cómo se consulta y qué falta para operarlo.

> **Estado actual:** Drive está integrado y seleccionado por la infraestructura como destino de staging. AWS staging está eliminado y no hay un despliegue de producción. La configuración de IaC no significa que exista un entorno activo.

Qué cambia en la práctica

Organización

Las fotos se agrupan por cliente, unidad/serie y orden. La firma y el PDF se guardan una sola vez por orden.

Acceso privado

El personal consulta desde Bull Dog Solution. La API autoriza cada acceso; no se comparten enlaces públicos ni credenciales de Google.

Responsabilidad

Drive es un servicio externo. Capacidad, retención, recuperación y permisos deben quedar acordados antes del uso real.

Qué está implementado y qué no

La aplicación soporta cargas y lecturas autenticadas de fotografías, firmas y reportes PDF con referencias persistidas. El adaptador de Drive y su integración están en el código; la IaC declara Drive para un futuro staging.

> **Antes del uso real:** recrear staging, configurar credenciales y hojas de prueba, validar fotos, firmas, PDFs, permisos y recuperación en un entorno controlado. Después, aprobar por separado cualquier despliegue productivo.

Archivos históricos: los objetos S3 existentes fueron datos de prueba y no se migran automáticamente a Drive. Esta guía no autoriza su eliminación física.

Cómo se organizan los documentos

La aplicación resuelve los archivos mediante referencias guardadas en la base de datos; no depende de que alguien reconstruya la ruta a mano.



Ejemplo de lectura

Para el cliente Transportes del Norte, la evidencia de la unidad VIN 1HGBH41JXMN109186 queda bajo la carpeta de esa unidad y el folio de servicio. La firma y las versiones del reporte de la misma orden se consultan en el área Órdenes, bajo el mismo folio.

> **Importante:** el nombre visible de una carpeta no autoriza el acceso. El permiso se valida en la aplicación y las rutas autenticadas entregan el archivo.

Quién valida cada parte

La integración no sustituye las decisiones de seguridad, cumplimiento ni operación del entorno.

Personal autorizado
Envía una foto o solicita ver un documento desde la orden.

›

Aplicación
Comprueba identidad y acceso a la orden; registra referencias y auditoría.

›

Drive compartido
Guarda el archivo privado en la carpeta correspondiente.

La consulta recorre el camino de regreso a través de la API; el navegador no recibe credenciales de Drive ni una URL pública.

Responsabilidades principales

Administración de Workspace / TI
Aprobar el Drive compartido, compartirlo con la identidad de servicio y definir capacidad y acceso.

Seguridad y operación
Resguardar y rotar credenciales; acordar monitoreo, respaldos y recuperación.

Negocio y cumplimiento
Aprobar acceso, retención y eliminación definitiva de fotos, firmas y reportes.

Equipo Bull Dog Solution
Recrear el ambiente y validar cargas, descargas, reintentos y permisos antes de habilitarlo.

Lista previa al uso real

- ☐ Entorno desplegado y configuración verificada
- ☐ Acceso de Drive y capacidad aprobados
- ☐ Fotos, firmas y PDF probados extremo a extremo
- ☐ Retención y borrado aprobados
- ☐ Recuperación y monitoreo definidos
- ☐ Reversión y responsables documentados

Code Style and Engineering Standards

This document defines the mandatory coding standards for the entire codebase. These rules apply to application code, tests, scripts, configuration, migrations, API contracts, logs, documentation embedded in code, commit-ready fixtures, and generated examples maintained by the team.

1. Language

- Write all source code in English.
- Use English for file names, directory names, identifiers, types, interfaces, classes, functions, variables, database objects, API fields, error codes, logs, tests, and technical comments.
- Use English for internal developer documentation and OpenAPI operation identifiers.
- User-facing text may be localized according to product requirements, but it must not be used as an internal identifier.
- Do not mix English and Spanish in names such as `createOrden`, `tecnicoService`, or `clienteId`.
- Prefer clear domain terms over literal or ambiguous translations.

```
// Correct
const assignedTechnicianId = order.technicianId;

// Incorrect
const tecnicoAsignadoId = orden.technicianId;
```

2. Production-ready baseline

Every committed change must be suitable for production unless it is explicitly isolated as a prototype.

- Do not commit placeholder implementations, unexplained hardcoded values, debug statements, dead code, or commented-out code.
- Do not use `TODO`, `FIXME`, or `HACK` without a tracked issue, a reason, and a clear removal condition.
- Handle expected failure modes explicitly.
- Validate all external input and configuration at system boundaries.
- Use secure defaults and least-privilege access.
- Keep secrets, credentials, tokens, and private URLs outside source control and logs.
- Make operations that can be retried idempotent when duplication could cause harm.
- Preserve backward compatibility for public contracts unless a breaking change is intentional and documented.
- Include tests proportional to the risk and behavior introduced by the change.

- Update technical documentation when behavior, configuration, or architecture changes.

3. Formatting and automated checks

- All files must be formatted before they are committed.
- Use the repository formatter and linter as the source of truth; do not maintain personal formatting conventions.
- Do not disable formatter or linter rules without a documented technical reason.
- Keep exceptions as narrow as possible and place them next to the affected code.
- Run formatting, linting, type checking, and relevant tests before considering work complete.
- CI must reject unformatted code, lint errors, type errors, failing tests, and invalid OpenAPI documents.
- Use UTF-8, Unix line endings, and a final newline in text files.
- Remove trailing whitespace.
- Let automated tools decide whitespace, indentation, quotes, semicolons, and line wrapping.

Expected validation commands:

```
pnpm format
pnpm lint
pnpm typecheck
pnpm test
pnpm build
```

4. Naming

- Choose names that describe intent, domain meaning, and units when relevant.
- Avoid vague names such as `data`, `item`, `value`, `object`, `thing`, `manager`, `helper`, or `utils` when a more precise term exists.
- Do not use unexplained abbreviations.
- Use `camelCase` for variables, functions, object properties, and module instances.
- Use `PascalCase` for types, interfaces, classes, Vue components, and enum-like schemas.
- Use `UPPER_SNAKE_CASE` only for true constants and environment variable names.
- Use `kebab-case` for general file and directory names unless a framework convention requires otherwise.
- Name Vue component files with `PascalCase.vue`.
- Name tests after the unit or behavior under test.
- Use boolean names that read as predicates, such as `isActive`, `hasEvidence`, `canComplete`, or `shouldRetry`.
- Include units in names when ambiguity is possible, such as `timeoutMs`, `sizeBytes`, or `ttlSeconds`.
- Use plural nouns for collections and singular nouns for individual values.

5. Code structure

- Organize the codebase by business domain using vertical slices.
- Keep module boundaries explicit and expose a small public API.
- Do not import private internals from another module through deep paths.
- Keep controllers thin: translate HTTP input into one application call and translate its result into HTTP output.
- Keep business rules in domain or application services, not in controllers, routes, Vue views, or persistence adapters.
- Keep infrastructure integrations behind interfaces or ports.
- Keep Prisma models internal to the API; use shared contracts for communication with the frontend.
- Keep remote server state in TanStack Query and reserve Pinia for session and truly global UI state.
- Extract shared code only after real reuse or when it represents a global architectural rule.
- Prefer focused modules with high cohesion over broad `common`, `helpers`, or `utils` collections.
- Avoid circular dependencies.
- Keep dependency direction toward domain and application logic.

6. Functions and control flow

- Write small, focused functions with one clear responsibility.
- Prefer early returns and guard clauses over deeply nested conditionals.
- Keep side effects visible and localized.
- Separate validation, decision-making, persistence, and external I/O when doing so clarifies behavior.
- Avoid boolean parameters when they make a call ambiguous; prefer an options object or separate functions.
- Do not mutate inputs unless the API explicitly communicates mutation.
- Prefer immutable transformations for application data.
- Avoid premature abstractions and generic frameworks built for a single use case.
- Avoid clever code that saves lines at the cost of readability or debuggability.
- Do not catch errors unless the code can add context, translate them, recover safely, or perform required cleanup.
- Never leave an empty `catch` block.

7. TypeScript

- Keep TypeScript strict mode enabled.
- Do not use `any`. Use `unknown` at untrusted boundaries and narrow it through validation or type guards.

- Do not use unsafe type assertions to bypass compiler errors.
- Prefer inferred local types and explicit types at public boundaries.
- Model meaningful domain states with discriminated unions, enums, or validated schemas.
- Avoid optional fields when absence is not a valid domain state.
- Use `readonly` where mutation is not expected.
- Derive DTO types from Valibot schemas instead of defining parallel types manually.
- Do not expose Prisma-generated types as API contracts.
- Handle nullable and optional values deliberately.
- Use exhaustive checks for finite state transitions and discriminated unions.

8. Validation and API contracts

- Treat every external value as `unknown` until validated.
- Validate request parameters, query strings, bodies, environment variables, Sheets data, storage metadata, and external API responses.
- Define reusable request and response contracts with Valibot.
- Keep JSON fields in `camelCase` and timestamps in ISO 8601 UTC.
- Use stable, machine-readable error codes and safe user-facing messages.
- Never expose stack traces, internal exception text, SQL details, credentials, or provider secrets to clients.
- Declare allowed filters and sortable fields explicitly; never map arbitrary query parameters to Prisma.
- Keep OpenAPI documentation synchronized with runtime validation and actual responses.
- Make breaking API changes explicit and versioned.

9. Error handling and debuggability

Code must be easy to diagnose without exposing sensitive information.

- Use typed domain and application errors for expected failures.
- Add actionable context when translating an error between layers.
- Preserve the original error as a cause when the runtime supports it.
- Assign and propagate a `requestId` for every HTTP request.
- Include relevant identifiers in structured logs, such as `requestId`, `userId`, `orderId`, `module`, `action`, duration, and outcome.
- Log events at the appropriate level: `debug`, `info`, `warn`, or `error`.
- Do not use `console.log` in application code; use the shared logger.
- Never log passwords, JWTs, cookies, authorization headers, private keys, signed URLs, binary content, or unnecessary personal data.
- Avoid generic messages such as `Something failed` when a safe and specific error code can identify the operation.
- Do not use logs as a substitute for correct error propagation or monitoring.

- Ensure background jobs report attempts, result, duration, retry state, and sanitized failure reason.

10. Comments and documentation in code

- Write comments only when they provide necessary technical context that the code cannot express clearly.
- Comments must explain why a constraint, workaround, invariant, or non-obvious decision exists.
- Do not write vague, decorative, conversational, or redundant comments.
- Do not narrate what the next line already says.
- Do not use comments to compensate for poor naming or excessive complexity; improve the code first.
- Keep comments accurate when behavior changes.
- Reference an issue, external limitation, or specification when a workaround depends on it.
- Use JSDoc only for public APIs or behavior that cannot be understood from the signature and types.
- Do not add file header comments that repeat the file name or module name.

```
// Correct: the row can move when users sort the spreadsheet, so it must be
// resolved again from its external ID and IMEI immediately before writing.
const currentRow = await equipmentSheet.findCurrentRow(equipmentIdentity);

// Incorrect: get the current row
const currentRow = await equipmentSheet.findCurrentRow(equipmentIdentity);
```

11. Security

- Apply authentication and authorization in the backend, regardless of frontend visibility.
- Check both role capability and resource ownership where required.
- Use parameterized database queries.
- Restrict CORS to an explicit allowlist when credentials are enabled.
- Set secure cookie attributes according to the deployment environment.
- Validate JWT signature, issuer, audience, purpose, and expiration.
- Hash passwords with Argon2id and store only refresh-token hashes.
- Give Google Sheets and S3 credentials the minimum required permissions.
- Keep object storage private and use short-lived signed URLs.
- Validate uploaded object existence, type, and size before marking it complete.
- Protect against user enumeration and excessive authentication attempts.
- Review dependencies and container images for known vulnerabilities in CI.

12. Database and persistence

- Use Prisma migrations for schema changes.
- Give every migration a clear, reviewable purpose.
- Do not edit a migration that has already been applied to a shared environment.
- Use transactions for local changes that must succeed or fail together.
- Keep external calls outside long-running database transactions.
- Store domain changes and critical outbox operations in the same transaction.
- Enforce important invariants at both application and database levels when possible.
- Add indexes based on actual query patterns and verify their effect.
- Avoid unbounded queries and N+1 access patterns.
- Use optimistic concurrency where concurrent administrative changes can conflict.
- Make destructive migrations reversible or provide a tested backup and rollback procedure.
- Do not store image binaries or Base64 payloads in PostgreSQL.

13. External integrations

- Access Google Sheets, S3, and other providers only through dedicated adapters.
- Set explicit timeouts for every external request.
- Retry only operations classified as safe to retry.
- Use exponential backoff with jitter and a configurable attempt limit.
- Do not retry an ambiguous write until the remote state has been verified.
- Use idempotency keys for assignments, completion, and synchronization operations.
- Use circuit breakers per integration rather than one global breaker.
- Sanitize provider errors before persisting, logging, or returning them.
- Never use a spreadsheet row number as a stable identity.
- Update only explicitly authorized cells in Google Sheets.

14. Vue frontend

- Use Vue 3 Composition API and `<script setup lang="ts">`.
- Keep views focused on rendering and user interaction.
- Place UI orchestration in composables and HTTP details in module API files.
- Keep critical business rules in the backend; frontend validation improves usability but is not authoritative.
- Use TanStack Query for remote data, caching, invalidation, and mutation state.
- Use Pinia only for authenticated session data and truly global UI state.
- Keep persistent filters in the URL.
- Provide explicit loading, empty, error, success, and disabled states.
- Ensure forms show field-level validation and preserve entered data after recoverable failures.

- Build technician workflows mobile-first and verify keyboard and touch interaction.
- Use semantic HTML and accessible labels, focus states, and contrast.
- Do not cache authenticated API responses in the service worker.
- Do not introduce deep imports into another frontend module.

15. Testing

- Test observable behavior, business rules, contracts, and failure modes.
- Use unit tests for pure rules and state transitions.
- Use integration tests for routes, persistence, storage, outbox processing, idempotency, and authorization.
- Use end-to-end tests for critical administrator and technician workflows.
- Include negative tests for permissions, invalid transitions, stale versions, external conflicts, and incomplete orders.
- Keep tests deterministic and independent.
- Do not depend on test execution order.
- Do not use production services or production Google Sheets in automated tests.
- Use factories or builders with meaningful defaults instead of large duplicated fixtures.
- Avoid arbitrary sleeps; wait for observable conditions.
- Name tests as behavior statements.
- A bug fix must include a regression test whenever practical.
- Do not change or weaken a test merely to make an incorrect implementation pass.

16. Dependencies

- Add a dependency only when it provides clear value over a small, maintainable implementation.
- Verify compatibility, maintenance status, license, bundle impact, and security before adoption.
- Use one library for one responsibility when possible; remove redundant packages.
- Pin resolved versions through the lockfile.
- Do not import internal paths from third-party packages unless their public documentation requires it.
- Isolate vendor-specific behavior behind adapters when replacement cost would otherwise spread through the codebase.

17. Change discipline

- Keep each change focused on one coherent outcome.
- Preserve existing behavior unless the change explicitly modifies it.
- Do not include unrelated refactors in a feature or bug fix.
- Remove unused code introduced or made obsolete by the change.

- Update tests, contracts, migrations, environment examples, and documentation together with the implementation.
- Review the final diff for secrets, debug output, generated noise, accidental file changes, and missing error handling.
- Ensure the codebase builds and the relevant validation commands pass before handoff.

18. Prohibited practices

- No emojis in source code, comments, logs, user-facing copy maintained in the repository, documentation, commit-ready examples, or test names.
- No mixed-language identifiers.
- No vague or redundant comments.
- No `any`, silent error swallowing, empty `catch` blocks, or ignored promises.
- No production `console.log` statements.
- No secrets, credentials, tokens, signed URLs, or private keys in the repository.
- No business rules implemented only in the frontend.
- No direct provider SDK usage outside its integration adapter.
- No arbitrary SQL string construction.
- No unbounded list endpoints.
- No committed dead code, commented-out code, temporary debug code, or unexplained magic values.
- No disabling of type checks, lint rules, security controls, or tests merely to make a change pass.

19. Definition of done

A change is complete only when all applicable statements are true:

- [ ] Code and technical identifiers are written in English.
- [ ] The implementation follows module boundaries and has a clear production-ready structure.
- [ ] External input is validated and permissions are enforced in the backend.
- [ ] Expected failures are handled and diagnosable through safe structured logs.
- [ ] Comments are technical, necessary, specific, and current.
- [ ] No emojis, temporary code, secrets, or vague placeholders remain.
- [ ] Formatting, linting, type checking, relevant tests, and build pass.
- [ ] Tests cover new behavior and important failure paths.
- [ ] API contracts, migrations, configuration examples, and documentation are updated when affected.
- [ ] The final diff contains only intentional changes.

Bull Dog Solution

Sistema de Gestión de Órdenes de Servicio

Manual de Usuario y Guía Operativa Paso a Paso

> Este documento sirve como **presentación interactiva** (compatible con Marp / Slides) y **manual de usuario vivo** para el equipo administrativo y técnico. Actualízalo conforme se integren nuevas características (*features*).

Índice General

1. Visión General y Propósito
2. Arquitectura y Roles del Sistema
3. Flujo General del Servicio
4. Módulo Administrador: Preparación y Asignación
5. Módulo Técnico: Operación en Campo (Mobile First)
6. Reglas del Checklist y Evidencias Fotográficas
7. Manejo de Discrepancias y Citas Fallidas
8. Módulo de Reportes y Descarga de Actas
9. Sincronización con Google Sheets y Resiliencia
10. Preguntas Frecuentes y Resolución de Problemas
11. Registro de Nuevas Funcionalidades (Changelog)

1. Visión General y Propósito

El sistema sustituye la operativa manual y las limitaciones de AppSheet por una plataforma web moderna, rápida y trazable.

- **Para la Oficina (Administradores):**
  - Control centralizado de órdenes, clientes, unidades y asignación de técnicos.
  - Conexión directa con Google Sheets para catálogos oficiales sin duplicidad.
  - Consulta de auditoría de órdenes y supervisión de reportes de servicio.
- **Para el Campo (Técnicos):**
  - Aplicación web optimizada para teléfonos móviles (PWA).
  - Verificación ágil de equipos telemáticos (GPS / Sensores).
  - Checklist digital obligatorio con fotografías y firma del cliente en pantalla.
  - Capacidad de reportar discrepancias sin detener el trabajo.

---

2. Arquitectura y Roles del Sistema

El sistema maneja dos perfiles de usuario claramente delimitados:

Rol	Entorno Principal	Responsabilidades Clave
ADMINISTRADOR	Web Desktop (Computadora)	• Crea órdenes en borrador. • Consulta clientes y equipos de Sheets. • Asigna y reasigna técnicos. • Consulta auditoría, reportes y supervisa sincronizaciones.
TÉCNICO	Web Mobile / PWA (Smartphone)	• Consulta órdenes asignadas a su cuenta individual o habilitadas para la cuenta compartida. • Inicia el servicio. • Valida equipos y registra discrepancias. • Completa checklist y evidencia. • Recoge firma del cliente y finaliza la orden.

3. Flujo General del Servicio

1. **Admin** crea Borrador
2. Asigna Unidades y Equipos
3. Asigna Técnico Responsable
4. **Técnico** inicia Servicio
5. Valida Hardware e IMEI
6. Checklist + evidencia mínima por unidad
7. Firma Digital Cliente
8. **Cierre y Reporte PDF**

Ruta de Excepción: En caso de unidad ausente o cliente no disponible → **Cierre como Cita Fallida (con motivo)**

Regla de oro: El técnico no puede cerrar un servicio normal si una unidad tiene menos de diez fotografías completadas, si falta alguna de las cinco categorías del checklist o si falta la firma con nombre del cliente. Las fotos se cuentan por unidad, no por categoría.

Administrar cuentas administradoras

Desde **Administración → Administradores**, una persona con perfil administrador puede crear una cuenta, editar nombre/correo, cambiar su contraseña o activar/desactivar otras cuentas. Al crearla, ingresa una contraseña inicial de al menos 12 caracteres y comunícala por un canal seguro; la app no la vuelve a mostrar. Al cambiar una contraseña, la persona debe usar la nueva en su próximo inicio de sesión.

La aplicación no permite desactivar tu propia cuenta ni dejar el sistema sin un administrador activo. Para desactivar una cuenta, confirma la acción en su fila. La sección es independiente de **Cuentas Técnicas**, que sigue dedicada a técnicos y a la cuenta compartida del taller.

Acceso técnico compartido

En **Cuentas Técnicas**, un administrador puede designar una cuenta activa con rol técnico y sin técnico individual asociado como cuenta compartida del taller. Luego puede habilitar para esa cuenta una sola orden asignada a un técnico individual y en estado ejecutable. La orden compartida aparece para la cuenta designada; no cambia su asignación ni concede acceso general a las demás órdenes. Cambiar la cuenta compartida revoca la orden habilitada anterior. El acceso se registra en auditoría.

4. Guía Administrador: 1. Crear Orden (Asistente de 5 Pasos)

1. Inicia sesión con tus credenciales de Administrador en el portal web.
2. Dirígete a la sección **Órdenes de Servicio** y haz clic en `+ Nueva Orden`. 3. **Paso 1: Cliente**:

- Selecciona el **Cliente** (catálogo oficial de Google Sheets).

4. **Paso 2: Servicios, Unidades y Equipos (Asignación Inteligente)**:
    - Agrega uno o varios bloques de servicio; una misma orden puede combinar:
        - `Instalación` : Colocación de nuevos equipos en vehículos.
        - `Revisión` : Diagnóstico, mantenimiento o inspección técnica de equipos ya instalados.
        - `Desinstalación` : Retiro de hardware instalado en la unidad.
        - `Sensor fuel` : Instalación de un equipo disponible en una unidad.
        - `Cambio de equipo` : Retiro del equipo instalado y colocación de uno nuevo.

- **En Revisiones y Desinstalaciones**:
  - El selector de equipos solo muestra los GPS con estatus **INSTALADO** que pertenecen al cliente seleccionado.
  - Al elegir un equipo, el sistema **autocompleta automáticamente** los datos de la unidad desde Google Sheets (`Económico` , `Marca/Modelo` , `Año` y `Número de Serie`), evitando capturas manuales innecesarias. El administrador puede revisar o afinar estos datos en pantalla.
- **En Instalaciones y Sensor fuel**:
  - El selector filtra **únicamente equipos en almacén/stock disponible**.
  - Se capturan los datos del vehículo donde se montará el nuevo equipo (Económico, Marca/Modelo, Año y Número de Serie/VIN).
- **En Cambios de equipo**:
  - Selecciona primero el equipo `INSTALADO` del cliente que será retirado.
  - Selecciona después el equipo disponible de almacén que lo reemplazará. La unidad se autocompleta a partir del equipo anterior.
- Permite agregar una o varias unidades/equipos según la flotilla a atender en la cita.

5. **Paso 3: Programación**:
   - Fecha y hora programada para la cita.
   - Indicador de servicio a domicilio / en sitio con captura de dirección física.
   - Notas generales e instrucciones para el técnico.
6. **Paso 4: Técnico Asignado**:
   - Selecciona obligatoriamente un técnico activo sincronizado desde `DIRECTORIO TECNICO` .
   - Puedes filtrar por ciudad o taller y buscar por nombre, taller o ciudad. La ficha muestra clave externa, ubicación y teléfono cuando están disponibles.
   - Si el directorio falla, pulsa **Reintentar**. Si está vacío, usa **Recargar directorio**. No podrás continuar hasta elegir un técnico disponible.
7. **Paso 5: Resumen y Confirmación**:
   - Revisa el resumen integral agrupado por servicio, con cliente, vehículos y hardware asignado.
   - Al pulsar **Crear Orden de Servicio** , el sistema genera un **Folio único**, crea las unidades y vincula los equipos atómicamente en estado **DRAFT (Borrador)**.

---

4. Guía Administrador: 2. Gestión de Unidades y Equipos en Detalle

Si requieres agregar o ajustar unidades y equipos posteriormente, puedes hacerlo desde el detalle de la orden:

1. Dentro de la orden en borrador, puedes agregar unidades adicionales o asociarles equipo mediante el selector [`EquipmentSelect`].
2. El selector mantiene las mismas reglas de negocio: muestra equipos disponibles para instalaciones y sensor fuel; instalados del cliente para revisiones y desinstalaciones; y ambos grupos para un cambio de equipo.
3. Las comparaciones de cliente aplican **normalización inteligente**: ignoran mayúsculas/minúsculas, acentos, espacios múltiples y variaciones de sufijos legales (ej. `"Transportes Bull Dog"` vs `"TRANSPORTES BULL DOG S.A. DE C.V."`), previniendo falsos conflictos.

---

4. Guía Administrador: 3. Asignación del Técnico y Sincronización

1. Con las unidades y equipos listos, selecciona el **Técnico Responsable** (obligatorio antes de confirmar).
2. Haz clic en el botón **Asignar y Notificar**:
   - El sistema realiza una revalidación fresca contra Google Sheets.
   - Si el equipo ya pertenece al cliente (incluso con diferencias de mayúsculas o sufijos), la verificación es exitosa.
   - La orden cambia a estado **ASSIGNED**.
   - La transacción crea operaciones Outbox y el worker actualiza en segundo plano las columnas autorizadas (`J`, `K`, `L`, `M`, `N`, `O`) en Google Sheets.
   - La orden aparece de inmediato en la bandeja de trabajo del técnico en su dispositivo móvil.

---

4. Guía Administrador: 4. Monitoreo y Estados

Desde la tabla principal de órdenes puedes filtrar por estado:

- `DRAFT` : En preparación por el administrador.
- `ASSIGNED` : Lista para que el técnico acuda a la cita.
- `IN_PROGRESS` : El técnico presionó "Iniciar Servicio" en campo.
- `COMPLETED` : Servicio concluido con checklist, fotos y firma.
- `REQUIRES_CORRECTION` : Observada por administración para revisión.
- `FAILED_APPOINTMENT` : Cita no realizada (con motivo registrado).
- `CANCELLED` : Cancelada antes de su ejecución.

4. Guía Administrador: 5. Tarifas Acordadas y Ajustes de Liquidación

En el detalle de la orden administrativa, dentro de la tarjeta de **Liquidación Técnica**:

1. **Configuración Previa (`ASSIGNED` , `IN_PROGRESS`):**
   - Pulsa el botón **`Configurar Tarifas Acordadas`** .
   - Asigna tarifas acordadas para los tipos de servicio requeridos y/o ajusta el costo de viáticos/salida.
   - Al guardar, las tarifas personalizadas quedan fijadas en la orden y se aplican de forma automática al completarse el servicio.
2. **Ajuste Post-Cierre (`COMPLETED` , `FAILED_APPOINTMENT`):**
   - Si tras la finalización se detectan diferencias o negociaciones extraordinarias, pulsa **`Ajustar Precios de Liquidación`** .
   - Modifica las tarifas unitarias o viáticos e ingresa una justificación para auditoría.
   - El sistema incrementa la versión contable, recalcula subtotales y encola la re-sincronización hacia la columna de costo en Google Sheets.

---

5. Guía Técnico: 1. Bandeja Móvil

1. Accede desde el navegador de tu celular al enlace de la aplicación (puedes agregarla a la pantalla de inicio como PWA).
2. Ingresa con tu correo y contraseña de técnico.
3. En la pantalla principal verás tu lista de **Servicios Asignados**:
   - Verás el folio, cliente, dirección o taller, y número de unidades.
4. Al llegar a la ubicación del vehículo, presiona la orden y pulsa el botón verde **Iniciar Servicio**:
   - El estado de la orden pasa a `IN_PROGRESS` registrando la hora de inicio.

---

5. Guía Técnico: 2. Verificación de Equipos

Por cada unidad asignada, debes inspeccionar físicamente el hardware:

1. Selecciona la unidad y entra a la sección **Equipos Asignados**.
2. Compara el IMEI y el Número de Serie físico contra lo indicado en la pantalla.
3. Si los datos coinciden exactamente, pulsa **Verificar (Coincide)**.
4. **¿Qué hacer si hay un error o el equipo es diferente?**
   - Pulsa **Reportar Discrepancia**.
   - Escribe el IMEI o serie real que tienes en mano y una breve nota.
   - El equipo se marca con alerta visual, pero **te permite continuar trabajando**. ¡No te bloquea el servicio!

---

6. Reglas del Checklist Post-Servicio

Al terminar los trabajos mecánicos/eléctricos, debes realizar la inspección de entrega en cada unidad. Existen **5 categorías obligatorias**:

1. **Tablero (`DASHBOARD`)**
2. **Cristales (`WINDOWS`)**
3. **Accesorios (`ACCESSORIES`)**
4. **Molduras (`TRIMS`)**
5. **Carrocería (`BODYWORK`)**

Estados permitidos por categoría:

- **`Bien`** (**`GOOD`**): Todo en orden sin daños causados ni preexistentes.
- **`Mal`** (**`BAD`**): **Requiere escribir obligatoriamente una observación detallada** del daño o anomalía.
- **`No aplica`** (**`NOT_APPLICABLE`**): La unidad no cuenta con dicho componente.

---

6. Reglas de Evidencias Fotográficas

> **Cada unidad requiere al menos diez fotografías completadas.** Las fotos son una galería general de la unidad y no se exigen por categoría del checklist.

1. Pulsa sobre el icono de cámara en la categoría correspondiente.
2. Toma una foto clara y bien iluminada desde tu teléfono. En Drive la carga se transmite de forma autenticada a la API y puede tardar más que una transferencia directa a un almacén de objetos.
3. La foto se carga automáticamente al almacenamiento privado mediante la aplicación.
4. El sistema mostrará una miniatura con confirmación verde.
5. Si alguna unidad no tiene las diez fotos requeridas, la aplicación indicará el requisito pendiente y no permitirá finalizar.

7. Firma del Cliente y Finalización

Cuando todas las unidades tengan sus equipos verificados y sus checklists completos con fotos:

1. Se habilitará el paso final: **Conformidad del Cliente**.
2. Ingresa el **Nombre Completo** de la persona que recibe el servicio.
3. Opcionalmente registra su cargo o documento de identidad.
4. Pídele al cliente que firme directamente en la pantalla de tu celular en el recuadro táctil.
5. En la sección **Notas de ejecución**, agrega las observaciones generales del servicio si aplica. Puedes guardarlas antes de finalizar; si quedan cambios pendientes, el sistema las guarda automáticamente al solicitar el cierre. Estas notas pertenecen a toda la orden, no a una unidad específica.
6. Pulsa **Finalizar Orden de Servicio**:
   - La orden pasa a **COMPLETED**.
   - Se sella la fecha y hora final.
   - El sistema solicita en segundo plano una versión privada del reporte PDF; su disponibilidad puede tardar mientras el worker lo procesa.
   - Se encolan el reporte técnico y el estado final de cada equipo en `SALIDAS`. En desinstalaciones y equipos retirados se limpian cliente y vehículo; en instalaciones se registra el nuevo vínculo.

---

7. Manejo de Citas Fallidas

Si acudes al lugar pero el servicio no se puede realizar (cliente ausente, unidad en viaje, vehículo descompuesto, negativa de acceso):

1. En la pantalla de la orden iniciada, selecciona el botón **Registrar Cita Fallida**.
2. Selecciona o escribe el **Motivo Detallado de la Falla** (obligatorio).
3. Confirma la acción:
    - La orden pasa al estado **FAILED_APPOINTMENT**.
    - No se exige checklist, fotografías ni firma del cliente.
    - La orden queda disponible para que administración revise, cancele o reprograme; el sistema no envía una notificación externa automática.

El administrador puede resolverla desde el detalle de la orden:

- **Reprogramar:** solicita la nueva fecha y crea una copia enlazada en estado `DRAFT`. La orden fallida original conserva su motivo, cobro, reporte e historial.
- **Cancelar servicio:** cierra administrativamente la cita fallida sin borrar el registro ni el cobro de la visita realizada.

---

8. Módulo de Reportes: Actas en PDF

Para cada orden completada, el sistema genera automáticamente un **Acta / Certificado de Servicio**:

- **¿Dónde consultarlo?**
  El administrador y el técnico con acceso a la orden pueden consultar o descargar el PDF privado desde el detalle. El cliente no tiene una cuenta ni un enlace público de descarga integrado.
- **¿Qué incluye el documento?**
  - Folio e identificación del servicio (la plantilla/logotipo oficial sigue pendiente de aprobación).
  - Tiempos de atención (inicio y término).
  - Datos de unidades y tabla de hardware verificado con notas de discrepancias.
  - Tabla del checklist vehicular (Bien / Mal / Observaciones).
  - Galería de fotografías incrustadas con marca de fecha y hora.
  - Firma digitalizada y nombre del cliente firmante.

8. Auditoría y sincronización

No hay un módulo de reportes ejecutivos/exportación XLSX o CSV en la interfaz actual. La auditoría se consulta desde el detalle administrativo de la orden y la sección **Sincronización** permite supervisar operaciones del reporte técnico y cambios de equipos en Google Sheets.

9. Sincronización con Google Sheets y Resiliencia

El sistema está diseñado para que una falla en Internet o en las cuotas de Google no detenga la operación:

- **Patrón Outbox:**
  Cuando se asigna un equipo o se actualiza una orden, la acción se guarda primero en la base de datos PostgreSQL y un worker la envía a Google Sheets en segundo plano.
- **Reintentos Automáticos:**
  Si Google Sheets no responde, la operación permanece registrada para reintento periódico; un administrador también puede reintentarla desde la sección de sincronización.
- **Monitor de Sincronización:**
  Los administradores cuentan con la sección **Sincronización**, donde pueden ver reportes programados, procesando, sincronizados o fallidos y ejecutar **Sincronizar ahora** cuando corresponda.
- **Corte semanal del reporte técnico:** La configuración inicial acepta órdenes cerradas hasta el martes a las 23:59:59 y publica el lote el miércoles a las 17:00, usando `America/Hermosillo`. El administrador puede cambiar el día de corte, el día de envío, la hora y la zona horaria. Las órdenes cerradas después del corte se publican en la ventana de la semana siguiente.
- **Diferencia entre órdenes y operaciones:** El resumen muestra por defecto solo el reporte técnico. El filtro **Equipos** muestra las actualizaciones internas de `SALIDAS`; una orden con varios equipos genera varias operaciones, pero no varias filas del reporte técnico. Los intentos se muestran dentro de cada operación y no se cuentan como órdenes nuevas.
- **Correcciones:** Si una corrección crea una versión nueva antes de la publicación, la versión pendiente anterior queda como `SUSTITUIDA`. Los fallos ocurridos al publicar se reintentan con backoff sin esperar hasta la siguiente semana.

---

10. Preguntas Frecuentes y Resolución de Problemas

- **¿Qué hago si no hay señal celular en el taller?**
  La aplicación no ofrece operación offline completa ni sincronización diferida garantizada. Conserva el trabajo localmente por medios aprobados y carga la información cuando tengas conexión; verifica que fotos y cambios aparezcan confirmados antes de cerrar.
- **¿Puedo cambiar de técnico una orden ya asignada?**
  Sí, el administrador puede reasignarla desde el panel web mientras la orden no haya sido finalizada.
- **¿Por qué no se habilita el botón de finalizar orden?**
  Verifica que: (1) Todas las unidades tengan las cinco categorías del checklist, (2) Cada unidad tenga al menos diez fotos completadas, (3) Todos los equipos estén verificados o con discrepancia reportada, y (4) Se haya ingresado el nombre y firma del cliente.

11. Registro de Nuevas Funcionalidades (Changelog)

> *Utiliza esta sección para registrar los cambios que se agreguen a la aplicación en fases posteriores.*

Fecha	Versión	Tipo	Descripción del Cambio / Nueva Feature	Responsable
2026-09-02	v1.0.0	Lanzamiento	MVP del Sistema: Órdenes, checklist de 5 categorías, evidencia privada, firma y sincronización Sheets.	Equipo Core
2026-09-11	v1.1.0	Mejora	Corte y envío semanal configurables para el reporte técnico, tablero global y reintento inmediato por operación.	Equipo Core
2026-09-10	v1.1.0	Feature	Órdenes multiservicio, directorio técnico sincronizado, tarifas por servicio, reporte de cobro y actualización final de inventario.	Equipo Core
2026-09-28	v1.2.0	Feature	Tarifas acordadas por orden, reajuste post-cierre y formateo multilínea de costo en reporte técnico.	Equipo Core
Propuesta no implementada	—	Idea	Modo offline progresivo y envío automático de reportes por correo requieren diseño, autorización y priorización; no están disponibles.	Por definir

¿Preguntas o Soporte Técnico?

Para reportar fallas en el sistema o sugerir mejoras:

- **Área de Operaciones:** operaciones@bulldogsolution.com
- **Soporte de Plataforma:** soporte-ti@bulldogsolution.com
- **Repositorio y Documentación Técnica:** SPECS.md | ERS.md

Bull Dog Solution — Órdenes de servicio

Sistema web responsivo para preparar, asignar, ejecutar y auditar órdenes multiservicio de equipos en unidades vehiculares.

> Este README resume el producto implementado en la rama actual. ROADMAP.md conserva el seguimiento de trabajo y SPECS.md la especificación funcional/técnica; para el estado de ejecución y despliegue, consulta la documentación técnica.

La documentación del estado técnico implementado, incluidos los diagramas de arquitectura y entidad-relación, está disponible en docs/technical/README.md.

> **Estado de AWS (verificado al 2026-10-09):** staging está eliminado/no desplegado y producción no está desplegada. No hay endpoint activo conocido. El estado y la conservación de datos históricos de RDS o archivos no se han confirmado; no asumir que se perdieron ni que siguen disponibles. `infra/cdk` describe la arquitectura prevista para un futuro redeploy, no recursos activos. Consultá Infraestructura y operación antes de usar procedimientos de despliegue.

Descripción

La plataforma sustituye el flujo operativo realizado en AppSheet y reúne en un solo lugar el trabajo administrativo y de campo:

- Los administradores crean órdenes, seleccionan clientes, registran unidades, asignan equipos y eligen al técnico responsable.
- Los técnicos consultan sus servicios, verifican equipos, documentan cada unidad con checklist y fotografías, reportan discrepancias y obtienen la firma del cliente.
- Los técnicos pueden guardar una nota general de ejecución antes del cierre; las notas no se separan por unidad.
- Google Sheets conserva la autoridad sobre los catálogos de clientes y equipos.
- PostgreSQL mantiene los datos transaccionales, el historial y el estado de sincronización.
- Las fotografías, firmas y reportes se almacenan de forma privada mediante un adaptador de proveedor. La IaC local configura Google Drive compartido, pero la integración aún requiere configuración y validación antes de un redeploy; el último inventario histórico observó S3. S3 permanece como adaptador alternativo en el código.
- Los reportes PDF de órdenes completadas pueden enviarse manualmente por Gmail a destinatarios guardados para el cliente. No se envían automáticamente; Gmail usa delegación de dominio y todavía no está conectado al task definition CDK.

Funcionalidades principales

Administración

- Autenticación segura y administración local de usuarios y técnicos.
- Creación de órdenes con varias unidades, varios equipos por unidad y distintos tipos de servicio en una misma orden.
- Consulta de clientes y equipos directamente desde Google Sheets.
- Asignación y reasignación de técnicos.
- Configuración de tarifas acordadas por orden (pre-ejecución) y reajuste de liquidación técnica post-cierre con auditoría y re-sincronización automática a Google Sheets.
- Seguimiento del estado y progreso de cada servicio.
- Detección visible de conflictos cuando una fila de Sheets cambia externamente.
- Consulta de evidencias, firma, discrepancias, auditoría y sincronización.
- Reporte PDF de cada orden completada con portada, detalle por unidad/equipo y fotografías en formato amplio.
- Reintento controlado de operaciones fallidas con Google Sheets.

Operación técnica

- Vista mobile-first de las órdenes asignadas.
- Verificación individual de equipos.
- Reporte de diferencias de ID, IMEI, serie o unidad sin bloquear el trabajo.
- Checklist posterior al servicio para tablero, cristales, accesorios, molduras y carrocería.
- Estados `Bien`, `Mal` y `No aplica`, con observación obligatoria para elementos en mal estado.
- Captura de al menos diez fotografías por unidad; la evidencia es una galería general y no se exige por categoría del checklist.
- Firma única del cliente y registro del nombre de la persona firmante.
- Cierre normal con validación integral o cierre simplificado como cita fallida.

Seguridad y trazabilidad

- Access y refresh tokens JWT con rotación y revocación de sesiones.
- Autorización por rol y pertenencia de la orden.
- Validación de todas las entradas mediante Valibot.
- Almacenamiento privado con autorización; Drive entrega archivos por la API y el adaptador de objetos usa URLs prefirmadas de corta duración.
- Auditoría de operaciones administrativas y cambios relevantes.
- Idempotencia, concurrencia optimista y outbox para efectos externos.
- Logs estructurados con redacción de secretos y datos sensibles.

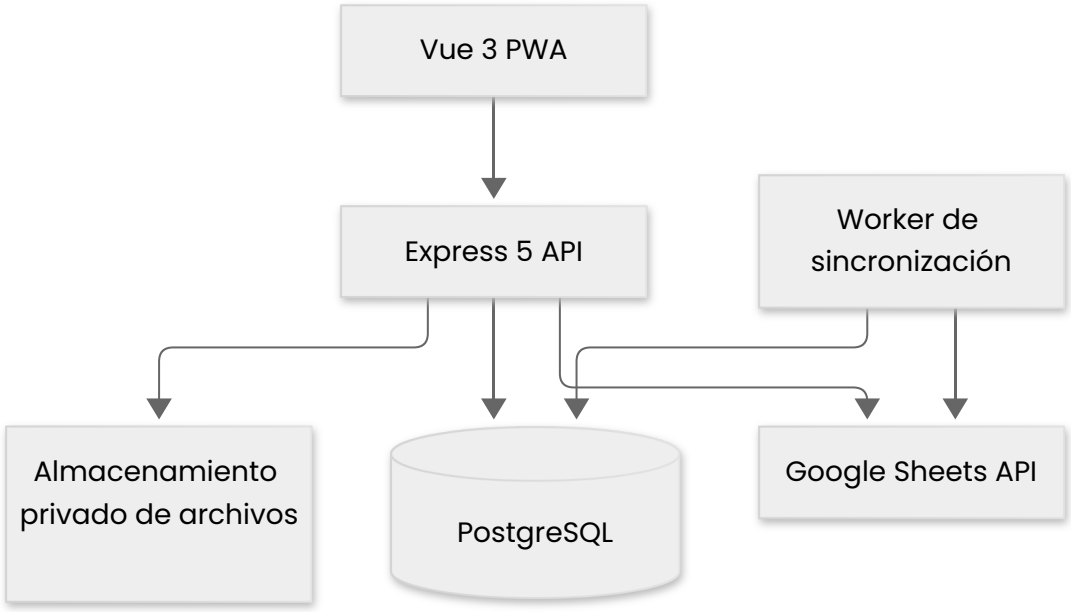
Flujo del servicio



Una discrepancia puede registrarse durante la verificación y no impide finalizar la orden cuando los demás requisitos están completos.

Arquitectura

El proyecto utiliza un monolito modular en un monorepo. El frontend y la API se desarrollan y construyen por separado, mientras que las reglas de negocio permanecen en el backend.



Capa	Tecnologías
Lenguaje	TypeScript
Frontend	Vue 3, Vite, Vue Router, Pinia, TanStack Query
Interfaz	Naive UI, Tailwind CSS, VueUse
API	Node.js, Express 5, Valibot
Persistencia	PostgreSQL, Prisma
Autenticación	JWT, Argon2id
Integraciones	Google Sheets API, Google Drive compartido privado para evidencias, firmas y reportes
Documentación	OpenAPI 3.1, Swagger UI
Pruebas	Vitest, Supertest, Playwright

Capa	Tecnologías
Infraestructura	Docker Compose local; AWS CDK para staging

Estructura del repositorio

```
.
├── apps/
│   ├── api/              # API Express y módulos de negocio
│   └── web/              # Aplicación Vue y PWA
├── packages/
│   ├── contracts/        # DTOs y esquemas Valibot compartidos
│   ├── eslint-config/    # Configuración común de lint
│   └── typescript-config/  # Configuración común de TypeScript
├── prisma/
│   ├── migrations/       # Historial de migraciones
│   ├── schema.prisma     # Modelo de datos
│   └── seed.ts           # Datos iniciales de desarrollo
├── infra/
│   ├── cdk/              # Stacks AWS de staging
│   ├── docker/           # Imágenes y configuración local/ECS
│   └── scripts/          # Operación, smoke tests y E2E
├── docker-compose.yml
├── pnpm-workspace.yaml
├── ROADMAP.md
└── SPECS.md
```

Los módulos siguen slicing vertical. Cada dominio contiene sus rutas, controladores, servicios, persistencia, contratos y pruebas. Google Sheets y los proveedores de almacenamiento se consumen a través de interfaces/adaptadores reemplazables.

Estados de una orden

Una orden normal sigue este recorrido:

```
DRAFT -> ASSIGNED -> IN_PROGRESS -> COMPLETED
              -> REQUIRES_CORRECTION -> IN_PROGRESS
    -> CANCELLED
```

Una cita fallida utiliza el flujo:

```
ASSIGNED -> IN_PROGRESS -> FAILED_APPOINTMENT
```

El administrador resuelve una cita fallida cancelando el servicio o reprogramándolo. La reprogramación conserva la orden fallida como historial y crea una copia enlazada en estado

`DRAFT`, con una nueva fecha y un nuevo folio.

Para completar un servicio normal, todas las unidades deben tener sus equipos verificados, checklist completo y fotografías requeridas. La orden también debe incluir el nombre y la firma del cliente. Una cita fallida requiere únicamente un motivo.

Integración con Google Sheets

Google Sheets permanece como fuente oficial de los catálogos externos:

- `Clientes`: lectura de `IDCliente` y `Cliente`.
- `SALIDAS`: lectura del catálogo de equipos y actualización limitada de su asignación.
- `DIRECTORIO TECNICO` y `2026`: directorio y reporte en el libro configurado por `GOOGLE_TECHNICIANS_SPREADSHEET_ID`. Cada cobro nuevo se inserta en la fila 2 del reporte y desplaza los cobros anteriores hacia abajo.

La aplicación solo puede escribir las columnas autorizadas:

Columna	Contenido
`J`	Cliente
`K`	Estado del servicio
`L`	Unidad
`M`	Marca/modelo de la unidad
`N`	Año
`O`	Número de serie
`P`	Tipo de servicio, cuando el negocio lo habilite

Las columnas `A` e `I` nunca son modificadas. Al asignar y al completar, la API vuelve a localizar el equipo por ID o IMEI y escribe únicamente las celdas necesarias. Instalación, revisión y sensor fuel dejan el equipo como `INSTALADO`; desinstalación y el equipo anterior de un cambio lo devuelven a `STOCK TECNICO`, limpian el cliente (`J`) y los datos del vehículo (`L:O`). El equipo nuevo de un cambio hereda el cliente y el vehículo. Una cita fallida no modifica el inventario.

Las actualizaciones de equipos se procesan inmediatamente. La publicación en el reporte técnico usa una ventana semanal configurable desde **Sincronización**: por defecto, acepta órdenes cerradas hasta el martes al finalizar el día y las publica el miércoles a las 17:00 en `America/Hermosillo`. Las órdenes cerradas después del corte pasan a la semana siguiente. El panel muestra por defecto operaciones del reporte técnico; las operaciones internas de equipos se pueden consultar con el filtro de tipo y no deben interpretarse como órdenes adicionales. En el reporte técnico (`2026`), cuando una orden cuenta con múltiples unidades, las columnas F (`economico`), G (`serie`), H (`imeiId`), K (`servicio`) y L (`costo`) se registran multilínea (`\n`) alineando cada fila por unidad en estricto orden cronológico.

Requisitos

- Node.js 24.20.0 LTS, fijado en `.nvmrc` y `.node-version`.
- pnpm 11.9.0 mediante Corepack.
- Docker y Docker Compose.
- Credenciales de Google Sheets y del proveedor de almacenamiento elegido para entornos no locales (en desarrollo local se cuenta con emuladores y mocks en memoria integrados).

Puesta en marcha local

1. Activar la versión fijada de pnpm e instalar las dependencias:

   ```
   corepack enable
   pnpm install --frozen-lockfile
   ```

2. Crear la configuración local a partir del archivo de ejemplo:

   ```
   cp .env.example .env
   ```

   > En desarrollo local, los marcadores `development-only…` activan el mock de Google Sheets y claves JWT efímeras. El almacenamiento local se configura por Docker Compose con MinIO; no implica que staging use MinIO.

3. Para trabajar con recarga en caliente, levantar PostgreSQL y MinIO:

   ```
   docker compose up -d postgres minio minio-init
   ```

4. Generar el cliente de Prisma, aplicar las migraciones y cargar datos iniciales:

   ```
   pnpm prisma:generate
   pnpm prisma:migrate
   pnpm prisma:seed
   ```

   El seed local crea los siguientes usuarios de prueba con la contraseña por defecto `development-only-change-me` (configurable mediante `SEED_ADMIN_PASSWORD`):

   - **Administrador:** `admin@example.invalid`
   - **Técnico:** `technician@example.invalid`

5. Iniciar API y web:

   ```
   pnpm dev
   ```

La aplicación web queda disponible por defecto en `http://localhost:5173` y la API en `http://localhost:3000`. `VITE_DEV_PORT`, `VITE_API_PROXY_TARGET` y `PORT` permiten usar puertos diferentes sin modificar código. El comando compila primero los contratos compartidos (`@puk/contracts`) antes de levantar ambos servidores en recarga reactiva. La interfaz no ofrece modo offline completo: las respuestas autenticadas no se cachean.

Como alternativa, el stack completo compilado se levanta con:

```
docker compose up --build -d
```

En esta modalidad, la web se publica por defecto en `http://localhost:4173`, la API en `http://localhost:3000` y la consola de MinIO en `http://localhost:9001`. Los puertos pueden sobrescribirse en `.env` con las variables documentadas en `.env.example`.

Configuración

La aplicación valida su configuración al iniciar y se detiene si falta una variable obligatoria. Los grupos principales son:

Grupo	Variables principales
Aplicación	`NODE_ENV`, `PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL`
JWT	Claves pública/privada, issuer, audience y TTL de cada token
Cookies	Dominio, modo seguro y política SameSite
Objetos	Configuración y credenciales del proveedor de almacenamiento
Google Sheets	IDs de catálogos y técnicos, cuenta de servicio y pestañas
Límites	Tamaños de evidencia/firma, TTL de URLs y caché de catálogos
Resiliencia	Timeouts, reintentos, circuit breaker y frecuencia del outbox
Programación	Día de corte, día/hora de envío y zona del reporte técnico
Logs	`LOG_LEVEL`

Los secretos no deben guardarse en el repositorio. En producción se utilizan identidades administradas o un gestor de secretos cuando el proveedor lo permita.

Comandos disponibles

Comando	Descripción
`pnpm dev`	Inicia API y web en desarrollo
`pnpm build`	Construye todas las aplicaciones y paquetes

Comando	Descripción
`pnpm format`	Formatea los archivos del repositorio
`pnpm format:check`	Verifica el formato sin escribir cambios
`pnpm lint`	Ejecuta el análisis estático
`pnpm typecheck`	Comprueba TypeScript estricto
`pnpm test`	Ejecuta pruebas unitarias y de integración
`pnpm test:e2e`	Ejecuta escenarios Playwright
`pnpm test:e2e:mock`	Ejecuta escenarios Playwright con dependencias simuladas
`pnpm docs:build`	Genera OpenAPI 3.1
`pnpm docs:validate`	Valida el contrato de la API
`pnpm prisma:generate`	Genera el cliente de Prisma en `apps/api`
`pnpm prisma:migrate`	Aplica migraciones de base de datos
`pnpm prisma:seed`	Carga datos iniciales de desarrollo
`pnpm prisma:validate`	Valida la configuración y el esquema Prisma

API y documentación

La API utiliza el prefijo `/api/v1` y responde en JSON con un `requestId` para trazabilidad.

- Especificación OpenAPI: `/api/v1/openapi.json`.
- Swagger UI: `/api/docs`.
- Estado de proceso: `/health/live`.
- Estado de dependencias: `/health/ready`.

Los endpoints abarcan autenticación, usuarios, técnicos, catálogos externos, órdenes, unidades, equipos, checklist, evidencias, firma, auditoría y sincronización.

Pruebas

La estrategia combina:

- Pruebas unitarias para validaciones, transiciones, autorización, rotación de tokens e identificación de equipos.
- Pruebas de integración para Express, PostgreSQL, almacenamiento de objetos, Sheets simulado, outbox e idempotencia.
- Pruebas de contrato para comparar OpenAPI con respuestas reales de la API.
- Pruebas end-to-end de los recorridos completos de administrador y técnico.

Los escenarios automatizados nunca escriben en la hoja de producción.

Despliegue

La estrategia objetivo separa desarrollo, staging y producción. Actualmente sólo está disponible el desarrollo local; staging no está desplegado y producción no cuenta con infraestructura propia:

- Frontend estático servido mediante CDN o reverse proxy.
- API ejecutada como contenedor Node.js.
- PostgreSQL administrado con respaldos automáticos y restauración probada.
- Google Drive compartido privado para evidencias, firmas y reportes.
- Reverse proxy con TLS y, preferentemente, API bajo el mismo dominio público.
- Google Sheets distinto para staging y producción.

CI ejecuta instalación reproducible, lint, typecheck, pruebas, generación de OpenAPI, builds, escaneos y síntesis de CDK. El workflow declarado despliega después de un CI exitoso sobre `main` sólo si `AWS_STAGING_READY` vale exactamente `true`; actualmente staging no está desplegado.

La IaC local define staging en AWS `us-east-2` con ECS sobre EC2, RDS PostgreSQL, Secrets Manager y CloudWatch; no confirma que esos recursos existan actualmente. CDK configura Google Drive compartido para evidencias, firmas y reportes, e inyecta sus credenciales desde Secrets Manager; su configuración debe validarse antes de redeploy. La topología declarada usa HTTP en una IPv4 efímera, sin CloudFront ni ALB, y no debe usarse con datos reales.

Durante el build Docker, las imágenes de API y migración generan Prisma Client con una URL ficticia de compilación; no se conectan a la base real para generarlo. Al iniciar una nueva tarea ECS, el contenedor de migración ejecuta `prisma migrate deploy` y luego el seed; la API espera que termine con éxito. Sólo se aplican migraciones pendientes versionadas en `prisma/migrations/`: cambiar únicamente `prisma/schema.prisma` no modifica la base de datos. Producción aún no tiene stacks IaC propios en la rama actual. Consulte Infraestructura y operación para el estado, diagramas objetivo, persistencia, CI/CD y brechas antes de producción, y Staging y despliegue para los pasos de un futuro redeploy.

Alcance del MVP

El MVP no incluye operación offline completa, aplicaciones móviles nativas, edición de los catálogos externos, facturación, cuentas bancarias ni microservicios. No modifica automáticamente `Fecha de salida` ni `Estado` del equipo en Google Sheets; sí administra `Estado del servicio` conforme al resultado de la orden.

Documentación del proyecto

- Especificación técnica
- Ruta de implementación

Ruta de trabajo — Sistema de órdenes de servicio

Ruta de implementación del MVP basada en `SPECS.md`. Cada fase debe terminar con sus criterios de salida cumplidos antes de avanzar a la siguiente.

> **Estado actual (2026-10-02):** las casillas de fases anteriores registran trabajo histórico. El producto implementa los flujos principales, pero AWS staging fue eliminado y no hay despliegue actual; producción aún no tiene infraestructura propia. La IaC de staging selecciona Google Drive, no S3, para evidencias, firmas y reportes. Ver infraestructura y operación.

0. Definiciones y preparación

- [x] Confirmar la versión LTS de Node.js y fijar las versiones compatibles mediante `pnpm-lock.yaml`.
- [x] Obtener una copia de Google Sheets exclusiva para desarrollo y staging.
- [x] Validar en la hoja los encabezados y permisos de `Clientes` y `SALIDAS`.
- [x] Revisar duplicados, valores incompletos y casos especiales de ID e IMEI.
- [x] Confirmar si la columna `P` se escribirá y, en su caso, definir su catálogo válido (decisión temporal documentada).
- [x] Acordar qué ocurre en Sheets al cancelar una orden ya sincronizada (decisión temporal documentada).
- [x] Acordar qué ocurre si una unidad cambia después de asignar la orden (decisión temporal documentada).
- [x] Definir la regla de disponibilidad mientras `Estado del Servicio` (columna `K`) no cambie.
- [x] Definir límites finales de cantidad y tamaño de fotografías para desarrollo.
- [x] Documentar variables de entorno, secretos requeridos y responsables de cada credencial.
- [x] Crear una matriz de trazabilidad entre reglas de negocio, endpoints y pruebas de aceptación.

Criterio de salida

- [x] Las decisiones que bloquean la integración con Sheets están resueltas o registradas explícitamente con una alternativa temporal para desarrollo.
- [x] El equipo dispone de credenciales y recursos de desarrollo que no apuntan a producción.

1. Base del monorepo

- [x] Crear el workspace de `pnpm` con `apps/web`, `apps/api`, `packages/contracts`, configuraciones compartidas, `prisma` e `infra/docker`.
- [x] Configurar TypeScript estricto para API, web y paquetes compartidos.
- [x] Configurar ESLint, formato, aliases y límites de importación entre módulos.
- [x] Crear la aplicación Express 5 con `app.ts` y `server.ts` separados.
- [x] Crear la aplicación Vue 3 con Vite, Vue Router, Pinia y TanStack Query.
- [x] Integrar Naive UI y Tailwind CSS mediante el plugin de Vite.
- [x] Configurar Docker Compose con PostgreSQL, MinIO, API y web.
- [x] Implementar validación de variables de entorno con Valibot y fallo temprano al iniciar.
- [x] Añadir scripts raíz para desarrollo, build, lint, typecheck, pruebas, documentación y Prisma.
- [x] Crear archivos de ejemplo de configuración sin incluir secretos reales.

Criterio de salida

- [x] `pnpm dev`, `pnpm build`, `pnpm lint` y `pnpm typecheck` funcionan desde la raíz.
- [x] API, web, PostgreSQL y MinIO pueden levantarse en desarrollo local.

2. Fundamentos transversales de la API

- [x] Implementar `requestId`, respuesta estándar de éxito y manejador global de errores.
- [x] Configurar Helmet, CORS con allowlist, límite JSON, cookies y `trust proxy`.
- [x] Configurar rate limiting general y límites particulares para autenticación y operaciones costosas.
- [x] Integrar Winston con logs estructurados y Morgan mediante un stream sanitizado.
- [x] Redactar cookies, tokens, contraseñas, secretos, URLs firmadas y datos privados en logs.
- [x] Implementar middleware Valibot para `params`, `query` y `body`.
- [x] Implementar paginación, filtros y ordenamiento mediante allowlists por endpoint.
- [x] Crear el modelo de capacidades de código y políticas contextuales por recurso.
- [x] Preparar OpenAPI 3.1, Swagger UI y generación de esquemas compatibles desde Valibot.
- [x] Añadir `/health/live` y `/health/ready`.
- [x] Definir puertos para Google Sheets y S3, sin acoplar casos de uso a sus SDK.
- [x] Configurar timeouts, clasificación de errores, reintentos y circuit breakers por integración.

Criterio de salida

- [x] Las rutas de ejemplo entregan el formato común, validan entradas y propagan `requestId`.
- [x] Los logs y errores no exponen información sensible.
- [x] El documento OpenAPI se genera y valida sin referencias rotas.

3. Persistencia y modelo de dominio

- [x] Modelar en Prisma usuarios, técnicos, órdenes, unidades, equipos asignados, checklist e items.
- [x] Modelar evidencias, firmas, discrepancias, sesiones de refresh, auditoría y operaciones de sincronización.
- [x] Añadir enumeraciones y restricciones de unicidad descritas en la especificación.
- [x] Crear la restricción SQL parcial que impide asignar un equipo a dos órdenes activas.
- [x] Implementar `Order.version` para concurrencia optimista.
- [x] Incorporar tablas de outbox o la persistencia equivalente para efectos externos críticos.
- [x] Crear migraciones iniciales, índices de consulta y un seed mínimo de desarrollo.
- [x] Definir helpers transaccionales para cambios de dominio, auditoría y outbox en una sola transacción.

Criterio de salida

- [x] La base puede crearse desde cero ejecutando migraciones y seed.
- [x] Las restricciones de unicidad, propiedad y concurrencia cuentan con pruebas automatizadas.

4. Autenticación, usuarios y técnicos

- [x] Implementar hash de contraseñas con Argon2id y política configurable de longitud mínima.
- [x] Implementar login con respuesta genérica ante credenciales inválidas.
- [x] Emitir access tokens de corta duración con `sub`, `role`, `iss` y `aud`.
- [x] Emitir refresh tokens en cookie segura y almacenar únicamente su hash.
- [x] Implementar rotación de refresh token, revocación individual y cierre de todas las sesiones.
- [x] Implementar `authenticate`, capacidades por rol y políticas de pertenencia.
- [x] Crear endpoints administrativos de usuarios y técnicos.
- [x] Crear las pantallas de login, sesión y directorio de técnicos.
- [x] Configurar la instancia Axios con token en memoria, refresh único compartido y un solo reintento.

- ☑ Añadir auditoría para creación, edición, activación y desactivación de usuarios y técnicos.
- ☑ Agregar administración separada de cuentas ADMIN, con protección contra auto-bloqueo y desactivación del último administrador.
- ☑ Agregar acceso compartido de taller: cuenta TECHNICIAN designada y una orden habilitada a la vez, con revisión optimista y auditoría.

Criterio de salida

- ☑ Un administrador y un técnico pueden iniciar y cerrar sesión según su rol.
- ☑ La rotación, expiración y revocación de tokens están cubiertas con pruebas.
- ☑ El navegador no almacena JWT en `localStorage` ni `sessionStorage`.

5. Catálogos de Google Sheets

- ☑ Implementar el adaptador autenticado de Google Sheets exclusivamente en la API.
- ☑ Leer y validar clientes de la pestaña `Clientes`.
- ☑ Leer, normalizar y validar equipos de la pestaña `SALIDAS`.
- ☑ Implementar la identidad por ID e IMEI, incluyendo `PENDIENTE`, `N/A` y datos incompletos.
- ☑ Implementar caché de corta duración con fecha de obtención e invalidación manual.
- ☑ Forzar una lectura fresca antes de cualquier asignación.
- ☑ Crear endpoints de búsqueda, detalle/revalidación y refresco de catálogos.
- ☑ Crear selectores administrativos de cliente y equipos con estados de carga, error y recarga.
- ☑ Simular respuestas de Sheets en pruebas; ninguna prueba automatizada debe escribir en producción.

Criterio de salida

- ☑ La UI permite buscar clientes y equipos y muestra cuándo se obtuvo el catálogo.
- ☑ Un equipo sin identidad válida no puede seleccionarse.
- ☑ Los casos de filas movidas, duplicadas o modificadas están cubiertos por pruebas.

6. Creación administrativa de órdenes

- ☑ Implementar los servicios y endpoints para crear y editar órdenes en borrador.
- ☑ Implementar alta, edición y eliminación de unidades mientras la orden esté en borrador.
- ☑ Impedir números de serie duplicados dentro de una orden.
- ☑ Implementar asignación y desasignación de varios equipos por unidad.
- ☑ Guardar snapshots del cliente y de cada equipo externo.

- [x] Implementar asignación y reasignación de técnico.
- [x] Implementar cancelación según las reglas acordadas para Sheets.
- [x] Aplicar control de versión y devolver `409` ante ediciones obsoletas.
- [x] Crear el flujo administrativo por pasos: cliente, servicio, unidades, equipos, técnico y revisión.
- [x] Crear lista, filtros y detalle de órdenes según las convenciones de la API.
- [x] Registrar auditoría de cambios relevantes de la orden.

Criterio de salida

- [x] Un administrador crea una orden con varias unidades y varios equipos por unidad.
- [x] Otro administrador no puede sobrescribir silenciosamente una versión obsoleta.
- [x] Un técnico no puede crear, editar, asignar ni cancelar órdenes.

7. Asignación, sincronización y outbox

- [x] Revalidar cada equipo por ID/IMEI y comparar su snapshot inmediatamente antes de asignar.
- [x] Comprobar en PostgreSQL que cada equipo no pertenezca a otra orden activa.
- [x] Devolver `409 SHEET_ROW_CONFLICT` sin sobrescribir cambios externos.
- [x] Escribir únicamente `J`, `L`, `M`, `N`, `O` y, si se aprueba, `P` mediante `batchUpdate`.
- [x] Proteger explícitamente las columnas `A`, `I`, `K` y cualquier rango no autorizado.
- [x] Crear `SheetSyncOperation` en la misma transacción local que la asignación.
- [x] Implementar claves de idempotencia para asignación y sincronización.
- [x] Implementar worker con bloqueo seguro, backoff exponencial, jitter y máximo de intentos.
- [x] Verificar el estado remoto antes de reintentar escrituras con resultado ambiguo.
- [x] Añadir estado visible de sincronización y reintento manual para administradores.
- [x] Programar el reporte técnico con corte semanal, día/hora y zona horaria configurables.
- [x] Mantener inmediatas las operaciones de equipos y separar sus conteos de los reportes técnicos.
- [x] Añadir tablero global con filtros, estados, errores, intentos y reintento inmediato.
- [x] Sustituir versiones pendientes del reporte cuando una corrección genera una versión nueva.
- [x] Registrar métricas y logs del worker, cola y circuit breaker.

Criterio de salida

- [x] Una asignación correcta modifica exclusivamente las celdas autorizadas.
- [x] Una operación fallida permanece visible, se puede reintentar y no se duplica.

- ☑ La orden no aparece como completamente sincronizada mientras existan operaciones fallidas.
- ☑ El resumen principal cuenta reportes técnicos y no confunde operaciones de equipos con órdenes.

8. Flujo de trabajo del técnico

- ☑ Implementar consulta de órdenes filtrada por el técnico autenticado.
- ☑ Crear las vistas mobile-first de “Mis órdenes”, detalle, unidades y progreso.
- ☑ Implementar la transición `ASSIGNED -> IN_PROGRESS` solo para el técnico asignado.
- ☑ Implementar verificación de cada equipo como coincidencia.
- ☑ Implementar reporte de discrepancias para IMEI, ID, serie, unidad y otros tipos.
- ☑ Permitir continuar y cerrar aunque existan discrepancias no resueltas.
- ☑ Implementar el checklist único por unidad con sus cinco categorías iniciales.
- ☑ Validar `Bien` , `Mal` o `No aplica` y exigir observación cuando el resultado sea `Mal` .
- ☑ Mostrar el progreso y los requisitos pendientes de cada unidad.
- ☑ Impedir cambios del técnico en órdenes completadas o que no le pertenezcan.

Criterio de salida

- ☑ Un técnico solo puede consultar y modificar sus órdenes asignadas.
- ☑ Todas las reglas de transición, verificación, discrepancia y checklist tienen pruebas unitarias y de integración.

9. Evidencias y firma

- ☑ Implementar el adaptador S3 y configuración local con MinIO.
- ☑ Crear el flujo de evidencia `PENDING -> COMPLETED` con URL prefirmada.
- ☑ Generar las claves de objeto en el servidor según la estructura definida.
- ☑ Validar existencia, tipo MIME, tamaño y metadatos antes de confirmar una subida.
- ☑ Permitir JPEG, PNG y WebP dentro de los límites configurados.
- ☑ Implementar eliminación lógica autorizada y limpieza de subidas huérfanas.
- ☑ Crear captura móvil desde cámara/galería y compresión previa en Vue.
- ☑ Exigir al menos una foto completada por categoría, incluso para `No aplica` .
- ☑ Implementar captura del nombre y una firma PNG por orden.
- ☑ Servir evidencias y firma mediante URLs temporales autorizadas o proxy seguro.
- ☑ Auditar reemplazos o eliminaciones sin guardar binarios ni URLs firmadas en logs.

Criterio de salida

- [x] Los bytes de fotografías y firma existen solo en almacenamiento privado, no en PostgreSQL.
- [x] No se puede confirmar una subida inexistente, demasiado grande o con tipo no permitido.
- [x] Un usuario sin acceso a la orden no puede obtener URLs de sus objetos.

10. Cierre, cita fallida y correcciones

- [x] Implementar el cálculo de completitud de todas las unidades en el backend.
- [x] Exigir equipos verificados, checklist completo, fotos por categoría, nombre y firma para el cierre normal.
- [x] Implementar cierre idempotente y transición a `COMPLETED`.
- [x] Implementar cita fallida exigiendo únicamente un motivo no vacío.
- [x] Implementar solicitud administrativa de corrección y retorno a trabajo técnico.
- [x] Conservar evidencias y firmas anteriores al reabrir y auditar cualquier reemplazo.
- [x] Implementar cancelación y reglas de inmutabilidad por estado.
- [x] Resolver citas fallidas mediante cancelación administrativa o una nueva orden enlazada en estado `DRAFT`.
- [x] Crear resumen previo al cierre y mensajes claros sobre requisitos faltantes.

Criterio de salida

- [x] Una orden normal incompleta no puede cerrarse aunque la UI sea omitida.
- [x] Una discrepancia no bloquea el cierre si se satisfacen los demás requisitos.
- [x] Una cita fallida no solicita checklist, fotografías ni firma.

11. Frontend completo y PWA básica

- [x] Completar dashboard, listado, filtros, detalle, auditoría y sincronización para administrador.
- [x] Completar navegación, checklist, evidencias, firma, cierre y cita fallida para técnico.
- [x] Cargar rutas de módulos de forma diferida y exponer cada módulo desde su `index.ts`.
- [x] Mantener datos remotos en TanStack Query y reservar Pinia para sesión y UI global.
- [x] Guardar en la URL los filtros que deban sobrevivir a recargas o enlaces compartidos.
- [x] Normalizar estados de carga, vacío, error, confirmaciones y notificaciones.
- [x] Aplicar diseño responsivo y accesible, priorizando el uso móvil del técnico.
- [x] Crear manifest e instalación PWA.
- [x] Cachear solo assets estáticos seguros y excluir respuestas autenticadas del service worker.

- [ ] Verificar que la interfaz no prometa funcionamiento offline.

Criterio de salida

- [ ] Los flujos principales funcionan en viewport móvil y de escritorio.
- [ ] Una recarga de la aplicación conserva la sesión mediante refresh seguro, no mediante almacenamiento del JWT.
- [ ] Ningún dato autenticado sensible queda cacheado por el service worker.

12. Calidad, seguridad y observabilidad

- [ ] Completar pruebas unitarias de reglas de negocio, estados, autorización, filtros, tokens y resiliencia.
- [ ] Completar pruebas de integración de API, PostgreSQL, S3 simulado, Sheets simulado, outbox e idempotencia.
- [ ] Añadir pruebas de contrato entre OpenAPI y respuestas reales de Supertest.
- [ ] Automatizar escenarios E2E de administrador, técnico, varias unidades, discrepancia, firma, cierre y cita fallida.
- [ ] Probar restricciones de acceso entre dos técnicos diferentes.
- [ ] Exponer métricas de HTTP, autenticación, subidas, Sheets, outbox, breakers y órdenes por estado.
- [ ] Revisar TLS, CORS, cookies, claves, permisos de Google y bloqueo público del bucket.
- [ ] Revisar SQL crudo parametrizado, límites de entrada y redacción central de secretos.
- [ ] Ejecutar escaneo de dependencias y contenedores.
- [ ] Realizar pruebas de restauración de PostgreSQL y definir limpieza de objetos abandonados.

Criterio de salida

- [ ] Los criterios técnicos de aceptación de `SPECS.md` están automatizados o documentados como verificación manual.
- [ ] No quedan vulnerabilidades críticas conocidas ni datos sensibles visibles en logs o respuestas.

13. CI/CD y staging

- [ ] Crear pipeline reproducible: instalación, lint, typecheck, pruebas, OpenAPI, build y escaneos.
- [ ] Ejecutar migraciones una sola vez por despliegue antes de activar la nueva versión.
- [ ] Preparar rollback y respaldo para migraciones destructivas.
- [ ] Seleccionar PostgreSQL, Google Drive compartido para archivos de staging y hosting AWS de staging; S3 se conserva en código como adaptador alternativo.

- ☑ Definir configuración separada por entorno; los secretos de staging deben recrearse tras la eliminación del ambiente.
- ☑ Usar un documento de Google Sheets exclusivo para staging.
- ☑ Definir IaC de staging para frontend/API, PostgreSQL, Drive y reverse proxy. **No está desplegada actualmente**; el stack no crea bucket S3 ni configura TLS público.
- ☑ Preferir `/api` en el mismo sitio público para simplificar cookies y CORS.
- ☑ Incorporar smoke tests al workflow para un redeploy futuro; no hay endpoint vivo actualmente.

Criterio de salida

- ☑ Staging se despliega de manera repetible desde CI y supera smoke tests y E2E críticos.
- ☑ Los respaldos, restauración y rollback cuentan con procedimientos probados.

14. Piloto y migración desde AppSheet

> Las casillas documentan el estado histórico registrado al preparar el MVP. No prueban que el cutover de producción siga activo ni que lo esté hoy: al 2026-10-02 AWS staging está eliminado y no existe despliegue productivo. La migración operativa debe revalidarse con Operaciones; ver el plan de AppSheet.

- ☑ Inventariar usuarios, técnicos, estados y automatizaciones existentes en AppSheet.
- ☑ Cargar y validar usuarios y técnicos iniciales.
- ☑ Definir la retención operativa o legal de evidencias y firmas.
- ☑ Definir dominio productivo, recuperación de contraseña y canal de notificación.
- ☑ Ejecutar un piloto en paralelo con un grupo pequeño de administradores y técnicos.
- ☑ Comparar órdenes, asignaciones y evidencias entre ambos sistemas.
- ☑ Corregir hallazgos del piloto y repetir los escenarios críticos.
- ☑ Preparar capacitación, soporte y procedimiento de contingencia.
- ☑ Congelar nuevas altas en AppSheet en la fecha acordada.
- ☑ Cambiar la operación al sistema nuevo y conservar AppSheet temporalmente en modo consulta.
- ☑ Supervisar errores, sincronizaciones pendientes, tiempos de respuesta y adopción durante la estabilización.

Criterio de salida del MVP

- ☑ Todos los criterios técnicos de aceptación de `SPECS.md` están cumplidos.
- ☑ El flujo completo de administrador y técnico funciona con datos y almacenamiento productivos.
- ☑ No existen sincronizaciones fallidas sin procedimiento de recuperación.

- [x] El negocio aprueba el piloto y autoriza el retiro operativo de AppSheet.

Trabajo posterior al MVP

- [x] Implementar el reporte PDF de orden de servicio definido en `docs/service-order-report-spec.md`.
- [ ] Aprobar el recurso oficial de logotipo y la plantilla visual del reporte.
- [x] Generar y versionar automáticamente el reporte al completar o corregir una orden.
- [x] Añadir descarga privada, reintentos y auditoría del PDF.
- [ ] Aprobar documentos de referencia y añadir pruebas de regresión visual del PDF.
- [ ] Evaluar modo offline completo solo con una estrategia explícita de conflictos y sincronización.
- [ ] Evaluar una columna protegida `SISTEMA_ID` en Sheets para identidad estable.
- [ ] Evaluar RBAC persistido cuando aparezcan más perfiles configurables.
- [ ] Incorporar recuperación de contraseña según el canal aprobado.
- [ ] Añadir exportación o reportes de auditoría si el negocio los requiere.
- [x] Implementar tarifas dinámicas por orden, personalización acordada y reajuste post-cierre definido en `docs/service-pricing-customization-spec.md`.
- [x] Integrar Google Drive para nuevas evidencias, firmas y reportes; no se migran objetos de prueba S3. La IaC lo selecciona para staging, pendiente de redeploy y validación externa.
- [ ] Evaluar aplicaciones nativas, facturación y cuentas bancarias como iniciativas separadas.

Especificación técnica — Sistema de órdenes de servicio

> Esta especificación mantiene los requisitos funcionales y de diseño del proyecto. No sustituye el estado implementado: el sistema ya incluye administración de usuarios, acceso técnico compartido, tarifas por orden, reportes PDF y Google Drive para archivos. AWS staging fue eliminado y no hay despliegue productivo; los pendientes de infraestructura/operación se mantienen como tales. Para el estado ejecutable y los detalles actuales, consulta la documentación técnica.

Organización: Bull Dog Solution

Documento: Especificación técnica del MVP

Fecha: 2026-09-10

Versión: 0.3

Estado: Implementado y actualizado

1. Objetivo

Construir un sistema web responsivo que sustituya el flujo operativo actual en AppSheet para administrar órdenes multiservicio de equipos en unidades vehiculares.

El sistema debe permitir que un administrador prepare completamente una orden, asigne un técnico, seleccione el cliente, registre las unidades y asigne uno o varios equipos a cada unidad. El técnico debe consultar sus órdenes, verificar la información, documentar el estado de cada unidad mediante checklist y fotografías, reportar discrepancias, obtener una firma final del cliente y cerrar el servicio.

Google Sheets continuará siendo la fuente oficial de los catálogos de clientes y equipos. La aplicación consultará esos datos mediante Google Sheets API y solo modificará las columnas expresamente autorizadas de la pestaña `SALIDAS`.

2. Alcance del MVP

2.1 Incluido

- Autenticación mediante JWT.
- Roles de administrador y técnico.
- Administración local de usuarios y técnicos.
- Consulta del catálogo de clientes en Google Sheets.

- Consulta y asignación de equipos desde Google Sheets.
- Creación, edición, asignación y seguimiento de órdenes.
- Varias unidades por orden.
- Varios equipos por unidad.
- Servicios de instalación, revisión, desinstalación, sensor fuel, cambio de equipo y cita fallida.
- Combinación de varios servicios en una misma orden mediante el tipo de cada unidad.
- Checklist por unidad.
- Mínimo de diez fotografías por unidad.
- Reporte de discrepancias sin bloquear el trabajo del técnico.
- Una firma del cliente por orden.
- Reporte PDF de la orden de servicio completada, conforme a docs/service-order-report-spec.md.
- Almacenamiento privado de archivos, con proveedor intercambiable.
- Auditoría de acciones relevantes.
- Registro y reintento de sincronizaciones con Google Sheets.
- Directorio técnico sincronizado, configuración de tarifas pactadas por orden, reajuste post-cierre y congelación de tarifas al cierre.
- Publicación del cobro en el libro de técnicos.
- Interfaz web responsiva y PWA básica.

2.2 Fuera del MVP

- Modo offline completo.
- Aplicaciones móviles nativas.
- Edición de clientes dentro de la aplicación.
- Alta o edición de equipos dentro de la aplicación.
- Facturación.
- Administración de cuentas bancarias.
- Pago o dispersión de tarifas a técnicos.
- Modificación automática de `Fecha de salida` o `Estado` del equipo en Google Sheets.
- Microservicios.

3. Reglas de negocio confirmadas

1. Una orden pertenece a un cliente y se asigna a un técnico.
2. Una orden contiene una o más unidades.
3. Una unidad puede tener varios equipos.
4. El administrador registra previamente la orden, las unidades y los equipos.
5. El técnico no crea unidades ni equipos desde cero.
6. El checklist se realiza después del servicio.
7. Las categorías iniciales del checklist son:

    - Tablero.
    - Cristales.
    - Accesorios.
    - Molduras.
    - Carrocería.

8. Cada categoría admite `Bien`, `Mal` o `No aplica`.
9. Cada unidad requiere al menos diez fotografías completadas en una galería propia; no se asignan a las categorías del checklist.
10. Cuando una categoría se marque `Mal`, la observación es obligatoria.
11. Para finalizar un servicio normal, todas las unidades deben tener:
    - Equipos verificados.
    - Checklist completo.
    - Al menos diez fotografías por unidad.
12. Se requiere una sola firma del cliente por toda la orden.
13. La firma requiere el nombre de la persona que firma.
14. Si el IMEI, ID, serie o unidad no coincide, el técnico reporta una discrepancia y continúa.
15. Una cita fallida exige solamente un motivo y no requiere checklist, fotografías ni firma.
16. Google Sheets conserva la autoridad sobre los catálogos de clientes y equipos.
17. La aplicación no modifica `Fecha de salida` ni `Estado` del equipo; administra `Cliente`, `Estado del servicio` y los datos del vehículo conforme al resultado.

4. Stack tecnológico

Capa	Tecnología
Lenguaje	TypeScript
Frontend	Vue 3 + Vite
Enrutamiento	Vue Router
Estado de cliente	Pinia
Estado de servidor y caché	TanStack Query para Vue
Cliente HTTP	Axios
Validación	Valibot
Componentes UI	Naive UI
Estilos	Tailwind CSS mediante plugin de Vite
Backend	Node.js + Express 5
ORM	Prisma

Capa	Tecnología
Base de datos	PostgreSQL
Autenticación	JWT con access y refresh tokens
Hash de contraseñas	Argon2id
Archivos privados	Google Drive o proveedor de almacenamiento configurado
API de Google	`googleapis` / Google Sheets API v4
Seguridad HTTP	Helmet, CORS y rate limiting
Logs HTTP	Morgan
Logs de aplicación	Winston con salida estructurada JSON
Documentación API	OpenAPI 3.1 + Swagger UI
Resiliencia externa	Timeouts, reintentos controlados y circuit breaker
Utilidades Vue	VueUse cuando evite implementaciones propias
Pruebas	Vitest, Supertest y Playwright
Contenedores	Docker
Gestor de paquetes	pnpm

Las versiones exactas se fijarán en el lockfile al crear el proyecto. Se usarán versiones estables compatibles con la versión LTS de Node.js seleccionada al iniciar la implementación.

4.1 Paquetes principales previstos

```
# Compartidos
typescript valibot @valibot/to-json-schema axios

# API
express prisma @prisma/client jose argon2
cors helmet cookie-parser express-rate-limit morgan winston
swagger-ui-express opossum googleapis
@aws-sdk/client-s3 @aws-sdk/s3-request-presigner

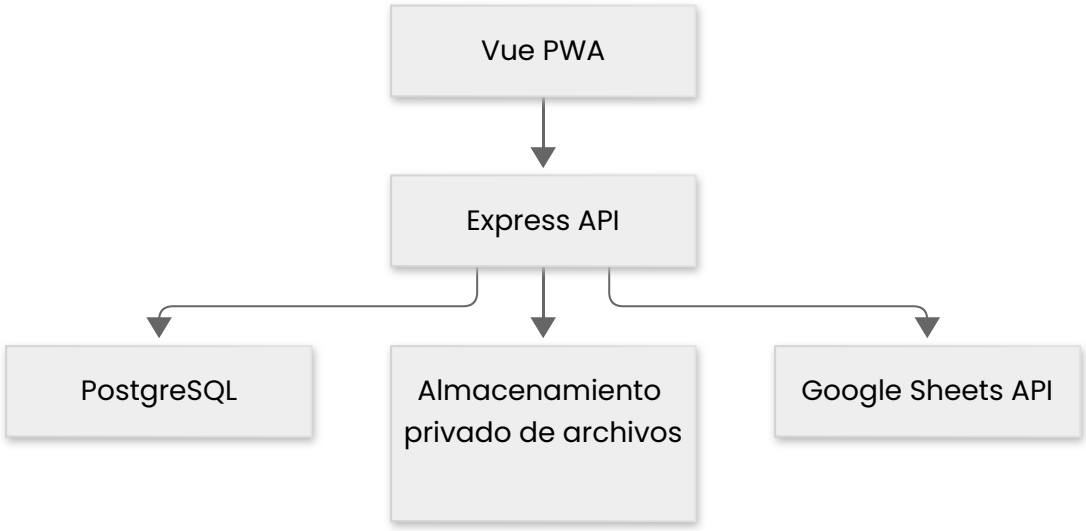
# Web
vue vue-router pinia @tanstack/vue-query
naive-ui tailwindcss @tailwindcss/vite @vueuse/core

# Pruebas
vitest supertest @playwright/test
```

La lista es una intención arquitectónica; antes de instalar se eliminarán paquetes redundantes y se comprobará compatibilidad entre versiones.

5. Arquitectura

Se implementará un monolito modular. El frontend y el backend vivirán en un monorepo, pero se construirán como aplicaciones separadas.



5.1 Responsabilidades

Vue: interfaz, navegación, captura de cámara, firma, validación inmediata y consumo de la API.

Express: autenticación, autorización, validación final, reglas de negocio, Prisma, Google Sheets, URLs firmadas para objetos, auditoría y manejo de errores.

PostgreSQL: datos transaccionales, órdenes, snapshots externos, checklist, referencias de evidencias, sesiones, auditoría y estado de sincronización.

Almacenamiento privado: bytes de fotografías, firmas y reportes; el proveedor se configura por entorno y PostgreSQL conserva solo referencias y metadatos.

Google Sheets: catálogo oficial de clientes y equipos y datos actuales de asignación permitidos.

5.2 Principios de diseño

- **Monolito modular:** un solo despliegue de API, dividido por dominios de negocio.
- **Slicing vertical:** cada módulo contiene sus rutas, casos de uso, persistencia, contratos y pruebas.
- **Dependencias hacia adentro:** controladores y adaptadores dependen de servicios/casos de uso, no al revés.
- **Controladores delgados:** traducen HTTP a una llamada de aplicación; no contienen reglas de negocio.
- **Integraciones detrás de puertos:** Google Sheets y los proveedores de almacenamiento se consumen mediante interfaces reemplazables y adaptadores concretos.
- **Contratos compartidos:** Valibot define los DTOs; Prisma define persistencia y no se expone al frontend.

- **Estado remoto separado del estado de UI:** TanStack Query administra datos del servidor; Pinia solo estado global de interfaz y sesión.
- **Abstracción progresiva:** una vista o componente se mueve a `shared` después de comprobar reutilización real, no por anticipación.
- **Sin tiempo real innecesario:** el MVP no incluye Socket.IO. Se añadirá únicamente si aparece un caso que no pueda resolverse con invalidación de consultas o polling moderado.

6. Estructura del repositorio

```
service-orders/
├── apps/
│   ├── web/
│   │   ├── src/
│   │   │   ├── app/
│   │   │   ├── modules/
│   │   │   │   ├── auth/
│   │   │   │   ├── orders/
│   │   │   │   ├── technicians/
│   │   │   │   ├── external-catalogs/
│   │   │   │   └── service-execution/
│   │   │   ├── shared/
│   │   │   │   ├── components/
│   │   │   │   ├── composables/
│   │   │   │   ├── http/
│   │   │   │   ├── layouts/
│   │   │   │   └── utils/
│   │   │   ├── router/
│   │   │   ├── stores/
│   │   │   └── main.ts
│   │   └── vite.config.ts
│   └── api/
│       ├── src/
│       │   ├── config/
│       │   ├── docs/
│       │   ├── events/
│       │   ├── middleware/
│       │   ├── modules/
│       │   ├── resilience/
│       │   ├── shared/
│       │   ├── app.ts
│       │   └── server.ts
│       └── tsconfig.json
├── packages/
│   ├── contracts/
│   ├── eslint-config/
│   └── typescript-config/
├── prisma/
```

```
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── infra/
│   └── docker/
├── docker-compose.yml
├── pnpm-workspace.yaml
└── package.json
```

7. Módulos del backend

```
modules/
├── auth
├── users
├── technicians
├── external-clients
├── external-equipment
├── orders
├── units
├── order-equipment
├── checklists
├── evidence
├── signatures
├── discrepancies
├── google-sheets
├── object-storage
├── audit
└── health
```

Cada módulo podrá contener `router`, `controller`, `service`, `repository`, esquemas Valibot, tipos y pruebas. Los controladores serán delgados; las reglas vivirán en servicios.

7.1 Estructura estándar de un módulo backend

```
modules/{domain}/
├── {domain}.router.ts
├── {domain}.controller.ts
├── {domain}.service.ts
├── {domain}.repository.ts
├── {domain}.policy.ts
├── {domain}.schemas.ts
├── {domain}.types.ts
├── {domain}.errors.ts
├── {domain}.openapi.ts
└── __tests__/
```

- `router` : rutas y composición de middleware.
- `controller` : adaptación HTTP, sin decisiones de negocio.
- `service` : casos de uso y transacciones.
- `repository` : consultas Prisma específicas del dominio.
- `policy` : autorización contextual, por ejemplo pertenencia de una orden al técnico.
- `schemas` : contratos Valibot del módulo.
- `openapi` : operación, ejemplos y respuestas para la documentación.
- `errors` : errores de dominio traducidos por el manejador global.

No se crearán repositorios genéricos universales. Prisma se encapsulará solo cuando la abstracción proteja reglas, facilite pruebas o evite consultas duplicadas.

8. Modelo de datos

8.1 Enumeraciones

```
UserRole:
  ADMIN
  TECHNICIAN

OrderServiceType:
  INSTALLATION
  REVIEW
  UNINSTALLATION
  FAILED_APPOINTMENT

OrderStatus:
  DRAFT
  ASSIGNED
  IN_PROGRESS
  REQUIRES_CORRECTION
  COMPLETED
  FAILED_APPOINTMENT
  CANCELLED

UnitStatus:
  PENDING
  IN_PROGRESS
  COMPLETED

ChecklistCategory:
  DASHBOARD
  WINDOWS
  ACCESSORIES
  TRIMS
  BODYWORK
```

```
ChecklistCondition:
  GOOD
  BAD
  NOT_APPLICABLE

EquipmentVerificationStatus:
  PENDING
  MATCHED
  DISCREPANCY_REPORTED

UploadStatus:
  PENDING
  COMPLETED
  REJECTED
  DELETED

SyncStatus:
  PENDING
  PROCESSING
  SYNCED
  FAILED
```

8.2 Entidades principales

User

- `id: UUID`
- `email: string`, único y normalizado.
- `passwordHash: string`.
- `name: string`.
- `role: UserRole`.
- `technicianId: UUID?`.
- `active: boolean`.
- `createdAt`, `updatedAt`.

Technician

- `id: UUID`.
- `externalKey: string?`, clave durable `TEC-XXXX` del directorio.
- `name: string`.
- `workshop: string?`.
- `legalName: string?`.
- `address: string?`.
- `city: string?`.
- `state: string?`.

- `phone: string?`.
- Datos administrativos del directorio y tarifas decimales para salida, instalación, revisión, desinstalación, cita fallida, sensor fuel y cambio de equipo.
- `active: boolean`.
- `createdAt`, `updatedAt`.

Order

- `id: UUID`.
- `folio: string`, único.
- `serviceType: OrderServiceType`.
- `status: OrderStatus`.
- `clientSheetId: string`.
- `clientNameSnapshot: string`.
- `technicianId: UUID`.
- Snapshots del técnico, precio unitario, cantidades, costos, viáticos, total, moneda, fecha y versión del cálculo.
- `pricingBreakdown: jsonb?`, desglose por servicio de las órdenes mixtas.
- `pricingOverrides: jsonb?`, sobreescritura personalizada de tarifas y viáticos pactados para la orden.
- `scheduledAt: timestamptz`.
- `isHomeService: boolean`.
- `serviceAddress: string?`.
- `generalNotes: string?`.
- Las notas de ejecución son generales de la orden, no independientes por unidad. Pueden capturarse durante la ejecución y se guardan antes del cierre si tienen cambios pendientes.
- `failedReason: string?`.
- `version: integer`, para concurrencia optimista.
- `createdById: UUID`.
- `startedAt`, `completedAt`, `cancelledAt`.
- `createdAt`, `updatedAt`.

El número de semana se deriva de `scheduledAt` y no se almacena como dato editable.

OrderUnit

- `id: UUID`.
- `orderId: UUID`.
- `serviceType: OrderServiceType`.
- `label: string`.
- `makeModel: string`.
- `year: integer?`.
- `serialNumber: string`.

- `status: UnitStatus`.
- `createdAt`, `updatedAt`.

El número de serie debe ser único dentro de una orden. La unicidad global se evaluará con los datos reales antes de imponerla.

OrderEquipment

- `id: UUID`.
- `orderUnitId: UUID`.
- `sheetName: string`, inicialmente `SALIDAS`.
- `sheetRowHint: integer?`, solo como optimización, nunca como identidad.
- `externalId: string?`, columna D.
- `imei: string?`, columna E.
- `externalKey: string`, identidad interna derivada.
- `brandSnapshot: string?`.
- `modelSnapshot: string?`.
- `lineSnapshot: string?`.
- `iccidSnapshot: string?`.
- `carrierSnapshot: string?`.
- `equipmentConditionSnapshot: string?`.
- `serviceStatusSnapshot: string?`.
- `rawSnapshot: jsonb`.
- `verificationStatus: EquipmentVerificationStatus`.
- `verifiedAt: timestamptz?`.
- `verifiedById: UUID?`.
- `createdAt`, `updatedAt`.

Un equipo no puede estar asignado a dos órdenes activas dentro de la aplicación. Se agregará un índice único parcial mediante una migración SQL cuando la restricción no pueda expresarse completamente en Prisma.

Checklist

- `id: UUID`.
- `orderUnitId: UUID`, único.
- `completedById: UUID?`.
- `completedAt: timestamptz?`.
- `createdAt`, `updatedAt`.

ChecklistItem

- `id: UUID`.
- `checklistId: UUID`.
- `category: ChecklistCategory`.

- condition: ChecklistCondition? .
- notes: string? .
- createdAt , updatedAt .

Restricción única: checklistId + category .

Evidence

- id: UUID .
- checklistItemId: UUID .
- objectKey: string , único.
- mimeType: string .
- sizeBytes: integer .
- checksum: string? .
- width: integer? .
- height: integer? .
- status: UploadStatus .
- uploadedById: UUID .
- createdAt , completedAt , deletedAt .

Signature

- id: UUID .
- orderId: UUID , único.
- signerName: string .
- objectKey: string , único.
- mimeType: string .
- sizeBytes: integer .
- signedAt: timestamptz .
- capturedById: UUID .
- createdAt .

Discrepancy

- id: UUID .
- orderId: UUID .
- orderUnitId: UUID .
- orderEquipmentId: UUID? .
- type: string .
- description: string .
- expectedSnapshot: jsonb .
- observedSnapshot: jsonb .
- reportedById: UUID .
- reportedAt: timestamptz .

- resolvedAt: timestamptz? .
- resolvedById: UUID? .
- resolutionNotes: string? .

La discrepancia no impide que el técnico continúe ni que cierre la orden si el resto de los requisitos se cumple.

RefreshSession

- id: UUID .
- userId: UUID .
- jti: UUID , único.
- tokenHash: string .
- userAgent: string? .
- ipAddress: inet? .
- expiresAt: timestamptz .
- revokedAt: timestamptz? .
- replacedByJti: UUID? .
- createdAt .

AuditLog

- id: UUID .
- actorUserId: UUID? .
- action: string .
- entityType: string .
- entityId: string? .
- before: jsonb? .
- after: jsonb? .
- requestId: string? .
- ipAddress: inet? .
- createdAt .

No se registrarán contraseñas, JWT, credenciales, contenido binario ni URLs firmadas.

SheetSyncOperation

- id: UUID .
- orderId: UUID? .
- operationType: string .
- idempotencyKey: string , único.
- payload: jsonb .
- status: SyncStatus .
- attempts: integer .
- lastErrorCode: string? .

- `lastErrorMessage: string?` sanitizado.
- `nextAttemptAt: timestamptz?` .
- `createdAt` , `updatedAt` , `syncedAt` .

9. Estados y transiciones

9.1 Orden normal

```
DRAFT -> ASSIGNED -> IN_PROGRESS -> COMPLETED
                  -> REQUIRES_CORRECTION -> IN_PROGRESS
    -> CANCELLED
```

9.2 Cita fallida

```
ASSIGNED -> IN_PROGRESS -> FAILED_APPOINTMENT
                        |-> resolución CANCELLED
                        `-> resolución RESCHEDULED -> nueva orden DRAFT
```

El cierre como cita fallida requiere únicamente `failedReason` no vacío. Una cita fallida conserva su cierre, tarifa y reporte. El administrador debe resolverla mediante cancelación administrativa o reprogramación. Reprogramar crea una nueva orden enlazada en estado `DRAFT` , copiando la configuración operativa pero no los resultados de ejecución de la visita anterior.

9.3 Reglas de transición

- Solo un administrador crea, edita, asigna, reasigna o cancela órdenes.
- Un técnico solo inicia y trabaja órdenes asignadas a él.
- Una orden completada es inmutable para el técnico.
- Un administrador puede marcarla `REQUIRES_CORRECTION` conservando auditoría.
- Reabrir una orden no elimina evidencias ni firmas anteriores; cualquier reemplazo queda auditado.
- Las transiciones se validan en el servicio del backend y nunca dependen solo del frontend.

10. Integración con Google Sheets

10.1 Autenticación

- La integración se ejecuta exclusivamente en el backend.
- Se utilizará una cuenta de servicio de Google Cloud.
- El documento se compartirá con el correo de la cuenta de servicio.
- Las credenciales se guardarán como secretos del entorno.
- El frontend nunca recibirá credenciales ni tokens de Google.

- Se otorgará el alcance mínimo necesario para leer y actualizar el documento configurado.

10.2 Pestaña `Clientes`

Columna	Encabezado	Uso
A	`IDCliente`	Identificador externo estable
B	`Cliente`	Nombre visible

La pestaña será de solo lectura. La orden guardará `IDCliente` y un snapshot del nombre.

10.3 Pestaña `SALIDAS`

Columna	Encabezado	Permiso
A	Fecha de salida	Solo lectura
B	Marca	Solo lectura
C	Modelo	Solo lectura
D	ID	Solo lectura
E	IMEI	Solo lectura
F	Línea	Solo lectura
G	ICCID	Solo lectura
H	Compañía	Solo lectura
I	Estado	Solo lectura; valores `NUEVO` o `USADO`
J	Cliente	Escritura al asignar/completar; se limpia al retirar
K	Estado del Servicio	Escritura controlada: `INSTALADO` o `STOCK TECNICO`
L	UNIDAD	Escritura al asignar/completar; se limpia al retirar
M	MARCA/MODELO DE LA UNIDAD	Escritura al asignar/completar; se limpia al retirar
N	AÑO	Escritura al asignar/completar; se limpia al retirar
O	NUMERO DE SERIE	Escritura al asignar/completar; se limpia al retirar
P	Tipo de servicio	Escritura solo si forma parte explícita de la asignación

La aplicación nunca escribirá rangos completos de una fila. Solo actualizará `J`, `K`, `L`, `M`, `N`, `O` y, cuando corresponda, `P`.

La asignación crea la sincronización inicial y el cierre vuelve a encolar el estado final de cada equipo. Instalación, revisión y sensor fuel escriben `INSTALADO` con cliente y vehículo; desinstalación limpia `J` y `L:O` y escribe `STOCK TECNICO`; cambio de equipo aplica la salida al anterior y la instalación al nuevo. Una cita fallida no altera `SALIDAS`.

El valor de P no se derivará automáticamente del enum `OrderServiceType` hasta definir su catálogo real. En la hoja se observaron valores como `ESTANDAR` y `OBD/SCAN`, que representan un concepto diferente a instalación, revisión, desinstalación o cita fallida.

10.4 Identificación de equipos

La fila no es una identidad estable porque puede cambiar al ordenar o insertar registros.

Regla de identidad:

1. Usar `ID` cuando tenga un valor válido y no sea `PENDIENTE` o `N/A`.
2. Validar además el IMEI cuando exista.
3. Si el ID no es válido, usar el IMEI como llave temporal.
4. Si no hay ID ni IMEI único, mostrar `Datos incompletos` y no permitir asignar.
5. Guardar `sheetRowHint` solo para acelerar la búsqueda.
6. Antes de escribir, volver a localizar la fila y comprobar ID, IMEI y snapshot esperado.

Se recomienda agregar en el futuro una columna protegida `SISTEMA_ID` con UUID inmutable, pero no es requisito del MVP.

10.5 Lectura y caché

- Los clientes y equipos se consultan desde Google Sheets.
- El backend podrá usar caché de corta duración para búsquedas.
- La caché no será autoritativa.
- Antes de asignar se realizará una lectura fresca.
- La UI mostrará cuándo se obtuvo el catálogo y permitirá recargarlo.

10.6 Escritura y concurrencia

Antes de modificar una fila:

1. Localizar nuevamente el registro por ID/IMEI.
2. Comparar las columnas relevantes con el snapshot mostrado al administrador.
3. Comprobar que el equipo no esté en otra orden activa de PostgreSQL.
4. Si cambió externamente, responder `409 SHEET_ROW_CONFLICT`.
5. Actualizar únicamente las celdas autorizadas mediante `spreadsheets.values.batchUpdate`.
6. Guardar la respuesta y el nuevo snapshot.

PostgreSQL y Google Sheets no comparten una transacción distribuida. Se utilizará `SheetSyncOperation` con idempotencia, estados y reintentos. Una orden no se mostrará como completamente sincronizada mientras exista una operación fallida.

10.7 Reintentos

- Reintentos con backoff exponencial y jitter.
- Máximo configurable de intentos automáticos.

- Los errores de autenticación o permisos no se reintentan indefinidamente.
- El administrador puede reintentar una operación fallida.
- Cada operación incluye una `idempotencyKey`.
- Los errores se sanitizan antes de guardarse o mostrarse.

11. API REST

Prefijo: `/api/v1`.

11.1 Convenciones

- JSON en `camelCase`.
- Fechas en ISO 8601 UTC.
- IDs internos UUID.
- Paginación mediante `page` y `pageSize`; los listados de crecimiento alto podrán migrar a cursor sin cambiar el resto del contrato.
- Filtros comunes: `search`, `status`, `from`, `to`, `sortBy` y `sortOrder`.
- Cada endpoint declara una allowlist de filtros y campos ordenables; nunca se traduce una query arbitraria a Prisma.
- Un middleware compartido normaliza paginación, límites y ordenamiento.
- `pageSize` tendrá un máximo configurable, inicialmente 100.
- `requestId` en cada respuesta.
- Valibot valida params, query y body.
- Los errores no exponen stack traces en producción.

Respuesta exitosa:

```
{
 "data": {},
 "meta": {
  "requestId": "uuid",
  "pagination": {
   "page": 1,
   "pageSize": 25,
   "total": 120,
   "totalPages": 5
  }
 }
}
```

Respuesta de error:

```
{
 "error": {
  "code": "VALIDATION_ERROR",
```

```
    "message": "La solicitud contiene datos inválidos",
    "fields": {
      "email": ["Correo inválido"]
    },
    "requestId": "uuid"
  }
}
```

11.2 Autenticación

Método	Ruta	Descripción
POST	`/auth/login`	Iniciar sesión
POST	`/auth/refresh`	Rotar refresh token y emitir access token
POST	`/auth/logout`	Revocar sesión actual
POST	`/auth/logout-all`	Revocar todas las sesiones
GET	`/auth/me`	Usuario autenticado

11.3 Usuarios y técnicos

Método	Ruta	Rol
GET	`/users`	Admin
POST	`/users`	Admin
PATCH	`/users/:id`	Admin
GET	`/technicians`	Admin
POST	`/technicians`	Admin
PATCH	`/technicians/:id`	Admin

11.4 Catálogos externos

Método	Ruta	Descripción
GET	`/external/clients?search=`	Buscar clientes en `Clientes`
GET	`/external/equipment?search=`	Buscar equipos en `SALIDAS`
GET	`/external/equipment/:externalKey`	Obtener y revalidar equipo
POST	`/external/refresh`	Invalidar caché; admin

11.5 Órdenes

Método	Ruta	Descripción
GET	`/orders`	Listar según rol
POST	`/orders`	Crear borrador; admin
GET	`/orders/:id`	Obtener detalle
PATCH	`/orders/:id`	Editar borrador; admin
POST	`/orders/:id/assign`	Asignar y sincronizar; admin
POST	`/orders/:id/start`	Iniciar; técnico asignado
POST	`/orders/:id/cancel`	Cancelar; admin
POST	`/orders/:id/reschedule`	Reprogramar cita fallida
POST	`/orders/:id/require-correction`	Solicitar corrección; admin
POST	`/orders/:id/complete`	Finalizar servicio normal
POST	`/orders/:id/fail-appointment`	Cerrar como cita fallida
PATCH	`/orders/:id/pricing`	Configurar o ajustar tarifas pactadas; admin
GET	`/orders/:id/audit`	Historial; admin

11.6 Unidades y equipos

Método	Ruta	Descripción
POST	`/orders/:id/units`	Agregar unidad; admin
PATCH	`/orders/:id/units/:unitId`	Editar unidad; admin
DELETE	`/orders/:id/units/:unitId`	Eliminar en borrador; admin
POST	`/orders/:id/units/:unitId/equipment`	Asignar equipos; admin
DELETE	`/orders/:id/units/:unitId/equipment/:equipmentId`	Desasignar en borrador; admin
POST	`/orders/:id/units/:unitId/equipment/:equipmentId/verify`	Confirmar coincidencia
POST	`/orders/:id/units/:unitId/equipment/:equipmentId/discrepancy`	Reportar diferencia

11.7 Checklist y evidencias

Método	Ruta	Descripción
PUT	`/orders/:id/units/:unitId/checklist`	Guardar resultados

Método	Ruta	Descripción
POST	`/orders/:id/units/:unitId/checklist/:category/evidence/presign`	Crear subida autorizada
POST	`/evidence/:id/complete`	Confirmar subida
DELETE	`/evidence/:id`	Eliminación lógica autorizada

11.8 Firma

Método	Ruta	Descripción
POST	`/orders/:id/signature/presign`	Preparar subida de firma
POST	`/orders/:id/signature/complete`	Confirmar firma y nombre
GET	`/orders/:id/signature/url`	URL temporal autorizada

11.9 Sincronización

Método	Ruta	Descripción
GET	`/orders/:id/sync`	Estado de sincronización de una orden
POST	`/orders/:id/sync/retry`	Reintentar fallos de la orden; admin
GET	`/sync`	Monitor global paginado; admin
GET	`/sync/schedule`	Consultar corte y próximo envío; admin
PUT	`/sync/schedule`	Actualizar programación semanal; admin
POST	`/sync/:id/retry`	Sincronizar una operación ahora; admin

El reporte técnico se programa a partir de la fecha real de cierre. Los valores iniciales son corte el martes al finalizar el día, envío el miércoles a las 17:00 y zona `America/Hermosillo`. El día de corte, el día y hora de envío y la zona son configurables. Un cierre posterior al corte pertenece al lote de la semana siguiente. Las actualizaciones de equipos permanecen inmediatas.

11.10 OpenAPI y Swagger UI

- La API publicará OpenAPI 3.1 en `/api/v1/openapi.json`.
- Swagger UI estará disponible en `/api/docs` y podrá deshabilitarse en producción.
- Los esquemas JSON reutilizables se generarán desde Valibot mediante `@valibot/to-json-schema` cuando la conversión sea compatible.
- Operaciones, códigos HTTP, seguridad, ejemplos y errores se declararán junto al módulo correspondiente.
- Los casos que no puedan convertirse automáticamente se documentarán de forma explícita y se cubrirán con pruebas de contrato.

- CI validará el documento OpenAPI y detectará referencias rotas.
- El contrato documentado nunca reemplaza la validación en runtime.

12. Valibot y contratos compartidos

`packages/contracts` contendrá DTOs y esquemas reutilizables:

```
LoginSchema
CreateOrderSchema
UpdateOrderSchema
AssignOrderSchema
CreateUnitSchema
AssignEquipmentSchema
VerifyEquipmentSchema
ReportDiscrepancySchema
SaveChecklistSchema
PresignEvidenceSchema
CompleteEvidenceSchema
CompleteSignatureSchema
CompleteOrderSchema
FailAppointmentSchema
OrderPricingOverridesSchema
```

Reglas:

- Todo dato externo se considera `unknown` hasta pasar por Valibot.
- El backend vuelve a validar aunque Vue ya lo haya hecho.
- Los tipos se infieren desde los esquemas.
- Los esquemas incluyen descripciones y metadatos útiles para formularios y OpenAPI.
- `@valibot/to-json-schema` genera los componentes JSON Schema compatibles para la especificación OpenAPI.
- Los modelos Prisma no se exportan como contratos del frontend.
- Las variables de entorno también se validan con Valibot al iniciar.
- La aplicación falla al arrancar si falta una variable obligatoria.

13. JWT y sesiones

13.1 Tokens

Access token:

- JWT firmado.
- Duración sugerida: 15 minutos.
- Se conserva en memoria en Vue.
- Se envía como `Authorization: Bearer <token>`.

- Claims mínimos: `sub`, `role`, `iat`, `exp`, `iss`, `aud`.

Refresh token:

- JWT firmado con clave distinta o propósito criptográfico distinto.
- Duración sugerida: 30 días.
- Cookie `HttpOnly`, `Secure` y `SameSite` según el despliegue.
- Incluye `jti`.
- Su hash se registra en `RefreshSession`.
- Se rota en cada uso.

13.2 Axios

Se creará una instancia única de Axios que:

- Define `baseURL`.
- Usa `withCredentials: true`.
- Agrega el access token.
- Al recibir `401`, ejecuta un único refresh compartido.
- Reintenta una sola vez la solicitud original.
- Evita ciclos infinitos.
- Limpia el estado cuando la renovación falla.
- Normaliza los errores de la API.
- Permite cancelar peticiones con `AbortController`.

No se guardarán JWT en `localStorage` ni `sessionStorage`.

13.3 Contraseñas

- Hash Argon2id.
- Nunca se registran en logs.
- Política inicial configurable de longitud mínima.
- El login responde de forma genérica para usuario inexistente o contraseña incorrecta.
- Rate limiting por IP y por cuenta.

14. Autorización

14.1 Administrador

- Administrar usuarios y técnicos.
- Consultar catálogos externos.
- Crear y editar órdenes.
- Registrar unidades.
- Asignar equipos y técnicos.
- Consultar todas las órdenes.

- Cancelar, reabrir o solicitar corrección.
- Consultar auditoría y sincronización.

14.2 Técnico

- Consultar únicamente órdenes asignadas.
- Iniciar una orden.
- Ver cliente, unidades y equipos asignados.
- Verificar equipos.
- Reportar discrepancias.
- Completar checklist.
- Adjuntar y eliminar sus evidencias mientras la orden esté abierta.
- Capturar nombre y firma del cliente.
- Finalizar o marcar cita fallida.

La autorización se aplica en backend y las consultas Prisma incluyen el filtro de pertenencia correspondiente.

14.3 Modelo de permisos

El MVP conservará dos roles (`ADMIN` y `TECHNICIAN`) y definirá permisos como capacidades de código, por ejemplo `orders:create` , `orders:read:any` , `orders:read:assigned` , `orders:complete:assigned` y `sync:retry` .

- Las rutas declaran la capacidad requerida.
- Las políticas del dominio validan además contexto y propiedad del recurso.
- No se crearán inicialmente tablas genéricas `Module` , `Operation` , `Action` y `Permission` .
- Si aparecen más perfiles configurables, las capacidades podrán migrarse a RBAC persistido sin cambiar los controladores.
- JWT incluye el rol, no una lista larga de permisos potencialmente obsoleta.

15. Evidencias y almacenamiento privado

15.1 Flujo de subida

1. Vue prepara la imagen y envía el archivo mediante el protocolo del proveedor configurado.
2. Express verifica usuario, orden, unidad y categoría.
3. Express crea `Evidence` con estado `PENDING` y reserva una referencia idempotente.
4. La API carga el archivo privado y valida tamaño, tipo y resultado de la operación.
5. La API persiste la referencia/metadatos y cambia el estado a `COMPLETED` solo tras confirmar la carga.

15.2 Reglas

- Archivos privados; el acceso se autoriza desde la aplicación.

- No almacenar blobs o Base64 en PostgreSQL.
- Tipos iniciales: JPEG, PNG y WebP.
- Tamaño máximo configurable; sugerido 10 MiB por evidencia.
- La firma se almacena como PNG, con máximo sugerido de 1 MiB.
- Usar credenciales del proveedor solo en servidor; las cargas directas, si se habilitan para un proveedor, usan URLs temporales.
- Nombres generados por el servidor; nunca confiar en el nombre del usuario.
- Verificar `Content-Type`, longitud y existencia antes de completar.
- Aplicar política de limpieza para objetos huérfanos.
- Descargas mediante el proveedor configurado o proxy autorizado.

15.3 Estructura de claves

```
orders/{orderId}/units/{unitId}/checklist/{category}/{evidenceId}.{ext}
orders/{orderId}/signature/{signatureId}.png
```

16. Frontend Vue

El frontend seguirá una arquitectura modular por dominios. Cada módulo será autónomo y solo publicará su API pública desde un archivo `index.ts`; otros módulos no importarán archivos internos por rutas profundas.

16.1 Estructura estándar de un módulo

```
modules/{domain}/
├── api/
│   └── {domain}.api.ts
├── components/
├── composables/
├── routes/
│   └── index.ts
├── schemas/
├── views/
├── types.ts
└── index.ts
```

- `api`: funciones Axios específicas del dominio.
- `components`: UI exclusiva del módulo.
- `composables`: orquestación de formularios, consultas y acciones.
- `routes`: rutas cargadas de forma diferida.
- `schemas`: reexporta o extiende contratos Valibot compartidos.
- `views`: pantallas asociadas a rutas.
- `index.ts`: superficie pública del módulo.

No se mantendrá una segunda copia completa de datos remotos en Pinia. Las funciones de `api` alimentarán a TanStack Query y las mutaciones invalidarán las claves de consulta correspondientes.

16.2 Vistas de administrador

- Inicio de sesión.
- Dashboard.
- Lista y filtros de órdenes.
- Creación de orden en pasos.
- Selección de cliente desde Google Sheets.
- Registro de unidades.
- Selección de varios equipos por unidad.
- Asignación de técnico.
- Detalle, evidencias, firma y auditoría de orden.
- Directorio de técnicos.
- Tablero global de sincronización con configuración semanal, filtros y reintento inmediato.

16.3 Vistas del técnico

- Mis órdenes.
- Detalle de orden.
- Inicio del servicio.
- Lista y progreso de unidades.
- Equipos esperados y verificación.
- Formulario de discrepancia.
- Checklist móvil por unidad.
- Cámara y galería de evidencias.
- Captura de nombre y firma.
- Resumen y cierre.
- Cita fallida.

16.4 Estado

- Pinia: usuario, access token y estado estrictamente global de UI.
- TanStack Query: órdenes, catálogos y datos remotos.
- Estado temporal de formularios: componentes/composables.
- No duplicar respuestas completas de API en Pinia.
- Los formularios editan un borrador local y conservan un snapshot para cancelar cambios sin volver a consultar.
- Los filtros pertenecen a la URL cuando deban sobrevivir recargas, navegación o enlaces compartidos.

- Los loaders globales se reservan para transiciones amplias; cada consulta muestra su propio estado de carga.
- Errores y confirmaciones se normalizan mediante un servicio de notificaciones compartido.

16.5 Sistema visual y componentes reutilizables

- Naive UI proveerá tablas, formularios, modales, paginación, notificaciones y controles accesibles.
- Tailwind CSS se utilizará para layout, espaciado y ajustes responsivos, evitando sobrescribir internamente componentes de Naive UI sin necesidad.
- El tema de Naive UI centralizará colores, tipografía y estados de la marca.
- Se crearán wrappers delgados únicamente para patrones repetidos, por ejemplo `BaseListPage`, `BaseDetailPage`, `PageHeader`, `AsyncState` y `ConfirmAction`.
- Los wrappers usarán props y slots; no conocerán dominios como órdenes o técnicos.
- Un componente pasará a `shared` después de tener al menos dos consumidores reales o representar una regla visual global.
- Se preferirá `<script setup lang="ts">` y Composition API.

16.6 Flujo de datos del frontend

```
View -> Composable -> Query/Mutation -> API module -> Axios -> Express
  <- estado reactivo y errores normalizados <-
```

- La vista presenta información y eventos.
- El composable contiene coordinación de UI, no reglas de negocio críticas.
- El módulo API conoce rutas y DTOs.
- La instancia Axios aplica autenticación, refresh y normalización de errores.
- La API Express continúa siendo la autoridad final.

16.7 PWA

- Manifest y posibilidad de instalar.
- Diseño mobile-first para técnico.
- Cachear únicamente assets estáticos seguros.
- No prometer operación offline.
- No cachear respuestas autenticadas con datos sensibles en el service worker.

17. Middleware de Express

Orden recomendado:

```
1. proxy/trust configuration
2. requestId
3. helmet
```

4. cors
5. express.json con límite
6. cookie-parser
7. morgan sanitizado
8. rate limiter
9. authenticate
10. authorize
11. pagination/query normalizer
12. validate con Valibot
13. controller
14. notFound
15. errorHandler

El middleware de query solo acepta campos declarados por cada ruta y produce una estructura tipada para el controlador. Morgan no registrará cuerpos, cookies, encabezados de autorización, URLs firmadas ni datos sensibles. En producción escribirá mediante un stream al logger estructurado.

18. Seguridad

- TLS obligatorio en producción.
- CORS con allowlist; nunca `*` cuando hay credenciales.
- Helmet.
- Cookies seguras.
- JWT con issuer y audience validados.
- Claves separadas y rotables.
- Principio de menor privilegio para Google y el proveedor configurado.
- Archivos privados y acceso autorizado por la aplicación.
- Validación de todos los inputs con Valibot.
- Consultas mediante Prisma; SQL crudo solo parametrizado.
- Rate limiting en autenticación y endpoints costosos.
- Límite de tamaño JSON.
- Prevención de enumeración de usuarios.
- Auditoría de acciones administrativas.
- Sanitización de errores.
- Secretos fuera del repositorio.
- Dependencias bloqueadas y escaneadas en CI.
- Respaldos cifrados.

19. Concurrencia e idempotencia

- `Order.version` implementa concurrencia optimista.
- Actualizaciones administrativas incluyen la versión conocida.

- Un cambio con versión obsoleta devuelve HTTP `409`.
- Asignación, cierre y sincronización aceptan `Idempotency-Key`.
- Prisma agrupa escrituras locales relacionadas en transacciones.
- Google Sheets se coordina mediante `SheetSyncOperation`; no se asume atomicidad entre sistemas.
- El backend revalida equipos inmediatamente antes de asignar.

19.1 Eventos internos y outbox

- Los módulos pueden publicar eventos de dominio como `OrderAssigned`, `ChecklistCompleted` y `OrderCompleted`.
- Los efectos críticos externos no se ejecutan directamente desde el controlador.
- En la misma transacción Prisma se guarda el cambio de dominio y el registro de outbox/sincronización.
- Un worker procesa operaciones pendientes con bloqueo seguro para evitar doble ejecución.
- Los handlers son idempotentes.
- El bus en memoria se limita a efectos no críticos del mismo proceso; no sustituye la outbox.

19.2 Resiliencia de servicios externos

- Toda llamada a Google Sheets y al proveedor de almacenamiento tiene timeout explícito.
- Las lecturas idempotentes pueden usar reintentos limitados con backoff y circuit breaker.
- Las escrituras a Sheets se reintentan exclusivamente mediante `SheetSyncOperation` e idempotency key.
- No se reintenta automáticamente una escritura cuyo resultado sea ambiguo hasta verificar primero el estado remoto.
- El circuit breaker se configura por integración, no globalmente para todas las llamadas.
- Cuando el circuito de Google Sheets esté abierto, se rechazan asignaciones nuevas con un error temporal y se conserva acceso a órdenes ya guardadas.
- Apertura, cierre y estado half-open del breaker se registran como métricas y logs.

20. Observabilidad

- `requestId` por solicitud.
- Winston será el logger central de aplicación.
- Logs JSON en producción y formato legible en desarrollo.
- Morgan enviará access logs a Winston.
- Niveles `debug`, `info`, `warn`, `error`.
- Cada log estructurado podrá incluir `requestId`, `userId`, `module`, `action`, `orderId`, duración y resultado.
- Se aplicará redacción central de cookies, autorización, contraseñas, JWT, claves, URLs firmadas y datos privados.

- Métricas mínimas:
  - Latencia y errores HTTP.
  - Logins fallidos.
  - Subidas fallidas.
  - Operaciones de Sheets pendientes o fallidas.
  - Estado y aperturas de circuit breakers.
  - Cola/outbox pendiente y antigüedad del evento más viejo.
  - Órdenes por estado.
- Endpoint `/health/live` sin dependencias.
- Endpoint `/health/ready` comprobando PostgreSQL y configuración crítica.
- No incluir secretos o PII innecesaria en logs.

21. Variables de entorno

```
NODE_ENV
PORT
APP_ORIGIN
API_ORIGIN
TRUST_PROXY

DATABASE_URL
DIRECT_URL

JWT_ACCESS_PRIVATE_KEY
JWT_ACCESS_PUBLIC_KEY
JWT_REFRESH_PRIVATE_KEY
JWT_REFRESH_PUBLIC_KEY
JWT_ISSUER
JWT_AUDIENCE
JWT_ACCESS_TTL
JWT_REFRESH_TTL

COOKIE_DOMAIN
COOKIE_SECURE
COOKIE_SAME_SITE

S3_ENDPOINT
S3_REGION
S3_BUCKET
S3_ACCESS_KEY_ID
S3_SECRET_ACCESS_KEY
S3_FORCE_PATH_STYLE

GOOGLE_SPREADSHEET_ID
GOOGLE_TECHNICIANS_SPREADSHEET_ID
```

```
GOOGLE_CLIENT_EMAIL
GOOGLE_PRIVATE_KEY
GOOGLE_CLIENTS_SHEET=Clientes
GOOGLE_EQUIPMENT_SHEET=SALIDAS
GOOGLE_TECHNICIANS_SHEET_NAME="DIRECTORIO TECNICO"
GOOGLE_TECHNICIAN_REPORT_SHEET_NAME=2026

MAX_EVIDENCE_BYTES
MAX_SIGNATURE_BYTES
PRESIGNED_URL_TTL_SECONDS
CATALOG_CACHE_TTL_SECONDS

LOG_LEVEL
SHEETS_TIMEOUT_MS
SHEETS_RETRY_LIMIT
CIRCUIT_BREAKER_ERROR_THRESHOLD
CIRCUIT_BREAKER_RESET_TIMEOUT_MS
OUTBOX_POLL_INTERVAL_MS
TECHNICAL_REPORT_CUTOFF_WEEKDAY=2
TECHNICAL_REPORT_SYNC_WEEKDAY=3
TECHNICAL_REPORT_SYNC_TIME=17:00
TECHNICAL_REPORT_SYNC_TIMEZONE=America/Hermosillo
```

En producción se preferirán identidades del proveedor y gestores de secretos sobre claves estáticas cuando estén disponibles.

22. Desarrollo local

Docker Compose incluirá:

- PostgreSQL.
- MinIO como almacenamiento compatible con S3.
- API.
- Opcionalmente una interfaz de correo de desarrollo si se añade recuperación de contraseña.

Google Sheets utilizará un documento de prueba separado. Las pruebas automatizadas nunca escribirán en la hoja de producción.

Comandos esperados:

```
pnpm install
pnpm dev
pnpm build
pnpm lint
pnpm typecheck
pnpm test
pnpm test:e2e
```

```
pnpm docs:build
pnpm docs:validate
pnpm prisma:migrate
pnpm prisma:seed
```

23. Pruebas

23.1 Unitarias

- Validaciones Valibot.
- Transiciones de estado.
- Reglas de checklist.
- Cálculo de completitud.
- Identificación de filas de Sheets.
- Autorización por rol.
- Capacidades y políticas contextuales.
- Rotación de tokens.
- Normalización y allowlist de filtros.
- Circuit breaker y clasificación de errores reintentables.

23.2 Integración

- Rutas Express con Supertest.
- PostgreSQL aislado para pruebas.
- Transacciones Prisma.
- Presign y confirmación de objetos con almacenamiento local.
- Adaptador de Sheets con respuestas simuladas.
- Conflictos e idempotencia.
- Worker de outbox y recuperación después de fallo.
- Contrato OpenAPI contra respuestas reales de Supertest.

23.3 End-to-end

- Login de administrador.
- Creación y asignación de orden.
- Login de técnico.
- Checklist de varias unidades.
- Varias fotografías por categoría.
- Reporte de discrepancia.
- Firma y cierre.
- Cita fallida.
- Restricciones de acceso entre técnicos.

24. CI/CD

Pipeline mínimo:

1. Instalación reproducible con lockfile.
2. Lint.
3. Typecheck.
4. Pruebas unitarias.
5. Pruebas de integración.
6. Generación y validación de OpenAPI.
7. Build de API y web.
8. Escaneo de dependencias y contenedores.
9. Aplicación controlada de migraciones.
10. Despliegue.
11. Smoke test de health checks.

Las migraciones se ejecutan una vez por despliegue, antes de activar la nueva versión. Se requiere respaldo y procedimiento de rollback para cambios destructivos.

25. Despliegue recomendado

- Frontend compilado como archivos estáticos y servido desde CDN o reverse proxy.
- API como contenedor Node.js.
- PostgreSQL administrado con respaldos automáticos.
- Almacenamiento privado administrado, con proveedor compatible con el adaptador elegido.
- Reverse proxy con TLS.
- Preferir `/api` bajo el mismo sitio público cuando sea posible para simplificar cookies y CORS.
- Entornos separados: desarrollo, staging y producción.
- Documento de Google Sheets distinto para staging.

26. Respaldo y retención

- Respaldos automáticos de PostgreSQL con restauración probada.
- Versionado del bucket si el proveedor y costo lo permiten.
- Política de retención de evidencias definida por negocio antes de producción.
- Eliminación lógica en base de datos antes de purgar objetos.
- Jobs de limpieza para subidas pendientes abandonadas.
- Exportación o reporte de auditoría disponible para administradores en una fase posterior.

27. Migración desde AppSheet

1. Inventariar usuarios, técnicos, estados y automatizaciones actuales.
2. Crear ambiente de staging.
3. Configurar una copia de Google Sheets.
4. Validar encabezados y permisos exactos.
5. Probar lectura y escritura limitada de `J`, `L:O` y opcionalmente `P`.
6. Cargar usuarios y técnicos.
7. Ejecutar ambos sistemas en paralelo con un grupo pequeño.
8. Comparar órdenes y evidencias.
9. Congelar nuevas altas en AppSheet.
10. Cambiar la operación al sistema nuevo.
11. Mantener AppSheet temporalmente en modo consulta.

28. Criterios técnicos de aceptación

- Un administrador puede crear una orden con varias unidades y varios equipos por unidad.
- Los clientes se obtienen de `Clientes` y los equipos de `SALIDAS`.
- La asignación solo modifica las columnas permitidas del Sheet.
- La aplicación nunca modifica A o I; K solo cambia entre los estados de servicio autorizados.
- Una fila cambiada externamente produce un conflicto visible y no se sobrescribe silenciosamente.
- Un técnico solo ve sus órdenes.
- Cada categoría acepta `Bien`, `Mal` o `No aplica`.
- No se puede cerrar una orden normal sin checklist, fotos, verificación de equipos y firma.
- Una discrepancia puede reportarse sin bloquear el cierre.
- Una cita fallida solo exige motivo.
- Las imágenes no se guardan en PostgreSQL.
- Las credenciales no llegan al navegador.
- Los refresh tokens se rotan y pueden revocarse.
- Todas las entradas se validan con Valibot.
- Las operaciones críticas dejan auditoría.
- La API, frontend y contratos compilan con TypeScript estricto.
- Existen pruebas automatizadas de los flujos críticos.

29. Decisiones pendientes antes de producción

> Esta lista conserva preguntas de la especificación inicial. Estado al 2026-10-02: Drive está seleccionado en la IaC de staging, pero no existe un despliegue; la elección/configuración de producción, retención y recuperación ante pérdida siguen pendientes. Las decisiones locales de cancelación, cambios de unidad y columnas de Sheets están registradas en

> development-decisions.md y deben ratificarse con Operaciones antes de un cutover. Esta lista no afirma que cada decisión de producto siga sin resolverse.

1. Definir qué ocurre en Google Sheets al cancelar una orden ya sincronizada.
2. Definir qué ocurre cuando un administrador cambia una unidad después de asignarla.
3. Confirmar si la columna P debe escribirse y proporcionar su catálogo válido.
4. Definir retención legal u operativa de evidencias y firmas.
5. Seleccionar proveedor de PostgreSQL y proveedor de archivos privados para producción.
6. Definir dominio, ambiente y estrategia de despliegue de producción.
7. Definir recuperación de contraseña y canal de notificación.
8. Revisar duplicados o valores incompletos de ID e IMEI.

30. Orden sugerido de implementación

Fase 1 — Base

- Monorepo, TypeScript, Vue, Express, Prisma y PostgreSQL.
- Slicing vertical y límites entre módulos.
- Naive UI, Tailwind CSS y componentes base.
- Configuración validada con Valibot.
- Autenticación JWT y roles.
- Usuarios y técnicos.
- Logger central, OpenAPI y pipeline de middleware.

Fase 2 — Google Sheets y órdenes

- Adaptador de Google Sheets.
- Catálogo de clientes y equipos.
- Modelo de órdenes, unidades y equipos.
- Creación y asignación administrativa.
- Sincronización e idempotencia.

Fase 3 — Ejecución técnica

- Vista móvil del técnico.
- Verificación y discrepancias.
- Checklist.
- Evidencias privadas mediante el proveedor configurado.
- Firma.
- Cierre y cita fallida.

Fase 4 — Operación

- Auditoría.
- Reporte PDF de orden de servicio.
- Observabilidad.
- Pruebas end-to-end.
- Seguridad y hardening.
- Staging y migración desde AppSheet.

31. Decisiones reutilizadas de la arquitectura de referencia

Se revisó `ARQUITECTURA.md` del proyecto Despachador. Las decisiones se clasifican según su valor para este sistema.

31.1 Adoptadas

Decisión	Aplicación en este proyecto
Arquitectura modular por dominios	Slicing vertical en Vue y Express
Controladores, servicios y persistencia separados	Controladores delgados y reglas en servicios
Módulos frontend autónomos	`api` , componentes, composables, rutas, schemas y vistas por dominio
Componentes y vistas base	Wrappers delgados con props/slots después de comprobar reutilización
Axios centralizado	Instancia compartida y módulos API por dominio
Snapshot al editar	Borrador local que permite cancelar sin perder el dato original
Middleware de paginación y query	Normalización tipada con allowlist por endpoint
OpenAPI/Swagger	OpenAPI 3.1, Swagger UI y validación en CI
Logger de aplicación	Winston estructurado integrado con Morgan
Circuit breaker	Solo en integraciones externas y con reglas distintas para lectura/escritura
Event bus	Eventos de dominio y outbox para efectos críticos
Naive UI + Tailwind CSS	Componentes consistentes y layout responsivo

31.2 Adaptadas

Decisión original	Adaptación
Stores Pinia por módulo con copia de datos remotos	TanStack Query mantiene estado remoto; Pinia queda para sesión/UI
Repositorios Axios por CRUD	Módulos API pequeños, tipados y específicos del dominio
RBAC relacional granular	Dos roles y capacidades de código, preparado para migrar a RBAC persistido
Bus de eventos en memoria	Outbox persistida para Google Sheets y otros efectos críticos
Wrappers de vistas extensos	Componentes base pequeños para evitar acoplamiento y abstracción prematura

31.3 No adoptadas en el MVP

Elemento	Motivo
Socket.IO	No existe telemetría ni requisito de tiempo real; polling/invalidation es suficiente
Multi-tenancy cliente/sucursal	El sistema actual opera para una sola organización
API Keys para hardware	No hay dispositivos enviando telemetría a la API
Motor de anomalías y calibraciones	Pertenece al dominio del proyecto de referencia
Gráficas, Excel y exportaciones generales	No forman parte del flujo operativo inicial
Permisos configurables en base de datos	Dos roles cubren el MVP; se conserva una ruta de evolución

Estas decisiones evitan copiar tecnología por similitud superficial y conservan únicamente patrones que reducen duplicación, aclaran responsabilidades o aumentan resiliencia.

Especificación de Requisitos de Software (ERS)

Sistema de Gestión de Órdenes de Servicio — Bull Dog Solution

Control del Documento

- **Proyecto:** Bull Dog Solution — Órdenes de servicio
- **Versión:** 1.0.0
- **Fecha:** 2026-09-02
- **Autor / Organización:** Bull Dog Solution
- **Estado:** Propuesta de Especificación Formal (ERS / IEEE 830 / ISO/IEC/IEEE 29148)

1. Introducción

1.1 Propósito

El propósito de este documento es especificar formalmente los requisitos funcionales y no funcionales para el desarrollo del **Sistema de Gestión de Órdenes de Servicio de Bull Dog Solution**. Este documento sirve como contrato técnico y funcional entre los interesados de negocio, administradores de operaciones, técnicos en campo y el equipo de ingeniería de software.

1.2 Alcance del Producto

El sistema es una plataforma web responsiva (PWA básica) orientada a sustituir la operativa manual basada en AppSheet. Centraliza la preparación, asignación, ejecución técnica en campo, verificación de equipamiento telemático, checklist vehicular, captura de evidencia multimedia, firmas de conformidad, auditoría de eventos y sincronización bidireccional controlada con Google Sheets.

Objetivos clave:

1. Eliminar inconsistencias en la captura de datos en campo.
2. Garantizar la trazabilidad del estado físico de las unidades antes y después del servicio.
3. Mantener a Google Sheets como catálogo maestro de clientes y almacén de inventario (`SALIDAS`), sin comprometer la integridad transaccional del sistema.
4. Proveer reportes operativos, técnicos y gerenciales en tiempo real y formatos descargables (PDF, Excel, CSV).

1.3 Definiciones, Acrónimos y Abreviaturas

- **ERS:** Especificación de Requisitos de Software.
- **PWA:** Progressive Web Application.
- **IMEI:** International Mobile Station Equipment Identity.
- **Proveedor de archivos:** Servicio privado configurado para evidencias, firmas y reportes.
- **JWT:** JSON Web Token.
- **Outbox Pattern:** Patrón arquitectónico para publicación garantizada y asíncrona de eventos transaccionales.
- **Checklist Post-Servicio:** Inspección del estado de la unidad vehicular realizada al concluir los trabajos técnicos.
- **Discrepancia:** Diferencia detectada en campo entre el equipo físico asignado (IMEI, serie, ID, unidad) y lo registrado en la orden.

1.4 Referencias

- Documento de Especificaciones Técnicas del Proyecto: `SPECS.md`
- Hoja de Ruta de Implementación: `ROADMAP.md`
- Estándar IEEE 830-1998 / ISO/IEC/IEEE 29148:2018 para Requisitos de Software.

1.5 Resumen del Documento

El resto del documento detalla la perspectiva del producto, actores, casos de uso, requisitos funcionales (incluyendo un módulo integral de reportes y analítica), requisitos no funcionales, interfaces de usuario y externas, y restricciones tecnológicas.

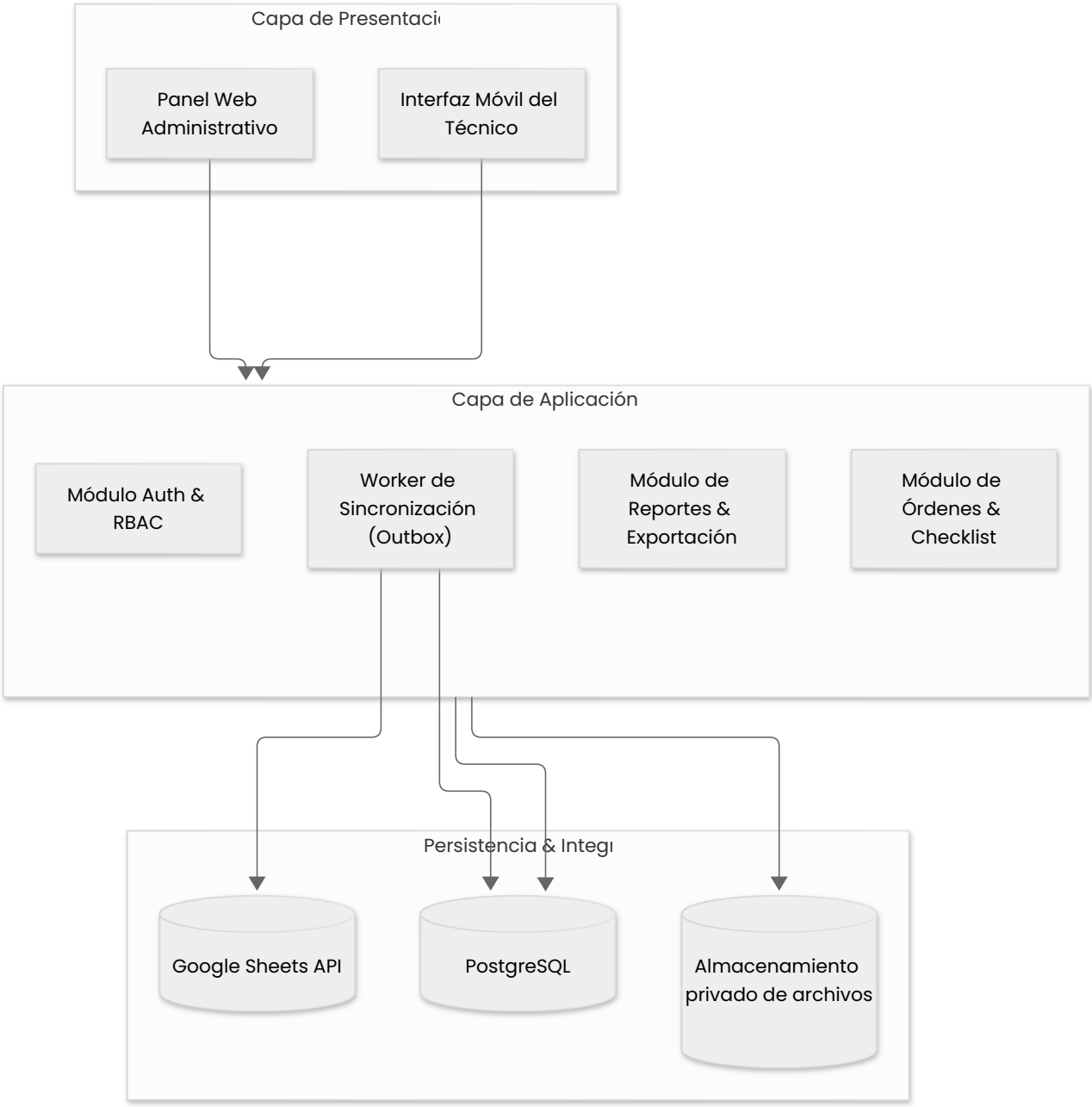
---

2. Descripción General

2.1 Perspectiva del Producto

El sistema opera bajo una arquitectura de monorepo con backend modular en Node.js (Express 5, TypeScript, Prisma ORM, PostgreSQL) y frontend reactivo en Vue 3 (Vite, Pinia, Naive UI, Tailwind CSS). Se integra de forma externa con:

- **Google Sheets API:** Autoridad de catálogos de Clientes y Equipos (`SALIDAS`).
- **Almacenamiento privado:** Repositorio de evidencias, firmas y reportes. Google Drive está integrado; el proveedor se configura por entorno y la aplicación controla el acceso.



2.2 Perfiles de Usuario (Roles)

Rol	Descripción	Nivel de Acceso
Administrador (`ADMIN`)	Personal de operaciones y logística en oficina. Prepara órdenes, vincula clientes/unidades/equipos, asigna técnicos, gestiona catálogos locales, audita y consulta reportes globales.	Acceso total al sistema y módulos administrativos/reportes.
Técnico (`TECHNICIAN`)	Personal operativo en campo. Visualiza órdenes asignadas en interfaz móvil, valida hardware, llena checklist con fotos, levanta discrepancias, recoge firmas y finaliza servicios.	Acceso restringido únicamente a sus órdenes asignadas y flujo de ejecución.

2.3 Restricciones Generales

1. **Autoridad de Catálogos:** La creación y edición de clientes y equipos no se realiza en la aplicación; residen y se administran en Google Sheets.
2. **Columnas Protegidas en Sheets:** El sistema tiene prohibido modificar `Fecha de salida` y `Estado` del equipo. Puede actualizar `Cliente`, `Estado del servicio` y los datos del vehículo conforme al servicio ejecutado.
3. **Almacenamiento Seguro:** No se almacenan imágenes en la base de datos; únicamente referencias y metadatos. El acceso a los archivos privados se autoriza mediante la aplicación.
4. **Modo Conectado:** El MVP asume conectividad a Internet para sincronización; no contempla soporte offline completo ni colas persistentes en IndexedDB.

---

3. Requisitos Específicos

3.1 Requisitos Funcionales (RF)

Módulo 1: Autenticación y Gestión de Usuarios

- **RF-01 (Autenticación Segura):** El sistema debe autenticar usuarios mediante correo electrónico y contraseña con hash Argon2id, retornando Access Token JWT (15 min) y Refresh Token (7 días) rotativo.
- **RF-02 (Control de Acceso Basado en Roles):** El sistema debe restringir endpoints y vistas según el rol (`ADMIN` o `TECHNICIAN`). Un técnico solo podrá consultar e interactuar con órdenes asignadas a su identificador.
- **RF-03 (Administración de Técnicos):** El administrador debe poder crear, editar y activar/desactivar perfiles de técnicos y asociarlos a cuentas de usuario del sistema.

Módulo 2: Integración con Google Sheets

- **RF-04 (Consulta de Catálogos en Sheets):** El sistema debe consultar el catálogo de clientes y de equipos disponibles (`SALIDAS`) mediante caché controlada con Redis/memoria y Google Sheets API.
- **RF-05 (Detección de Conflictos de Concurrencia):** Si la fila de un equipo o cliente cambia en Google Sheets durante la preparación de una orden, el sistema debe alertar al administrador sobre el conflicto de versión antes de confirmar.
- **RF-06 (Outbox y Reintentos de Sincronización):** Las actualizaciones dirigidas a Google Sheets deben registrarse en una tabla Outbox transaccional y procesarse asíncronamente con reintentos exponenciales y registro de errores.

Módulo 3: Gestión de Órdenes de Servicio

- **RF-07 (Creación de Órdenes Multilote):** El administrador debe poder crear una orden en estado `DRAFT`, asociarla a un cliente, registrar una o múltiples unidades vehiculares, y asignar uno o varios equipos por unidad.
- **RF-08 (Tipos de Servicio):** El sistema debe soportar `INSTALLATION`, `REVIEW`, `UNINSTALLATION`, `FAILED_APPOINTMENT`, `FUEL_SENSOR` y `EQUIPMENT_REPLACEMENT`, permitiendo varios servicios

por orden mediante el tipo de cada unidad.

- **RF-09 (Asignación y Reasignación):** El administrador debe poder asignar y reasignar una orden a un técnico activo, transicionando su estado a `ASSIGNED`.
- **RF-10 (Ciclo de Vida de la Orden):** El sistema debe hacer cumplir la máquina de estados: `DRAFT` $\to$ `ASSIGNED` $\to$ `IN_PROGRESS` $\to$ `COMPLETED` / `REQUIRES_CORRECTION` / `FAILED_APPOINTMENT` / `CANCELLED`.

Módulo 4: Ejecución en Campo (Operación Técnica)

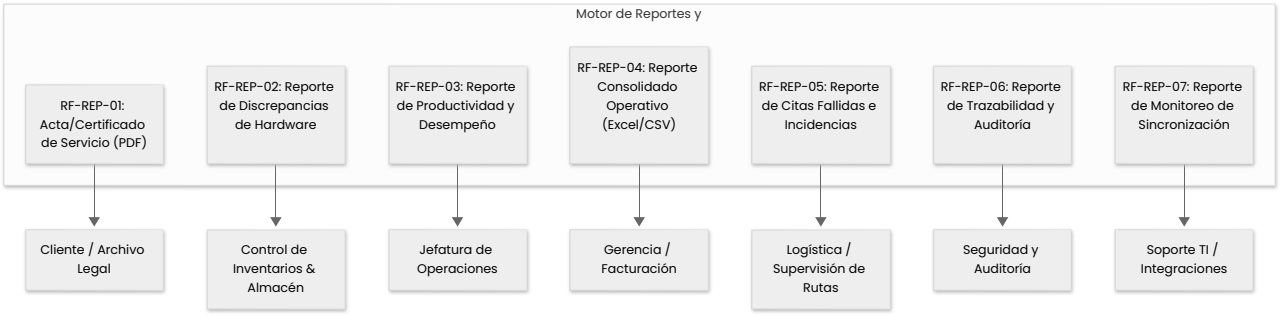
- **RF-11 (Vista Móvil del Técnico):** El técnico debe visualizar su listado de órdenes asignadas con detalles de cliente, ubicación, unidades y estatus.
- **RF-12 (Verificación de Equipamiento):** El técnico debe verificar individualmente cada equipo asignado a una unidad, marcándolo como `MATCHED` o `DISCREPANCY_REPORTED`.
- **RF-13 (Reporte de Discrepancias en Campo):** Si un equipo difiere físicamente (IMEI, serie, ID de equipo o unidad no coincidente), el técnico debe poder reportar los valores observados y continuar el servicio sin bloqueos.
- **RF-14 (Checklist Post-Servicio Obligatorio):** Para cada unidad, el técnico debe registrar el estado de 5 categorías obligatorias:
  1. Tablero (`DASHBOARD`)
  2. Cristales (`WINDOWS`)
  3. Accesorios (`ACCESSORIES`)
  4. Molduras (`TRIMS`)
  5. Carrocería (`BODYWORK`)

  - Estados permitidos: `GOOD` (Bien), `BAD` (Mal) o `NOT_APPLICABLE` (No aplica).
  - Si el estado es `BAD`, el campo de observación explicativa es **obligatorio**.
- **RF-15 (Evidencias Fotográficas Obligatorias):** Cada unidad requiere al menos diez fotografías completadas en su galería propia (no vinculadas a categorías del checklist) y cargadas al proveedor privado configurado.
- **Notas de ejecución:** El técnico puede capturar una nota general de la orden durante la ejecución. La nota puede guardarse antes del cierre o automáticamente al solicitar la finalización; no se divide por unidad.
- **RF-16 (Captura de Firma del Cliente):** La orden finalizada exige la captura digital de una única firma del cliente en pantalla, junto con el nombre completo y documento de la persona firmante.
- **RF-17 (Cierre de Cita Fallida):** Si la unidad no está disponible o el cliente no se presenta, el técnico puede registrar un cierre como `FAILED_APPOINTMENT` indicando obligatoriamente el motivo, omitiendo fotos, checklist y firma.
- **RF-18 (Resolución Administrativa de Cita Fallida):** El administrador puede cancelar el servicio o reprogramarlo. La cancelación conserva el cierre, tarifa y reporte de la visita. La reprogramación crea una nueva orden enlazada, con nuevo folio y fecha, en estado `DRAFT`; copia la configuración operativa y reinicia evidencias, firma, checklist, verificaciones y datos de cobro.
- **RF-19 (Tarifas Dinámicas y Ajuste de Liquidación):** El sistema debe permitir a los administradores (`orders:manage`) configurar tarifas acordadas (`rates`) y viáticos

(travelExpenses) para una orden específica previo a su cierre, así como ajustar importes de liquidación técnica en órdenes ya finalizadas (COMPLETED , FAILED_APPOINTMENT). El reajuste post-cierre debe recalcular los importes y desgloses con aritmética de precisión arbitraria, registrar auditoría formal (ORDER_PRICING_ADJUSTED), incrementar la versión contable de la orden e invalidar y encolar la re-sincronización hacia el reporte técnico de Google Sheets.

3.2 Requisitos Funcionales del Módulo de Reportes y Analítica (RF-REP)

Este módulo atiende la supervisión operativa, la auditoría técnica, la rendición de cuentas al cliente y el control administrativo.



RF-REP-01: Acta de Entrega y Certificado Técnico de Servicio (Formato PDF)

- **Descripción:** El sistema debe generar automáticamente un documento PDF formal y descargable al completarse la orden de servicio.
- **Entrada / Filtro:** Folio o ID único de la orden completada.
- **Contenido Requerido:**
  - Encabezado con logotipo corporativo de Bull Dog Solution, folio único, fecha y hora de inicio/cierre.
  - Datos del cliente, técnico responsable y taller/ubicación.
  - Detalle por unidad vehicular: identificación, placas/VIN, estado final.
  - Tabla de equipamiento verificado: IDs, números de serie, IMEIs y notas de discrepancia (si existieran).
  - Resultados del checklist vehicular clasificado en las 5 categorías con sus observaciones.
  - Mosaico de evidencias fotográficas incrustadas con sello de fecha/hora y categoría correspondiente.
  - Imagen de la firma digitalizada del cliente, nombre completo legible y leyenda de conformidad del servicio.
- **Formato de Salida:** PDF estándar A4/Carta descargable y consultable mediante la aplicación con autorización sobre la orden.

RF-REP-02: Reporte de Discrepancias de Equipamiento en Campo

- **Descripción:** Reporte especializado para auditoría de almacén e inventarios que consolida todos los desfases detectados entre lo programado en Google Sheets y lo encontrado físicamente en las unidades.

- **Criterios de Filtrado:** Rango de fechas, cliente, técnico, estado de la orden, tipo de discrepancia (IMEI, Serie, ID, Unidad).
- **Columnas Requeridas:**
  - Folio de la orden.
  - Fecha del reporte y nombre del técnico reportante.
  - Nombre del cliente y unidad vehicular afectada.
  - Datos esperados (según catálogo `SALIDAS`) vs. Datos reales capturados en campo.
  - Observaciones y estatus de atención administrativa.
- **Formato de Salida:** Vista tabular interactiva en pantalla con opción de exportación a CSV y Excel (XLSX).

RF-REP-03: Reporte de Productividad y Desempeño Operativo de Técnicos

- **Descripción:** Panel y reporte consolidado para evaluar el rendimiento del personal operativo.
- **Criterios de Filtrado:** Periodo de tiempo (diario, semanal, mensual), técnico individual o todos los técnicos.
- **Métricas e Indicadores:**
  - Total de órdenes asignadas vs. completadas.
  - Tasa de efectividad de visitas (Completadas vs. Citas Fallidas / Canceladas).
  - Tiempo promedio transcurrido entre asignación, inicio del servicio y cierre (`IN_PROGRESS` a `COMPLETED`).
  - Total de unidades y equipos instalados/revisados/desinstalados por técnico.
  - Conteo de discrepancias levantadas por técnico.
- **Formato de Salida:** Tablero con gráficos de barras/progreso y descarga de resumen en formato CSV/XLSX.

RF-REP-04: Reporte Consolidado de Órdenes y Servicios (Exportación General)

- **Descripción:** Reporte maestro para administración y finanzas que consolida todas las transacciones operativas del sistema.
- **Criterios de Filtrado:** Rango de fechas de creación/finalización, estado de la orden (`DRAFT`, `ASSIGNED`, `IN_PROGRESS`, `COMPLETED`, `FAILED_APPOINTMENT`, `CANCELLED`), cliente, tipo de servicio.
- **Columnas Requeridas:** Folio, Cliente, Tipo de Servicio, Técnico Asignado, Cantidad de Unidades, Cantidad de Equipos, Estado Actual, Fecha de Creación, Fecha de Cierre, Duración Total.
- **Formato de Salida:** Exportación asíncrona o sincrónica en formato Excel (XLSX) y CSV con soporte para UTF-8.

RF-REP-05: Reporte de Citas Fallidas y Cancelaciones

- **Descripción:** Informe operativo orientado a identificar causas recurrentes de no ejecución de servicios para cobro de visita o reasignación logística.
- **Criterios de Filtrado:** Rango de fechas, cliente, técnico responsable, motivo registrado.

- **Columnas Requeridas:**
  - Folio de la orden.
  - Cliente y ubicación de la cita.
  - Técnico asignado.
  - Motivo de la cita fallida (justificación obligatoria registrada por el técnico).
  - Fecha y hora exacta de registro.
- **Formato de Salida:** Vista en pantalla y exportación en CSV/Excel.

RF-REP-06: Reporte de Auditoría y Trazabilidad de Acciones

- **Descripción:** Registro inmutable de eventos del sistema para cumplimiento y control de cambios.
- **Criterios de Filtrado:** Rango de fechas, usuario responsable, entidad afectada (`Order` , `Unit` , `Equipment` , `User`), tipo de acción (`CREATE` , `UPDATE` , `STATUS_CHANGE` , `REASSIGN`).
- **Columnas Requeridas:** Timestamp ISO-8601, Usuario (ID y nombre), Dirección IP (si aplica), Acción realizada, Estado previo vs. Estado nuevo (Diff JSON).
- **Formato de Salida:** Visor de auditoría paginado con exportación de registros a CSV.

RF-REP-07: Reporte y Monitoreo de Sincronización con Google Sheets

- **Descripción:** Tablero y reporte técnico para soporte y administración que expone la salud del canal de sincronización bidireccional.
- **Métricas y Datos:**
  - Total de transacciones encoladas en Outbox (`PENDING` , `PROCESSING` , `SYNCED` , `FAILED`).
  - Detalle de fallas: mensaje de error de la API de Google, código HTTP, número de reintentos ejecutados.
  - Conflictos de versión detectados en catálogos de clientes o equipos.
  - Botón de acción para reintento manual asistido de tareas fallidas.
- **Formato de Salida:** Panel de diagnóstico técnico en tiempo real en la interfaz de administración.

3.3 Requisitos No Funcionales (RNF)

3.3.1 Rendimiento y Eficiencia

- **RNF-01 (Tiempos de Respuesta de la API):** El 95% de las peticiones REST estándar deben responder en menos de 300 ms bajo condiciones normales de red.
- **RNF-02 (Generación de Reportes PDF):** El tiempo de generación del PDF del acta de servicio no debe exceder los 3.5 segundos para órdenes con hasta 5 unidades y 25 imágenes.
- **RNF-03 (Optimización Móvil):** La aplicación móvil para técnicos debe tener un tamaño de bundle inicial menor a 1.5 MB y tiempos de renderizado menores a 1 segundo en dispositivos de gama media.

3.3.2 Seguridad y Protección de Datos

- **RNF-04 (Cifrado en Tránsito y Reposo):** Toda comunicación debe efectuarse sobre HTTPS/TLS 1.3. Las contraseñas deben cifrarse con Argon2id.
- **RNF-05 (Privacidad de Evidencias):** Los buckets de almacenamiento de imágenes y firmas deben ser estrictamente privados. La consulta de fotos se realizará exclusivamente mediante URLs firmadas con expiración máxima de 15 minutos.
- **RNF-06 (Prevención de Inyecciones y Validación):** Todas las cargas útiles entrantes (payloads) deben validarse de forma estricta en el servidor mediante esquemas Valibot.

3.3.3 Disponibilidad y Resiliencia

- **RNF-07 (Tolerancia a Fallos de Google Sheets):** La caída temporal o limitación de cuota de la API de Google Sheets no debe impedir la ejecución en campo de órdenes ya asignadas. Los eventos pendientes deben persistir en PostgreSQL mediante el patrón Outbox.
- **RNF-08 (Concurrencia Optimista):** El sistema debe prevenir sobreescrituras concurrentes en órdenes y sincronización mediante tokens de versión o timestamps de actualización.

3.3.4 Usabilidad y Accesibilidad

- **RNF-09 (Diseño Mobile-First):** La interfaz para técnicos debe ser 100% operable en pantallas táctiles de smartphones con diagonales desde 4.7 pulgadas, con botones y áreas táctiles de al menos 48x48 px.
- **RNF-10 (Claridad en Reporte de Checklist):** La captura de checklist y fotos debe presentar retroalimentación visual clara e inequívoca del progreso (p. ej., barra de avance de fotos capturadas).

---

4. Requisitos de Interfaz Externa

4.1 Interfaces de Usuario

- **Interfaz Web de Administración:** Desarrollada con Naive UI y Tailwind CSS, con diseño optimizado para escritorio (1280px+), tablas paginadas, filtros de búsqueda avanzada, modales de visualización de evidencias y tableros de reportes.
- **Interfaz Móvil del Técnico:** Aplicación web responsiva con comportamiento PWA, navegación guiada paso a paso, componente de lienzo táctil (`HTML5 Canvas`) para captura de firma y selector nativo de cámara para evidencias fotográficas.

4.2 Interfaces de Software y Terceros

- **Google Sheets API v4:**
  - Lectura de hoja de `Clientes` (Nombres, identificadores, contratos).
  - Lectura y actualización selectiva de hoja `SALIDAS` (catálogo de equipos, series e IMEIs).
  - Lectura de `DIRECTORIO TECNICO` y publicación idempotente del cobro en la pestaña anual configurada del libro de técnicos.
- **Google Drive y almacenamiento de objetos:**

    - Persistencia privada de evidencias, firmas y reportes mediante el proveedor configurado.
    - Acceso y transferencia de archivos mediados por la aplicación o por URLs temporales cuando el adaptador las requiera.

- **Motor de Renderizado PDF:**
    - Librería de servidor (p. ej., PDFKit o Puppeteer ligero) para la compilación de actas de servicio firmadas.

---

5. Matriz de Trazabilidad de Requisitos

ID Requisito	Descripción Sintética	Módulo Responsable	Componente de Sistema	Prioridad
RF-01	Autenticación JWT y sesiones	Seguridad	API Express / Auth Module	Alta
RF-02	RBAC (`ADMIN` vs `TECHNICIAN`)	Seguridad	API Express / Middleware	Alta
RF-04	Consulta a catálogos Sheets	Integración	API Express / Google Sheets Service	Alta
RF-07	Creación de órdenes multilote	Operaciones	API Express & Admin UI	Alta
RF-10	Máquina de estados de la orden	Núcleo	API Express / Orders Module	Crítica
RF-12	Verificación de equipos	Operación Campo	Technician Mobile UI	Alta
RF-13	Reporte de discrepancias	Operación Campo	Mobile UI & Orders Service	Media
RF-14	Checklist vehicular (5 categorías)	Operación Campo	Mobile UI & Checklist Service	Crítica
RF-15	Evidencias fotográficas privadas	Operación Campo	Storage Adapter & Mobile UI	Crítica
RF-16	Firma digital del cliente	Operación Campo	Canvas Signature & Storage Adapter	Crítica
RF-17	Cierre como cita fallida	Operación Campo	Orders Module / Mobile UI	Alta
RF-REP-01	Acta técnica de servicio (PDF)	Reportes	Reports Engine & PDF Service	Crítica
RF-REP-02	Reporte de discrepancias	Reportes	Reports Engine & Admin UI	Media
RF-REP-03	Productividad de técnicos	Reportes	Reports Engine & Admin UI	Media

ID Requisito	Descripción Sintética	Módulo Responsable	Componente de Sistema	Prioridad
RF-REP-04	Consolidado general (XLSX/CSV)	Reportes	Reports Engine & Export Service	Alta
RF-REP-05	Reporte de citas fallidas	Reportes	Reports Engine & Admin UI	Media
RF-REP-06	Reporte de auditoría y trazabilidad	Reportes	Audit Service & Admin UI	Alta
RF-REP-07	Monitoreo de sincronización	Reportes	Outbox Worker & Admin UI	Alta
RNF-05	Acceso privado a archivos	Seguridad	Storage Service	Crítica
RNF-07	Resiliencia con Outbox	Arquitectura	Outbox Worker & PostgreSQL	Crítica

---

6. Apéndice: Estructura de Datos Base para Reportes

```
// Estructura de payload para generación del Reporte RF-REP-01 (Acta de Servicio PDF)
interface ServiceOrderReportData {
  folio: string;
  serviceType:
    | 'INSTALLATION'
    | 'REVIEW'
    | 'UNINSTALLATION'
    | 'FAILED_APPOINTMENT'
    | 'FUEL_SENSOR'
    | 'EQUIPMENT_REPLACEMENT';
  completedAt: string;
  client: {
    id: string;
    name: string;
  };
  technician: {
    id: string;
    name: string;
    workshop?: string;
  };
  units: Array<{
    id: string;
    serviceType: ServiceOrderReportData['serviceType'];
    identifier: string;
    plate?: string;
    vin?: string;
    checklist: Array<{
      category: 'DASHBOARD' | 'WINDOWS' | 'ACCESSORIES' | 'TRIMS' | 'BODYWORK';
      condition: 'GOOD' | 'BAD' | 'NOT_APPLICABLE';
```

```
    observation?: string;
    photos: Array<{
      url: string;
      uploadedAt: string;
    }>;
  }>;
  equipments: Array<{
    id: string;
    expectedImei: string;
    expectedSerial: string;
    verificationStatus: 'MATCHED' | 'DISCREPANCY_REPORTED';
    discrepancyNote?: string;
  }>;
 }>;
 signature: {
  signerName: string;
  signatureImageUrl: string;
  signedAt: string;
 };
}
```

Plan de Piloto y Migración desde AppSheet

Este plan establece la estrategia de transición gradual desde la aplicación legacy en AppSheet hacia el nuevo sistema web y PWA de Bull Dog Solution conforme a la Fase 14 del roadmap.

> **Estado (2026-10-02):** este archivo conserva el plan propuesto, no evidencia de que el piloto, cutover o retiro de AppSheet hayan ocurrido. El código del producto está implementado, pero AWS staging está eliminado y producción no está desplegada. Cualquier cambio operativo debe planificarse y verificarse de nuevo con Operaciones antes de tratar una etapa como completada.

1. Inventario del Sistema Legacy

- **Usuarios y Roles:**
  - Administradores de operaciones (creación de órdenes, asignación de técnicos, validación de evidencia y sincronización).
  - Técnicos de campo (ejecución de servicio, verificación de equipos, captura de checklist, fotografías y firmas).
- **Catálogos y Hojas de Cálculo:**
  - Pestaña `Clientes` : catálogo maestro de clientes (`IDCliente` , `Cliente`).
  - Pestaña `SALIDAS` : registro de equipos instalados, retirados o revisados con columnas `(ID, IMEI)` .
- **Estados Operativos Mapeados:**
  - `Borrador` -> `DRAFT`
  - `Asignada` -> `ASSIGNED`
  - `En Progreso` -> `IN_PROGRESS`
  - `Completada / Para Revisión` -> `COMPLETED`
  - `Corrección Requerida` -> `REQUIRES_CORRECTION`
  - `Cancelada` -> `CANCELLED`
  - `Cita Fallida` -> `FAILED_APPOINTMENT`

2. Carga Inicial de Datos y Técnicos

1. **Usuarios Administradores:** Creación inicial de cuentas con contraseñas seguras y verificación de perfil `ADMIN` .
2. **Directorio de Técnicos:** Registro de técnicos en la tabla `Technician` y vinculación uno a uno con sus correspondientes registros de usuario `User` con rol `TECHNICIAN` .

3. **Cuentas de Acceso:** Entrega segura de credenciales temporales solicitando cambio en primer inicio.

3. Propuesta de políticas de retención (requiere aprobación)

- **Fotografías de Servicio:** Se propuso una retención mínima operativa de 12 meses en el proveedor privado de archivos. No está aprobada; archivado y borrado deben ajustarse a los requisitos legales y contractuales.
- **Firmas Digitales y Nombres de Receptores:** Se propuso conservación indefinida asociada a la orden y auditoría. No está aprobada ni configurada como política automática.

4. Ejecución del Piloto en Paralelo

1. **Grupo Piloto:** Selección de 2 administradores y un subconjunto de 3 a 5 técnicos de campo.
2. **Operación en Paralelo:**
    - Durante 2 semanas, el grupo piloto registrará órdenes reales en el nuevo sistema mientras se mantiene la continuidad operativa.
    - Verificación de consistencia entre las actualizaciones reflejadas en Google Sheets y los registros locales.
3. **Puntos de Validación Diaria:**
    - Comprobación de errores de sincronización en la pestaña de `Sincronización Sheets`.
    - Validación de la calidad de fotografías tomadas en campo desde dispositivos móviles.
    - Detección de posibles desconexiones o dificultades de captura de firma.

5. Procedimiento de Corte Operativo (Cutover)

1. **Congelamiento de Altas en AppSheet:**
    - Fecha y hora pactadas con la dirección de operaciones.
    - Restricción de permisos en AppSheet a modo solo lectura (`read-only`).
2. **Conmutación Operativa:**
    - Todas las nuevas órdenes de servicio se crean y gestionan exclusivamente a través de Bull Dog Solution.
    - Monitoreo activo de la cola outbox y métricas operativas (`/metrics`).
3. **Estabilización:**
    - Periodo de hiper-cuidado de 10 días hábiles con soporte directo a técnicos y administradores.
    - Retiro formal y desmantelamiento de la aplicación AppSheet tras cumplir satisfactoriamente los criterios de salida del MVP.

Decisiones temporales para desarrollo

Registro de decisiones temporales tomadas durante el desarrollo. Las decisiones que ya se incorporaron al producto se mantienen como historial; no todas son pendientes vigentes. AWS staging se eliminó y no existe producción desplegada. Ver la guía de staging antes de interpretar referencias a entornos.

Google Sheets

- El desarrollo y el staging histórico usaron documentos separados, nunca el documento productivo. Staging ya fue eliminado; cualquier redeploy debe recrear referencias/secretos y seguir manteniendo libros separados.
- La pestaña `Clientes` debe exponer `IDCliente` y `Cliente`; `SALIDAS` debe conservar los encabezados y posiciones descritos en `SPECS.md`.
- La identidad estable usa `ID` cuando es válido y, en su ausencia, IMEI como respaldo temporal. Valores `PENDIENTE`, `N/A`, vacíos o identidades duplicadas/incompletas no son seleccionables.
- La columna `P` queda deshabilitada por defecto. Si el negocio la habilita, solo acepta `INSTALACION`, `REVISION` o `DESINSTALACION`.
- Cancelar una orden sincronizada no revierte Sheets ni crea una compensación automática; el caso requiere revisión administrativa.
- Las unidades y equipos solo se modifican mientras la orden permanece en `DRAFT`; una corrección reabre la ejecución, no la configuración de unidades.
- La columna `K` se actualiza a `INSTALADO` o `STOCK TECNICO` según el resultado. La base local también evita asignaciones concurrentes mientras la orden está activa.
- Antes de asignar se fuerza una lectura fresca y se compara la fila actual con el snapshot. Para la columna J (`Cliente`), la comparación aplica `isMatchingClient` y `normalizeString` (insensible a mayúsculas, acentos, espacios y sufijos legales como `S.A. de C.V.`) contra el nombre y el ID del cliente de la orden. Una diferencia de cliente con una empresa ajena produce `409 SHEET_ROW_CONFLICT`.

Evidencias

- Cada fotografía admite JPEG, PNG o WebP, con máximo de 10 MiB.
- Se exigen al menos diez fotografías completadas por unidad. La evidencia es una galería general de unidad, no se asigna a categorías del checklist.
- La firma PNG tiene un máximo de 1 MiB.
- Los límites se configuran por entorno y el backend siempre los vuelve a validar.

Revisión previa a integración

Antes de conectar un documento real se debe registrar, sin copiar datos sensibles al repositorio:

1. propietario y ambiente del documento;
2. cuenta de servicio con acceso mínimo;
3. encabezados observados y fecha de validación;
4. conteos de duplicados, vacíos, `PENDIENTE`, `N/A` e identidades incompletas;
5. aprobación o reemplazo explícito de las decisiones temporales anteriores.

Variables, secretos y responsables

Los valores locales no sensibles se documentan en `.env.example`. Los secretos reales viven en el gestor de secretos de cada ambiente y nunca se registran en Git ni en logs.

Grupo	Variables	Sensible	Responsable propuesto
Aplicación	`NODE_ENV`, `PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`	No	Plataforma
PostgreSQL	`DATABASE_URL`, `DIRECT_URL`	Sí	Plataforma
JWT	`JWT_*_PRIVATE_KEY`, `JWT_*_PUBLIC_KEY`, issuer, audience y TTL	Claves privadas	Seguridad/Plataforma
Cookies	`COOKIE_DOMAIN`, `COOKIE_SECURE`, `COOKIE_SAME_SITE`	No	Backend
Archivos	Drive: `GOOGLE_DRIVE_CLIENT_EMAIL`, `GOOGLE_DRIVE_PRIVATE_KEY`, `GOOGLE_DRIVE_SHARED_DRIVE_ID`; adaptador S3 opcional con `S3_*`	Credenciales e IDs	Plataforma / Workspace
Sheets	spreadsheet ID, cuenta de servicio, clave privada y pestañas	Credenciales e ID	Operaciones/Plataforma
Límites	tamaños, TTL de URLs y caché	No	Backend/Producto
Resiliencia	timeouts, reintentos, breaker y outbox	No	Backend/SRE
Logs	`LOG_LEVEL`	No	SRE

Desarrollo puede usar PostgreSQL y MinIO locales con las credenciales ficticias de `.env.example`. Google Drive es el proveedor elegido en IaC de staging, pero el ambiente AWS está eliminado y sus secretos deben recrearse; producción no está desplegada. La integración de Sheets puede usar el adaptador simulado local o credenciales específicas del entorno.

Plan de implementación de las issues de GitHub

Fecha: 2026-09-08. Repositorio: `luistorres-idt/puk`.

> **Estado del documento (2026-10-02):** la tabla final registra verificación local histórica de las issues #4–#10; no refleja su estado actual en GitHub ni un despliegue vigente. Las funcionalidades enumeradas están implementadas en la rama de trabajo actual según contratos/código. Staging AWS se eliminó después de aquellas pruebas; no hay endpoint de staging activo. Los status remotos no se cambiaron ni se volvieron a consultar al actualizar esta documentación.

Orden de entrega

1. #7: contrato del libro y decisiones funcionales.
2. #6: directorio técnico sincronizado.
3. #4: técnico obligatorio al crear órdenes.
4. #8: servicios nuevos y política de altas posteriores.
5. #5: snapshot de precios.
6. #9: publicación del cobro.
7. #10: validación integral del épico.

Las validaciones de técnico activo de #4 y el enum, las etiquetas y las restricciones de #8 pueden adelantarse. La ejecución específica de los servicios nuevos y los cobros requieren las decisiones pendientes de #7. No se considera terminada una issue por haber implementado solamente esa parte independiente.

#7 - Contrato del libro

Fuente: https://github.com/luistorres-idt/puk/issues/7.

- Mantener la especificación versionada en `docs/technician-workbook-contract.md`, distinguiendo requisitos de las issues de datos verificados contra el libro.
- Obtener el libro de desarrollo/staging, encabezados y clave externa estable; verificar encabezados antes de importar o escribir. No utilizar números de fila como identidad ni inferir identidades por nombre, correo o teléfono.
- Acordar moneda, separadores, fecha, semana, zona horaria, unidad de cobro, congelación de tarifas, viáticos, citas fallidas y correcciones.
- Confirmar los flujos de Sensor fuel/Cambio de equipo; `nombre` y `rfc` quedan explícitamente fuera de la escritura de la aplicación y bajo captura manual.
- Preparar fixtures ficticios con vacíos, cero, duplicados, filas movidas y errores.

- Documentar configuración por ambiente. Validar permisos en staging sin escribir datos de prueba en producción.

Aceptación: contrato revisado con el negocio y prueba de acceso al libro de staging. Pendiente externo: libro, encabezados reales y reglas aún no confirmadas.

#6 - Directorio técnico

Fuente: https://github.com/luistorres-idt/puk/issues/6.

- Añadir un parser de directorio detrás del puerto de Sheets y el circuit breaker existentes. Separar lectura remota, validación y persistencia.
- Agregar clave externa única nullable para técnicos históricos, datos de contacto, facturación, importes `Decimal`, validez y metadatos del origen.
- Sincronizar transaccionalmente por clave externa. Rechazar duplicados antes de mutar; preservar UUIDs, enlaces a usuarios e historial. Desactivar retirados.
- Definir el enlace inicial de registros históricos con claves aprobadas, evitando asociaciones automáticas ambiguas por nombre.
- Exponer un endpoint específico de selección con proyección explícita de UUID, nombre, taller y ciudad. No devolver datos fiscales ni tarifas.
- Incorporar caché con caducidad y recarga explícita. Una caída de Sheets debe mostrarse y no interpretarse como catálogo vacío ni desactivar todos los técnicos.

Verificación: parser, decimales, duplicados, importación idempotente, conservación de UUID/enlaces, retirados, errores, expiración y recarga. Depende de #7.

#4 - Técnico obligatorio

Fuente: https://github.com/luistorres-idt/puk/issues/4.

- Exigir UUID en `CreateOrderSchema` y verificar existencia/actividad en el caso de uso. Impedir también quitar el técnico mediante una actualización del borrador.
- Conectar la validación de selección con el directorio sincronizado de #6 cuando esté disponible, para rechazar técnicos retirados entre selección y envío.
- Mantener nullable la relación en PostgreSQL para preservar lectura histórica. Auditar órdenes sin técnico, regularizar individualmente con asignaciones verificadas y registrar auditoría. No asignar técnicos por defecto masivamente.
- Mostrar carga, error, catálogo vacío y reintento en el wizard. Exigir selección vigente antes del resumen y del envío, y mostrar el nombre seleccionado.
- Actualizar OpenAPI, fixtures, pruebas del servicio, API y recorrido del wizard.

Aceptación: la API rechaza ausente/null/UUID inválido/inexistente/inactivo; una creación válida conserva técnico desde `DRAFT`. La exigencia de origen Sheets depende de #6.

#8 - Servicios y altas posteriores

Fuente: https://github.com/luistorres-idt/puk/issues/8.

- Agregar `FUEL_SENSOR` y `EQUIPMENT_REPLACEMENT` mediante migración aditiva.
- Centralizar etiquetas y opciones tipadas de los seis servicios para frontend, filtros, PDF y contratos. Mantener exhaustividad comprobada por TypeScript.
- Compartir la política de altas posteriores: cualquier servicio configurado mientras la orden permanezca en `DRAFT`. Aplicarla a los botones y endpoints de alta.
- Conservar lectura, edición y eliminación existentes. La carga inicial del wizard sigue entrando por `POST /orders` con unidades/equipos anidados.
- Implementar los flujos aprobados: sensor fuel usa equipo de stock y datos manuales de unidad; cambio de equipo vincula un equipo instalado y su reemplazo de stock.

Verificación: matriz por servicio/estado, endpoints de alta, creación inicial de revisión/desinstalación, visibilidad de componentes, etiquetas, OpenAPI y migración en una base aislada. El flujo nuevo depende de #7.

#5 - Snapshot de tarifa

Fuente: https://github.com/luistorres-idt/puk/issues/5.

- Crear un resolvedor puro y exhaustivo de servicio a columna de tarifa.
- Usar aritmética decimal y persistencia `Decimal`, moneda y redondeo documentados.
- Persistir identidad/nombre del técnico, servicio, costo base, salida del taller, viáticos aplicados, total, fecha de captura y versión del origen.
- Calcular/recalcular dentro de la transacción del cierre permitido y auditar el cambio. Congelar al pasar a `COMPLETED`; una corrección genera una versión nueva.
- Distinguir tarifa ausente de formato inválido y cero explícito. Emitir errores accionables y métricas sin datos personales ni payloads fiscales.
- Tratar correcciones mediante un flujo explícito con versión y auditoría; jamás sobrescribir históricos por una actualización del directorio.

Verificación: los seis servicios, taller/domicilio, redondeo, nulos, cero, inválidos, reintentos concurrentes y cambios posteriores en Sheets. Depende de #7, #6 y #8.

#9 - Reporte técnico de cobro

Fuente: https://github.com/luistorres-idt/puk/issues/9.

- Ampliar el outbox existente con una unión discriminada por `operationType` y payloads validados. Reutilizar worker, circuit breaker, reintentos y métricas.
- Encolar en la misma transacción del cierre cobrable, con clave única por orden y versión. La publicación remota nunca revierte el cierre confirmado.
- Serializar escrituras al reporte por libro/folio, incluyendo exclusión entre procesos. Buscar por folio, rechazar duplicados y volver a verificar identidad antes de escribir. No asumir que

un número de fila sigue identificando la orden.

- Escribir solo las columnas administradas por la aplicación, preservando `D` (`nombre`), `E` (`rfc`) y `O` (`envio dhl`) en inserciones, actualizaciones y reintentos. Validar encabezados y datos antes de la primera escritura.
- Generar celdas multilínea en orden estable y alinear económicos/series; agrupar equipos por unidad con secuencia documentada.
- Mostrar sincronización de cobro separada de equipos y permitir reintentos observables con errores sanitizados.

Verificación: inserción/actualización/reintento, O intacta, filas movidas, duplicados, encabezados inválidos, concurrencia entre workers, timeout después de escritura, caída de Sheets y correcciones. Depende de #7 y #5.

#10 - Épico

Fuente: https://github.com/luistorres-idt/puk/issues/10.

- Integrar las seis entregas sin duplicar lógica de dominio ni crear workers paralelos para el mismo mecanismo de outbox.
- Ejecutar lint, typecheck, pruebas, validación OpenAPI, build y E2E representativos.
- Probar migraciones sobre datos históricos en una base aislada.
- Validar el recorrido completo con el libro de staging y registrar evidencia de un solo cobro por folio y conservación de DHL.
- Marcar cada criterio por evidencia real; mantener pendientes los que requieran confirmación del negocio o validación externa.

Límites de implementación

Las reglas contables y los mapas de las issues #7 y #8 ya fueron confirmados e implementados. La validación automatizada usa datos ficticios; cualquier prueba manual adicional debe realizarse en el libro de staging, nunca en producción. Permanecen fuera de alcance los pagos a técnicos y la escritura de las columnas manuales `D`, `E` y `O` del reporte de cobro.

Estado de esta entrega

Actualización del 2026-09-10: se implementaron y verificaron las issues #4, #5, #6, #7, #8, #9 y #10 conforme a los contratos y decisiones acordadas.

Issue	Implementado y verificado localmente	Estado en GitHub
#7	Contrato del libro v1.0 en `technician-workbook-contract.md`, 88 identidades `TEC-0001` a `TEC-0088` verificadas en staging, reglas de tarifas, cálculo y exclusión de columnas D, E y O	Sin modificar status en GitHub (pendiente de autorización explícita)

Issue	Implementado y verificado localmente	Estado en GitHub
#6	Directorio técnico: parser A1:W1, IDs `TEC-XXXX`, sincronización transaccional y automática con TTL/fallback, tarifas, desactivación de retirados y selector con filtros de ciudad/taller	Sin modificar status en GitHub (pendiente de autorización explícita)
#4	Técnico obligatorio: validado en creación (`POST /orders`) y actualización (`PATCH /orders/:id`), wizard con loading/error/reintento, columna en Postgres nullable para compatibilidad histórica, snapshots de técnico persistidos	Sin modificar status en GitHub (pendiente de autorización explícita)
#8	Seis servicios habilitados, varios servicios por orden mediante `OrderUnit.serviceType`, altas en cualquier borrador y flujos finales de `SALIDAS` para instalación, revisión, sensor fuel, desinstalación y cambio de equipo	Sin modificar status en GitHub (pendiente de autorización explícita)
#5	Snapshot de tarifas: módulo puro `pricing.ts`, aritmética `Prisma.Decimal`, redondeo half-up a 2 decimales en MXN, base por equipo (1 para reemplazo), viáticos únicos en servicio a domicilio, fallback a 0.00 en cita fallida sin tarifa y congelación al completar	Sin modificar status en GitHub (pendiente de autorización explícita)
#9	Reporte técnico: formateador `technical-report-formatter.ts` con fecha/semana en `America/Hermosillo` y multilíneas en F, G, H; outbox discriminado `SHEET_TECHNICAL_REPORT`, reintentos, cliente Google Sheets escribiendo estrictamente A:C, F:N y P (preservando D, E y O intactas), reabierto y recálculo en corrección por ADMIN	Sin modificar status en GitHub (pendiente de autorización explícita)
#10	Épico integrado: outbox unificado con métricas por `operationType`, circuit breaker decorando nuevos métodos, auditoría completa de transacciones, validación OpenAPI sin errores y suite de pruebas unitarias/integración	Sin modificar status en GitHub (pendiente de autorización explícita)

No se modificó el estado de las issues en GitHub, conforme a las instrucciones del usuario.

Evidencia de validación

- `pnpm typecheck`: 6 proyectos de workspace verificados exitosamente (`@puk/contracts`, `@puk/api`, `@puk/web`, `infra/cdk`, etc.) con 0 errores.
- `pnpm test`: 202 pruebas pasando en `@puk/api` (28 archivos, más 2 archivos de integración omitidos), 22 en `@puk/contracts` y 40 en `@puk/web`.
- `pnpm docs:validate`: Documento OpenAPI generado y validado contra especificaciones sin discrepancias.
- Migraciones Prisma: además del directorio y snapshots, `20260909174050_multi_service_orders_and_unit_service_type` agrega el desglose de precios y el tipo por unidad con backfill desde la orden.

- Reglas de secreto y seguridad: Redacción activa en métricas y logs para datos sensibles, fiscales y bancarios; nunca se toca producción ni se exponen credenciales.

Implementación de almacenamiento en Google Drive

Este documento resume la integración con Google Drive y mediciones de rendimiento realizadas durante su implementación. Las pruebas descritas son evidencia histórica acotada a sus entornos; no equivalen a una medición del tiempo total de uso en producción.

Resumen

- Google Drive compartido es el destino configurado por la IaC para evidencias, firmas y reportes en un futuro staging. El ambiente AWS fue eliminado y no está desplegado. El código conserva el adaptador S3 y su fallback/default según entorno; staging configura `STORAGE_PROVIDER=drive` y el script seguro exige credenciales dedicadas de Drive. Producción no tiene IaC propia ni está desplegada.
- En modo Drive, los archivos pasan por la API para mantener autenticación y permisos; el navegador no recibe credenciales de Google ni enlaces públicos.
- Las evidencias pertenecen a la unidad del vehículo, no a una categoría del checklist.
- No se migran los objetos de prueba que quedaron en S3.

Dónde se guarda cada archivo

```
Drive compartido/
└── Cliente/
    ├── VIN/
    │   └── Folio de orden/      ← fotografías de evidencia: evidencia-<id>.<ext>
    └── Órdenes/
        └── Folio de orden/      ← firma y versiones del reporte PDF
```

Las carpetas se crean cuando hacen falta. La API conserva un mapeo entre la identidad lógica de cada carpeta y su ID de Drive, de modo que los reintentos reutilicen la misma carpeta y no dependan de que los nombres sean únicos. La carpeta de orden bajo `Órdenes` permite guardar allí la firma y los PDF compartidos por todas las unidades de esa orden.

Cómo fluye una evidencia en modo Drive

1. El navegador envía la imagen a una ruta autenticada de la API, identificando orden y unidad.
2. La API valida el acceso, que la unidad pertenezca a la orden, el formato y el tamaño de la imagen.
3. Se reserva en la base de datos la evidencia con estado pendiente y una clave de idempotencia; también se reserva un ID de archivo de Drive.

4. La API crea la carpeta que falte y carga el archivo en Drive con un nombre genérico `evidencia-…`, sin asociarlo a BODYWORK.
5. Al completar la carga, la evidencia queda asociada directamente a la unidad. Si una respuesta se pierde, el reintento puede reconciliar la reserva y el archivo en vez de generar otra evidencia.

Las firmas y los reportes PDF también se sirven mediante rutas autenticadas de la API. Para mostrar imágenes privadas, el cliente solicita el archivo y lo presenta usando una URL temporal del navegador; esas URLs se liberan cuando dejan de usarse.

Checklist y evidencias son conceptos separados

Las secciones del checklist (tablero, ventanas, accesorios, molduras y carrocería) conservan sus requisitos de revisión. Las fotos, en cambio, forman una galería general por unidad: no se pide una foto para cada sección. El mínimo de evidencia configurado se cuenta en el total de fotos de la unidad.

La migración `20260925120000_evidence_unit_owner` conserva las filas de evidencia existentes y las reasigna a su unidad antes de retirar la relación con el elemento del checklist. Ya se aplicó en la base local `puk` de `localhost:5433`; **no copia archivos de S3 a Drive ni borra objetos de S3**.

Diferencia de velocidad frente a S3

En el flujo S3, el navegador puede subir directamente a una URL prefirmada. En Drive, el flujo actual envía la imagen a la API y la API la carga a Drive; al leerla, la API también obtiene el archivo para entregarlo al cliente. Ese recorrido adicional es una explicación plausible de la lentitud observada, pero no sustituye una medición: latencia de red, tamaño de imagen y respuestas de los proveedores también influyen.

Impacto conocido: la experiencia de carga y visualización de imágenes con Drive es significativamente más lenta que con S3. Antes de cambiar el diseño, conviene medir por separado subida y descarga con imágenes comparables y registrar tamaño, tiempo total y percentiles. No hay en este cambio una optimización de rendimiento ni una cifra de benchmark.

Medición local

Se ejecutaron 10 muestras medidas por tamaño y proveedor, en secuencia alternada, con una precarga por proveedor/tamaño excluida del resultado. Cada muestra subió y descargó los mismos bytes de una imagen JPEG sintética; se comprobó que los bytes recibidos fueran idénticos. Los tiempos cubren el I/O del almacenamiento, no el renderizado en pantalla. Drive usa `getFile` (consulta de metadatos y descarga); S3 usa una URL prefirmada y una petición GET directa.

Tamaño del archivo	Operación	Drive mediana (p90)	S3 mediana (p90)	Razón de medianas Drive/S3
0.52 MiB (547,209 bytes)	Subida	1,588 ms (1,706 ms)	22.7 ms (28.2 ms)	70×

Tamaño del archivo	Operación	Drive mediana (p90)	S3 mediana (p90)	Razón de medianas Drive/S3
0.52 MiB (547,209 bytes)	Lectura	845 ms (951 ms)	3.4 ms (4.2 ms)	249×
2.55 MiB (2,671,488 bytes)	Subida	3,151 ms (4,923 ms)	33.5 ms (40.3 ms)	94×
2.55 MiB (2,671,488 bytes)	Lectura	1,112 ms (1,185 ms)	5.5 ms (5.9 ms)	202×

Límite importante: el endpoint S3 configurado para esta prueba resuelve a `localhost`; no es un bucket AWS remoto. Por lo tanto, estos números describen la configuración local y no deben interpretarse como comparación entre Google Drive y S3 en producción: el recorrido de red no es equivalente. Sí confirman que, en este entorno, el flujo de Drive tarda mucho más. Para una comparación de producción habría que repetirla contra el bucket S3 real desde la misma región o máquina donde corre la API. Los archivos temporales de ambos proveedores se eliminaron al terminar.

Prueba contra el bucket S3 remoto

Después se repitió la prueba contra el bucket remoto indicado, en `us-east-2`, usando el endpoint regional estándar de AWS. La prueba corrió desde la misma máquina para ambos proveedores, con los mismos JPEG sintéticos, 10 muestras medidas por proveedor/tamaño, orden alternado y una precarga excluida. AWS documenta el endpoint regional como `s3.us-east-2.amazonaws.com` (referencia de endpoints).

Tamaño del archivo	Operación	Drive mediana (p90)	S3 remoto mediana (p90)	Razón de medianas Drive/S3
0.52 MiB (547,209 bytes)	Subida	1,518 ms (1,884 ms)	410 ms (645 ms)	3.70×
0.52 MiB (547,209 bytes)	Lectura	903 ms (1,199 ms)	512 ms (1,283 ms)	1.76×
2.55 MiB (2,671,488 bytes)	Subida	1,883 ms (2,238 ms)	553 ms (893 ms)	3.41×
2.55 MiB (2,671,488 bytes)	Lectura	1,136 ms (1,211 ms)	880 ms (1,801 ms)	1.29×

En esta medición, Drive tuvo medianas más altas en ambas operaciones y ambos tamaños. La diferencia de lectura fue menor con el archivo grande; además, el p90 de descarga S3 tuvo variación notable, así que conviene no concluir solo a partir de medianas. La prueba mide I/O de almacenamiento y no incluye interfaz, autenticación de la aplicación, base de datos ni resolución de carpetas; tampoco representa el tiempo total percibido por una persona en producción.

El bucket tiene versionado habilitado. Por eso, el borrado normal deja marcadores y conserva versiones anteriores: se eliminaron de forma explícita las 22 versiones y sus 22 marcadores

creados por esta prueba, y se verificó que no quedaran registros para el prefijo temporal. También se retiraron la carpeta y los archivos temporales de Drive.

Otros cambios relacionados aún sin commit

- Separación de la evidencia del checklist a la unidad, incluyendo API, contratos, vistas, consultas, reportes y pruebas.
- Rutas de evidencia con alcance por unidad y validación de que la unidad pertenezca a la orden.
- Reintentos idempotentes para cargas de Drive y nombre de archivo neutral para las fotos.
- Lectura privada de evidencia, firma y PDF desde Drive; conservación del adaptador S3 sólo por compatibilidad/legado.
- Ajuste de la respuesta de firma para que la vista previa pueda mostrar el nombre de quien firmó.
- Limpieza de operaciones del cliente de Drive que no tenían uso.
- Actualizaciones de `docs/google-drive-integration-plan.md` y `spec.md`, además de pruebas asociadas.

Estos cambios permanecen sin commit en el árbol de trabajo. La generación del cliente Prisma y la migración local ya se ejecutaron. La última verificación reportada para el código fue: 91 pruebas de API aprobadas, typecheck de API y web aprobados, y `git diff --check` limpio.

Referencias

- Plan de integración de Google Drive
- Especificación de la integración
- Implementación: `apps/api/src/integrations/google-drive/` y `apps/api/src/modules/execution/`

Plan de integración con Google Drive

Estado: implementación integrada y validada en desarrollo, incluida carga de evidencia y lectura privada de archivos; la infraestructura de staging ya configura Drive, pero AWS staging está eliminado y no hay despliegue actual. Producción no está desplegada. La validación local no equivale a una validación operativa en producción. La implementación se contrasta con `spec.md`.

Alcance: almacenar en un Drive compartido las nuevas fotografías de evidencia, firmas y reportes PDF de las órdenes de servicio, con acceso privado desde la aplicación. Los archivos actuales de S3 son exclusivamente de prueba: se pueden descartar y **no se migrarán** a Drive.

Decisión vigente: Google Drive compartido es el proveedor configurado para un futuro staging y el destino previsto para evidencias, firmas y reportes. El adaptador S3 permanece en el código como alternativa técnica/legado; no se provisiona en el stack de staging. AWS staging fue eliminado y no hay recursos actuales; antes de redeploy se deben recrear infraestructura y secretos. Ver staging y despliegue.

> Las secciones siguientes conservan el diseño y los criterios originales del plan. La implementación de aplicación (carga/lectura privada de evidencias y firmas, generación y almacenamiento de PDF en Drive) está completada e integrada. Lo pendiente es operativo: recrear staging, volver a validar con Drive en ese entorno y definir respaldo/retención para producción. No ejecutar operaciones de AWS ni limpieza de datos como parte de este documento.

Resumen de la propuesta

1. Confirmar con TI el Drive compartido de producción, su capacidad, las políticas de acceso y la identidad de servicio aprobada.
2. Completar la validación de extremo a extremo de fotografías, firmas, PDF, descargas privadas, reintentos y permisos en un entorno controlado.
3. Desplegar la configuración de Drive con secretos gestionados fuera del repositorio y verificar referencias de prueba antes del corte.
4. Activar escrituras en producción solo después de cumplir los criterios de aceptación y acordar observabilidad y reversión. El borrado físico de objetos S3 es una operación posterior y separada.

No se decide aquí eliminar firmas originales. Hoy se consultan por separado y se necesitan para generar o regenerar PDF. Además, el plan de migración desde AppSheet indica conservación indefinida de firmas; cualquier cambio requiere una decisión funcional y de retención explícita.

Estado actual y alcance técnico

Archivo	Estado implementado	Consideración operativa
Fotografías	La web prepara JPEG y envía la carga autenticada a la API; la API valida, persiste y confirma la referencia privada en Drive. El adaptador S3 permanece en el código como legado.	Validar tipos de archivo, permisos y rendimiento de extremo a extremo en Drive antes del despliegue.
Firmas	La API recibe PNG autenticado, registra la referencia y mantiene una vista previa privada en Drive.	Preservar la firma original conforme a la política de retención.
PDF	El worker genera la versión del reporte y `report-storage.ts` la persiste en Drive.	Comprobar lectura de evidencias/firma, regeneración y descarga privada en Drive.

El adaptador S3 implementa `ObjectStoragePort` y utiliza URLs temporales; Drive usa IDs de archivo, referencias persistidas y autorización propia. El stack de staging selecciona Drive con `STORAGE_PROVIDER=drive`; el entorno desplegado no existe actualmente. Las referencias de evidencia, firma y reporte se guardan en PostgreSQL y no deben mezclarse entre proveedores. Los registros antiguos de prueba no requieren migración histórica a Drive.

Requisitos previos de Google Workspace

Responsable	Validación requerida
TI / administrador de Workspace	Confirmar edición, almacenamiento compartido libre, política de miembros externos y posibilidad de usar una identidad `@...iam.gserviceaccount.com` en el Drive compartido.
TI / administrador de Google Cloud	Drive API ya fue habilitada para la configuración local usada en la prueba confirmada. Para producción, crear o aprobar una cuenta de servicio **dedicada** y conceder solo el acceso necesario al Drive de la aplicación. Validar el alcance OAuth mínimo para crear, leer y descargar archivos. No conceder delegación de todo el dominio por defecto.
Seguridad / operación	Definir el mecanismo de autenticación de la carga en AWS. Preferir credenciales de corta duración si la integración lo permite; si se requiere una clave, custodiarla fuera del repositorio y gestionar su rotación. Definir auditoría, respaldo/exportación y recuperación.
Negocio / cumplimiento	Confirmar plazos de conservación de fotografías, firmas y cada versión de PDF, así como los requisitos de borrado definitivo y acceso de personal autorizado.

Una cuenta de servicio no debe guardar archivos en **Mi unidad**: Google indica que no puede ser propietaria de archivos y que debe usar un Drive compartido o actuar en nombre de un usuario mediante OAuth. El Drive compartido pertenece a la organización y consume su capacidad agrupada. La aplicación ya usa una cuenta de servicio para Sheets, pero con alcance de Sheets; se propone una identidad separada para reducir el impacto de una credencial comprometida.

Diseño propuesto para validar en el piloto

Organización de carpetas

Los archivos nuevos se ubicarán en el Drive compartido con esta jerarquía visible:

```
Nombre del cliente/
  Serie o VIN de la unidad/
    Folio de la orden de servicio/
      Evidencias de esa unidad
  Órdenes/
    Folio de la orden de servicio/
      firma.png
      reporte-v1.pdf
      reporte-v2.pdf
```

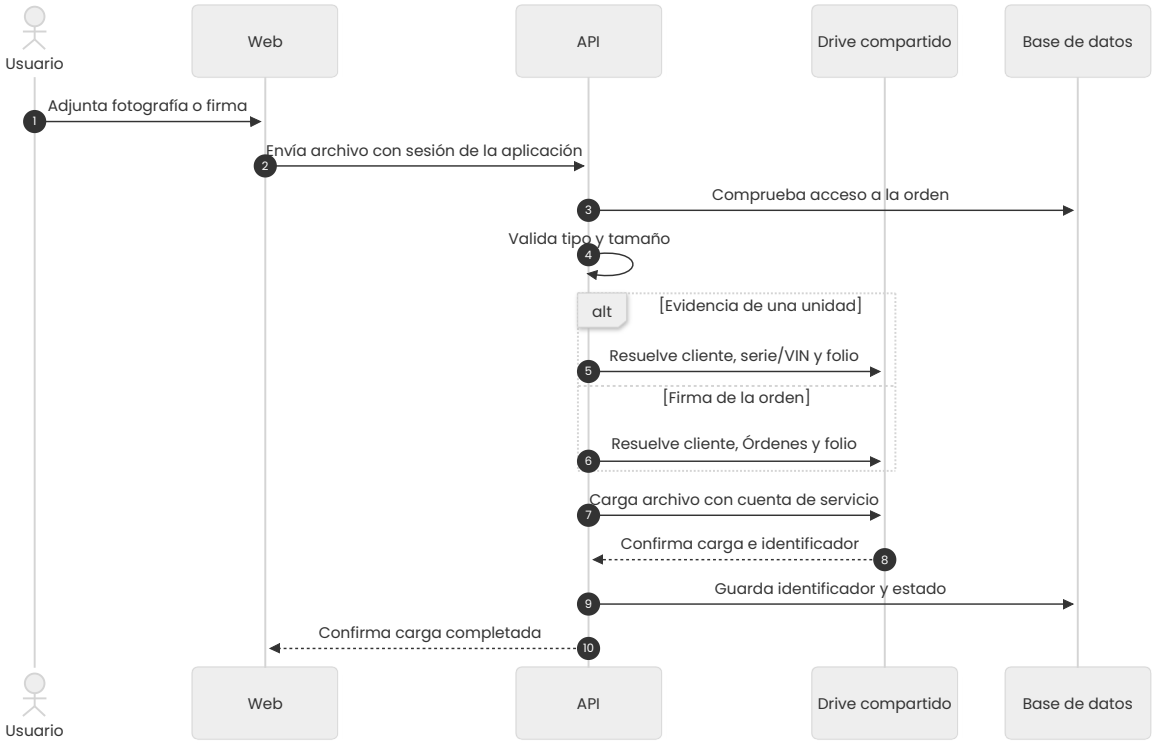
La carpeta del cliente se reutiliza para sus unidades; la de cada unidad se reutiliza para sus distintas órdenes. Dentro de la última carpeta de cada unidad van únicamente sus evidencias. Los nombres visibles pueden salir de `clientNameSnapshot`, `OrderUnit.serialNumber` y `folio`, respectivamente. Para encontrar o crear la carpeta de una unidad se usará la combinación de `clientSheetId` y serie/VIN normalizada; se conservarán los identificadores de carpeta y archivo de Drive, sin depender solo del nombre visible. `OrderUnit.id` identifica una unidad **dentro de una orden**, no el mismo auto en órdenes posteriores. El piloto debe validar el formato y la consistencia de la serie/VIN antes de reutilizar una carpeta.

La firma y el PDF son recursos de la **orden completa**, no de una unidad. Se guardarán una sola vez en `Nombre del cliente/Órdenes/Folio de la orden de servicio/`, incluso si la orden tiene una sola unidad. Cada versión del PDF tendrá su propio archivo y referencia persistida. Esta rama separada evita elegir arbitrariamente una unidad o duplicar archivos compartidos; a cambio, los archivos de una orden quedan repartidos entre su carpeta común y las carpetas de evidencias de sus unidades. La aplicación debe resolverlos desde las referencias de la base de datos, no inferir su ubicación a partir de una ruta visible.

Carga y lectura privadas

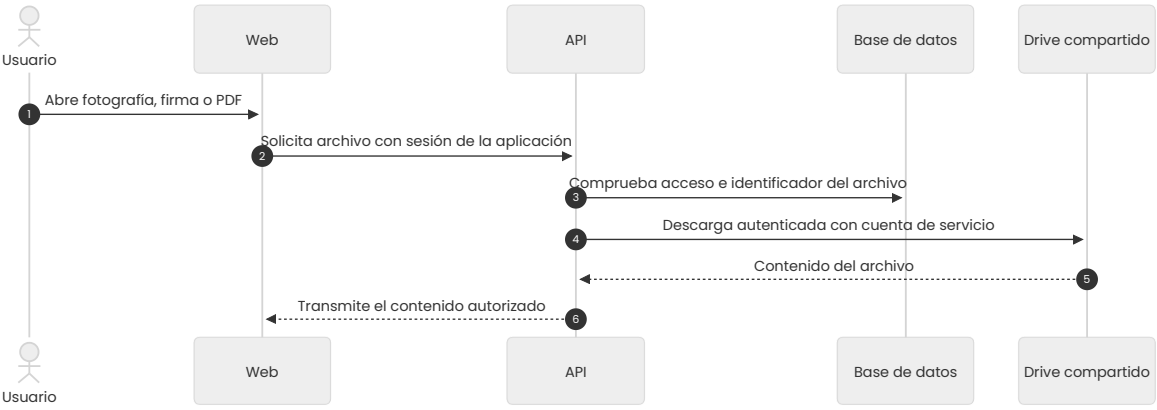
La integración con Drive ya está implementada. En ese modo la aplicación conserva la autorización: el navegador no recibe credenciales ni enlaces públicos del proveedor. El adaptador de almacenamiento de objetos mantiene su propio protocolo alternativo; no es el modelo conceptual único de almacenamiento.

Flujo implementado: carga privada en Drive



Si Drive falla, la API no debe marcar la carga como completada. Los PDF generados por la API se guardarán en la carpeta común de la orden mediante el mismo tramo API → Drive, sin pasar por el navegador.

Flujo implementado: lectura privada desde Drive



El navegador no recibe credenciales de Google ni un enlace público de Drive. Este diseño añade transferencia de datos y trabajo a la API, por lo que el piloto debe medir latencia y consumo de recursos.

- **Carga:** mantener la autenticación y las reglas de acceso de la API. Como punto de partida, recibir el archivo en un endpoint autenticado de la API y transmitirlo a Drive, con límite de bytes, validación de tipo real, confirmación de carga y limpieza de cargas incompletas. Probar el consumo de memoria, el tiempo de respuesta y la conectividad de ECS a Google

antes de fijar esta vía. Las sesiones de carga resumible son una opción a evaluar si las mediciones lo requieren.

- **Lectura:** resolver la referencia de la base de datos y transmitir fotografías, firmas y PDF a través de la API tras verificar acceso a la orden. No usar enlaces públicos ni exigir que cada técnico inicie sesión en Google. Evitar exponer tokens o identificadores como mecanismo de autorización.
- **Metadatos:** conservar una clave lógica estable para la aplicación y registrar los identificadores de archivo y carpeta de Drive. No interpretar una ruta S3 como identificador de Drive. Conservar MIME, tamaño y checksum cuando corresponda.
- **Fallas:** reintentos acotados con espera progresiva para errores transitorios de Drive; no marcar evidencia como `COMPLETED` ni PDF como `READY` antes de comprobar la carga. Registrar fallas sin incluir credenciales ni contenido sensible.
- **Idempotencia:** exigir una clave UUID por carga de evidencia y firma, reservar el ID de Drive en la base antes de transferir, y reservar `objectKey` del reporte antes de generar/subir. Los reintentos verifican nombre, padre, MIME y checksum del archivo reservado. Las firmas sustituidas conservan su referencia anterior; no se ejecuta borrado físico.

Google documenta cargas resumibles, descarga de archivos binarios mediante `files.get` con `alt=media` y el parámetro `supportsAllDrives=true` para operaciones sobre Drives compartidos. El piloto debe confirmar el alcance OAuth exacto y las políticas del tenant antes de fijar el diseño definitivo.

Alcance de los cambios en el repositorio

- API y contratos: `ObjectStoragePort`, servicios de `ejecución` y `reportes`, rutas y contratos de carga/descarga, composición en `server.ts`.
- Persistencia: modelos y migración Prisma para localizar los nuevos archivos y carpetas por sus identificadores de Drive y retirar las referencias de prueba a S3 sin dejar registros que aparenten archivos disponibles.
- Web: captura de evidencias, firma y vistas previas/descarga; mantener mensajes de progreso y errores.
- Configuración y operación: entornos local, pruebas y staging; infraestructura CDK solo después de aprobar el mecanismo de autenticación y salida de red. MinIO/S3 puede seguir sirviendo a pruebas locales, pero no es una dependencia de lectura histórica en producción.
- Mantenimiento: no ejecutar `cleanup-orphaned-objects.ts` como atajo para descartar S3. Hoy considera referencias de evidencia y firma, **pero no las de reportes PDF**; revisar su alcance antes de cualquier limpieza autorizada. No reutilizarlo para Drive sin rediseñar inventario, límites y borrado.

Secuencia de trabajo y criterios de salida

La prueba local confirmada cubre una carga de fotos y la ubicación de sus carpetas. No demuestra por sí sola la preparación de producción. Las fases siguientes conservan controles de despliegue y corte; la issue #28 da trazabilidad a la implementación.

Fase	Trabajo	Criterio para avanzar
1. Piloto	Medir tamaños esperados y capacidad libre del tenant; validar identidad, permisos, consistencia de serie/VIN, jerarquía de carpetas, carga, lectura y descarga con archivos de prueba.	TI confirma acceso al Drive compartido y se documentan resultados sin datos reales expuestos.
2. Adaptación	Implementar referencias de Drive, creación/reutilización de ambas ramas de carpetas y pruebas de autorización para nuevas cargas y lecturas.	Una orden de prueba con varias unidades funciona de extremo a extremo, con firma y PDF únicos en la carpeta común; no se depende de lectura S3.
3. Corte y observación	Identificar y descartar de forma controlada los registros y objetos de prueba de S3; activar Drive para nuevas órdenes y vigilar errores, latencia y cuota.	No quedan referencias de prueba que aparenten archivos disponibles y el flujo nuevo supera los criterios de aceptación.

Reversión: antes de crear datos reales, se puede detener el corte y descartar los archivos de prueba. Después del corte, no se promete volver a S3 con un cambio de configuración: los archivos reales nuevos estarán en Drive y cualquier retorno requeriría un plan explícito para conservar su acceso.

Verificación mínima

- Cargar y consultar fotografías JPEG/HEIC convertidas, firmas PNG y PDF sin acceso anónimo a Drive.
- Rechazar archivos fuera de límite o tipo permitido; recuperar cargas interrumpidas sin duplicar registros activos.
- Generar y volver a generar reportes con fotografías y firma de Drive; probar correcciones y versiones nuevas de PDF.
- Verificar que la jerarquía cliente/serie o VIN/orden reutiliza las carpetas correctas entre órdenes, sin mezclar clientes ni unidades; probar diferencias de formato en la serie/VIN.
- Verificar que una orden con varias unidades guarda cada evidencia en su carpeta de unidad y una sola firma y cada versión del PDF en `Cliente/Órdenes/Folio`.
- Comprobar autorización por orden, tratamiento de errores `403` / `404` / `429`, reintentos, métricas y registros sin datos sensibles.
- Comprobar que los mecanismos de limpieza de Drive respetan la política de retención; no ejecutar la limpieza de S3 sin autorización y alcance verificado.
- Verificar que el descarte de datos de prueba no afecta órdenes reales ni deja referencias rotas antes del corte.

Decisiones pendientes

1. ¿Qué edición y capacidad libre tiene el Workspace, y permite TI agregar la cuenta de servicio al Drive compartido?

2. ¿Qué mecanismo de credenciales se aprueba para la API que corre en AWS y qué alcance OAuth mínimo funciona en el piloto?
3. ¿Durante cuánto tiempo deben conservarse originales y versiones de PDF? La eliminación de firmas después de `READY` contradice la retención documentada y hoy impediría su vista previa y regeneración.
4. ¿Quién confirma que todos los registros y objetos actuales de S3 son descartables y autoriza su eliminación, incluidas las versiones anteriores si existen?

El volumen comunicado es de aproximadamente 15 reportes semanales y, normalmente, una unidad con al menos 10 fotografías por reporte. Esto orienta el piloto, pero no reemplaza medir tamaños reales, crecimiento, lecturas y capacidad libre. El costo incremental de Drive depende del contrato y la capacidad agrupada del tenant; el esfuerzo de integración y operación también forma parte de la comparación con S3.

Referencias externas

- Google Drive: Drives compartidos y propiedad de archivos.
- Google Workspace: cuentas de servicio y acceso acotado.
- Google Drive: cargas de archivos y descargas.
- Google Drive: soporte de Drives compartidos, alcances OAuth y límites de uso.
- Amazon S3: borrado de objetos con versionado.

Auditoría de mantenibilidad y limpieza

Estado: auditoría y limpieza de los límites prioritarios cerradas. El reporte distingue la verificación realizada de la deuda residual; no afirma que cada archivo del repositorio necesite o haya recibido una refactorización.

Corte de evidencia (2026-10-02): cifras de líneas, conteos de pruebas y resultados de comandos abajo son el registro de la ejecución original de la auditoría, no una ejecución nueva en esta actualización documental. Las tareas de documentación no alteraron código ni cambian esa evidencia.

Diagnóstico

El principal problema no es la cantidad de líneas por sí sola, sino la mezcla de responsabilidades en archivos que coordinan reglas de negocio, acceso a datos, estado de interfaz y presentación. La medición excluye código generado y `dist`.

Archivo	Líneas antes de esta limpieza	Responsabilidades mezcladas
`apps/web/src/modules/orders/views/OrderDetailView.vue`	1.887	Consultas y mutaciones, formularios de unidades/equipos, reporte, medios privados, auditoría y modales.
`apps/api/src/modules/execution/execution.service.ts`	1.344	Ejecución de órdenes, checklist, medios privados, cierre y corrección.
`apps/api/src/modules/orders/orders.service.ts`	1.102	Creación y edición de órdenes, unidades, equipos, asignación y reprogramación.
`apps/web/src/modules/orders/views/OrderCreateWizardView.vue`	760	Orquestación del asistente y presentación de pasos; ya delega parte de la lógica en composables probados.

Archivo	Líneas antes de esta limpieza	Responsabilidades mezcladas
`apps/web/src/modules/service-execution/views/OrderExecutionView.vue`	752	Estado de ejecución, notas, medios, checklist y acciones de ciclo de vida.

La vista de detalle administrativo no tenía pruebas unitarias o de componente directas; `apps/web/e2e/orders.spec.ts` sí cubría sus puntos de entrada con API simulada. En backend existen pruebas extensas de órdenes y ejecución que deben proteger los invariantes durante futuras extracciones.

Limpieza aplicada

Unidad	Resultado	Evidencia
Medios privados del detalle	`useOrderMedia` concentra precarga, previsualización y liberación de URLs de evidencias y firma.	Tres pruebas enfocadas en precarga, reintento, previsualización y liberación.
Agregar unidad	`useAddOrderUnit` concentra estado, selecciones, validación y mutación; `AddOrderUnitModal` contiene su interfaz.	Cinco pruebas del composable; prueba de navegador para apertura del modal.
Secciones del detalle	`OrderEvidenceTab`, `OrderSyncTab`, `OrderReportCard` y `OrderAuditTab` separan presentación mediante propiedades y eventos tipados.	Typecheck, build y prueba de navegador para las pestañas de evidencia/sincronización.
Notas de ejecución	`useExecutionNotes` centraliza persistencia, indicador de guardado y bandera de cambios sin mezclar el guardado manual con el cierre.	Tres pruebas del composable, incluidos errores de persistencia.
Reprogramación de cita fallida	`reschedule-failed-appointment.ts` separa resolución, clonación de unidades/equipos y auditoría de `OrdersService`.	Pruebas de una transacción, orden de escrituras, conflicto de versión y snapshots inactivos.
Finalización de órdenes	`OrderFinalizationService` concentra completitud, cierre, cita fallida y corrección; `ExecutionService` conserva la API pública y delega.	Pruebas existentes de estados, precios y outbox; prueba nueva del fallo de encolado del PDF posterior al commit.
Procesamiento de evidencias	`image-preprocessing.ts` separa conversión HEIC/HEIF y redimensionado JPEG de la UI y del protocolo de carga.	Cinco pruebas de dimensiones, formato, metadatos y errores de lectura/decodificación.
Reintentos de evidencias Drive	`evidence-upload-retry.ts` encapsula la clave de idempotencia, su persistencia de sesión y el respaldo en memoria; el	Cinco pruebas de identidad, persistencia, prioridad en

Unidad	Resultado	Evidencia
	adaptador de objetos conserva su flujo independiente.	memoria, almacenamiento inaccesible y limpieza.
Galería de evidencias	`useEvidenceGallery` administra cargas, liberación de URLs privadas y respuestas tardías al eliminar evidencias o cambiar de orden/unidad.	Cinco pruebas de carga, retirada, cambio de contexto, borrado y desmontaje.
Asignación de equipo	`useAttachOrderEquipment` y `AttachOrderEquipmentModal` separan el formulario y su mutación del detalle; conservan el payload original.	Cuatro pruebas de permisos, reinicio, mutación y errores.
Medios de ejecución	`ExecutionMediaService` concentra evidencias y firmas; `ExecutionService` mantiene su API pública y delega. Drive y el adaptador de objetos siguen rutas explícitas.	Pruebas existentes de Drive y dos nuevas para confirmación y errores del proveedor de objetos.
Almacenamiento de reportes	`report-storage.ts` separa reserva/reutilización en Drive y escritura del proveedor de objetos de la carga de datos y transacción READY/auditoría.	Pruebas de reserva concurrente, desaparición del PDF remoto y clave/checksum del proveedor de objetos.
Pestaña de unidades	`OrderUnitsTab` separa la presentación mediante propiedades y eventos tipados; el detalle conserva mutaciones, permisos y modales.	Prueba de navegador del evento Quitar hasta DELETE y apertura de Asignar Equipo.
Consultas de órdenes	`OrdersQueryService` concentra listado, detalle administrativo/técnico y auditoría; `OrdersService` conserva delegados públicos compatibles.	Cuatro pruebas nuevas de acceso denegado y lecturas sin consultas innecesarias.

`OrderDetailView.vue` pasó de **1.887 a 764 líneas** (-1.123, -59,5 %); `OrderExecutionView.vue` pasó de **752 a 744**; `orders.service.ts` pasó de **1.102 a 791** (-311, -28,2 %); `execution.service.ts` de **1.344 a 396** (-948, -70,5 %); y `EvidenceCapture.vue` de **456 a 322**. `ReportsService` quedó en **357 líneas**. En el árbol actual no queda ningún archivo de producción TypeScript/Vue de más de 1.000 líneas si se excluyen pruebas, código generado, `dist` y artefactos de CDK. La reducción refleja límites de responsabilidad: no se comprimió código para alcanzar una cifra. Los nuevos módulos siguen siendo archivos revisables por separado.

Verificación actual

- `pnpm typecheck`, `pnpm lint` y `pnpm build`: correctos.
- `pnpm test`: 25 pruebas de contratos, 310 de API y 124 de web correctas; 6 pruebas de API permanecen omitidas.
- Playwright `e2e/orders.spec.ts` en Chromium de escritorio: 16 pruebas correctas, incluidas las acciones de la pestaña de unidades y la navegación por las pestañas extraídas.
- Prettier en los archivos modificados y `git diff --check`: correctos.

- `pnpm format:check` global: falla en 13 archivos **no modificados** por esta limpieza. Es deuda de formato previa, no un fallo atribuido a los cambios de esta unidad.
- Build web: conserva el aviso de un chunk mayor de 500 kB; esta limpieza no abordó tamaño de bundles.

Duplicación y límites de abstracción

- `useExecutionNotes` eliminó la persistencia duplicada de notas; el refresco y mensaje de éxito siguen siendo exclusivos del guardado manual.
- `useAddOrderUnit` eliminó los restablecimientos duplicados del formulario de unidad sin introducir un framework genérico de modales.
- `orders.service.ts` repite parte de la validación de disponibilidad de equipos dentro de `assignEquipment` y `assertEquipmentAvailable`. Los mensajes de error actuales difieren; cualquier unificación debe conservar el contrato observable y la protección de unicidad en la base de datos.
- **No abstraer por inercia:** `useOrderMutations` ya centraliza la invalidación de caché. Drive y el adaptador de objetos tienen protocolos de carga, idempotencia y errores distintos; un wrapper genérico ocultaría esas diferencias.
- La firma Drive usa el mismo mecanismo básico de persistencia de claves que la evidencia, pero su identidad depende del firmante normalizado y del checksum, y vive en el módulo API. Compartir el mecanismo podría reducir duplicación; unir ambos flujos o sus identidades cambiaría sus contratos de reintento.

Límites conservados y deuda residual

- `OrderDetailView.vue` (764 líneas), `OrderCreateWizardView.vue` (760) y `OrderExecutionView.vue` (744) conservan orquestación y plantillas extensas. El asistente ya delega bloques, reemplazos y envío en composables; extraer secciones adicionales solo por tamaño trasladaría complejidad sin clarificar responsabilidades.
- `OrdersService` (791 líneas) conserva operaciones administrativas CRUD y asignación; las lecturas están en `OrdersQueryService` con delegados compatibles, sin introducir otro cableado en router/app. `assignEquipment` y `assertEquipmentAvailable` comparten validaciones, pero difieren en mensajes y contexto. No se unificaron sin caracterizar esos contratos y la restricción de unicidad de base de datos.
- `ExecutionMediaService` (600 líneas) mantiene juntas las operaciones de medios, pero conserva las diferencias entre Drive y el adaptador de objetos. Un split adicional por proveedor exigiría pruebas de paridad de permisos, reintentos y confirmación.
- La interfaz de carga de evidencias no tiene una prueba de navegador que ejecute de extremo a extremo ambos proveedores. Las pruebas unitarias, de router y de servicio protegen los límites refactorizados, pero no reemplazan una integración real con almacenamiento.
- Las cargas simultáneas desde ambos controles de `EvidenceCapture` no se serializaron: cambiar esa política sería comportamiento nuevo, no limpieza mecánica.
- `pnpm format:check` sigue fallando en 13 archivos preexistentes no tocados; formatear los HTML de salida generaría cientos de líneas de churn ajeno a esta limpieza. El chunk

`heic2any` aún supera 500 kB y las 6 pruebas API omitidas siguen siendo deuda independiente.

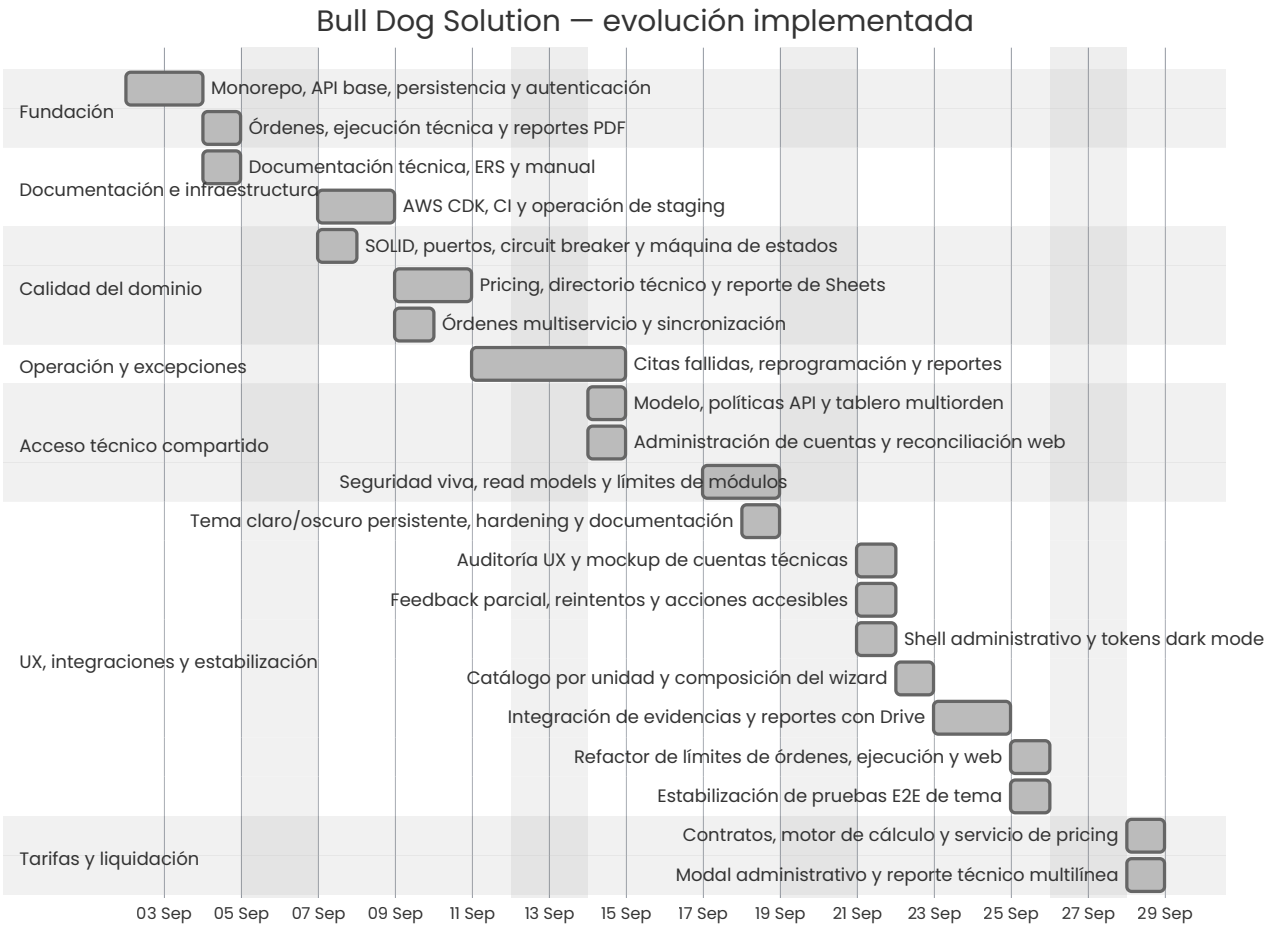
El cierre significa que los límites prioritarios identificados se separaron y verificaron, no que todo riesgo del sistema haya desaparecido. Los cambios quedaron sin commit para revisión del responsable del proyecto.

Línea de tiempo del proyecto

Esta línea de tiempo resume la evolución implementada en el repositorio hasta el 30 de septiembre de 2026. Las fechas históricas y los commits representativos se obtienen del historial Git. El trabajo UX de cuentas técnicas y la limpieza de mantenibilidad ya están integrados en la rama actual; no queda un plan laboral pendiente reflejado aquí.

Estado de lectura (2026-10-02): la cronología es un registro histórico cerrado al 2026-09-30; no representa disponibilidad de staging. La documentación operativa vigente confirma que staging fue eliminado y producción no está desplegada. Esta actualización documental no añade hitos de código.

Diagrama de Gantt



El Gantt muestra fases por día laboral. Los fines de semana quedan fuera del calendario de trabajo y no se agregan tareas para esos días.

Fases y commits representativos

Fecha	Fase	Commits principales	Resultado
2026-09-02	Fundación del producto	`cad9005`, `ffc8048`, `19d5b14`, `de75bec`, `d37c5bc`, `d2edb3a`, `4409750`	Monorepo, API, persistencia, autenticación, órdenes, ejecución técnica y reportes PDF.
2026-09-04	Documentación base	`35886f4`	Paquete inicial de documentación técnica.
2026-09-07	Infraestructura y arquitectura	`3a90b0b`, `7154fad`, `8c93a69`, `62d37e4`, `09624b4`, `c67a47f`	Staging AWS, CI, operación, SOLID, inversión de dependencias y máquina de estados.
2026-09-09	Dominio multiservicio	`0e43ed1` – `0239c88`, `d447a9a`, `d6b967a`	Tarifas, snapshots de pricing, directorio técnico, transiciones de equipos y sincronización de reportes.
2026-09-11	Excepciones operativas	`64d77e3`, `ae79473`, `9742216`, `b31bedb`, `871ac21`, `e73f53a`	Programación de reportes, citas fallidas, reprogramación y reintentos consistentes.
2026-09-14	Acceso compartido	`5cefdbe`, `1acd4ed`, `39e7b19`, `b65621c`, `34d9cb2`, `7b9a9c3`, `6df839a`	Cuenta técnica compartida, tablero multiorden, permisos y administración de cuentas.
2026-09-17	Endurecimiento y mantenibilidad	`6fb7580`	Principal vigente, read models seguros, reconciliación de sesión, límites de módulos y refactor frontend.
2026-09-18	Staging y tema	`2abece9`, `46ffb5e`, `bf285db`	Tema persistente claro/oscuro, hardening de despliegue y documentación operativa.
2026-09-21	UX de cuentas técnicas	`48d127d`, `1754850`, `6508887`	Auditoría y mockup, feedback/reintentos, acciones accesibles y superficies administrativas del tema.
2026-09-22	UX del asistente de órdenes	`82cfe23`, `8f1a98b`	Estado del wizard en composables y explicación de ejecución por unidad.
2026-09-23	Google Drive	`0f7c455`, `820b47f`, `a6e386d`	Evidencias, firmas y reportes privados en Drive, organización y documentación/benchmark.
2026-09-25	Mantenibilidad y estabilización	`68b769b`, `365c7b4`, `24b91fd` – `cfd7856`	Separación de consultas, flujos y presentación; auditoría actualizada y aserciones E2E de tema estabilizadas.
2026-09-28	Tarifas y liquidación dinámica	`bfb0608` – `48c05cc`	Contratos de pricingOverrides, motor decimal puro, OrderPricingService, modal administrativo y costo multilínea en reporte.

Hitos actuales

- **Producto funcional:** órdenes administrativas, ejecución móvil, evidencias, firma, reportes y sincronización con Sheets.
- **Operación avanzada:** pricing por servicio, citas fallidas y tablero de taller multiorden.
- **Seguridad:** autorización por capacidad y propiedad, principal resuelto en vivo y respuestas operativas sin datos administrativos.
- **Calidad:** contratos compartidos, pruebas unitarias/integración/E2E, OpenAPI validado, build y auditoría de dependencias de producción sin vulnerabilidades conocidas.
- **UX y estabilización:** los cambios de cuentas técnicas y del asistente están integrados; las pruebas E2E del tema fueron ajustadas para tolerar diferencias de Chromium.
- **Mantenibilidad:** limpieza prioritaria reportada y verificada; las limitaciones y deuda residual permanecen explícitas en `docs/maintainability-audit.md`.

Criterio de lectura

El Gantt representa días de trabajo observables en Git y tareas UX planificadas. Para una auditoría histórica, usar `git log --date=iso` y revisar cada hash listado en la tabla. El trabajo del 21 de septiembre se identifica explícitamente como rama activa sin commit.

Seguridad, Calidad y Observabilidad

Este documento consolida la arquitectura de seguridad, calidad y observabilidad implementada en el sistema Bull Dog Solution conforme a `SPECS.md` y la Fase 12 del roadmap.

> **Alcance vigente:** describe controles del código, no una garantía de despliegue. AWS staging está eliminado y no existe producción desplegada. TLS, retención/backups de Drive y operación segura del ambiente deben verificarse al recrear un entorno; la IaC de staging expone HTTP en una IPv4 efímera y no crea TLS/ALB/CloudFront.

1. Seguridad de Red y Transporte

- **TLS / HTTPS:** Todas las comunicaciones externas deben realizarse exclusivamente bajo TLS 1.3/1.2.
- **CORS:** Configurado mediante lista blanca explícita de orígenes (`APP_ORIGIN`). Se rechazan orígenes no autorizados y se habilitan credenciales (`credentials: true`) únicamente para el origen frontend de la aplicación.
- **Cabeceras de Seguridad:** Uso de `helmet` con políticas de Content-Security-Policy, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, y eliminación de cabecera `X-Powered-By` .

2. Autenticación y Gestión de Sesiones

- **Tokens Asimétricos:** Emisión de JWT de corta duración (15 minutos) firmados con par de claves asimétricas RSA/EdDSA.
- **Refresh Tokens Rotativos:** Los refresh tokens se almacenan en PostgreSQL con hash y revocación inmediata. Se transmiten mediante cookie HTTP-only con atributos `Secure` , `SameSite=Strict` y ruta restringida (`/api/v1/auth`).
- **Acceso técnico:** La política individual limita al técnico a órdenes de su asignación. Una cuenta TECHNICIAN designada para taller puede acceder solo a la única orden explícitamente habilitada; esa concesión no cambia el técnico asignado y se audita.

3. Protección de Datos y Almacenamiento

- **Inyección SQL:** No se utiliza SQL crudo no parametrizado. Todas las consultas se canalizan a través del ORM Prisma y PostgreSQL con parámetros vinculados.
- **Límites de Entrada:**
  - Payload JSON de Express acotado a 1 MiB.
  - Rate limiting central (`express-rate-limit`) con cuotas por IP y ventana de tiempo.
  - Validación de esquemas en todas las entradas mediante Valibot.

- **Archivos privados:**
  - Google Drive compartido es el proveedor seleccionado por la IaC de staging; S3/MinIO se conserva en el código para otros entornos.
  - En modo Drive, fotografías, firmas y PDFs se transfieren por rutas autenticadas de la API, que comprueba autorización y devuelve archivos privados sin cache. El adaptador de objetos utiliza URLs prefirmadas donde corresponde.
  - Verificación de metadatos (tipo MIME JPEG/PNG/WebP, tamaño máximo por imagen y dimensiones) antes de confirmar el registro.
- **Redacción de Secretos en Logs:**
  - Filtro central en Winston y Morgan que anonimiza y redacta automáticamente tokens de portador, contraseñas, cookies de refresh, claves de API, URLs firmadas y datos privados.

4. Observabilidad y Métricas

- **Endpoint Prometheus (`/metrics`):**
  - `http_requests_total` : Conteo de peticiones HTTP por método, ruta normalizada y código de estado.
  - `http_request_duration_seconds` : Latencia media por ruta.
  - `auth_login_failures_total` : Conteo de intentos de autenticación fallidos.
  - `uploads_failed_total` : Conteo de subidas fallidas.
  - `sheets_operations_total` : Operaciones hacia Google Sheets clasificadas por estado (`COMPLETED` , `FAILED` , `PENDING`).
  - `circuit_breaker_state` : Estado del interruptor de circuito (`CLOSED` , `OPEN` , `HALF_OPEN`).
  - `outbox_events_pending_total` y `outbox_oldest_event_age_seconds` : Monitorización de cola de sincronización.
  - `orders_by_status` : Conteo en tiempo real de órdenes por estado.
- **Health Checks:**
  - `/health/live` : Comprobación de proceso en ejecución sin dependencias.
  - `/health/ready` : Comprobación de conectividad a PostgreSQL y dependencias críticas.
  - `/health/metrics` : Instantánea JSON para supervisión administrativa.

5. Respaldo, Restauración y Limpieza de Almacenamiento

- **Respaldos de Base de Datos:** Script automatizado `infra/scripts/backup.sh` con compresión `gzip` , suma de verificación SHA256 y rotación de 30 días.
- **Restauración y Recuperación ante Desastres:** Script `infra/scripts/restore.sh` que valida la integridad del archivo y ejecuta la restauración en una sola transacción (`--single-transaction`).

- **Limpieza de Objetos Abandonados:** existe `infra/scripts/cleanup-orphaned-objects.ts` ; su alcance debe revisarse antes de usarlo (no asumir que cubre todas las referencias, en particular reportes Drive). No ejecutarlo para limpieza histórica sin autorización específica.

Especificación funcional — Reporte de orden de servicio

Organización: Bull Dog Solution

Documento: Reporte PDF de orden de servicio

Fecha: 2026-09-02

Versión: 0.1

Estado al 2026-10-02: Implementación del reporte PDF privada y versionada; plantilla/logotipo oficial y pruebas de regresión visual siguen pendientes de aprobación.

1. Objetivo

Generar un reporte PDF formal por cada orden de servicio completada. El documento debe conservar la información que actualmente se presenta en AppSheet, permitir identificar con claridad el servicio realizado y mostrar las evidencias fotográficas con un tamaño suficiente para su revisión.

El reporte funciona como comprobante de atención para Bull Dog Solution y para el cliente. No sustituye los datos transaccionales de la aplicación: es una representación inmutable de la versión de la orden con la que fue generado.

2. Alcance

2.1 Incluido

- Generación automática al completar una orden de instalación, revisión o desinstalación.
- Descarga y consulta del PDF desde el detalle de la orden.
- Portada con identificación del servicio, datos del cliente, técnico, observaciones generales y firma.
- Una sección por cada unidad de la orden.
- Detalle de todos los equipos relacionados con cada unidad.
- Checklist y observaciones de la unidad.
- Evidencias fotográficas agrupadas por unidad; son una galería general y no se vinculan a una categoría del checklist.
- Regeneración controlada cuando una orden pase por corrección y vuelva a completarse.
- Acceso privado y auditado al documento.

2.2 Fuera de alcance inicial

- Edición manual del PDF después de generarlo.
- Plantillas diferentes por cliente.
- Envío automático por correo o mensajería.
- Reportes consolidados de varias órdenes.
- Reporte equivalente para citas fallidas.
- Firma criptográfica o sello fiscal del PDF.

3. Disparador y estados

1. El sistema solicita la generación cuando una orden normal cambia a `COMPLETED`.
2. La generación ocurre en segundo plano para que el cierre de la orden no dependa del tiempo de armado del PDF.
3. El reporte pasa por los estados `PENDING`, `PROCESSING`, `READY` o `FAILED`.
4. Si falla la generación, la orden permanece completada y un administrador puede reintentarla.
5. Si la orden entra a `REQUIRES_CORRECTION`, el último PDF deja de ser la versión vigente, pero se conserva para auditoría.
6. Cuando la orden corregida vuelve a `COMPLETED`, se genera una nueva versión.
7. Una versión `READY` nunca se sobrescribe; cada regeneración crea un archivo nuevo.

4. Estructura del documento

El documento se genera en tamaño Carta y orientación vertical. Debe respetar márgenes seguros de impresión, usar tipografía legible y presentar numeración de página en el pie, excepto cuando interfiera con la firma o una fotografía.

4.1 Portada

La primera página debe contener:

- Logotipo oficial de Bull Dog Solution.
- Título `REPORTE DE ORDEN DE SERVICIO`.
- Folio de la orden como elemento visual principal.
- Nombre del cliente.
- Tipo de servicio: instalación, revisión o desinstalación.
- Fecha programada del servicio.
- Fecha y hora de inicio y conclusión, cuando estén disponibles.
- Nombre del técnico responsable.
- Taller o domicilio del servicio, cuando aplique.
- Observaciones generales de la orden. Si no existen, mostrar `Sin observaciones generales`.
- Nombre de la persona que firma por parte del cliente.

- Imagen de la firma del cliente.
- Fecha y hora de firma.
- Leyenda de conformidad aprobada por el negocio.

El folio, cliente y logotipo deben poder identificarse sin ampliar el documento. La firma se escala conservando su proporción y nunca debe deformarse.

Las observaciones generales incluyen las notas capturadas durante la creación o ejecución de la orden. Se trata de un único campo por orden; las notas específicas por unidad quedan fuera de este alcance.

4.2 Detalle por unidad

Después de la portada se crea una sección independiente por cada unidad, en el mismo orden en que aparecen en la orden. Cada sección debe comenzar con un encabezado que repita el folio e identifique la unidad.

Datos requeridos:

- Nombre o etiqueta de la unidad (`label`).
- Marca y modelo (`makeModel`).
- Año, o `No registrado` cuando no exista.
- VIN o número de serie (`serialNumber`).
- Estado final de la unidad.

4.3 Equipos de la unidad

Cada unidad debe incluir todos sus equipos, aun cuando alguno tenga una discrepancia. Por cada equipo se presentan, cuando existan:

- Marca.
- Modelo.
- IMEI.
- Resultado de verificación: coincidente o discrepancia reportada.
- Descripción de la discrepancia y valores observados, cuando aplique.

El reporte omite expresamente el ID externo, ICCID, línea telefónica, operador, condición del equipo y estado del servicio del equipo.

Los campos opcionales sin información se muestran como `No registrado` ; no se deben imprimir valores técnicos como `null` , `undefined` o claves internas.

4.4 Checklist de la unidad

Se incluye el resultado de las cinco categorías:

- Tablero.
- Cristales.
- Accesorios.

- Molduras.
- Carrocería.

Para cada categoría se muestra la condición en español (`Bien` , `Mal` o `No aplica`) y su observación. Cuando no exista observación, se muestra `Sin observaciones` .

4.5 Evidencias fotográficas

Las fotografías aparecen inmediatamente después de la información de la unidad a la que pertenecen. La categoría del checklist no se imprime en la página de evidencia, porque la fotografía puede documentar un elemento distinto a la categoría usada durante su captura.

Reglas de presentación:

- Una fotografía por página siempre que sea técnicamente posible.
- La imagen ocupa el área útil máxima de la página, conservando su proporción y sin recortarse.
- No se estira ni se deforma la imagen.
- Se permite rotarla automáticamente para aprovechar mejor la hoja según su orientación y metadatos EXIF.
- Una imagen de baja resolución no se amplía más allá de un límite que la vuelva ilegible; se centra sobre fondo blanco.
- Cada página muestra folio, unidad, número consecutivo de evidencia y fecha/hora de captura o carga disponible.
- Las evidencias se numeran consecutivamente dentro de la unidad.
- Solo se incluyen evidencias con estado `COMPLETED` y no eliminadas.
- Si una imagen no puede leerse, la generación falla de forma visible; el PDF no debe omitirla silenciosamente.

El objetivo visual es que la evidencia abarque casi toda la hoja para facilitar la inspección de detalles. No se usa un mosaico de miniaturas en esta primera versión.

5. Correspondencia con el modelo actual

Contenido del reporte	**Fuente actual**
Folio	`Order.folio`
Cliente	`Order.clientNameSnapshot`
Tipo de servicio	`Order.serviceType`
Fecha programada	`Order.scheduledAt`
Inicio y conclusión	`Order.startedAt` , `Order.completedAt`
Técnico	`Order.technician.name`
Ubicación	`Order.serviceAddress` o datos del taller del técnico

Contenido del reporte	Fuente actual
Observaciones generales	`Order.generalNotes`
Nombre y firma del cliente	`Signature.signerName`, imagen en `Signature.objectKey`
Fecha de firma	`Signature.signedAt`
Nombre/modelo/año/VIN de unidad	`OrderUnit.label`, `makeModel`, `year`, `serialNumber`
Datos de equipos	Snapshots de `OrderEquipment`
Checklist	`ChecklistItem.category`, `condition`, `notes`
Fotografías	`Evidence.objectKey`, `createdAt`, dimensiones y tipo MIME
Discrepancias	`Discrepancy.description`, `expectedSnapshot`, `observedSnapshot`

El reporte usa los snapshots guardados en la orden y no vuelve a consultar Google Sheets al generarse. Esto evita que una modificación posterior del catálogo cambie un acta ya emitida.

6. Reglas funcionales

- Solo se genera para órdenes normales en estado `COMPLETED`.
- Debe existir una firma válida antes de generar el reporte.
- Todas las unidades y todos los equipos de la orden deben aparecer.
- Una unidad sin equipos conserva su sección e indica `Sin equipos registrados`.
- No existe un límite funcional de páginas; el número depende de unidades, equipos y fotografías.
- Todas las fechas se presentan en la zona horaria configurada para la operación y con formato legible en español.
- Los textos largos deben continuar en la página siguiente, sin superponerse ni quedar cortados.
- El PDF debe representar caracteres en español correctamente.
- Los datos sensibles incluidos se limitan a los requeridos por el acta; no se muestran IDs UUID, claves S3, filas de Sheets ni información de auditoría interna.

7. Acceso y experiencia de usuario

- El administrador puede consultar y descargar cualquier reporte.
- El técnico puede consultar y descargar reportes de sus propias órdenes.
- El detalle de la orden muestra el estado de generación y las acciones `Descargar reporte` o `Reintentar generación`, según permisos.
- La descarga usa una URL temporal o un proxy autenticado; el objeto no es público.
- El nombre sugerido del archivo es `orden-servicio-{folio}-v{version}.pdf`, normalizando caracteres no seguros.

- Cada consulta, descarga, reintento y generación queda registrada en auditoría.

8. Persistencia propuesta

Se requiere una entidad de reporte versionado con, al menos:

- `id`.
- `orderId`.
- `orderVersion`.
- `status`.
- `objectKey` opcional hasta completar la generación.
- `mimeType` con valor `application/pdf`.
- `sizeBytes`.
- `checksum`.
- `generatedAt`.
- `failureReason` sanitizado.
- `createdAt` y `updatedAt`.

La clave sugerida en almacenamiento es:

```
orders/{orderId}/reports/{reportId}.pdf
```

La restricción de unicidad debe evitar dos reportes vigentes para la misma combinación de orden y versión. Los reintentos deben ser idempotentes.

9. API propuesta

Método	Ruta	Propósito
`GET`	`/orders/:id/report`	Obtener estado y metadatos del reporte vigente
`GET`	`/orders/:id/reports`	Consultar versiones anteriores permitidas
`POST`	`/orders/:id/report/generate`	Solicitar generación o reintento idempotente
`GET`	`/orders/:id/report/download`	Obtener descarga autenticada o URL temporal

La generación automática reutiliza el mismo caso de uso que el endpoint manual. La respuesta de generación debe ser asíncrona (`202 Accepted`) cuando el archivo todavía no esté listo.

10. Requisitos no funcionales

- El PDF debe abrir correctamente en navegadores actuales, visores móviles y lectores estándar.
- La generación no debe cargar las imágenes completas en memoria simultáneamente.

- El proceso debe tolerar órdenes extensas mediante lectura y composición por flujo o por lotes.
- El acceso a fotografías, firma y reporte debe respetar el almacenamiento privado.
- Los errores no deben exponer claves de objetos, rutas internas ni URLs firmadas en logs.
- El checksum permite verificar que el archivo descargado corresponde a la versión generada.
- La salida debe ser determinista para una misma versión de orden, plantilla y recursos de marca.

11. Criterios de aceptación

1. Al completar una orden normal se crea automáticamente una solicitud de reporte.
2. La portada muestra logotipo, folio, cliente, fechas, técnico, observaciones generales, nombre y firma del cliente.
3. El PDF contiene una sección por cada unidad con nombre, marca/modelo, año y VIN o número de serie.
4. Cada sección contiene todos los equipos de la unidad y sus datos disponibles.
5. Las discrepancias se muestran junto al equipo correspondiente y no sustituyen los valores esperados.
6. El checklist muestra las cinco categorías, su condición traducida y sus observaciones.
7. Cada fotografía válida aparece una sola vez, asociada a su unidad; la categoría del checklist no se imprime en la página de evidencia.
8. Las fotografías se presentan, por defecto, una por página, grandes, sin recorte y sin deformación.
9. Una orden con varias unidades y varios equipos conserva correctamente la agrupación de datos y evidencias.
10. Los campos opcionales vacíos usan textos legibles y nunca muestran valores técnicos.
11. Un técnico sin acceso individual o compartido a la orden no puede consultar ni descargar el reporte.
12. El archivo se almacena de forma privada y la URL de descarga expira.
13. Un fallo de generación es visible y permite reintento sin duplicar la versión vigente.
14. Después de una corrección se genera una versión nueva y la anterior se conserva para auditoría.
15. Una prueba de regresión visual compara portada, detalle y páginas fotográficas contra documentos de referencia aprobados.

12. Casos mínimos de prueba

- Una unidad, un equipo y cinco fotografías verticales.
- Varias unidades y varios equipos por unidad.
- Fotografías verticales, horizontales y con orientación EXIF.
- Varias fotografías dentro de la misma categoría.
- Equipo con campos opcionales vacíos.

- Equipo con discrepancia y valores observados.
- Observaciones generales y de checklist de varias líneas.
- Imagen dañada o faltante.
- Reintento después de un fallo temporal de almacenamiento.
- Corrección y generación de una segunda versión.
- Intento de descarga por administrador, técnico asignado, técnico con acceso compartido a la orden y técnico sin acceso.

13. Definiciones pendientes de negocio y diseño

Antes de aprobar la plantilla final se deben confirmar:

1. Archivo oficial del logotipo, variantes permitidas y colores de marca.
2. Texto exacto de la leyenda de conformidad junto a la firma.
3. Si la fecha principal de portada será la programada, la de inicio o la de conclusión.
4. Si `serialNumber` siempre representa el VIN o si se requiere un campo separado.
5. Nombre comercial de cada dato proveniente de `rawSnapshot` que también deba mostrarse.
6. Política de retención y visibilidad de versiones anteriores.
7. Si el cliente tendrá posteriormente un canal directo de descarga o envío.
8. Si se requiere simultáneamente una variante A4 además del formato Carta.

Especificación de Requerimientos y Diseño Técnico (ERS / RFC)

Tarifas Dinámicas y Ajuste de Liquidación de Órdenes de Servicio

- **Proyecto:** Bull Dog Solution
- **Documento:** Especificación técnica y de requerimientos
- **Fecha original:** 2026-09-28
- **Revisión de estado:** 2026-10-02
- **Estado:** Implementado y verificado
- **Autor:** Equipo de Arquitectura de Software

---

1. Contexto y Justificación del Negocio

En la operación de campo actual, los costos que cobran los técnicos varían frecuentemente en función de:

- El día en que se ejecuta la orden (fines de semana, días festivos, urgencias).
- Negociaciones puntuales en el sitio de trabajo o condiciones logísticas extraordinarias.
- Desviaciones detectadas con posterioridad a la finalización del servicio que requieren corrección contable antes de liquidar el pago.

Actualmente, las tarifas se encuentran definidas en la hoja de cálculo de Google Sheets (`DIRECTORIO TECNICO`), las cuales se sincronizan hacia la base de datos en la tabla `Technician` . Sin embargo, al completar una orden, el sistema toma de forma rígida esas tarifas maestras del técnico.

Principio Rector

Las tarifas del directorio en Google Sheets se conservan como valores base/por defecto. No se eliminan ni se altera su sincronización. No obstante, el sistema debe proveer la capacidad de sobreescribir las tarifas para una orden específica, tanto al inicio de su ejecución como tras haber sido finalizada.

---

2. Alcance y Reglas de Negocio

2.1 Control de Acceso y Roles (MANDATORIO)

- **Técnicos (`TECHNICIAN`):**

    - **No tienen permiso de modificar tarifas ni viáticos.**
    - Su interfaz móvil de ejecución (`OrderExecutionView.vue`) mantiene el flujo limpio habitual. No se añaden pasos bloqueantes ni campos de precio a su pantalla.

- **Administradores (`ADMIN`):**
    - Poseen control total sobre las tarifas y liquidación (`orders:manage`).
    - Pueden definir o modificar tarifas acordadas mientras la orden no esté finalizada ni cancelada (incluye `DRAFT` , `ASSIGNED` , `IN_PROGRESS` y órdenes devueltas a `REQUIRES_CORRECTION`).
    - Pueden ajustar precios y viáticos en órdenes ya finalizadas (`COMPLETED` , `FAILED_APPOINTMENT`).

2.2 Ciclo de Vida de los Precios en la Orden

1. **Valores por defecto:** Al asignar un técnico a una orden, las tarifas base del servicio y viático corresponden a las configuradas en su registro.
2. **Definición previa o al inicio:** Un administrador puede registrar tarifas personalizadas (`pricingOverrides`) para esa orden específica. Si un servicio no tiene tarifa personalizada, hereda la tarifa base del técnico.
3. **Cálculo al finalizar (`completeOrder` / `recordFailedAppointment`):** El motor de liquidación toma prioritariamente las tarifas de `pricingOverrides` si existen; en su defecto, utiliza las tarifas maestras del técnico.
4. **Ajuste post-cierre:** Si una orden ya está `COMPLETED` o `FAILED_APPOINTMENT` , el administrador puede editar directamente los precios unitarios o viáticos. Esto genera una nueva versión de precios (`pricingVersion`), recalcula los subtotales/totales, actualiza el snapshot y encola la actualización automática hacia Google Sheets.

---

3. Arquitectura y Principios de Diseño

Para evitar degradar la mantenibilidad del proyecto y no generar archivos monolíticos ("archivotes con muchas dependencias"), se adoptan los siguientes principios de arquitectura limpia:

1. **Responsabilidad Única (SRP):**
    - `pricing.ts` : Función pura de cálculo sin dependencias de base de datos ni I/O.
    - `OrderPricingService` : Nuevo servicio desacoplado encargado exclusivamente de la gestión, validación, actualización y auditoría de tarifas de órdenes.
    - `orders.service.ts` y `order-finalization.service.ts` : No se sobrecargan de lógica contable accesoria.
2. **Auditoría e Inmutabilidad de Versiones:**
    - Todo cambio de tarifas registra un evento en la tabla `AuditLog` (`ORDER_PRICING_CONFIGURED` o `ORDER_PRICING_ADJUSTED`).
    - Cada ajuste post-cierre incrementa `pricingVersion` , preservando la trazabilidad.

3. **Sincronización Idempotente con Google Sheets:**
    - La sincronización del reporte técnico (`SHEET_TECHNICAL_REPORT`) utiliza `idempotencyKey: REPORT:${orderId}:${pricingVersion}` . Cualquier ajuste sustituye las operaciones pendientes (`SUPERSEDED`) y programa la escritura de la nueva liquidación en la hoja de cálculo.

---

4. Especificación Técnica de Componentes

4.1 Modelo de Datos (Prisma)

Se agrega el campo `pricingOverrides` en el modelo `Order` de `prisma/schema.prisma` :

```
model Order {
  // ... campos existentes ...
  pricingUnitRate          Decimal?          @db.Decimal(10, 2)
  pricingAppliedQuantity   Int?
  pricingBaseCost          Decimal?          @db.Decimal(10, 2)
  pricingDepartureCost     Decimal?          @db.Decimal(10, 2)
  pricingTravelExpenses    Decimal?          @db.Decimal(10, 2)
  pricingTotal             Decimal?          @db.Decimal(10, 2)
  pricingCurrency          String?           @default("MXN")
  pricingSnapshotAt        DateTime?         @db.Timestamptz(3)
  pricingVersion           Int?              @default(1)
  pricingBreakdown         Json?             @db.JsonB
  pricingOverrides         Json?             @db.JsonB   // <--- NUEVO CAMPO
  // ... relaciones y auditoría ...
}
```

Estructura de `pricingOverrides` (`Json`):

```
{
  "rates": {
    "INSTALLATION": "650.00",
    "REVIEW": "300.00"
  },
  "travelExpenses": "400.00",
  "reason": "Tarifa convenida por día festivo"
}
```

4.2 Estrategia de Tipado y Precisión Financiera (Type Safety & Precision)

Para prevenir discrepancias contables y errores de redondeo por punto flotante (IEEE-754 en JavaScript, ej. `0.1 + 0.2 = 0.30000000000000004`), el manejo de tipos se estructura estrictamente en 4 capas:

1. **Transporte y Contrato (`@puk/contracts`):**
   - **Fuente de verdad única (SSOT)** definida con Valibot.
   - Todo importe monetario viaja como `string` decimal formateado con 2 dígitos decimales (`/^\d+(\.\d{1,2})?$/`). Cero tipos `number` en JSON de transporte para precios.
   - Las claves de servicios usan el enum fuertemente tipado `OrderServiceType` .
2. **Dominio y Cálculo (`apps/api` - `pricing.ts`):**
   - Los strings se convierten a instancias de `Prisma.Decimal` (basado en `decimal.js`).
   - Toda la aritmética (`plus` , `times`) opera en precisión arbitraria con redondeo explícito (`ROUND_HALF_UP` a 2 decimales).
   - Los snapshots se generan llamando a `.toFixed(2)` para reingresar al contrato como string inmutable.
3. **Persistencia (PostgreSQL / Prisma):**
   - `Order.pricingOverrides` : Almacenado como `Json?` (`@db.JsonB`), validado en runtime mediante el schema de Valibot al deserializar.
   - Columnas monetarias calculadas (`pricingTotal` , `pricingBaseCost` , etc.): Preservadas como `Decimal(10, 2)` .
4. **Presentación e Interfaz (`apps/web`):**
   - Componentes de formulario de Naive UI (`NInputNumber`) manipulan temporalmente valores numéricos en el estado local reactivo.
   - La capa de cliente API (`api.ts`) es responsable de transformar `number | null` a `string` (`val.toFixed(2)`) antes de emitir la petición HTTP, garantizando coherencia con el contrato.

Esquemas Valibot en `@puk/contracts` (`packages/contracts/src/orders.ts`):

```
export const PricingRateOverrideSchema = v.pipe(
  v.string(),
  v.regex(/^\d+(\.\d{1,2})?$/, 'Debe ser un importe válido (ej. 450.00)'),
);

export const OrderPricingOverridesSchema = v.object({
  rates: v.optional(v.record(OrderServiceTypeSchema, PricingRateOverrideSchema)),
  travelExpenses: v.optional(v.nullable(PricingRateOverrideSchema)),
  reason: v.optional(v.nullable(v.pipe(v.string(), v.maxLength(500)))),
});

export type OrderPricingOverrides = v.InferOutput<typeof OrderPricingOverridesSchema>;
```

4.3 Motor de Cálculo Puro (`pricing.ts`)

Se extiende la entrada de cálculo para admitir tarifas personalizadas:

```
export interface CalculateOrderPricingInput {
  readonly technician: TechnicianPricingSource;
  readonly serviceType?: OrderServiceType | null;
  readonly equipmentCount?: number;
  readonly isHomeService: boolean;
  readonly pricingVersion?: number;
  readonly snapshotAt?: Date;
  readonly items?: ReadonlyArray<{
    serviceType: OrderServiceType;
    equipmentCount?: number;
  }>;
  readonly overrides?: OrderPricingOverrides | null; // <--- NUEVO
}
```

- En la resolución de tarifa (`resolveServiceRate` o interna): si `overrides?.rates?.[serviceType]` existe, se utiliza ese valor y se omite la tarifa del técnico.
- En viáticos: si `overrides?.travelExpenses` está provisto, se aplica directamente dicho importe.

4.4 Servicio Dedicado: `OrderPricingService`

Ubicación: `apps/api/src/modules/orders/order-pricing.service.ts`

Responsabilidades:

1. `setOrderPricingOverrides(actorUserId: string, orderId: string, input: OrderPricingOverrides)` :
    - Permite a administradores configurar o actualizar tarifas pactadas en órdenes que no estén finalizadas ni canceladas; la vista ofrece el control a administradores en cualquier estado excepto `CANCELLED` .
    - Persiste en `order.pricingOverrides` .
    - Registra auditoría `ORDER_PRICING_CONFIGURED` .
2. `adjustFinalizedOrderPricing(actorUserId: string, orderId: string, input: OrderPricingOverrides)` :
    - Valida que la orden esté en `COMPLETED` o `FAILED_APPOINTMENT` .
    - Carga la orden con sus unidades, equipos y técnico.
    - Ejecuta `calculateOrderPricing` con los nuevos `overrides` .
    - Dentro de una transacción de dominio (`domainTransaction`):
        - Incrementa `pricingVersion` .
        - Actualiza `pricingBaseCost` , `pricingTravelExpenses` , `pricingTotal` , `pricingBreakdown` , `pricingSnapshotAt` y `pricingOverrides` .
        - Marca operaciones pendientes de `SHEET_TECHNICAL_REPORT` como `SUPERSEDED` .
        - Encola nueva operación `SHEET_TECHNICAL_REPORT` con `formatTechnicalReportRow` .
        - Registra auditoría `ORDER_PRICING_ADJUSTED` .

4.5 Endpoints HTTP (`apps/api`)

En `orders.router.ts` :

- `PATCH /api/orders/:id/pricing` :
  - Middleware: `authorize('orders:manage')` .
  - Validación del cuerpo con `OrderPricingOverridesSchema` .
  - Enruta según el estado de la orden a `setOrderPricingOverrides` o `adjustFinalizedOrderPricing` .

4.6 Frontend (`apps/web`)

1. **Componente Reutilizable (`OrderPricingModal.vue`):**
   - Modal de Naive UI (`NModal` , `NForm` , `NInputNumber`).
   - Lista cada tipo de servicio presente en la orden con su tarifa base actual y el campo para ingresar tarifa personalizada.
   - Campo para costo de viáticos / salida.
   - Campo opcional de justificación o nota del ajuste.
2. **Integración en `OrderDetailView.vue` (Vista Administrativa):**
   - En la sección "Desglose de Liquidación Técnica":
     - Si la orden está en estado `ASSIGNED` o `IN_PROGRESS` : botón "Configurar Tarifas Acordadas".
     - Si la orden está en estado `COMPLETED` o `FAILED_APPOINTMENT` : botón "Ajustar Precios de Liquidación".
   - Al guardar, refresca reactivamente la orden y muestra notificación de éxito.

4.7 Formateo Multilínea en Reporte Técnico de Google Sheets (`technical-report-formatter.ts`)

En órdenes con múltiples unidades y servicios multilínea (salvo citas fallidas):

- Las columnas F (`economico`), G (`serie`), H (`imeiId`), K (`servicio`) y L (`costo`) utilizan `\n` para alinear cada renglón por unidad en orden cronológico.
- La columna L (`costo`) resuelve el costo de cada unidad individual (`resolveUnitCost`) con base en el desglose congelado (`pricingBreakdown`), tarifas sobreescritas (`pricingOverrides`) o tarifa unitaria, multiplicado por el número de equipos de la unidad (o 1 para reemplazo de equipos).
- Las columnas M (`viaticos`) y N (`total`) conservan el valor global consolidado de la orden.

---

5. Matriz de Impacto y Mitigación de Riesgos

Componente	Nivel de Riesgo	Estrategia de Mitigación
Sincronización Google Sheets	Bajo	Se reutiliza el formateador `formatTechnicalReportRow` y el sistema de reintentos e idempotencia existente por versión.
Reportes PDF de Operación	Nulo	El reporte PDF no contiene datos financieros ni tarifas de técnicos; solo evidencias operativas.
Flujo de Técnicos en Campo	Nulo	No se altera la experiencia ni los endpoints utilizados por los técnicos en campo.
Integridad de Código Existente	Mínimo	No se modifica la firma base de funciones existentes; los overrides son estrictamente opcionales.

---

6. Fases de Implementación Propuestas

1. **Fase 1: Contratos y Persistencia**
   - Crear migración Prisma con la columna `pricingOverrides` en `Order`.
   - Definir esquemas Valibot en `packages/contracts`.
2. **Fase 2: Motor de Cálculo y Pruebas Unitarias**
   - Adaptar `calculateOrderPricing` en `pricing.ts` para consumir `overrides`.
   - Añadir batería de pruebas unitarias en `pricing.test.ts`.
3. **Fase 3: Servicio de Negocio y Endpoints**
   - Implementar `OrderPricingService` con auditoría y re-sincronización de Sheets.
   - Exponer endpoint `PATCH /api/orders/:id/pricing` protegido por `orders:manage`.
   - Añadir pruebas de integración en `orders.router.test.ts`.
4. **Fase 4: Frontend Administrativo**
   - Implementar `OrderPricingModal.vue`.
   - Integrar modal y botones de acción en `OrderDetailView.vue`.
5. **Fase 5: Verificación End-to-End**
   - Ejecutar suite de pruebas de regresión y prueba de flujo completo.
6. **Fase 6: Formateo Multilínea de Costos en Reporte Técnico**
   - Desglosar columna L (`costo`) renglón por renglón alineada con las unidades y servicios en Google Sheets.
   - Validar suite unitaria de formateo de reporte con servicios homogéneos y mixtos.

Guía de Despliegue, CI/CD y Ambiente de Staging

Este documento detalla los procedimientos de integración continua, despliegue y operación del ambiente de Staging y Producción para Bull Dog Solution.

> **Estado al 2026-10-09:** AWS staging está eliminado/no desplegado y producción no está desplegada. Los inventarios y procedimientos de AWS de este documento son referencias históricas o pasos para un futuro redeployment; verificá los recursos y el diff CDK antes de operar. Gmail tampoco está conectado al despliegue CDK.

1. Pipeline de Integración Continua (CI)

El archivo `.github/workflows/ci.yml` define el flujo de validación automática en cada `push` o `pull_request`:

1. **Instalación de dependencias:** Uso de `pnpm install --frozen-lockfile` con almacenamiento en caché.
2. **Validación Prisma:** Ejecución de `pnpm prisma:validate` y generación del cliente Prisma.
3. **Análisis Estático (Linting):** Ejecución de `pnpm lint` aplicando las reglas de ESLint de arquitectura modular.
4. **Comprobación de Tipos (Typecheck):** Ejecución de `pnpm typecheck` en todos los paquetes del monorepo.
5. **Pruebas Unitarias y de Integración:** Ejecución de `pnpm test` con Vitest.
6. **Validación de OpenAPI:** Ejecución de `pnpm docs:validate` y verificación de contratos OpenAPI contra endpoints.
7. **Compilación (Build):** Generación de artefactos optimizados de backend y frontend web.
8. **Auditoría de Dependencias:** Ejecución de `pnpm audit` verificando 0 vulnerabilidades conocidas.
9. **Pruebas E2E:** Ejecución de pruebas con Playwright en entornos de escritorio y móvil (`pnpm test:e2e`).

2. Configuración de Entornos y Secretos Separados

Se deben mantener tres ambientes completamente aislados:

- **Desarrollo (Local):** `docker-compose.yml` local con PostgreSQL, MinIO y catálogo simulado de Sheets.
- **Staging (pruebas):** no desplegado actualmente. La IaC describe ECS, PostgreSQL y Google Drive; verificar y desplegar explícitamente esa infraestructura antes de considerarla activa.

- **Producción:** Infraestructura productiva con políticas de respaldo continuo y hoja principal de clientes y salidas.

Cada entorno cuenta con sus propias claves criptográficas asimétricas para JWT (`JWT_ACCESS_PRIVATE_KEY` , `JWT_REFRESH_PRIVATE_KEY`), credenciales de base de datos y configuración de la cuenta de servicio y el Drive compartido que utilizará.

3. Topología objetivo de producción

Esta sección describe el objetivo general para producción. La arquitectura de staging y sus diferencias con el despliegue activo se documenta en la sección 5 y en Infraestructura y operación.

- **Frontend & API:** Despliegue bajo el mismo dominio público o mediante reverse proxy (ej. Nginx, Caddy o Cloudflare) mapeando `/api` directamente a la instancia de Express y `/` al frontend estático SPA/PWA. Esto elimina complejidades de cookies de terceros y simplifica CORS.
- **Base de Datos:** PostgreSQL gestionado (ej. Neon, Supabase o AWS RDS PostgreSQL) con migración ejecutada mediante `infra/scripts/migrate.sh` antes de conmutar tráfico.
- **Archivos de aplicación:** Google Drive compartido privado para evidencias, firmas y reportes. S3 permanece como adaptador alternativo en el código; el último inventario histórico observó S3 en staging y Drive es el proveedor seleccionado por la IaC local, pendiente de validación antes de redeploy.

4. Procedimiento de Migración y Despliegue

Staging mediante CDK

El despliegue automatizado de staging se ejecuta desde `.github/workflows/deploy-staging.yml` después de un CI exitoso sobre `main` , únicamente cuando `AWS_STAGING_READY` vale exactamente `true` . El workflow hace checkout del `head_sha` validado, ejecuta `pnpm infra:deploy` y luego el smoke test. Al 2026-10-09 staging está eliminado/no desplegado; el workflow no implica que haya recursos activos.

CDK construye las imágenes Docker. Las imágenes de API y migración generan el cliente Prisma durante el build con un `DATABASE_URL` ficticio de compilación; esa generación no conecta a la base de datos de producción ni ejecuta migraciones. Al iniciar una nueva tarea ECS, el contenedor `migration` ejecuta `prisma migrate deploy` y después `prisma/seed.ts` . La API depende de que ese contenedor termine con éxito (`SUCCESS`), y web espera la salud de la API.

`prisma migrate deploy` aplica las migraciones pendientes que estén versionadas en `prisma/migrations/` ; editar únicamente `prisma/schema.prisma` no cambia la base de datos. Crear y revisar la migración, incluirla en el commit y desplegarla junto con la versión de la aplicación. El contenedor de migración se ejecuta al iniciar una tarea nueva; si no hay migraciones pendientes, un despliegue de sólo código no modifica el esquema.

Las migraciones deben ser compatibles con la versión anterior de la aplicación mientras ECS reemplaza tareas. Para cambios incompatibles, usar una secuencia expand/contract en

despliegues separados. Si la migración falla, la API no inicia; el rollback de ECS no revierte automáticamente cambios ya aplicados en la base de datos.

Ejecución manual fuera del flujo CDK

Para un entorno que no use el contenedor de migración de CDK, el script manual es:

```
./infra/scripts/migrate.sh
```

No ejecutar este paso como si fuera un paso previo independiente del workflow CDK: en staging las migraciones y el seed corren dentro de la tarea ECS.

1. **Despliegue de versión:** actualización de las imágenes y del task definition mediante el mecanismo propio del entorno.
2. **Smoke Tests Automáticos:**

   ```
   TARGET_URL="http://<ipv4-publica-actual>" ./infra/scripts/smoke-test.sh
   ```

   El script verifica automáticamente liveness (`/health/live`), readiness (`/health/ready`), métricas (`/metrics`) y documentación OpenAPI.
3. **Procedimiento de Rollback:** En caso de fallo crítico detectado en smoke tests o métricas, se revierte el contenedor a la versión de imagen previa y se restaura el respaldo de base de datos previo mediante `infra/scripts/restore.sh` .

5. Diseño de staging e inventario histórico

La configuración CDK y el inventario histórico de esta sección no representan recursos actualmente activos. AWS staging está eliminado/no desplegado desde la verificación del 2026-10-09; producción tampoco está desplegada. No asumir que el workflow está habilitado: el valor de `AWS_STAGING_READY` no se verificó.

Último inventario histórico conocido (2026-10-05; no representa el estado actual)

Componente	Última observación histórica (2026-10-05)
Cuenta y región	`075566976700` , `us-east-2`
CloudFormation	Los stacks estaban activos en la fecha indicada; ahora eliminados
Cómputo	Un host EC2 `t3.small` , instancia `i-05dab87f12229d17c`
Acceso	`http://18.222.63.51` , IPv4 efímera en esa fecha
ECS	Cluster `puk-staging` , servicio y tarea de API/web ejecutándose
Persistencia	RDS separado del host; secreto runtime existente en esa fecha
Archivos	Revisión histórica de tarea 14 con configuración S3

Componente	Última observación histórica (2026-10-05)
Recursos de red	Sin ALB ni NAT Gateway en el inventario regional consultado

En la consulta del 2026-10-05, el bucket `EvidenceBucket` seguía existiendo y la aplicación desplegada lo referenciaba; devolvió cero versiones y marcadores de borrado. Es sólo una observación histórica, no evidencia del estado actual ni autorización para borrar recursos. No se ha comprobado ni declarado pérdida de datos de RDS o de archivos históricos.

La IaC local describe un host público `t3.small`, tarea ECS en modo `bridge`, migración/seed antes de API y web después de su health check. No declara ALB, CloudFront, NAT Gateway ni Elastic IP. RDS queda privado; los detalles están en Infraestructura y operación.

Los valores de acceso de los inventarios históricos son sólo referencias de pruebas y no implican que exista un endpoint activo. No ingresar datos reales ni información de producción en un futuro staging sin TLS.

Configuración de Sheets, Drive y Gmail para un futuro despliegue

Para un futuro despliegue, los valores de Google se almacenarían en AWS Secrets Manager; el secreto de staging anterior ya no está disponible. No se almacenan como variables de GitHub Actions ni en el repositorio. La configuración local de referencia está en `.env.example`; no contiene valores reales.

Campo del secreto runtime	Uso / origen
`GOOGLE_SPREADSHEET_ID`	Documento de pruebas para clientes y equipos
`GOOGLE_TECHNICIANS_SPREADSHEET_ID`	Documento de pruebas para directorio y reportes técnicos
`GOOGLE_CLIENT_EMAIL`	`client_email` del JSON de la cuenta de servicio de Sheets
`GOOGLE_PRIVATE_KEY`	`private_key` del mismo JSON
`GOOGLE_DRIVE_SHARED_DRIVE_ID`	ID de la raíz de la unidad compartida de pruebas
`GOOGLE_DRIVE_CLIENT_EMAIL`	`client_email` del JSON de la cuenta de servicio de Drive
`GOOGLE_DRIVE_PRIVATE_KEY`	`private_key` del mismo JSON de Drive
`GOOGLE_GMAIL_CLIENT_EMAIL`	`client_email` de la cuenta de servicio con delegación Gmail
`GOOGLE_GMAIL_PRIVATE_KEY`	`private_key` del mismo JSON de la cuenta Gmail
`GOOGLE_GMAIL_SENDER_EMAIL`	Buzón Workspace suplantado y encabezado `From`

Gmail no está conectado al task definition de CDK. `ApplicationStack` inyecta credenciales de Sheets y Drive, pero no `GOOGLE_GMAIL_*`; configurar esos valores en Secrets Manager no basta para habilitar envíos en AWS. Se requiere primero mapearlos en IaC/tarea y desplegar. En ejecución local, el envío manual funciona con las tres variables, delegación de dominio autorizada y API reiniciada al cambiar el remitente. El superadministrador de Workspace debe autorizar el scope `https://www.googleapis.com/auth/gmail.send`. La identidad de Gmail separada

es recomendada, pero la aplicación permite reutilizar la identidad de Sheets/Drive. El envío es manual; no se envía al completar órdenes automáticamente. Ver entrega de reportes por correo.

Para un documento de Sheets con URL `https://docs.google.com/spreadsheets/d/<ID>/edit`, usar el segmento `<ID>`, no el `gid` de una pestaña. Véase operaciones de Sheets. Para Drive, abrir **Unidades compartidas**, entrar a la **raíz** de la unidad de pruebas y tomar el ID de `https://drive.google.com/drive/folders/<ID>`. No usar el ID de una subcarpeta ni una carpeta compartida de **Mi unidad**; confirmar que el ID corresponda a la unidad mediante Drive API antes de desplegar. Obtener el ID desde la raíz.

Las pestañas de clientes/equipos están configuradas en `infra/cdk/lib/application-stack.ts` como `Clientes` y `SALIDAS`. La API usa por defecto `DIRECTORIO TECNICO` y `2026` para técnicos y reportes; sus variables son `GOOGLE_TECHNICIANS_SHEET_NAME` y `GOOGLE_TECHNICIAN_REPORT_SHEET_NAME`. Cambiar nombres de pestañas requiere ajustar la configuración del task definition y desplegar, no sólo cambiar un ID de documento.

Declarar una nueva cuenta de servicio de pruebas

1. Seleccionar un proyecto de Google Cloud dedicado a pruebas y habilitar **Google Sheets API** y **Google Drive API** en **APIs y servicios** → **Biblioteca**. Si se usan proyectos separados, habilitar la API correspondiente en cada uno. Habilitar APIs.
2. En **IAM y administración** → **Cuentas de servicio** → **Crear cuenta de servicio**, crear la identidad de pruebas. Se pueden usar identidades separadas para Sheets y Drive; no reutilizarlas de forma automática. Los permisos IAM del proyecto no dan acceso a documentos de Workspace y no se necesita delegación de todo el dominio para estos archivos compartidos explícitamente. Crear credenciales.
3. Abrir la cuenta → **Claves** → **Agregar clave** → **Crear nueva clave** → **JSON**. Descargar el archivo fuera del repositorio y protegerlo con permisos restrictivos. No pegar su contenido en chat, logs, tickets ni commits. Si la organización impide crear claves, solicitar el mecanismo aprobado; no desactivar esa política para continuar. Crear claves JSON.
4. Crear o copiar documentos **exclusivos de pruebas**, conservar sus pestañas y encabezados y eliminar datos reales. Compartir ambos Sheets directamente con el correo de su cuenta de servicio como **Editor**, porque la integración también escribe asignaciones y reportes. Compartir con cuentas de servicio.
5. Crear una **unidad compartida real de Google Workspace**, dedicada a pruebas; en **Administrar miembros**, añadir el correo de la cuenta de Drive como **Colaborador / Contributor** (`writer`), suficiente para las operaciones actuales de lectura y creación. No conceder Administrador por defecto; ampliar permisos sólo si una operación futura lo requiere. Las políticas de Workspace deben permitir esa membresía. Una carpeta compartida de Mi unidad no sustituye una unidad compartida: las cuentas de servicio no tienen cuota ni pueden ser propietarias de archivos. Unidades compartidas, roles de Drive.
6. Mediante la consola de Secrets Manager, abrir **Recuperar valor del secreto** → **Editar** y agregar o modificar **únicamente** los campos de Google de la tabla. Mantener intactos `JWT_*`, `SEED_*` y cualquier otro campo existente. Guardar sin copiar ni imprimir el JSON completo. Esta operación humana no requiere enviar claves al agente.

> **No usar `configure-staging-secrets.mjs --overwrite` para cambiar IDs o cuentas de servicio.** Es un reemplazo completo: genera nuevas claves JWT y nuevas contraseñas de los dos administradores del seed. Puede invalidar sesiones y desalinear las contraseñas guardadas con usuarios existentes. Sin `--overwrite`, el script sólo sirve para crear un secreto que aún no existe y rechaza el actual.

Validar y sincronizar antes de limpiar

El despliegue de Drive está pendiente de configuración segura y validación de Google. No usar IDs ficticios ni recursos de producción para superar esta condición.

1. Confirmar que ambos Sheets y la unidad compartida son de pruebas. Con sus identidades de servicio, verificar lectura de las pestañas esperadas y lectura de metadatos de la unidad mediante un mecanismo seguro que no imprima claves ni contenido de documentos.

2. Ejecutar `pnpm infra:test`, `pnpm --filter @puk/infra typecheck` y `pnpm infra:synth`. Revisar `pnpm infra:diff` con acceso AWS autorizado. La eliminación de referencias S3 exige preservar los exports de Foundation que todavía consume Application: primero actualizar el consumidor, luego retirar exports o recursos de red sin uso. No desplegar Foundation primero si intenta eliminar un export aún consumido. No reemplazar RDS, VPC ni secretos por accidente.

3. Con todos los campos de Drive presentes y requisitos cumplidos, desplegar **sólo Application** desde `infra/cdk`, sin actualizar dependencias automáticamente:

   ```
   pnpm exec cdk deploy PukStagingApplication --exclusively --app 'node --import tsx bin/puk.ts'
   ```

   No empezar con `pnpm infra:deploy` / `deploy --all`: el diff actual de Foundation retira exports S3 que todavía consume Application y modifica la política del bucket. Actualizar Foundation después de liberar los imports y validar la aplicación. Guardar una nueva versión del secreto no cambia contenedores existentes: para cambios sólo de secretos, usar **Force new deployment** en el servicio del cluster ECS `puk-staging`; para el primer cambio S3 → Drive, también debe actualizarse el task definition desde IaC.

4. Ejecutar el smoke test y una prueba funcional controlada de carga/lectura de un archivo sintético en Drive y escritura de una fila de pruebas en Sheets. `/health/ready` sólo comprueba PostgreSQL; un smoke test verde **no verifica** autenticación o permisos de Google.

5. Sólo después de validar la transición, retirar permisos y recursos obsoletos que no tengan consumidores. Antes de considerar el bucket vacío como candidato a limpieza, volver a comprobar objetos, versiones y marcadores de borrado, sin paginación pendiente. Conservar datos, RDS y secretos; no vaciar almacenamiento para eliminarlo. Cambiar `STORAGE_PROVIDER` o el ID de Drive no migra archivos existentes.

El workflow `.github/workflows/deploy-staging.yml` se ejecuta después de CI exitoso sobre `main` sólo si `AWS_STAGING_READY` vale exactamente `true`. Su valor actual no se verificó ni modificó. No habilitar despliegue automático hasta resolver la configuración de Google y el cambio de exports entre stacks. GitHub Actions utiliza OIDC y el rol `puk-staging-github-deploy`.

Consultar la IP y diagnosticar

La IP del inventario es una referencia temporal. Consultar su valor actual antes de operar:

```
ASG_NAME=$(aws cloudformation describe-stacks \
  --region us-east-2 \
  --stack-name PukStagingApplication \
  --query "Stacks[0].Outputs[?OutputKey=='EcsHostAutoScalingGroupName'].OutputValue" \
  --output text)
INSTANCE_ID=$(aws autoscaling describe-auto-scaling-groups --region us-east-2 \
  --auto-scaling-group-names "$ASG_NAME" \
  --query "AutoScalingGroups[0].Instances[?LifecycleState=='InService'].InstanceId | [0]" --output text)
PUBLIC_IP=$(aws ec2 describe-instances --region us-east-2 --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text)
echo "http://$PUBLIC_IP"
```

Diagnóstico de sólo lectura con sesión y destino autorizados; no imprimir secretos ni copiar logs que contengan información sensible:

```
aws cloudformation describe-stacks --region us-east-2 --stack-name PukStagingApplication
aws ecs describe-services --region us-east-2 --cluster puk-staging --services <service-name>
aws ecs list-tasks --region us-east-2 --cluster puk-staging --service-name <service-name>
aws logs tail /puk/staging/migration --region us-east-2 --since 30m
aws logs tail /puk/staging/api --region us-east-2 --since 30m
TARGET_URL="http://<ipv4-publica-actual>" ./infra/scripts/smoke-test.sh
```

El smoke test comprueba `/health/live`, `/health/ready`, `/metrics` y `/docs/`. Si falla una migración, la API no inicia: conservar los logs y evaluar la migración antes de reintentar. El circuit breaker de ECS puede revertir la revisión de aplicación; no restaura automáticamente la base de datos. Para restauraciones, usar sólo un respaldo verificado y aprobación operativa.

Los assets de CI corresponden a `workflow_run.head_sha`. En despliegues manuales, registrar el SHA, la revisión de ECS, los cambios locales incluidos y el operador. Un solo host y despliegues stop-before-start (0 % mínimo, 100 % máximo) implican una interrupción durante cada actualización.

Costos y limpieza

RDS, EC2, EBS, la IPv4 pública y los servicios asociados pueden generar costos aunque no haya tráfico. No hay ALB ni NAT Gateway en el inventario regional consultado. No eliminar recursos por nombre o por ausencia de tráfico: comprobar stack propietario, dependencias y datos primero. `RETAIN` y protección contra borrado no sustituyen un respaldo verificado.

Documentación técnica

Proyecto: Bull Dog Solution — Sistema de órdenes de servicio

Última actualización: 2026-10-02

Alcance: estado implementado en la rama actual

Este directorio describe la arquitectura y operación técnica del sistema. A diferencia de `SPECS.md` y `docs/ERS.md`, que también contienen requisitos y decisiones de producto, estos documentos reflejan los componentes que existen actualmente en el repositorio.

Contenido

1. Arquitectura: contexto, contenedores, componentes, dependencias y decisiones estructurales.
2. Modelo de datos: diagrama entidad-relación, agregados, restricciones y estados.
3. Backend y flujos: módulos de API, workers y secuencias críticas.
4. API HTTP: convenciones, autenticación y catálogo de endpoints.
5. Frontend: estructura Vue, estado, rutas y flujo de datos.
6. Infraestructura y operación: Docker, configuración, despliegue, observabilidad y comandos.
7. Seguridad y calidad: controles implementados y estrategia de pruebas.
8. Línea de tiempo del proyecto: evolución por fases, fechas y commits.

Fuentes de verdad

Tema	Fuente principal
Persistencia	`prisma/schema.prisma` y `prisma/migrations/`
Contratos de entrada	`packages/contracts/src/`
Rutas HTTP	Routers de `apps/api/src/modules/`
Documento OpenAPI	`apps/api/src/docs/openapi.ts` y `/docs`
Composición de servicios	`apps/api/src/server.ts` y `apps/api/src/app.ts`
Rutas de interfaz	`apps/web/src/router/` y módulos web
Infraestructura local	`docker-compose.yml` e `infra/docker/`
Infraestructura AWS	`infra/cdk/` y `.github/workflows/`
Reglas funcionales	`SPECS.md`, `docs/ERS.md` y servicios de dominio

Cuando la documentación y el comportamiento diverjan, el esquema, las migraciones, los contratos y el código ejecutable tienen precedencia. La documentación debe actualizarse en el mismo cambio que modifique esas superficies.

Plan: administrar usuarios administradores

Estado: Completado y verificado localmente (2026-10-02) · **Progreso:** 6/6 checkpoints

Este plan agrega una interfaz para que los administradores gestionen otras cuentas admin sin mezclar esa tarea con **Cuentas Técnicas**. El archivo es el registro de avance: marcar cada checkpoint solo cuando sus criterios estén verificados.

La implementación está en la rama actual. Su verificación es local; no implica que exista un ambiente AWS desplegado (staging fue eliminado y producción no está desplegada).

Decisión de diseño

- Crear una vista administrativa independiente, enlazada desde la navegación existente y protegida por `users:manage`.
- Mantener **Cuentas Técnicas** enfocada en técnicos y en la designación de la cuenta compartida.
- Mostrar únicamente usuarios `ADMIN`; crear admins con el rol fijo `ADMIN`, sin selector de roles.
- Reutilizar las rutas actuales `GET/POST/PATCH /api/v1/users` si la verificación de permisos y reglas de dominio confirma que cubren los casos. No agregar un modelo ni una migración Prisma salvo que aparezca un requisito que realmente lo necesite.
- Usar una tabla compacta con búsqueda/estado y acciones por fila. Crear, editar y cambiar contraseña se resuelven en modales, no en formularios permanentes que carguen la vista.

Alcance

Incluye: listar admins; crear una cuenta admin; editar nombre/correo; restablecer contraseña; activar o desactivar; autorización de API y ruta; pruebas y documentación.

No incluye: gestión de técnicos, designación de cuenta compartida, cambios de credenciales de AWS/Google, cambios al flujo de login ni políticas de invitación o recuperación por correo.

Checkpoints

C0 · Confirmar el estado actual — completado

- [x] Confirmado que el endpoint y el servicio de usuarios ya existen; la interfaz actual filtra `TECHNICIAN` y fija ese rol al crear.
- [x] Confirmado que el panel de cuenta compartida y las cuentas técnicas deben seguir separados de la administración de admins.

- ☑ No se modificó código de la funcionalidad.

C1 · Cerrar reglas de dominio y autorización

- ☑ Confirmado que `GET/POST/PATCH /api/v1/users` exige `users:manage` en servidor; la protección visual de la ruta no sustituye esta autorización.
- ☑ Creación fija con rol `ADMIN`; cambio de contraseña pasa por la política mínima vigente de 12 caracteres.
- ☑ Se bloquea el auto-bloqueo y la desactivación/degradación del último admin activo; las actualizaciones usan transacción serializable.
- ☑ Se preservan la cuenta técnica compartida, la auditoría y la revocación de sesiones existentes.

Criterio de salida: las reglas críticas tienen pruebas, y una llamada no autorizada no puede listar, crear ni modificar admins.

C2 · Agregar una entrada administrativa protegida

- ☑ Crear ruta `/admin/users`, protegida con `requiresRole: 'ADMIN'` (rol que posee `users:manage`).
- ☑ Mostrar la entrada de navegación solo a admins, en escritorio y móvil.
- ☑ Guard de ruta deniega el acceso directo a usuarios sin permiso; API también deniega listados y cambios.
- ☑ Se mantiene **Cuentas Técnicas** separada y sin cambios funcionales.

Criterio de salida: un admin llega a la vista desde navegación; un técnico es redirigido y además recibe rechazo de API si intenta llamar el endpoint.

C3 · Implementar la vista de admins sin sobrecargar otras pantallas

- ☑ Se agrega `modules/admin-users` para vista y operaciones; las operaciones genéricas se comparten desde `modules/user-management`.
- ☑ La lista se filtra a admins y ofrece búsqueda por nombre/correo y estado.
- ☑ Creación siempre fija `ADMIN`; edición, cambio de contraseña y estado están disponibles por fila.
- ☑ Formularios modales con estados de carga/error/vacío y confirmación antes de desactivar.
- ☑ Contraseñas nunca se vuelven a mostrar; campos sensibles se limpian al cerrar modal.
- ☑ Vista responsiva con navegación móvil, labels/aria-labels y tabla desplazable en pantallas angostas.

Criterio de salida: la pantalla tiene una tarea clara, no duplica la configuración de cuenta compartida y no expone acciones de técnicos.

C4 · Cubrir comportamiento y seguridad con pruebas

- [x] Pruebas del módulo verifican filtrado a `ADMIN` y rol fijo de creación; pruebas existentes validan API de cuentas técnicas.
- [x] Pruebas de router verifican denegación a técnicos; pruebas de API verifican denegación para GET/POST/PATCH.
- [x] Pruebas de servicio cubren último admin y auto-bloqueo; reglas existentes de cuenta compartida permanecen.
- [x] Pruebas de `TechnicalAccountsView` /API continúan pasando.

Criterio de salida: las pruebas enfocadas cubren las operaciones y los casos de acceso/regresión.

C5 · Documentar operación y soporte

- [x] `docs/technical/api.md` documenta permisos, semántica y códigos de error.
- [x] `MANUAL_USUARIO.md` documenta el ciclo de gestión de admins y contraseña segura.
- [x] `docs/technical/security-and-quality.md` documenta límites, auditoría y concurrencia.
- [x] No se documentan credenciales reales ni secretos.

Criterio de salida: una persona administradora puede completar el flujo siguiendo la documentación sin depender de credenciales compartidas en texto.

C6 · Verificación final y cierre

- [x] `pnpm --filter @puk/api test` y `pnpm --filter @puk/web test` aprobados.
- [x] `pnpm typecheck`, `pnpm lint` y `pnpm test` aprobados.
- [x] `git diff --check` aprobado; no se cambió el modelo ni las migraciones Prisma; suite de cuentas técnicas aprobada.
- [x] Plan marcado como completado; no hubo desviaciones de alcance.

Criterio de salida: comandos aprobados, documentación sincronizada y todos los checkpoints previos marcados.

Archivos previstos

La lista se confirma durante la implementación; no implica que estos cambios ya existan.

- `apps/web/src/modules/admin-users/` — nueva vista, componentes, API y pruebas de admins.
- `apps/web/src/router/` y navegación de `AppLayout` — ruta protegida y entrada visible por capacidad.
- `apps/api/src/modules/users/` y sus pruebas — reglas de dominio que falten; conservar endpoints actuales cuando sea posible.
- `apps/api/src/modules/administration/administration.router.test.ts` — verificar permisos/contratos existentes.

- `docs/technical/api.md` y guía de usuario existente — documentación según los cambios efectivos.

Registro de cambios del plan

- 2026-09-30 — Plan creado tras verificar que la UI gestiona cuentas técnicas, mientras el backend ya ofrece endpoints genéricos protegidos para usuarios.
- 2026-09-30 — Implementados los checkpoints C1-C6: controles para evitar auto-bloqueo/último admin, módulo y ruta separados, pruebas y documentación; sin migraciones Prisma.

API HTTP

Convenciones

- Base autenticada: `/api/v1`.
- Cuerpo: JSON, excepto las cargas de archivos del proveedor configurado.
- Validación: esquemas Valibot de `@puk/contracts`.
- Éxito: `{ "data": ..., "meta": { "requestId": ... } }`.
- Error: `{ "error": { "code": ..., "message": ..., "details"?: ... }, "meta": { "requestId": ... } }`.
- Fechas: ISO 8601 en UTC por HTTP; la presentación local pertenece al cliente/reporte.
- Documentación interactiva: `/docs`.

Autenticación y autorización

El access token se envía como `Authorization: Bearer <token>`. El refresh token se guarda en la cookie HTTP-only `puk_refresh`. El JWT prueba la identidad, pero cada request protegido resuelve desde PostgreSQL que el usuario siga activo y cuál es su rol vigente. Un usuario inexistente o inactivo recibe `401 SESSION_REVOKED`; si no puede consultarse la autoridad vigente, la API falla cerrada con `503 AUTHORITY_UNAVAILABLE`. Los roles son `ADMIN` y `TECHNICIAN`; además de capacidades, los servicios verifican que el técnico sea propietario de la orden.

Endpoints

Salud y observabilidad

Método	Ruta	Acceso	Uso
GET	`/health/live`	Público	proceso disponible
GET	`/health/ready`	Público	conexión a PostgreSQL
GET	`/metrics`	Infraestructura	formato Prometheus
GET	`/health/metrics`	Infraestructura	snapshot JSON

Sesión

Método	Ruta	Acceso	Uso
POST	`/api/v1/auth/login`	Público limitado	iniciar sesión
POST	`/api/v1/auth/refresh`	Cookie	rotar tokens
POST	`/api/v1/auth/logout`	Cookie	cerrar sesión actual

Método	Ruta	Acceso	Uso
POST	`/api/v1/auth/logout-all`	Autenticado	revocar todas las sesiones
GET	`/api/v1/auth/me`	Autenticado	usuario actual

Administración y catálogos

Método	Ruta	Acceso
GET/POST	`/api/v1/users`	Administrador
PATCH	`/api/v1/users/:id`	Administrador
GET/POST	`/api/v1/technicians`	Administrador
PATCH	`/api/v1/technicians/:id`	Administrador
GET	`/api/v1/technicians/selectable`	Administrador
POST	`/api/v1/technicians/sync`	Administrador
GET	`/api/v1/external/clients`	Autenticado
GET	`/api/v1/external/equipment`	Autenticado
GET	`/api/v1/external/equipment/:externalKey`	Autenticado
POST	`/api/v1/external/refresh`	Administrador

`/api/v1/users` requiere `users:manage` en cada operación. Las actualizaciones permiten editar nombre/correo, cambiar contraseña y activar/desactivar; el servicio registra auditoría y revoca sesiones al desactivar. No se permite que un administrador se desactive a sí mismo ni desactivar o degradar al último administrador activo (`ADMIN_SELF_LOCKOUT` , `LAST_ACTIVE_ADMIN`). La cuenta técnica compartida conserva sus restricciones de rol y designación.

Método	Ruta	Acceso / propósito
GET	`/api/v1/shared-technician-access`	Consultar configuración; administrador con `users:manage`
PUT	`/api/v1/shared-technician-access/account`	Designar o retirar cuenta TECHNICIAN compartida; `users:manage`
PUT	`/api/v1/shared-technician-access/order`	Habilitar o retirar una orden ejecutable; `orders:manage`
GET	`/api/v1/me/order-access`	Consultar modo individual/compartido de la sesión autenticada

El acceso compartido habilita como máximo una orden a la cuenta de taller designada; no cambia el técnico asignado a la orden. Las modificaciones requieren `expectedRevision` para evitar actualizaciones concurrentes obsoletas y se auditan.

Órdenes, unidades y equipos

`POST /api/v1/orders` requiere `technicianId` con UUID de un técnico activo. `PATCH` permite sustituirlo por otro técnico activo, pero no enviarlo como `null`. `GET /api/v1/technicians/selectable` sincroniza periódicamente `DIRECTORIO TECNICO`, devuelve técnicos activos con UUID, clave externa, nombre, taller, ciudad, estado y teléfono, y acepta `?fresh=true` para forzar una recarga. Si Sheets falla, utiliza el último directorio local disponible. `POST /api/v1/technicians/sync` permite la sincronización administrativa.

Las altas de unidades y equipos se permiten en cualquier orden `DRAFT`. Cada unidad puede declarar su propio `serviceType`, por lo que una orden puede combinar instalación, revisión, desinstalación, sensor fuel y cambio de equipo. `FAILED_APPOINTMENT` es un resultado de ejecución y no un bloque configurable en el asistente. El listado acepta los seis valores del dominio como filtro.

Método	Ruta	Acceso / propósito
GET/POST	`/api/v1/orders`	listar propias/todas; crear como administrador
GET/PATCH	`/api/v1/orders/:id`	detalle administrativo explícito; editar
GET	`/api/v1/orders/:id/execution`	detalle operativo mínimo para ejecutar
POST	`/api/v1/orders/:id/assign`	asignar y encolar sincronización inicial
POST	`/api/v1/orders/:id/cancel`	cancelar
POST	`/api/v1/orders/:id/reschedule`	reprogramar una cita fallida como nuevo borrador
GET	`/api/v1/orders/:id/audit`	auditoría administrativa
POST	`/api/v1/orders/:id/units`	agregar unidad
PATCH/DELETE	`/api/v1/orders/:id/units/:unitId`	modificar/eliminar unidad
POST	`/api/v1/orders/:id/units/:unitId/equipment`	asignar equipo
GET	`/api/v1/orders/:id/sync`	estado de sincronización
POST	`/api/v1/orders/:id/sync/retry`	reintento administrativo
PATCH	`/api/v1/orders/:id/pricing`	configurar o ajustar tarifas pactadas (admin)

`PATCH /api/v1/orders/:id/pricing` requiere permiso `orders:manage`. Admite `rates` (tarifas por servicio), `travelExpenses` (viáticos) y `reason` opcional. En órdenes no concluidas (`DRAFT`, `ASSIGNED`, `IN_PROGRESS`) almacena `pricingOverrides` y audita `ORDER_PRICING_CONFIGURED`. En órdenes finalizadas (`COMPLETED`, `FAILED_APPOINTMENT`) recalcula importes y desglose, incrementa `pricingVersion`, reemplaza la operación pendiente del reporte técnico en el outbox (`SUPERSEDED`), encola la nueva sincronización hacia Google Sheets y audita `ORDER_PRICING_ADJUSTED`.

Ejecución técnica

`GET /api/v1/orders/:id/execution` es el único detalle consumido por la vista técnica. Su respuesta omite datos fiscales y de contacto del técnico, tarifas, creador, auditoría y operaciones de sincronización. `GET /api/v1/orders/:id` queda reservado al administrador y también usa una proyección de respuesta explícita.

Método	Ruta	Propósito
GET	`/api/v1/orders/:id/execution`	cargar detalle operativo
POST	`/api/v1/orders/:id/start`	iniciar trabajo
POST	`/api/v1/orders/:id/units/:unitId/equipment/:equipmentId/verify`	verificar equipo
POST	`/api/v1/orders/:id/units/:unitId/equipment/:equipmentId/discrepancy`	reportar diferencia
PUT	`/api/v1/orders/:id/units/:unitId/checklist`	guardar checklist
POST	`/api/v1/orders/:id/units/:unitId/evidence/file`	subir evidencia (Drive)
POST	`/api/v1/orders/:id/units/:unitId/evidence/presign`	preparar carga (objetos)
POST	`/api/v1/evidence/:id/complete`	confirmar foto
DELETE	`/api/v1/evidence/:id`	eliminar lógicamente foto
GET	`/api/v1/evidence/:id/url`	URL temporal de foto
PUT	`/api/v1/orders/:id/signature/file`	subir firma PNG (Drive)
POST	`/api/v1/orders/:id/signature/presign`	preparar carga (objetos)
POST	`/api/v1/orders/:id/signature/complete?objectKey=...`	confirmar carga (objetos)
GET	`/api/v1/orders/:id/signature/file`	descargar firma privada (Drive)
GET	`/api/v1/orders/:id/signature/url`	URL temporal de firma
GET	`/api/v1/orders/:id/completeness`	requisitos pendientes
PATCH	`/api/v1/orders/:id/execution-notes`	guardar nota general de ejecución
POST	`/api/v1/orders/:id/complete`	completar orden

Método	Ruta	Propósito
POST	`/api/v1/orders/:id/fail-appointment`	registrar cita fallida
POST	`/api/v1/orders/:id/require-correction`	solicitar corrección

La nota de ejecución pertenece a la orden completa, no a una unidad individual. El endpoint permite guardarla durante `IN_PROGRESS`; la vista también la guarda automáticamente antes de solicitar el cierre cuando existen cambios pendientes.

Al completar una orden se congelan el precio y su desglose por servicio, se encola nuevamente el estado final de cada equipo en `SALIDAS` y se publica el renglón del reporte técnico. Estas tareas remotas son idempotentes y no revierten el cierre si Google Sheets está temporalmente indisponible.

Reportes

Método	Ruta	Acceso / propósito
GET	`/api/v1/orders/:id/report`	metadatos de versión vigente
GET	`/api/v1/orders/:id/reports`	historial de versiones
POST	`/api/v1/orders/:id/report/generate`	generar/reintentar como administrador
GET	`/api/v1/orders/:id/report/file`	descargar PDF privado
GET	`/api/v1/orders/:id/reports/:version/file`	descargar versión privada

Monitoreo de sincronización

Los siguientes endpoints requieren rol administrador:

Método	Ruta	Propósito
GET	`/api/v1/sync`	Listar y filtrar operaciones globales
GET	`/api/v1/sync/schedule`	Consultar corte, horario y siguiente ejecución
PUT	`/api/v1/sync/schedule`	Actualizar la programación semanal
POST	`/api/v1/sync/:id/retry`	Sincronizar inmediatamente una operación

Contrato y compatibilidad

Los DTOs de escritura deben añadirse primero a `packages/contracts`. Los cambios incompatibles requieren una nueva versión de ruta o una migración coordinada de web y API. OpenAPI se genera/valida con `pnpm docs:build` y `pnpm docs:validate`; actualmente documenta la superficie principal, pero los routers son la lista exhaustiva.

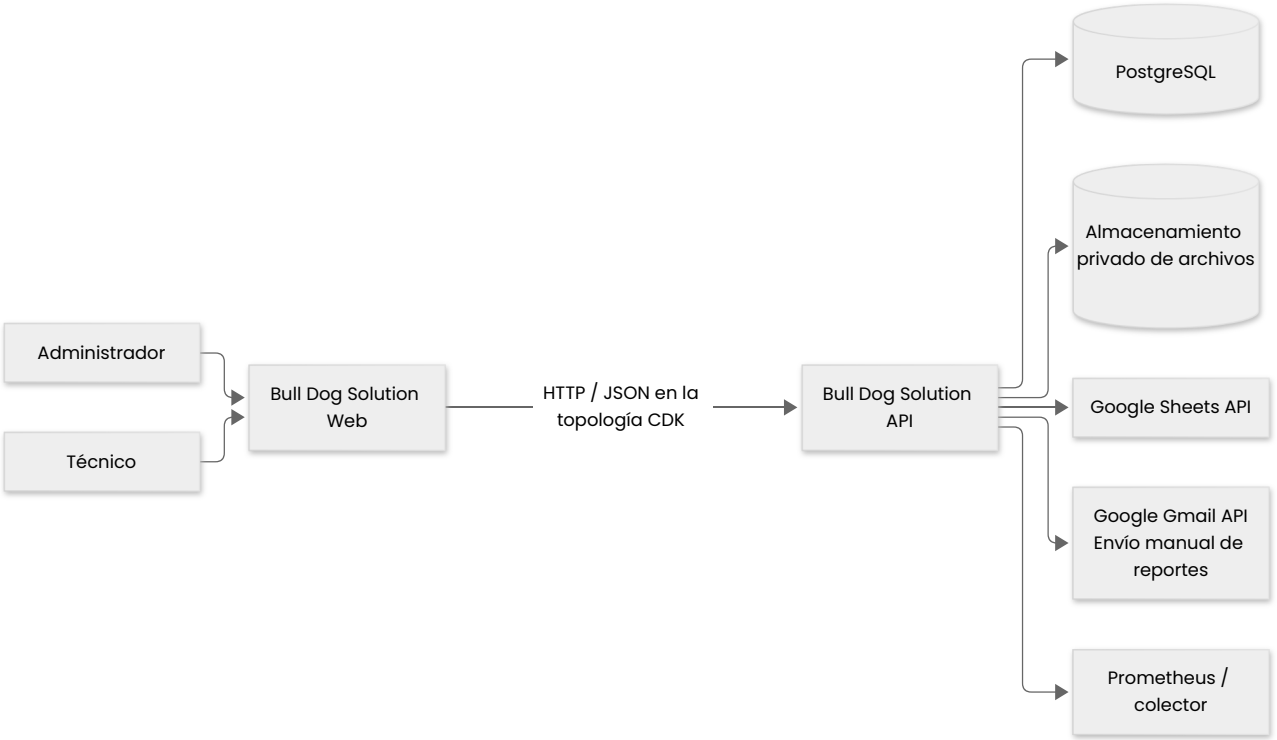
Arquitectura

Resumen

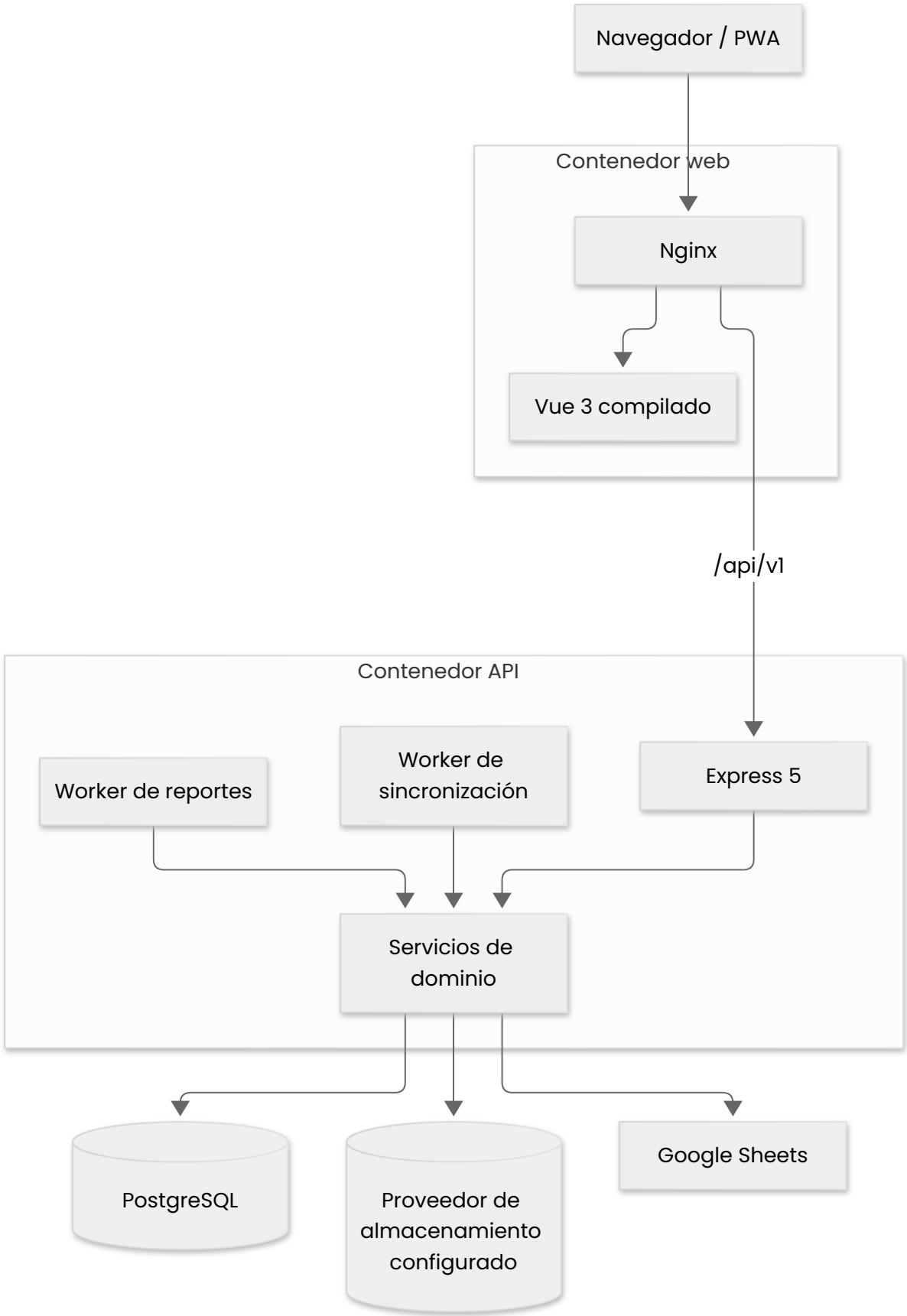
Bull Dog Solution es un monorepo TypeScript con un frontend Vue y una API Express desplegables por separado. La API implementa un monolito modular: autenticación, órdenes, ejecución técnica, catálogos, sincronización, archivos y reportes comparten proceso y base de datos, pero están separados por módulos de dominio.

PostgreSQL conserva el estado transaccional, incluyendo contactos de correo por cliente. Google Sheets sigue siendo la fuente externa de clientes y equipos. La IaC local configura Google Drive compartido para fotografías, firmas y PDFs, pero staging está eliminado/no desplegado y Drive aún requiere configuración y validación antes de volver a desplegar; el último inventario histórico observó S3. El adaptador de almacenamiento de objetos permanece en el código para compatibilidad y otros entornos. Gmail permite el envío manual del PDF cuando la API está configurada; no hay envío automático. Producción no tiene infraestructura propia y CDK todavía no inyecta credenciales Gmail.

Diagrama de contexto



Contenedores desplegables



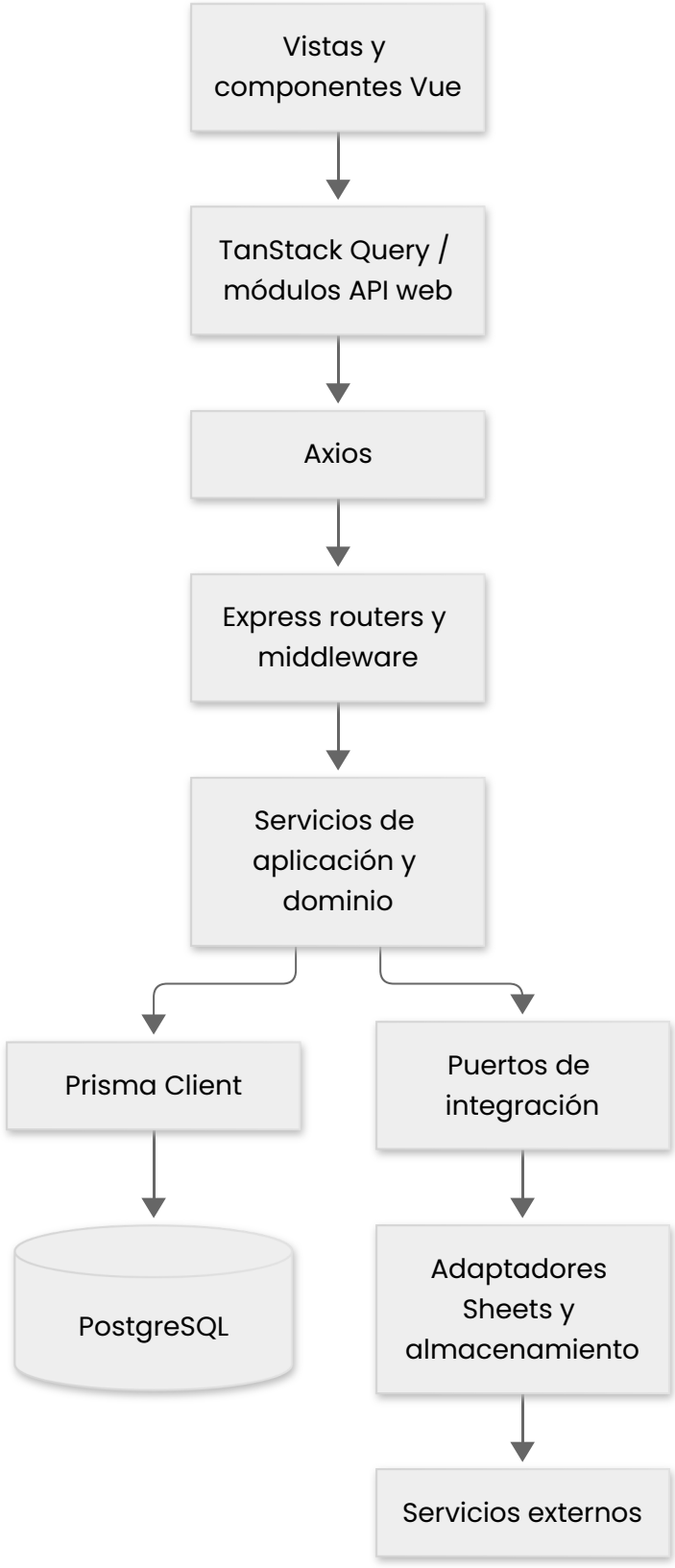
La topología CDK declarada usa ECS sobre EC2 con tareas en modo `bridge`: Nginx y Express comparten la tarea, y Nginx llega a la API mediante el enlace de contenedor `api` en el puerto 3000. El host publica HTTP por el puerto 80, sin TLS; esta infraestructura no está desplegada actualmente y no debe usarse con datos reales. Docker Compose ofrece MinIO como proveedor local de desarrollo, no como modelo del almacenamiento en staging. La topología detallada, su estado y las diferencias del proxy local están documentados en Infraestructura y operación.

Despliegue y migraciones

El workflow de staging espera un CI exitoso sobre `main`, y sólo despliega si la variable `AWS_STAGING_READY` vale exactamente `true`. En la revisión validada ejecuta `pnpm infra:deploy`; staging está eliminado/no desplegado, por lo que esta automatización no implica que haya un entorno disponible.

Durante el build Docker, las imágenes de API y migración generan Prisma Client usando una URL ficticia de compilación; la generación no se conecta a la base de datos real ni ejecuta migraciones. Cuando inicia una nueva tarea ECS, el contenedor de migración ejecuta `prisma migrate deploy` y luego el seed. La API espera que la migración termine con éxito y la web espera a que la API esté saludable. Sólo se aplican migraciones pendientes versionadas en `prisma/migrations/`: cambiar únicamente `prisma/schema.prisma` no modifica la base de datos. Un despliegue sólo de código no cambia el esquema si no hay migraciones pendientes. Las migraciones deben ser compatibles con la versión previa de la aplicación durante el reemplazo de tareas; un rollback de ECS no revierte los cambios ya aplicados a la base.

Capas lógicas



Responsabilidades

- El frontend coordina navegación, formularios, cámara, firma, estados de carga y consumo HTTP.

- Los routers autentican, autorizan, validan y traducen HTTP; no deben contener reglas de negocio.
- Los servicios validan estados, propiedad, concurrencia y efectos del dominio.
- Prisma implementa persistencia y transacciones.
- Los puertos aíslan SDKs y siguen segregación de interfaces (ISP): `SheetsPort`, `ObjectStoragePort`, `EquipmentCatalogPort`, `ClientCatalogPort`, `CatalogServicePort` y `ReportGeneratorPort`.
- La resiliencia de integración aplica el patrón Decorator con `CircuitBreaker` configurable y telemetría hacia Prometheus.
- `packages/contracts` comparte esquemas Valibot y tipos de entrada entre API y web.

Estructura del monorepo

```
apps/
├── api/              API, workers, integraciones y OpenAPI
└── web/              SPA/PWA Vue
packages/
├── contracts/        DTOs y validaciones compartidas
├── eslint-config/    reglas de lint
└── typescript-config/  configuraciones TS compartidas
prisma/               esquema, migraciones y seed
infra/docker/         imágenes de API/web y Nginx
infra/cdk/            stacks de staging en AWS
infra/scripts/        migración, respaldo, restore, smoke y E2E
.github/workflows/    CI y despliegue continuo de staging
docs/                 requisitos, decisiones y documentación técnica
```

Decisiones arquitectónicas vigentes

- Monolito modular antes que microservicios.
- Catálogos externos con snapshots locales dentro de la orden.
- Escrituras a Sheets mediante outbox persistente y reintentos.
- Almacenamiento privado de evidencias, firmas y reportes detrás de adaptadores de proveedor.
- Reportes PDF generados en segundo plano y versionados mediante abstracción `ReportGeneratorPort`.
- Envío manual por Gmail separado de la generación del PDF; destinatarios provienen de `ClientEmail`, y la identidad del remitente se configura en la API.
- Ciclo de vida y transiciones de orden gobernados por una máquina de estados de dominio explícita (`order-state-machine`).
- Validación de cierre y completitud técnica desacoplada mediante patrón de especificación pura (`order-completeness`).

- Tolerancia a fallos en adaptadores externos (Sheets y almacenamiento) protegida por Circuit Breaker con estados CLOSED, OPEN y HALF-OPEN.
- JWT de acceso en memoria del navegador; refresh token en cookie HTTP-only.
- TanStack Query para estado remoto; Pinia se limita a sesión y estado global de UI.
- Mocks de Sheets, almacenamiento y claves JWT únicamente en desarrollo con valores `development-only`.
- Staging descrito en dos stacks CDK: foundation para estado/red y application para cómputo/entrega. Los stacks no están desplegados actualmente; verificar estado y diff antes de operar.

Estado implementado al 2026-10-12

- El acceso técnico compartido se modela como configuración versionada y auditable; las políticas distinguen ejecución individual de tablero multiorden.
- Cada request protegido resuelve el principal vigente desde PostgreSQL. El JWT identifica la sesión, pero no es la fuente final de actividad ni de rol.
- Las respuestas de detalle de órdenes usan read models explícitos: el detalle administrativo y el detalle operativo tienen selects, mapeadores y contratos separados.
- La web reconcilia la identidad con `/auth/me` al navegar, recuperar foco y actualizar el tablero; las APIs de cuentas técnicas viven fuera del módulo `orders`.
- Las consultas de órdenes, la reprogramación de citas fallidas, la finalización y los medios de ejecución, el almacenamiento de reportes y varios flujos de la web tienen límites dedicados; sus servicios/vistas públicas conservan fachadas y contratos compatibles. El alcance y la deuda residual se detallan en la auditoría de mantenibilidad.
- Evidencias, firmas y reportes usan el proveedor configurado; Google Drive está integrado y el adaptador de objetos se conserva para entornos que lo seleccionen. Drive está configurado en la IaC local, pero falta validar sus credenciales y recursos antes de un redeploy. Cada proveedor mantiene sus protocolos de permisos, reintento y confirmación.
- El envío manual de reportes utiliza Gmail con delegación de dominio y las variables `GOOGLE_GMAIL_*`; `GOOGLE_GMAIL_SENDER_EMAIL` es el usuario suplantado y el encabezado `From`. CDK aún no pasa estas variables a la tarea, por lo que esta integración no está disponible en AWS hasta conectar el secret mapping.
- Las tarifas de órdenes admiten personalización acordada previa y ajuste contable post-cierre mediante `OrderPricingService` y cálculo puro en `pricing.ts`, con precisión arbitraria, auditoría, versiones contables y publicación multilínea en la columna de costo del reporte técnico de Sheets.

La secuencia histórica de implementación y sus commits está en la línea de tiempo del proyecto.

Backend y flujos

Módulos de la API

Módulo	Responsabilidad
`auth`	login, refresh, revocación y consulta de sesión
`administration`	gestión administrativa unificada de usuarios y técnicos (con DI)
`shared-technician-access`	configuración auditable del acceso compartido y consulta del modo de acceso
`catalogs`	consulta y caché de clientes/equipos de Sheets
`orders`	borradores, unidades, equipos, asignación, cancelación y outbox
`execution`	inicio, verificación, discrepancias, checklist, evidencias, firma y cierre
`reports`	generación, versionado y descarga privada del PDF
`health`	liveness y readiness
`observability`	logs estructurados y métricas

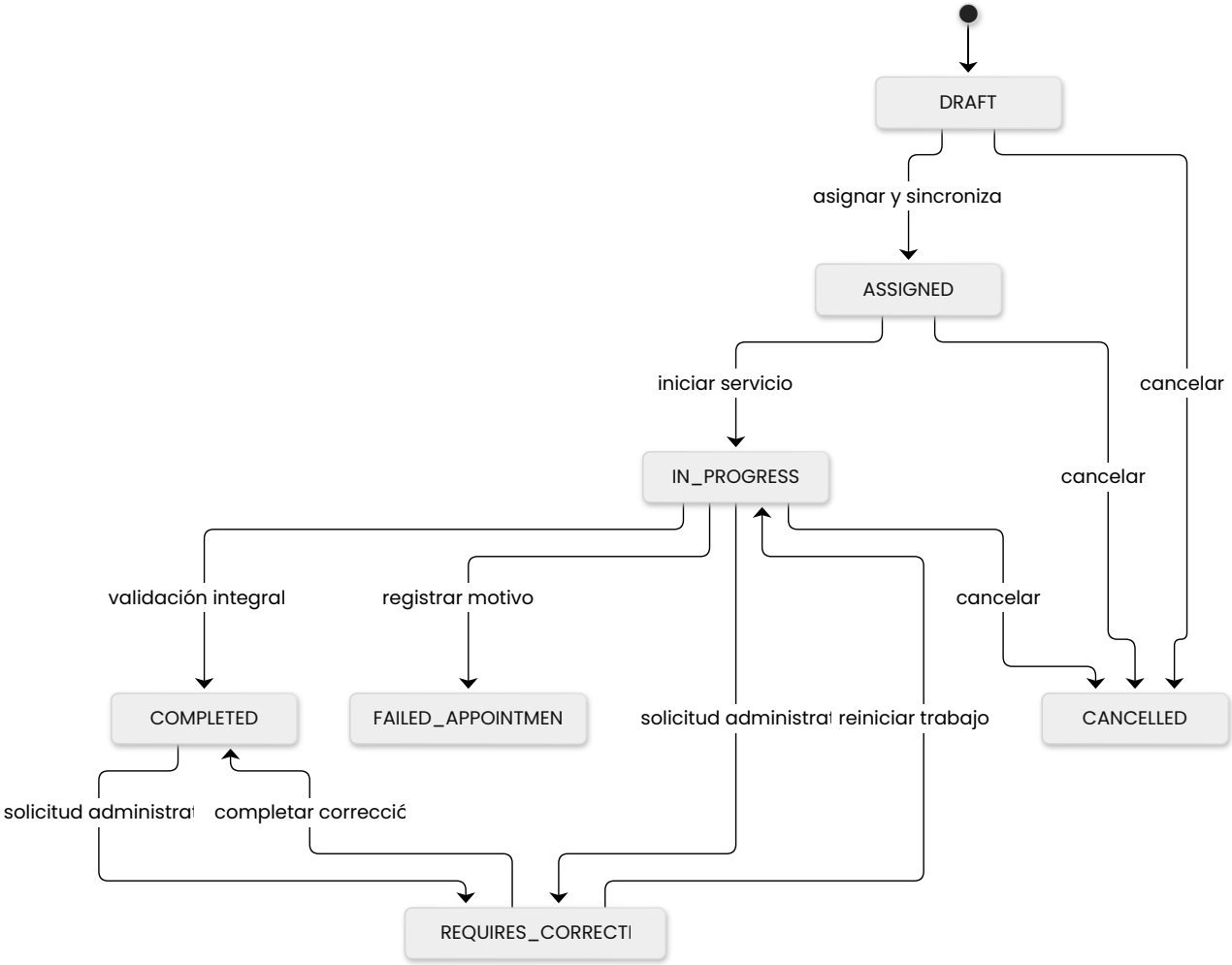
Dentro de `orders`, `reschedule-failed-appointment.ts` concentra la resolución de una cita fallida y la clonación de la orden dentro de una sola transacción de dominio; `OrdersService` conserva la API pública del módulo. En `execution`, `OrderFinalizationService` concentra validación de completitud, cierre, cita fallida y corrección, mientras `ExecutionService` conserva la fachada pública y los flujos de evidencias y firma. Ambos servicios comparten la carga de la orden con autorización mediante `execution-order-access.ts`.

Acceso a órdenes técnicas

La API aplica `assertOrderAccess` en operaciones de lectura y ejecución. El modo individual limita al técnico a sus propias órdenes. La cuenta TECHNICIAN compartida designada recibe acceso sólo a la orden explícitamente habilitada en `SharedTechnicianAccess`; habilitarla no cambia `Order.technicianId` y no abre el resto del catálogo. La configuración permite una sola orden, valida que sea ejecutable, usa `revision` para control optimista y registra cambios/revocaciones en auditoría.

Ciclo de una orden

El ciclo de vida está formalizado en una máquina de estados determinista (`order-state-machine.ts`), que define el grafo de transiciones permitidas y valida invariantes de dominio (`assertValidOrderTransition`, `assertCanEditOrder`, `assertCanModifyUnits`, `assertCanExecuteWork`).



Validación de completitud (Specification Pattern)

Para completar normalmente una orden se evalúa la regla de dominio pura mediante `evaluateOrderCompleteness`:

1. Al menos una unidad registrada.
2. Todos los equipos verificados (`MATCHED` o `DISCREPANCY_REPORTED`). Una discrepancia documentada no bloquea el cierre.
3. Checklist completo con las 5 categorías obligatorias (`DASHBOARD`, `WINDOWS`, `ACCESSORIES`, `TRIMS`, `BODYWORK`).
4. Mínimo 10 fotografías de evidencia completadas por unidad. Se almacenan como galería de unidad y no están ligadas a categorías del checklist.
5. Firma digital y nombre de conformidad del cliente.

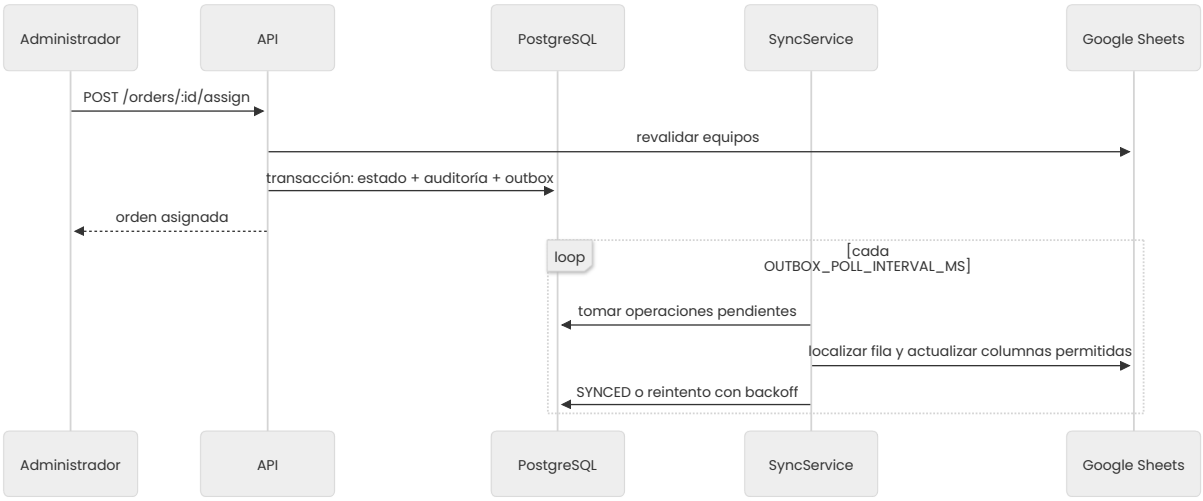
La cita fallida omite checklist, fotos y firma.

Notas de ejecución

Las notas capturadas durante el trabajo se almacenan en `Order.generalNotes` como una nota general de la orden. `PATCH /orders/:id/execution-notes` permite guardarlas mientras la orden está en `IN_PROGRESS`, genera la auditoría `EXECUTION_NOTES_UPDATED` y acepta hasta 5000

caracteres. Si la vista tiene cambios pendientes al solicitar el cierre, los guarda antes de ejecutar `POST /orders/:id/complete`. No existe todavía una nota independiente por unidad.

Asignación y sincronización con Sheets



Las operaciones fallidas almacenan intentos, código/mensaje y siguiente fecha. El administrador puede devolverlas a `PENDING`. Google Sheets no se consulta para reconstruir órdenes cerradas: los equipos conservan snapshots.

Al completar una orden, el backend vuelve a encolar el estado final de cada equipo de forma inmediata. El reporte técnico queda programado en el outbox para la ventana semanal configurada. Por defecto, el corte es el martes al final del día y la publicación ocurre el miércoles a las 17:00 en `America/Hermosillo`. Una orden cerrada después del corte pasa a la ventana de la semana siguiente. El administrador puede modificar corte, día, hora y zona horaria desde el tablero de sincronización; las operaciones todavía no intentadas se recalculan al guardar.

La sincronización final de equipos es idempotente y corrige una asignación inicial omitida o desactualizada: instalación, revisión y sensor fuel conservan cliente y vehículo con estado `INSTALADO`; desinstalación devuelve el equipo a `STOCK TECNICO` y limpia cliente y vehículo; y cambio de equipo hace lo mismo con el anterior antes de instalar el nuevo. Una cita fallida no modifica el inventario.

Los fallos posteriores al inicio de una ventana usan el backoff normal del outbox y no esperan otra semana. Una ejecución manual establece la operación como disponible inmediatamente. Cuando una corrección genera una versión nueva del reporte, las versiones pendientes anteriores se marcan como `SUPERSEDED` para no escribir dos veces el mismo folio.

Creación atómica y validación de conflictos

- `POST /orders` permite la creación atómica de la cabecera de la orden junto con sus unidades iniciales y equipos vinculados dentro de una misma transacción de dominio (`domainTransaction`).

- Antes de confirmar la asignación (`POST /orders/:id/assign`), el backend ejecuta una revalidación fresca contra Google Sheets.
- Para evitar falsos positivos de conflicto (`409 SHEET_ROW_CONFLICT`), la coincidencia de cliente entre la hoja y la orden aplica `isMatchingClient` y `normalizeString` . Esto asegura que diferencias de mayúsculas/minúsculas, acentos, espacios múltiples o sufijos corporativos (como `S.A. de C.V.`) no bloqueen la asignación de hardware que legítimamente pertenece al cliente. Solo se emite conflicto si el equipo en Sheets está asignado a una empresa completamente ajena.

Tarifas dinámicas y liquidación de órdenes

El cálculo de liquidación y costos de las órdenes se gestiona mediante componentes desacoplados de alta precisión:

Motor de cálculo financiero (`pricing.ts`)

- Utiliza aritmética de precisión arbitraria (`Prisma.Decimal` / `decimal.js`) con redondeo `ROUND_HALF_UP` a dos decimales.
- Determina el costo base por unidad multiplicando la tarifa del servicio por la cantidad de equipos (o tarifa única en reemplazo de equipos).
- Aplica viáticos / costo de salida según la configuración de la orden o tarifas del técnico.
- Genera el desglose estructurado `pricingBreakdown` congelado al completar la orden.

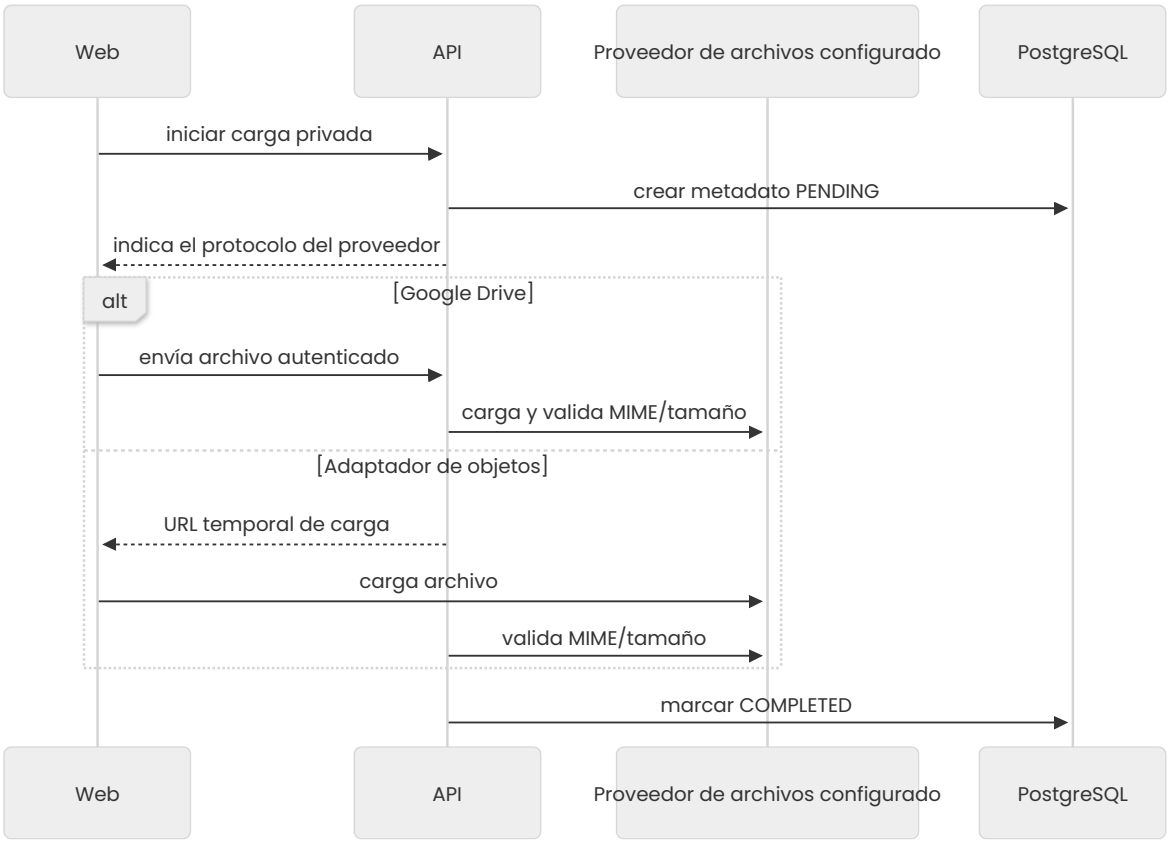
Servicio de ajuste: `OrderPricingService`

- Permite la sobreescritura de tarifas acordadas (`pricingOverrides`) por administradores (`orders:manage`).
- **Órdenes no finalizadas y no canceladas:** guarda los overrides para ser tomados durante el cierre y emite auditoría `ORDER_PRICING_CONFIGURED` .
- **Órdenes finalizadas (`COMPLETED` , `FAILED_APPOINTMENT`):** ejecuta recálculo financiero completo, incrementa `pricingVersion` , actualiza snapshots de precios y encola la re-sincronización hacia Google Sheets invalidando versiones previas (`SUPERSEDED`), registrando auditoría `ORDER_PRICING_ADJUSTED` .

Formato en Reporte Técnico de Google Sheets (`technical-report-formatter.ts`)

- Para órdenes multiservicio o con múltiples unidades (excluyendo citas fallidas), las columnas F (`economico`), G (`serie`), H (`imeild`), K (`servicio`) y L (`costo`) se sincronizan renglón por renglón usando saltos de línea (`\n`).
- La columna L (`costo`) desglosa el costo correspondiente a cada unidad en estricto orden cronológico alineado con el servicio, preservando la coherencia visual y contable en la hoja de cálculo.

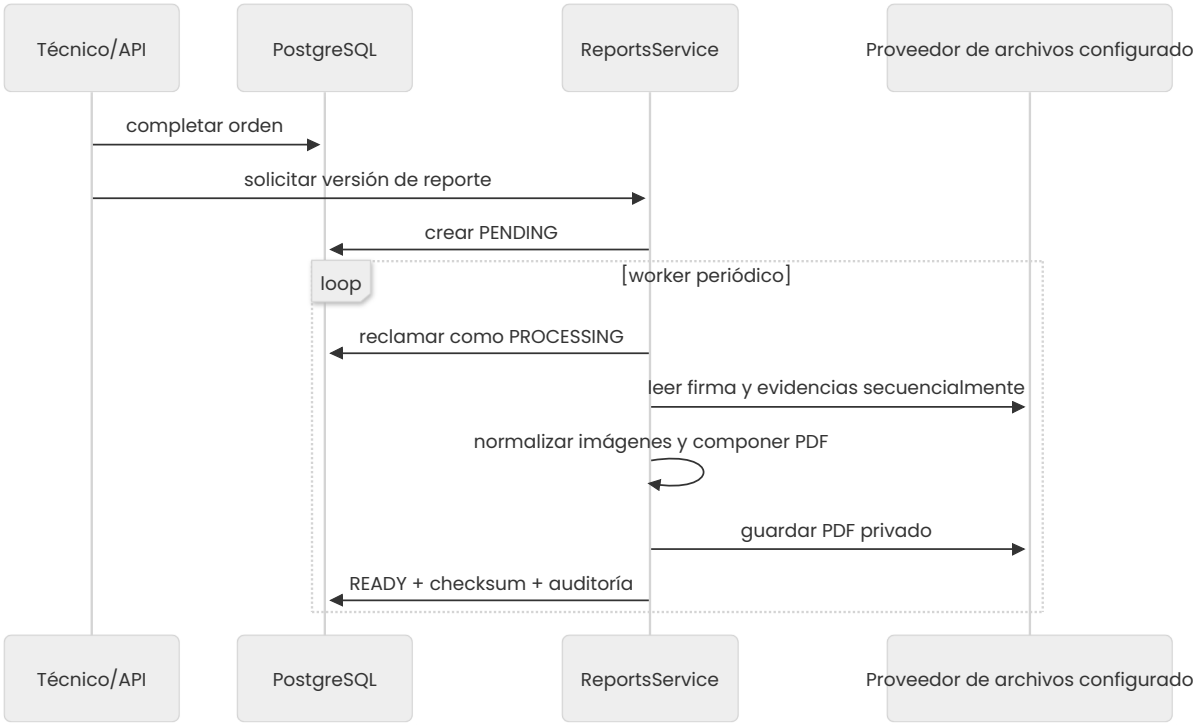
Carga de fotografías y firma



Las referencias siguen una identidad estable por evidencia, firma o reporte; el proveedor configurado resuelve su ubicación física. La base nunca almacena bytes ni Base64. En el modo Drive, la API recibe y transmite las cargas autenticadas; el adaptador de objetos conserva su propio protocolo.

En el estado actual (2026-10-02), Drive compartido es el proveedor configurado por la IaC de staging, pero AWS staging está eliminado y no existe producción desplegada. El adaptador S3/MinIO se mantiene para desarrollo local/entornos que lo seleccionen; sus rutas no son el flujo de Drive.

Generación de reporte PDF



PDFKit compone páginas Carta; Sharp corrige orientación y normaliza imágenes. Una evidencia por página ocupa el área útil sin mostrar la categoría del checklist. Un proceso `PROCESSING` abandonado por más de diez minutos vuelve a `PENDING`. Los errores públicos se sanitizan y un reintento no duplica la versión vigente. `ReportsService` carga la orden y conserva la transición `READY` y su auditoría; `report-storage.ts` gestiona la reserva/reutilización de IDs en Drive o el almacenamiento del proveedor de objetos. En Drive el checksum se calcula sobre los bytes remotos cuando se reutiliza un archivo.

Transacciones y auditoría

`domainTransaction` reúne mutaciones del dominio, auditorías y registros de outbox en una transacción Prisma. Acciones como creación, asignación, cierre, captura de evidencia, generación y descarga de reporte dejan eventos identificables. Los controladores no escriben directamente salvo consultas administrativas puntuales. `ExecutionService` conserva la API pública de ejecución; delega cierre y corrección a `OrderFinalizationService`, y evidencias/firmas a `ExecutionMediaService`. Los protocolos Drive y el adaptador de objetos permanecen explícitos para preservar permisos, reservas y confirmaciones diferentes. `OrdersQueryService` concentra las lecturas de listado, detalle administrativo y técnico, y auditoría. `OrdersService` conserva métodos públicos delegados para mantener los routers y consumidores existentes; el helper de creación de equipos permanece junto a los comandos que lo usan.

Resiliencia y Circuit Breaker

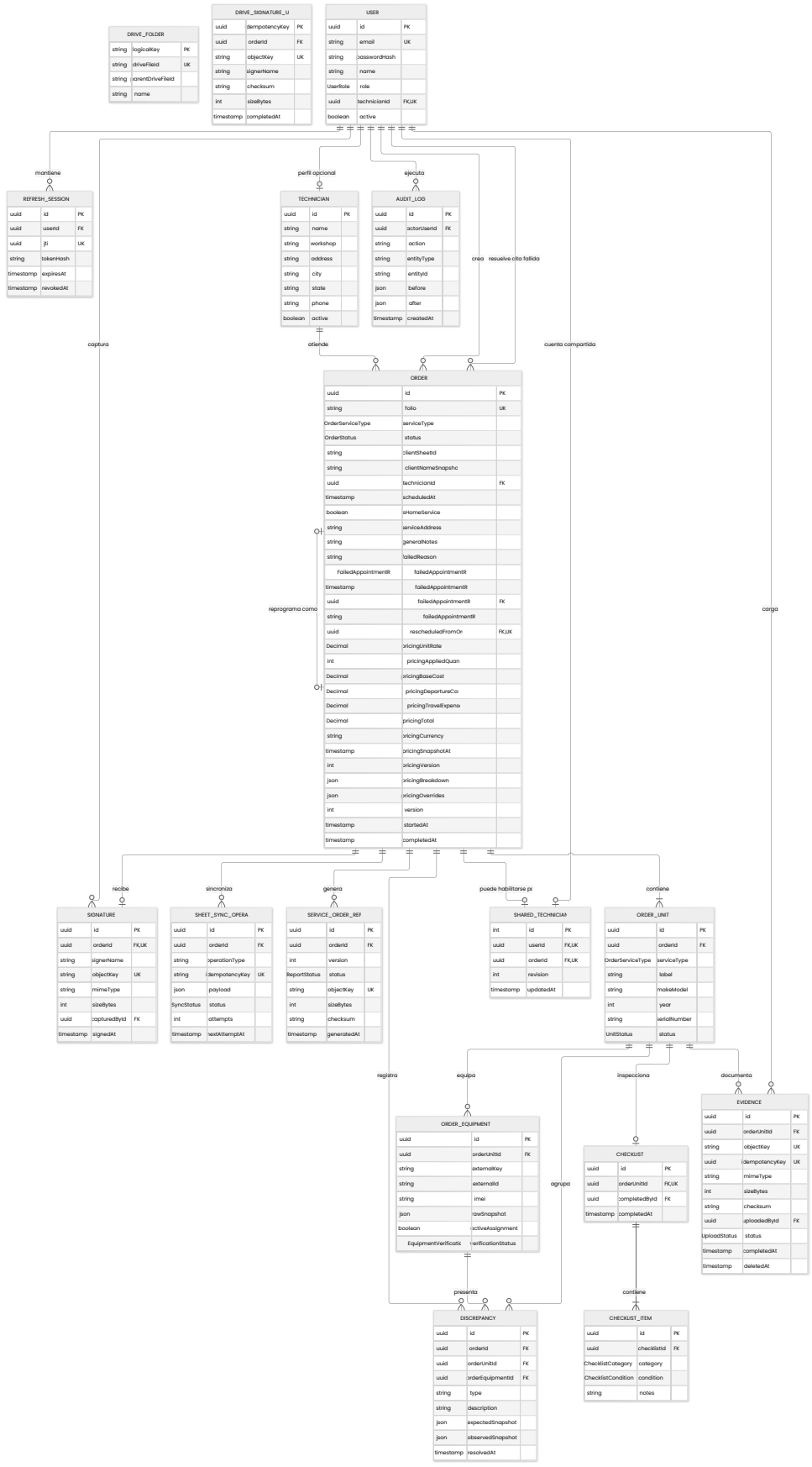
Para proteger la aplicación ante degradaciones o caídas en servicios de terceros (Google Sheets y almacenamiento), los adaptadores se envuelven mediante el patrón Decorator (`CircuitBreakerSheetsPort` y `CircuitBreakerStoragePort`):

- **Estados de circuito**: `CLOSED` (operación normal), `OPEN` (fallo rápido inmediato sin saturar la red externa) y `HALF_OPEN` (prueba controlada de recuperación).
- **Parámetros configurables**: Umbral de fallos consecutivos (`CIRCUIT_BREAKER_ERROR_THRESHOLD` , por defecto 5) y tiempo de espera antes de reintento (`CIRCUIT_BREAKER_RESET_TIMEOUT_MS` , por defecto 30,000 ms).
- **Telemetría**: Cada transición de estado actualiza la métrica Prometheus `puk_circuit_breaker_state{circuit="..."}` para observabilidad y alertas operativas.

Modelo de datos

Estado: esquema relacional documentado según `prisma/schema.prisma` (revisado 2026-10-02). Las referencias de almacenamiento son opacas y dependen del proveedor; Drive usa IDs de archivo y una tabla de mapeo de carpetas. La base conserva metadatos, no bytes.

Diagrama entidad-relación



Agregado principal

`Order` es la raíz del agregado operativo. Conserva el cliente y el técnico como snapshots, y contiene unidades. La API exige un técnico activo para crear/asignar, pero la relación puede ser nullable por compatibilidad con registros históricos. Cada unidad tiene su tipo de servicio, equipos, checklist, fotografías (`Evidence`) y discrepancias. Las evidencias pertenecen directamente a la unidad, no a los items del checklist; las cinco categorías del checklist se almacenan independientemente. La orden conserva los importes y el desglose congelados (`pricing*` , `pricingBreakdown`), así como los overrides acordados (`pricingOverrides`). El servicio administrativo puede reajustar una orden cerrada, incrementar `pricingVersion` , auditar y encolar una nueva publicación. La firma se captura una vez por orden. Sincronizaciones y reportes son efectos versionados relacionados con la orden.

`SharedTechnicianAccess` es una configuración de control de acceso singleton (ID 1), con una cuenta TECHNICIAN designada y, opcionalmente, una orden ejecutable habilitada. `revision` evita actualizaciones obsoletas; cambiar la cuenta también limpia la orden habilitada. La cuenta y la orden son relaciones opcionales para poder revocar el acceso.

Una cita fallida permanece como resultado histórico y facturable. Su resolución administrativa se registra mediante `FailedAppointmentResolution` : `CANCELLED` cierra el servicio y `RESCHEDULED` crea una nueva orden enlazada mediante `rescheduledFromOrderId` . La nueva orden comienza en `DRAFT` , copia cliente, servicio, técnico, unidades y snapshots de equipos, y reinicia todos los datos de ejecución.

Enumeraciones de estado

Entidad	Estados
Orden	`DRAFT` , `ASSIGNED` , `IN_PROGRESS` , `REQUIRES_CORRECTION` , `COMPLETED` , `FAILED_APPOINTMENT` , `CANCELLED`
Unidad	`PENDING` , `IN_PROGRESS` , `COMPLETED`
Verificación	`PENDING` , `MATCHED` , `DISCREPANCY_REPORTED`
Evidencia	`PENDING` , `COMPLETED` , `REJECTED` , `DELETED`
Sincronización	`PENDING` , `PROCESSING` , `SYNCED` , `FAILED`
Reporte	`PENDING` , `PROCESSING` , `READY` , `FAILED`
Resolución de cita fallida	`CANCELLED` , `RESCHEDULED`

Restricciones relevantes

- `Order.folio` es único.
- Un usuario solo puede vincularse con un técnico.
- El número de serie de una unidad es único dentro de su orden.

- Solo existe un checklist por unidad y una firma por orden.
- La configuración singleton de acceso compartido mantiene a lo sumo una cuenta y una orden activas; su elegibilidad y sincronización con el estado de la orden también se valida en el dominio.
- Cada categoría aparece una sola vez en cada checklist.
- Las claves de evidencia, firma y reporte son únicas en almacenamiento.
- El par `(orderId, version)` identifica una versión de reporte.
- `idempotencyKey` impide duplicar operaciones de sincronización.
- Una restricción parcial de migración impide asignar simultáneamente el mismo `externalKey` a más de una orden activa.
- `Order.version` implementa concurrencia optimista para edición administrativa.
- `Order.rescheduledFromOrderId` es único, por lo que una cita fallida solo puede originar una reprogramación.
- La base restringe las resoluciones administrativas a órdenes cuyo estado es `FAILED_APPOINTMENT`.

Eliminación y retención

Unidades, checklist, items, evidencias, firmas y reportes usan cascada al eliminar su padre. Usuarios y técnicos usan restricciones o `SET NULL` para conservar trazabilidad. La aplicación normalmente cancela órdenes o elimina lógicamente evidencias; no expone eliminación física de órdenes como operación habitual. `DriveFolder` persiste IDs de carpetas para resolver claves lógicas de forma idempotente; no representa una ruta física local ni almacena el contenido de archivos.

Riesgos y controles al usar servicios externos

Proyecto: Bull Dog Solution — Sistema de órdenes de servicio
Servicio analizado: Google Sheets
Objetivo: documentar los riesgos reales de depender de un servicio externo sin presentarlo como una mala solución por sí misma.

Estado (2026-10-02): el análisis de código incluye Google Sheets y Google Drive. Drive está integrado y seleccionado por la IaC para staging, pero ese ambiente AWS está eliminado; producción no está desplegada. Las recomendaciones sobre capacidad, retención, recuperación y observabilidad son tareas operativas, no evidencia de controles actualmente activos en un entorno desplegado.

Resumen ejecutivo

Google Sheets puede ser una solución práctica para compartir catálogos, directorios y reportes con equipos que ya trabajan allí. El problema no es usarlo, sino tratarlo como si tuviera las mismas garantías que una base de datos transaccional.

En Bull Dog Solution, Sheets debe considerarse una **dependencia externa y una superficie de intercambio**, no el único lugar donde vive el estado crítico del negocio. Si el servicio está caído, cambia su estructura o alguien edita datos manualmente, la aplicación debe poder detectar la situación, conservar la operación local y dejar la sincronización pendiente para resolverla después.

Riesgos principales

Riesgo	Qué puede pasar	Impacto	Controles recomendados
Disponibilidad, red y cuotas	Google Sheets, la red o la API pueden no responder, responder lento o limitar solicitudes.	Una consulta puede fallar y una sincronización puede quedar pendiente.	Timeout, reintentos acotados, estado `pending/failed`, reintento manual y alertas por antigüedad.
Datos desactualizados	Una hoja puede mostrar información anterior mientras otra operación todavía no se sincronizó.	Un usuario puede tomar decisiones con datos que ya cambiaron en Bull Dog Solution.	Definir la fuente de verdad, mostrar fecha/estado de última sincronización y evitar prometer tiempo real.
Ediciones manuales y cambios de estructura	Alguien puede renombrar una pestaña, mover columnas, cambiar	La integración puede rechazar la operación o, en el peor caso,	Contrato de columnas, validación de encabezados, permisos

Riesgo	Qué puede pasar	Impacto	Controles recomendados
	encabezados o pegar valores incompatibles.	interpretar mal los datos.	limitados y revisión antes de cambios.
Escrituras concurrentes	Dos usuarios o procesos pueden editar la misma fila casi al mismo tiempo.	Puede haber sobrescrituras, conflictos o resultados difíciles de reconstruir.	Identificadores estables, detección de duplicados, verificación posterior y evitar usar Sheets como mecanismo de bloqueo.
Escrituras parciales	La API puede aceptar parte de una operación y fallar antes de completarla o antes de que la aplicación reciba la respuesta.	Un reintento ingenuo puede duplicar o volver a modificar información.	Operaciones idempotentes, folios únicos, verificación posterior y conciliación de diferencias.
Credenciales y permisos	Una cuenta de servicio comprometida o demasiado amplia puede dar acceso a hojas sensibles.	Exposición o modificación no autorizada de información.	Cuenta dedicada, mínimo privilegio, secretos fuera del repositorio, rotación y auditoría de accesos.
Privacidad y gobierno de datos	La información operativa queda duplicada en un proveedor externo y puede compartirse, exportarse o conservarse más tiempo del esperado.	Pérdida de control sobre quién accede a los datos y durante cuánto tiempo.	Clasificar datos, limitar columnas, revisar compartición, definir retención y documentar responsables.
Cambios del proveedor	Google puede modificar cuotas, permisos, API o comportamiento de la plataforma.	Una integración que funcionaba puede degradarse sin un cambio en Bull Dog Solution.	Monitoreo, pruebas de contrato, dependencia encapsulada y plan de contingencia.
Soporte y diagnóstico	Algunos errores externos son genéricos o difíciles de reproducir localmente.	Aumenta el tiempo para saber si el problema está en Bull Dog Solution, la red, permisos o la hoja.	Correlation ID, logs sin secretos, métricas de latencia/error y runbook operativo.

Controles que ya existen en Bull Dog Solution

La integración actual ya reduce parte de estos riesgos:

- Usa timeout para no dejar una solicitud esperando indefinidamente.
- Registra operaciones de sincronización pendientes o fallidas y permite reintentarlas.
- Valida encabezados y estructura esperada de las hojas.
- Detecta folios duplicados antes de completar la sincronización de reportes.
- Verifica la fila después de escribirla.

- Protege columnas no administradas por la integración y comprueba que no hayan sido alteradas accidentalmente.
- Mantiene la integración detrás de un cliente específico, evitando repartir detalles de Google Sheets por todo el dominio.

Google Drive es una dependencia adicional para archivos privados. En modo Drive, la API intermedia las cargas y descargas autenticadas, lo que mantiene la autorización de la aplicación pero añade latencia y dependencia de disponibilidad/cuota de Drive. La documentación de rendimiento de Drive registra mediciones de prueba; no deben extrapolarse al tiempo total de uso ni considerarse SLA.

Estos controles **reducen la probabilidad y el impacto**, pero no convierten una API externa en una transacción distribuida. Siempre puede existir una ventana entre la escritura, la respuesta y la verificación.

Recomendaciones de operación

1. **Definir una fuente de verdad por dato.** El estado transaccional de órdenes, usuarios y permisos debe mantenerse en la persistencia de Bull Dog Solution; Sheets puede servir como catálogo, directorio o reporte según cada caso.
2. **No bloquear el negocio por una caída de Sheets.** Registrar la operación localmente y sincronizar después es más seguro que perder una orden o impedir su cierre.
3. **Monitorear la salud de la integración.** Como mínimo: última sincronización exitosa, operaciones pendientes, edad de la operación más antigua, tasa de fallos, errores de permisos y errores de cuota.
4. **Mantener un contrato de las hojas.** Documentar pestañas, encabezados, columnas administradas y columnas editables por personas.
5. **Preparar conciliación y recuperación.** Debe existir una forma de comparar Bull Dog Solution contra Sheets, corregir diferencias y saber cuándo no conviene reintentar automáticamente.
6. **Aplicar mínimo privilegio.** La cuenta usada por la API debe acceder únicamente a las hojas necesarias y sus credenciales deben rotarse sin modificar código.
7. **Definir un responsable.** El equipo debe saber quién puede cambiar la estructura, quién revisa fallos y cuándo se debe pausar la sincronización.

Cuándo dejar de usar Sheets para ese caso

No hace falta retirarlo por defecto. Conviene evaluar otra solución cuando el caso requiera varias escrituras concurrentes, auditoría inmutable, permisos por registro, consistencia inmediata, grandes volúmenes, búsquedas complejas o garantías transaccionales estrictas.

La decisión equilibrada para Bull Dog Solution es conservar Sheets donde aporta valor al flujo operativo y Drive para archivos privados, manteniendo fronteras claras: **la aplicación debe tolerar indisponibilidad, detectar inconsistencias y recuperarse sin perder el estado principal**. La disponibilidad real y los procedimientos de recuperación deben verificarse después de recrear un entorno.

Checklist antes de agregar otra integración externa

- [ ] ¿Qué dato se replica y cuál es su fuente de verdad?
- [ ] ¿Qué ocurre si el proveedor está caído durante una hora?
- [ ] ¿La operación puede reintentarse sin duplicar ni sobrescribir datos válidos?
- [ ] ¿Cómo se detectan cambios manuales o de estructura?
- [ ] ¿Qué información sensible sale de Bull Dog Solution y quién puede verla?
- [ ] ¿Cómo se monitorea, concilia y recupera un fallo?
- [ ] ¿Existe una alternativa temporal o un procedimiento manual?

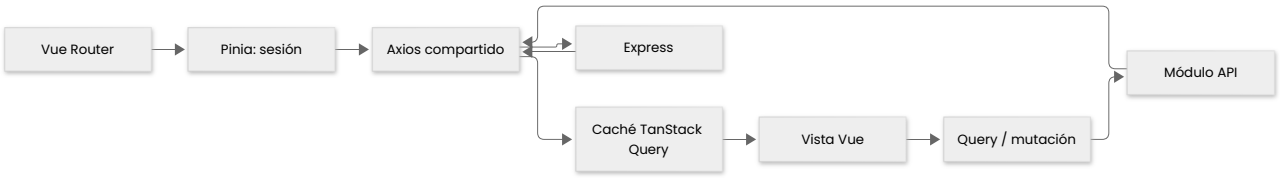
Frontend

Tecnologías y organización

La aplicación web usa Vue 3, Composition API, Vite, Vue Router, TanStack Query, Pinia, Axios, Naive UI y Tailwind CSS. Las rutas se cargan de forma diferida por módulo.

```
apps/web/src/
├── app/                tema global
├── modules/
│   ├── auth/           inicio de sesión
│   ├── home/           tablero inicial
│   ├── external-catalogs/selectores de Sheets
│   ├── orders/         lista, alta y detalle administrativo
│   ├── service-execution/flujo móvil, cámara y firma
│   ├── technicians/    directorio y cuentas técnicas
│   └── technical-accounts/ APIs de cuentas compartidas y usuarios técnicos
├── router/             composición y guardas
├── shared/             HTTP y layout
└── stores/             sesión
```

Flujo de datos



- TanStack Query mantiene datos remotos, caché, polling e invalidación.
- Pinia conserva usuario, access token en memoria e inicialización de sesión. La identidad se reconcilia con `/auth/me` mediante una promesa deduplicada y TTL de 30 segundos.
- Axios adjunta el access token y coordina un único refresh ante respuestas `401`.
- Los módulos de frontend (`orders`, `service-execution`, `technicians`) encapsulan sus peticiones en archivos `api.ts` dedicados que comparten la interfaz `ApiResponse<T>`.
- El manejo de errores de interfaz utiliza la utilidad `extractApiErrorMessage` (`@/shared/http/client.ts`) para presentar transparentemente al usuario los mensajes de error originados en el backend en lugar de códigos genéricos de Axios.
- Formularios y modales usan estado local del componente.
- Tras mutaciones se invalidan órdenes, auditoría, sincronización o reporte según corresponda.
- En el detalle administrativo, `useOrderMedia` administra el ciclo de vida de las URLs privadas y `useAddOrderUnit` encapsula el formulario de alta; `AddOrderUnitModal`, `OrderEvidenceTab`,

`OrderSyncTab`, `OrderReportCard` y `OrderAuditTab` separan la presentación de la orquestación de la vista. Asimismo, `OrderPricingModal` y `useOrderPricingModal` aíslan el formulario y validación para configurar tarifas acordadas y reajustar liquidaciones técnicas en formato tabular. La vista conserva las consultas y mutaciones de TanStack Query.

- `OrderUnitsTab` presenta unidades y equipos con propiedades/eventos tipados; `AttachOrderEquipmentModal` y `useAttachOrderEquipment` aíslan el formulario de vinculación. `useEvidenceGallery` administra URLs privadas y descarta respuestas obsoletas al cambiar de contexto. Estos límites no trasladan las mutaciones ni la autoridad de permisos fuera de la vista/backend.

Rutas

Ruta	Vista	Perfil
`/login`	inicio de sesión	público
`/`	dashboard / órdenes recientes	autenticado
`/orders`	listado y filtros	autenticado; datos filtrados en API
`/orders/new`	asistente de creación	administrador
`/orders/:id`	detalle, evidencias, reporte y auditoría	administrador
`/orders/:id/execute`	ejecución mobile-first	técnico autorizado o administrador
`/technicians`	administración de técnicos	administrador
`/admin/users`	administración de cuentas ADMIN	administrador

La guarda de navegación inicializa la sesión, revalida la identidad cuando vence el TTL, redirige según `orderAccessMode` y aplica `requiresRole`. Recuperar el foco y actualizar manualmente el tablero compartido fuerzan la misma reconciliación y reutilizan una única política de redirección. La autorización real siempre se repite en backend.

La administración de cuentas técnicas se compone desde `technicians/views/TechnicalAccountsView.vue`: `useTechnicalAccounts` concentra estado y mutaciones, mientras `SharedAccountPanel`, `TechnicalAccountTable` y `TechnicalAccountModals` mantienen la presentación separada. Las llamadas de usuarios y acceso compartido pertenecen a `modules/technical-accounts`. La vista de cuentas técnicas permite designar la cuenta de taller y habilitarle una orden ejecutable; la administración de cuentas ADMIN está separada y no debe confundirse con ese control. La vista de ejecución carga `/orders/:id/execution`, cuyo contrato omite información administrativa, fiscal y de sincronización. Los técnicos navegan a esa vista también para consultar su historial. Al final de la vista, la tarjeta **Notas de ejecución** permite guardar una nota general de la orden de hasta 5000 caracteres. Si hay cambios pendientes al pulsar **Finalizar y Cerrar Servicio**, la interfaz los guarda automáticamente antes de completar la orden; las notas no se separan por unidad. `useExecutionNotes` centraliza el estado y la persistencia; el guardado manual refresca la orden, mientras que un error al guardar antes del cierre impide finalizarla.

Asistente de creación de órdenes y catálogo de equipos

El asistente (`/orders/new`) guía al administrador en un flujo estructurado de 5 pasos:

1. **Cliente**: Selección del cliente desde Sheets. Los servicios se agregan por bloque en el paso siguiente.
2. **Unidades y Equipos**:
    - **Revisiones y Desinstalaciones**: El componente `EquipmentSelect` filtra automáticamente los equipos con estatus `INSTALADO` del cliente. Al seleccionar un hardware, la función `extractUnitFromEquipment` autocompleta la unidad (`unit` , `unitMakeModel` , `unitYear` , `unitSerialNumber`) reduciendo la captura manual.
    - **Instalaciones y Sensor fuel**: `EquipmentSelect` filtra exclusivamente hardware en stock/almacén (`STOCK TECNICO` , `DISPONIBLE` , etc.) y solicita los datos de la nueva unidad física.
    - **Cambio de equipo**: exige seleccionar el equipo instalado que se retira y uno disponible de almacén; ambos quedan asociados a la misma unidad.
    - **Órdenes mixtas**: cada unidad conserva su propio `serviceType` y el resumen agrupa unidades por servicio.
    - **Normalización**: Las comparaciones de clientes aplican `normalizeString` (case-insensitive, diacríticos removidos vía NFD y eliminación de sufijos legales tipo `S.A. de C.V.`).
3. **Programación**: Fecha, hora, tipo de servicio (en sitio o taller) y notas operativas.
4. **Técnico**: Selección obligatoria desde `DIRECTORIO TECNICO` , con filtros de ciudad y taller, búsqueda, ficha de contacto, marca de última sincronización y recarga explícita. La carga, el catálogo vacío y los errores bloquean el avance.
5. **Resumen y Creación**: Validación integral y envío atómico de la orden con sus unidades y equipos a la API.

En el detalle, las acciones Agregar Unidad, Registrar Primera Unidad y Asignar Equipo aparecen para administradores mientras la orden está en borrador y respetan el servicio seleccionado. La API valida la misma regla. La carga inicial anidada del wizard y las acciones de quitar/eliminar conservan sus reglas existentes. Los botones interactivos personalizados responden tanto a clic como a teclado. `AddOrderUnitModal` y `AttachOrderEquipmentModal` contienen los formularios; sus composables administran estado, validación y mutación sin trasladar al componente de detalle el payload de Sheets. `OrderUnitsTab` presenta unidades y equipos con propiedades y eventos tipados. El detalle conserva las mutaciones y los modales, de modo que la pestaña no crea otra fuente de estado de órdenes.

Evidencias y reportes

`image-preprocessing.ts` convierte fotografías HEIC/HEIF a JPEG y limita sus dimensiones a 1920 px antes de enviarlas. Las promesas rechazan explícitamente archivos no legibles o imágenes no decodificables para evitar bloqueos en la interfaz. El módulo transforma imágenes; `EvidenceCapture` conserva los protocolos del proveedor configurado. En modo Drive la API

recibe la carga autenticada; el adaptador de objetos puede usar URLs prefirmadas. La firma se captura con canvas y se envía como PNG. El detalle de la orden consulta el estado del reporte cada tres segundos mientras está pendiente o procesándose y solicita el archivo privado cuando queda listo. `evidence-upload-retry.ts` conserva la identidad de reintento de Drive usando la orden, la unidad y el archivo original. `useEvidenceGallery` libera las URLs privadas cuando una evidencia sale de la lista, cambia el contexto o se desmonta la vista, e ignora respuestas de cargas obsoletas.

PWA

El proyecto incluye manifest e iconos instalables. Solo se cachean assets estáticos; no se promete operación offline ni se cachean respuestas autenticadas.

Infraestructura y operación

Inventario histórico: 2026-10-05

Estado verificado después del inventario: staging eliminado/no desplegado al 2026-10-09; producción no desplegada.

Fuentes: inventario AWS verificado el 2026-10-05 para el estado observado entonces; `infra/cdk`, `docker-compose.yml`, `infra/docker` y `.github/workflows` para la configuración declarada.

El inventario del 2026-10-05 observó staging en la cuenta `075566976700`, región `us-east-2`: los stacks `PukStagingFoundation` y `PukStagingApplication`, una tarea ECS en EC2 `t3.small`, RDS independiente del host, la IPv4 efímera `18.222.63.51` y el secreto runtime `puk/staging/runtime`. Al 2026-10-09 staging está eliminado/no desplegado y producción no está desplegada. Los datos del inventario son referencias históricas, no evidencia de recursos activos. No se inspeccionaron datos de RDS.

Diferencia observada históricamente: la revisión de tarea 14 usaba S3; la IaC local selecciona Drive. El bucket `EvidenceBucket` estaba referenciado por esa revisión. La consulta de versiones del 2026-10-05 devolvió cero versiones y marcadores de borrado, sin paginación pendiente; no habilita su eliminación mientras siga siendo un consumidor activo. La sincronización espera configuración segura y validación de los recursos de Google de pruebas. No se han eliminado datos ni recursos como parte de esta sincronización. Producción no tiene stacks CDK propios.

Ambientes

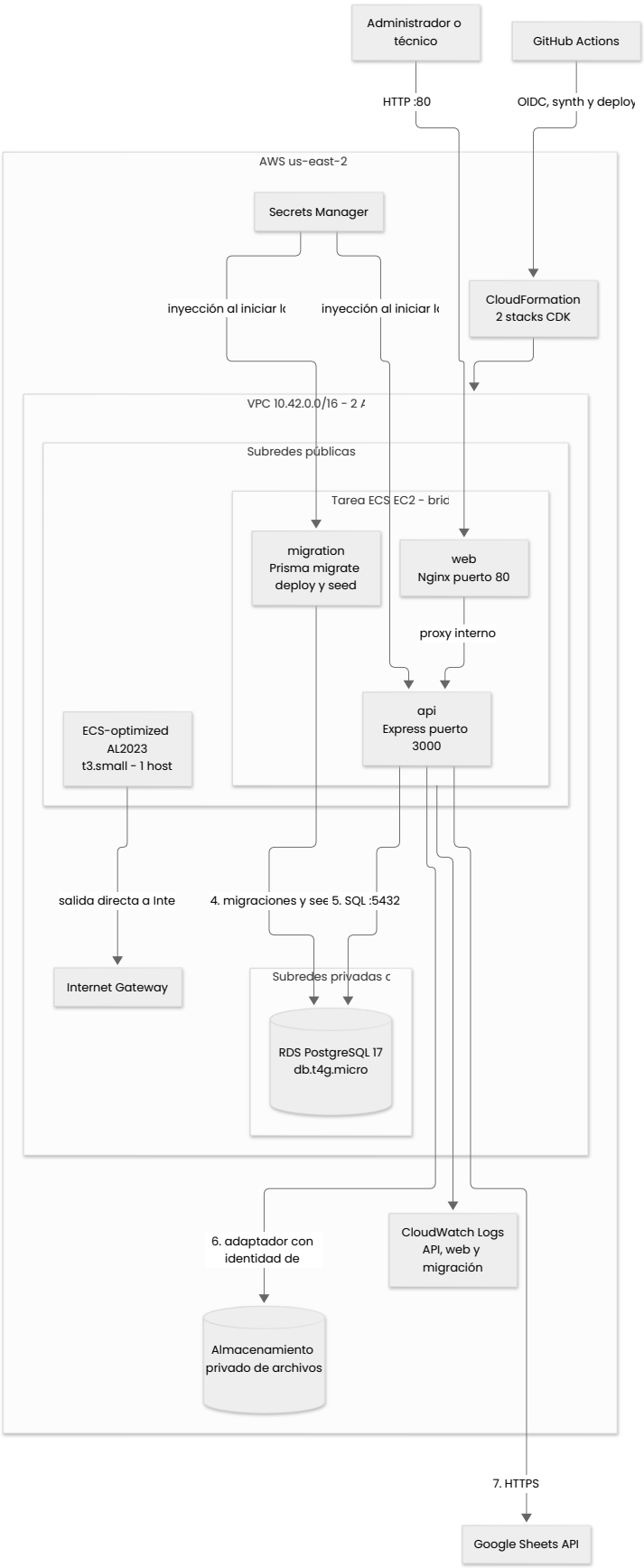
Ambiente	Implementación actual	Persistencia	Entrada
Desarrollo con hot reload	Vite y API en el host; PostgreSQL y MinIO en Docker Compose	Volúmenes locales	`http://localhost:5173` y `http://localhost:3000`
Desarrollo compilado	Contenedores `web`, `api`, `postgres`, `minio` y `minio-init`	Volúmenes locales	`http://localhost:4173` y `http://localhost:3000`
Staging	No desplegado al 2026-10-09; la IaC describe ECS sobre EC2, RDS, almacenamiento y logs	Por verificar	Sin endpoint activo
Producción	No implementado como IaC en la rama actual	Por definir	Por definir

Arquitectura de staging en AWS: topología declarada por CDK

El diagrama y las secciones siguientes describen la infraestructura declarada en CDK, no recursos activos. Staging está eliminado/no desplegado al 2026-10-09; verificar el diff y el estado real de AWS antes de cualquier operación.

Vista de despliegue

> **Topología declarada por IaC:** los detalles corresponden al diseño en código. El inventario histórico observó S3; Drive es el proveedor seleccionado por la IaC local y requiere configuración y validación antes de un redeployment.



El diagrama muestra la topología definida por CDK: subredes públicas, privadas para aplicación y privadas aisladas, una por zona de disponibilidad; un Auto Scaling Group fijo con una instancia EC2 `t3.small` en subred pública y una IPv4 efímera; HTTP directo por el puerto 80;

sin NAT Gateway, Elastic IP, ALB ni CloudFront. Nginx enruta internamente al API y RDS queda en subredes privadas aisladas. Esto no confirma la existencia actual de esos recursos.

Recorrido de una solicitud

1. El navegador accede a `http://<ipv4-publica>` directamente.
2. El security group acepta sólo HTTP TCP/80 desde Internet; no expone SSH ni el puerto 3000.
3. Nginx sirve el frontend Vue, aplica fallback a `index.html` y enruta `/api/` , `/health/` , `/docs/` y `/metrics` al API por la red interna de contenedores.
4. Express procesa API, salud, métricas y OpenAPI.

Esta entrada no usa TLS: las credenciales y cookies de sesión viajan sin cifrado. La IPv4 es efímera y puede cambiar al detener/reemplazar el host; no existe un dominio estable ni CDN, protección de borde, terminación TLS o caché CDN. No debe usarse para datos reales ni producción.

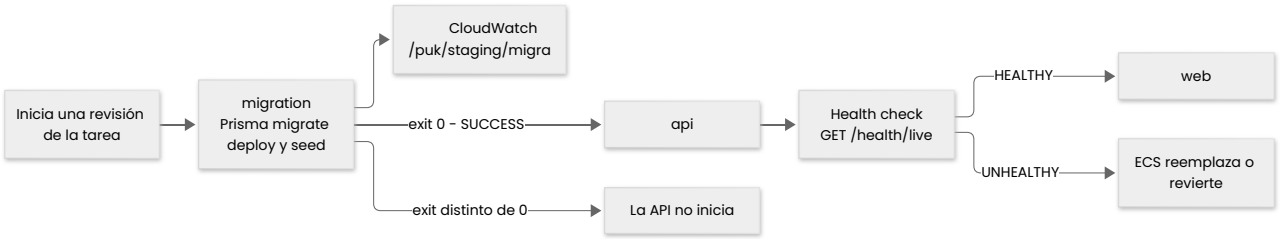
Red y límites de confianza

Zona	Recursos	Entrada permitida	Salida relevante
Pública	Host ECS EC2 con IPv4 pública efímera	TCP/80 desde cualquier IPv4	Internet Gateway, RDS y APIs AWS
Privada aislada	RDS PostgreSQL	TCP/5432 sólo desde el security group de la aplicación	Sin ruta directa a Internet
Fuera de la VPC	Google Drive, Secrets Manager y CloudWatch	Mediante APIs administradas	Según el servicio

El host EC2 recibe IPv4 pública efímera para conexiones entrantes y salientes sin NAT Gateway ni Elastic IP. Su security group sólo permite TCP/80 desde Internet; el API en el puerto 3000 no se publica. La API requiere salida HTTPS para Google Sheets, Google Drive y AWS. La tarea usa modo `bridge` y conserva sus roles ECS propios; IMDSv2 es obligatorio y el hop limit es 1 para impedir que contenedores alcancen las credenciales del instance profile. La instancia no recibe permisos de Secrets Manager ni de S3. Esta configuración evita el costo horario y de procesamiento del NAT, aunque la IPv4 pública sigue teniendo costo y la salida no pasa por un NAT controlado.

Tarea ECS sobre EC2 y orden de arranque declarado

> **Orden definido por IaC:** al iniciarse una tarea, el contenedor de migración termina antes de que pueda iniciar la API; no implica que haya una tarea activa actualmente.



Los tres contenedores comparten la tarea; ECS la ejecuta en modo `bridge` dentro del host EC2. Nginx se vincula al contenedor `api` por el alias `api`. Nginx publica el puerto de host 80; el API también mapea el puerto de host 3000, pero el security group bloquea el ingreso externo a ese puerto. Estas son propiedades de la definición CDK, no una observación del entorno actual:

- `migration` no es esencial. Cada vez que se inicia una tarea nueva, ejecuta `prisma migrate deploy` y después `prisma/seed.ts`; la API depende de que termine con éxito (`SUCCESS`). `migrate deploy` aplica sólo las migraciones pendientes presentes en `prisma/migrations/`. Un cambio únicamente en `prisma/schema.prisma` no modifica la base de datos: hay que crear y versionar la migración correspondiente. Las imágenes de API y migración generan el cliente Prisma durante el build Docker con una URL de base de datos ficticia de build; esa generación no se conecta a la base de datos de producción ni ejecuta migraciones. Un despliegue sólo de código puede iniciar el contenedor de migración, pero no cambia el esquema si no hay migraciones pendientes.
- `api` es esencial y expone el puerto 3000. Su health check interno consulta `/health/live`.
- `web` es esencial y expone el puerto 80. Sólo inicia cuando la API está saludable.
- El servicio usa `desiredCount: 1` en un ASG de capacidad fija (mínimo/máximo/deseado 1). El despliegue usa 0 % mínimo y 100 % máximo para detener la tarea anterior antes de iniciar la nueva: hay una interrupción breve durante cada despliegue y no se garantiza alta disponibilidad ante falla del host.
- El host es `t3.small` con CPU credits en modo `standard`: se limita el costo inesperado de créditos surplus, a cambio de que el rendimiento de CPU pueda bajar al agotarse el balance bajo carga sostenida.

La migración dentro de cada tarea mantiene el orden de arranque, pero con más de una tarea podría ejecutar migraciones o seed de forma concurrente. Antes de escalar horizontalmente se debe convertir la migración en un job único por despliegue o comprobar explícitamente su idempotencia y bloqueo.

Persistencia

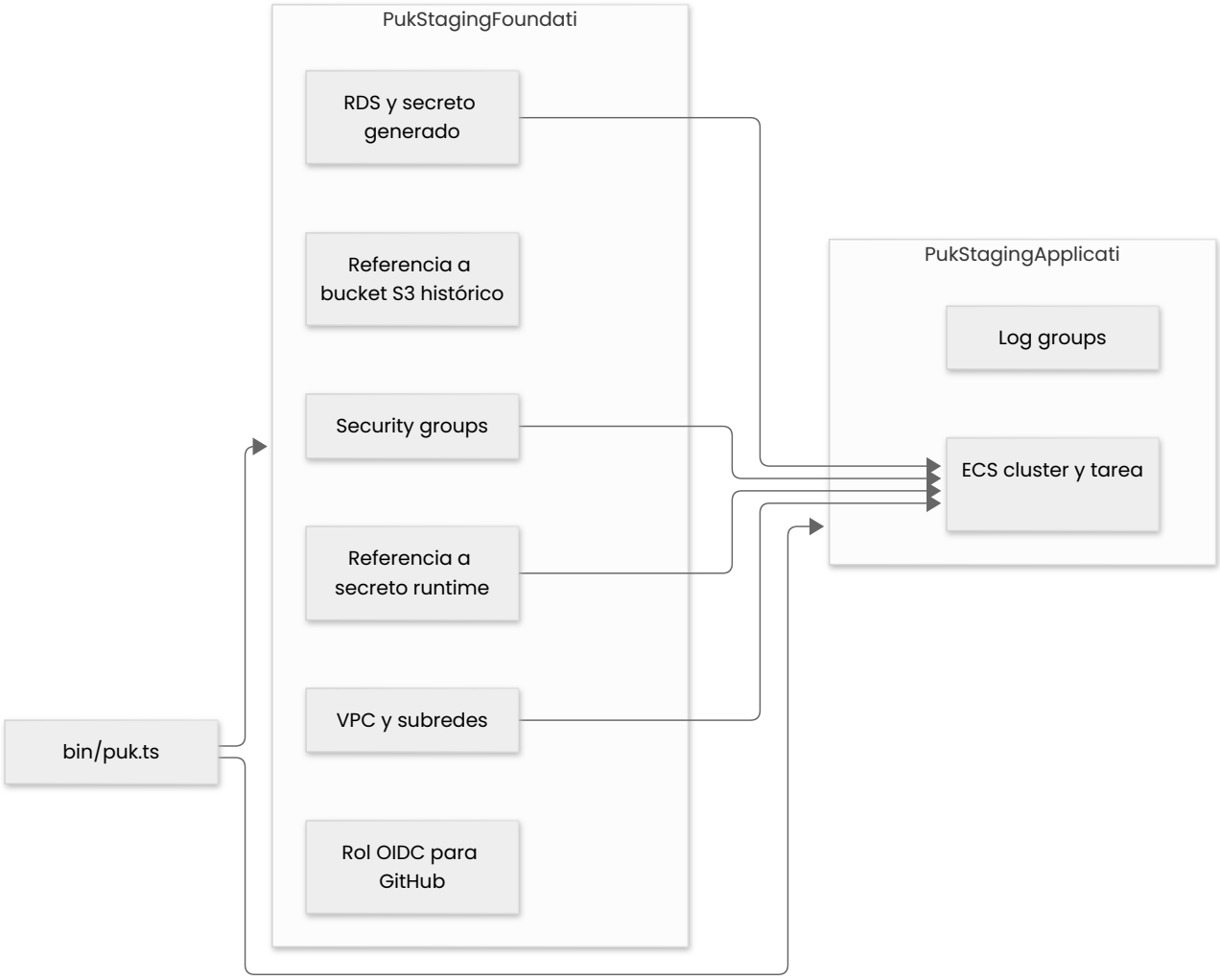
Recurso	Configuración declarada / diferencia histórica	Política IaC al eliminar en el futuro
RDS PostgreSQL 17	`db.t4g.micro`, 20 GiB, autoscaling hasta 100 GiB, cifrado, Single-AZ, 1 día de backup automático, sin acceso público	`RETAIN` y deletion protection
Almacenamiento de archivos	Históricamente S3; Drive seleccionado por la IaC local y pendiente de configuración para redeploy	Conservar archivos y bucket hasta validar transición

Recurso	Configuración declarada / diferencia histórica	Política IaC al eliminar en el futuro
Logs	Grupos separados para migración, API y web; retención de un mes	`RETAIN`

El objetivo de producto es usar una unidad compartida de Google Drive para evidencias, firmas y reportes; PostgreSQL conserva referencias y metadatos. La IaC local selecciona `STORAGE_PROVIDER=drive`, no declara buckets ni permisos S3 nuevos, y requiere los tres campos `GOOGLE_DRIVE_*`. La última revisión de tarea observada históricamente usaba S3; no hay una revisión desplegada actualmente. Conservar bucket, referencias y permisos necesarios hasta validar el reemplazo; cambiar de proveedor no migra archivos existentes. La política de respaldo independiente sigue pendiente antes de producción.

Stacks CDK

> **Dependencias declaradas entre stacks:** revisar los exports consumidos antes de desplegar cualquier limpieza o redeployment.



`PukStagingApplication` recibe referencias del stack Foundation, por lo que CDK crea dependencias y exports de CloudFormation. Foundation concentra recursos con estado o compartidos; Application concentra cómputo. Esta separación reduce el riesgo al reemplazar la capa de aplicación, aunque el stack foundation no activa `terminationProtection` a nivel de CloudFormation.

La IaC conserva una referencia a un security group legado del ALB para compatibilidad entre stacks. No hay ALB ni NAT Gateway en el inventario regional consultado. Tanto esa referencia como los exports S3 requieren limpieza ordenada: preservar el recurso/export original mientras Application lo consuma, actualizar primero el consumidor y retirar después sólo lo que quede sin uso. El diff actual de Foundation elimina exports S3 aún consumidos y modifica la política del bucket. Con Drive configurado y validado, desplegar primero Application con `cdk deploy PukStagingApplication --exclusively` desde `infra/cdk`, sin actualizar Foundation como dependencia; no empezar con `deploy --all`. Después de liberar imports y validar la aplicación, actualizar Foundation. No borrar datos como atajo; volver a comprobar versiones y marcadores de borrado antes de considerar el bucket vacío como candidato a limpieza.

No se deben renombrar construct IDs ni mover recursos persistentes sin revisar primero `pnpm infra:diff`: un cambio de logical ID puede provocar reemplazo. Tampoco se debe eliminar un export del stack foundation mientras Application lo consuma.

Identidades y configuración de ejecución

- ECS inyecta credenciales de RDS desde su secreto generado al iniciar `migration` y `api`.
- El inventario del 2026-10-05 observó el secreto runtime `puk/staging/runtime` en Secrets Manager, `us-east-2`; staging está actualmente eliminado/no desplegado.
- La configuración desplegada histórica contenía referencias de Sheets. La configuración CDK local de Drive agrega `GOOGLE_DRIVE_CLIENT_EMAIL`, `GOOGLE_DRIVE_PRIVATE_KEY` y `GOOGLE_DRIVE_SHARED_DRIVE_ID`; requiere configurar y verificar una identidad y una unidad compartida de pruebas.
- AWS staging está eliminado/no desplegado. Gmail funciona localmente sólo con las variables `GOOGLE_GMAIL_CLIENT_EMAIL`, `GOOGLE_GMAIL_PRIVATE_KEY` y `GOOGLE_GMAIL_SENDER_EMAIL`, más delegación Workspace para `https://www.googleapis.com/auth/gmail.send`; el remitente es tanto el usuario suplantado como el encabezado `From`, y cambiarlo requiere reiniciar la API.
- El CDK de Application no mapea `GOOGLE_GMAIL_*` al entorno de la tarea. Gmail no puede habilitarse en AWS agregando únicamente los valores al secreto: primero se debe actualizar la IaC/task secret mapping y desplegar. El envío de reportes es manual, no automático.
- Las identidades de Sheets y Drive pueden ser distintas; no inferir ni reutilizar sus claves automáticamente. Permisos IAM de Google Cloud no sustituyen compartir los recursos de Workspace.
- La IaC define el rol OIDC `puk-staging-github-deploy`, limitado a `main` del repositorio configurado.
- Las variables no sensibles se definen en el task definition; nombres de pestañas y `STORAGE_PROVIDER` no cambian editando sólo los IDs en Secrets Manager.

El procedimiento completo de cuentas de servicio, IDs, permisos y actualización parcial está en Configurar Sheets y Drive. No copiar claves o archivos JSON a chat, logs, tickets ni Git.

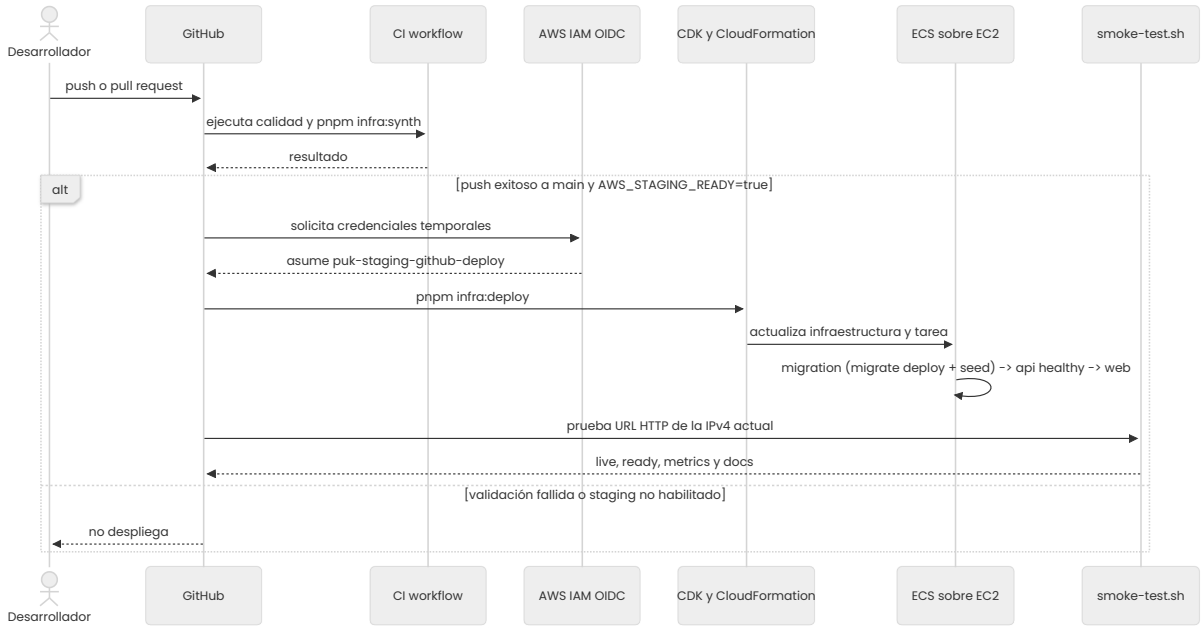
Observabilidad

Esta sección describe los checks y logs definidos por la IaC y la aplicación; no valida por sí sola el acceso a Google.

- ECS Container Insights está deshabilitado en el cluster `puk-staging` para evitar métricas adicionales; permanecen disponibles los health checks y logs de CloudWatch.
- El host usa ECS-optimized Amazon Linux 2023 x86_64, no expone SSH y acepta HTTP público sólo por TCP/80. No tiene Elastic IP; si la instancia se reemplaza, cambia su IPv4 pública de acceso y egreso.
- `GET /health/live` comprueba que el proceso de API responda.
- `GET /health/ready` comprueba sólo conectividad con PostgreSQL y se usa en el smoke test. No prueba Sheets, autenticación de Google ni permisos de Drive; validarlos por separado.
- `GET /metrics` expone métricas Prometheus, pero el CDK no crea un scraper, alarmas ni dashboards.
- `GET /health/metrics` ofrece un snapshot JSON desde la aplicación.
- Los logs de migración, API y web se envían a grupos separados de CloudWatch Logs con retención de un mes.
- Cada petición recibe un `requestId`; puede propagarse uno válido desde el cliente.

Integración y despliegue continuo

> **Workflow declarado:** `.github/workflows/deploy-staging.yml` corre después de un CI exitoso sobre `main` sólo si `AWS_STAGING_READY == 'true'`. El valor actual no se verificó ni modificó. Resolver configuración de Google y exports entre stacks antes de habilitar despliegues automáticos.



El workflow `.github/workflows/ci.yml` instala dependencias, valida Prisma, ejecuta lint, typecheck, pruebas, validación OpenAPI, build, `infra:synth`, auditoría y E2E. Después de un CI exitoso sobre `main`, `.github/workflows/deploy-staging.yml` hace checkout de `workflow_run.head_sha`, ejecuta `pnpm infra:deploy` y smoke-tests sólo si la variable de repositorio `AWS_STAGING_READY` vale exactamente `true`. La concurrencia está serializada y no cancela un despliegue en curso. Aunque éste sea el flujo declarado, staging actualmente no está desplegado.

Los assets Docker de CDK se construyen y publican mediante el bootstrap de CDK. El repositorio no define ECR repositories ni tags de aplicación explícitos.

El workflow de despliegue hace checkout de `github.event.workflow_run.head_sha`, por lo que CI despliega la revisión que pasó las validaciones. Un despliegue manual de assets o de CDK debe registrar explícitamente el SHA de Git, la revisión de ECS y el operador para conservar la trazabilidad de release.

Actualización de secretos y operación

Para cambiar IDs o una cuenta de servicio, usar la consola de Secrets Manager y editar únicamente los campos de Google de `puk/staging/runtime`, preservando todos los campos `JWT_*`, `SEED_*` y cualquier otro valor existente. La guía de staging explica el mapeo del JSON y los permisos de los documentos y la unidad compartida.

No usar `configure-staging-secrets.mjs --overwrite` para cambios parciales: reemplaza el contenido completo y regenera claves JWT y contraseñas del seed. El comando sin `--overwrite` rechaza el secreto existente y sólo es apropiado para creación inicial. Nunca mostrar claves en terminal, chat, tickets o logs. Una nueva versión del secreto no actualiza tareas ya iniciadas: requiere despliegue nuevo. La primera activación de Drive también necesita el task definition actualizado desde IaC, no sólo reiniciar la revisión 14.

Con una sesión y destino autorizados, estos diagnósticos son de sólo lectura; no copiar información sensible que pudiera aparecer en logs:

```
aws cloudformation describe-stacks --region us-east-2 --stack-name PukStagingApplication
aws ecs describe-services --region us-east-2 --cluster puk-staging --services <service-name>
aws ecs list-tasks --region us-east-2 --cluster puk-staging --service-name <service-name>
aws logs tail /puk/staging/migration --region us-east-2 --since 30m
aws logs tail /puk/staging/api --region us-east-2 --since 30m
TARGET_URL="http://<ipv4-publica-actual>" ./infra/scripts/smoke-test.sh
```

Consultar siempre la IPv4 actual con el procedimiento de la guía de staging. Antes de desplegar Drive, verificar lectura de las pestañas de pruebas y metadatos de la unidad compartida con las cuentas de servicio mediante un mecanismo seguro. Después, ejecutar smoke y pruebas funcionales con archivos y filas sintéticos. El smoke sólo comprueba `/health/live`, `/health/ready`, `/metrics` y `/docs/`; no certifica la integración con Google.

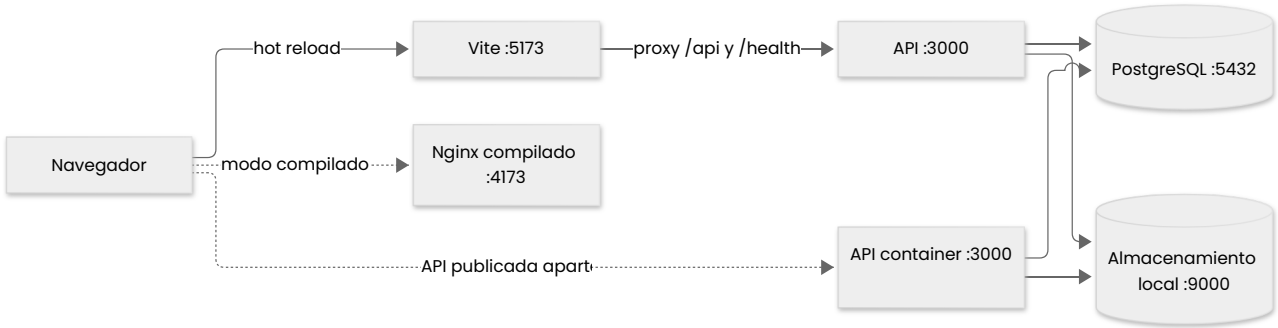
Si falla la migración, la API no inicia; conservar logs y evaluar la migración antes de reintentar. El circuit breaker de ECS puede revertir la aplicación, pero no restaura automáticamente RDS. `RETAIN` protege algunos recursos al eliminar un stack; no es un backup. Usar sólo respaldos verificados y autorización operativa para restaurar o decidir el destino de datos.

RDS, EC2, EBS, la IPv4 pública y los servicios asociados pueden generar costos incluso sin tráfico. La limpieza debe limitarse a recursos sin consumidores, después de validar Drive y revisar los exports entre stacks; no eliminar datos o el bucket para resolver una actualización.

Desarrollo local

Servicios Docker Compose

Servicio	Imagen o proceso	Puerto predeterminado	Persistencia
`web`	Nginx + Vue compilado	4173 externo / 80 interno	No
`api`	Node.js 24 + Express	3000	No
`postgres`	PostgreSQL 18	5432	Volumen `postgres-data`
`minio`	MinIO	9000 API, 9001 consola	Volumen `minio-data`
`minio-init`	Cliente `mc`	No aplica	Crea el bucket privado



En hot reload, el proxy de Vite envía `/api` y `/health` a `VITE_API_PROXY_TARGET`. En ECS, Nginx y API comparten tarea y `127.0.0.1:3000` es correcto. En Docker Compose están en contenedores distintos, pero la configuración Nginx también apunta a `127.0.0.1:3000`; por ello el proxy `/api` del contenedor web compilado no alcanza actualmente a `api`. Hasta corregir esa divergencia, use hot reload o el puerto publicado de la API para validar el modo local.

Inicio rápido

```
pnpm install
cp .env.example .env
docker compose up -d postgres minio minio-init
pnpm prisma:migrate
pnpm prisma:seed
pnpm dev
```

Para construir todos los contenedores:

```
docker compose up --build -d
```

Los Dockerfiles son multi-stage. La imagen de API se ejecuta como usuario `node`; el frontend se construye con Vite y se sirve desde Nginx. Las versiones admitidas son Node.js 24 a 26 y pnpm 11.

Variables de entorno

Grupo	Variables principales
HTTP	`PORT`, `APP_ORIGIN`, `API_ORIGIN`, `TRUST_PROXY`
Base de datos	`DATABASE_URL`, `DIRECT_URL` o variables `PUK_DATABASE_*` en ECS
JWT	Claves pública y privada, `JWT_ISSUER`, `JWT_AUDIENCE`, TTLs
Cookies	`COOKIE_DOMAIN`, `COOKIE_SECURE`, `COOKIE_SAME_SITE`
S3 (legado/local)	Endpoint, región, bucket, credenciales locales y path style; revisión histórica 14 usó S3, Drive está en IaC
Drive	Cuenta de servicio, clave privada e ID de unidad compartida (`GOOGLE_DRIVE_*`)
Gmail	Cuenta de servicio, clave privada y buzón suplantado (`GOOGLE_GMAIL_*`); no mapeado por CDK
Sheets	Spreadsheet, cuenta de servicio, clave privada y pestañas
Límites	Tamaños de evidencia y firma, TTL de URLs, caché
Resiliencia	Timeouts, reintentos, circuit breaker y polling
Seed	Correos y contraseña inicial

La API valida su configuración al arrancar. Los valores `development-only` activan adaptadores simulados fuera de producción y nunca deben usarse en producción.

Procesos periódicos

- `SyncService` consulta el outbox cada `OUTBOX_POLL_INTERVAL_MS`, procesa hasta 20 operaciones y aplica backoff exponencial con jitter.
- `ReportsService` procesa hasta dos PDFs por ciclo y recupera trabajos interrumpidos después de diez minutos.
- Ambos intervalos usan `unref()` y se cancelan durante el apagado ordenado.
- Con `desiredCount: 1` hay una sola instancia de cada worker. El modelo debe revisarse antes de escalar la cantidad de tareas para evitar trabajo duplicado.

Operación y validación

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm prisma:validate
pnpm docs:validate
pnpm infra:synth
pnpm infra:diff
```

Antes de desplegar cambios CDK se debe ejecutar `synth` y revisar `diff`, especialmente para RDS, S3, VPC y referencias entre stacks. Hotswap y express mode no forman parte del flujo y no deben usarse en producción.

La guía operativa y el primer despliegue están en staging-and-deployment.md. Los scripts `infra/scripts/backup.sh` y `restore.sh` realizan respaldo lógico y restauración de PostgreSQL, pero no están conectados al workflow de AWS; su programación, almacenamiento externo y prueba periódica son responsabilidad operativa.

Evaluación para producción

La arquitectura declarada de staging prioriza costo y simplicidad. AWS staging no está desplegado; Drive sigue pendiente de validación antes de un redeployment. La tabla compara la configuración objetivo con el trabajo previo a producción:

Pilar	Configuración objetivo de IaC	Trabajo pendiente
Seguridad	RDS cifrado; Drive privado pendiente; HTTP público; OIDC en CI; el inventario histórico observó S3	Configurar dominio y TLS; limitar ingreso; definir controles de Workspace; agregar WAF según riesgo; auditar permisos con cdk-nag

Pilar	Configuración objetivo de IaC	Trabajo pendiente
Confiabilidad	Dos AZ para subredes; rollback de ECS; política de retención prevista para RDS	RDS Multi-AZ; alarmas; estrategia de migración única; restauraciones probadas
Rendimiento	Sin CDN ni caché de borde	Autoscaling ECS, pruebas de carga y dimensionamiento de RDS
Costos	Sin NAT; host EC2 único con IPv4 pública y RDS micro	Presupuestos y alarmas de costo; revisar transferencia e IPv4 públicas
Operación	Logs por contenedor y smoke test	Dashboards, alarmas, trazas, runbooks y objetivos SLO
Sostenibilidad	Contenedores dimensionados explícitamente	Medir utilización y ajustar CPU, memoria y retenciones

Referencias oficiales AWS

- Conectar aplicaciones ECS a Internet desde subredes privadas
- Periodo de retención de backups automatizados de RDS
- Buenas prácticas de desarrollo y despliegue con AWS CDK

Seguridad y calidad

Controles implementados

Identidad y sesión

- Contraseñas con Argon2id.
- Access y refresh JWT firmados con pares de claves separados.
- Access token de corta duración y refresh token rotativo.
- Validación viva del principal en cada request protegido: el JWT aporta la identidad y PostgreSQL determina actividad y rol vigente.
- Solo se almacena el hash del refresh token en PostgreSQL.
- Cookie HTTP-only configurable con `Secure`, `SameSite` y dominio.
- Revocación por sesión y cierre de todas las sesiones.
- La desactivación bloquea inmediatamente access tokens vigentes con `SESSION_REVOKED`; una falla al consultar la autoridad produce `AUTHORITY_UNAVAILABLE` y no se confunde con credenciales inválidas.
- El cliente reconcilia `/auth/me` al navegar, recuperar el foco y actualizar el tablero compartido; un `401` limpia la sesión sin pasar por el interceptor de refresh. Los `403` no se reintentan ni disparan un interceptor global: la reconciliación ocurre en esos límites explícitos para evitar ciclos y efectos laterales sobre denegaciones que no implican un cambio de identidad.

API

- Helmet, CORS con allowlist y límite JSON de 1 MiB.
- Rate limiting general y específico para autenticación.
- Validación Valibot de body, params y query.
- Capacidades por rol más políticas de propiedad de la orden.
- Mensajes de autenticación genéricos y manejador uniforme de errores.
- `requestId` para correlación.

Archivos

- Google Drive compartido se usa para el almacenamiento seleccionado en IaC de staging; Drive transmite archivos mediante rutas autenticadas de la API.
- El adaptador de almacenamiento de objetos conserva carga/descarga directa mediante URLs prefirmadas según entorno.
- Validación posterior de existencia, MIME y tamaño.
- Nombres de objeto generados por servidor.
- Fotografías y firmas fuera de PostgreSQL.

- PDFs privados, checksum SHA-256 y errores de generación sanitizados.
- Las respuestas públicas de reporte no exponen `objectKey`.

Integraciones y datos

- Credenciales de Google y del proveedor de almacenamiento solo en servidor.
- Puertos abstraen integraciones externas.
- Timeouts, reintentos y circuit breaker (CLOSED/OPEN/HALF_OPEN) con métricas Prometheus para adaptadores de Sheets y almacenamiento.
- Outbox e idempotencia para efectos externos.
- Concurrencia optimista en órdenes y restricción de asignación activa.
- Auditoría de acciones críticas con snapshots antes/después cuando aplica.
- La gestión de usuarios exige `users:manage`; las actualizaciones administrativas impiden el auto-bloqueo y la desactivación/degradación del último administrador activo. Las actualizaciones serializables protegen la comprobación ante cambios concurrentes.

Roles

Capacidad	Administrador	Técnico
Usuarios y técnicos	Sí	No
Crear/editar/asignar órdenes	Sí	No
Consultar todas las órdenes	Sí	No
Ejecutar orden propia	Sí según endpoints	Sí
Consultar catálogos	Sí	Sí
Reintentar sincronización/reporte	Sí	No
Descargar reporte	Sí	Solo de orden asignada

Las capacidades declaradas son `users:manage`, `technicians:manage`, `orders:manage`, `orders:execute`, `catalogs:read` y `sync:retry`.

Estrategia de pruebas

Nivel	Herramientas	Cobertura principal
Unitario	Vitest	tokens, contraseñas, validación, máquina de estados, especificación de completitud, PDF, resiliencia, cliente web
Integración API	Supertest + Vitest	autenticación, catálogos, administración, permisos, formato HTTP
Integración DB	Prisma + PostgreSQL de prueba	restricciones, concurrencia, unicidad

Nivel	Herramientas	Cobertura principal
Contrato	OpenAPI + pruebas	health, respuestas y referencias
E2E	Playwright	flujos de administrador y técnico
Build estático	TypeScript/Vue TSC/Vite	compatibilidad estricta y bundles

Pipeline mínimo

1. Instalación con lockfile.
2. Formato y lint.
3. Typecheck.
4. Pruebas unitarias e integración.
5. Validación Prisma y OpenAPI.
6. Build de API y web.
7. Escaneo de dependencias e imágenes.
8. Migraciones, despliegue y smoke tests.

Aspectos que requieren política externa

- Rotación y custodia de claves JWT, Google y almacenamiento.
- Retención legal de fotografías, firmas, auditoría y reportes.
- Control de acceso a `/metrics` en el proxy productivo.
- Backups cifrados y pruebas periódicas de restauración.
- Aprobación del logotipo y leyenda de conformidad del PDF.
- Monitoreo y alertas sobre outbox fallido, generación de reportes y capacidad/retención de Drive o del proveedor configurado.

Contrato del libro de técnicos y cobro

Versión: `1.1-implemented`, 2026-09-10. Estado: directorio, tarifas, cálculo de snapshots, persistencia y sincronización del reporte técnico implementados. La validación externa de directorio y escritura en staging ocurrió históricamente; AWS staging fue eliminado después y no existe despliegue actual. Las pruebas automatizadas siguen cubriendo los contratos localmente.

Identidad y configuración

El libro se configura por ambiente; su ID no se incrusta en código ni en fixtures. El 2026-09-09 el usuario proporcionó el ID del libro y confirmó su uso para desarrollo/staging. Se verificó el directorio mediante la API con la cuenta de servicio configurada en la aplicación. El usuario confirmó los nombres exactos de las pestañas: `DIRECTORIO TECNICO` para el directorio y `2026` para el reporte del año actual. Los encabezados del directorio están en la fila 1; los del reporte siguen pendientes de verificación. Debe definirse qué fecha determina la pestaña anual en publicaciones atrasadas o correcciones de órdenes de años previos.

El usuario confirmó que actualmente no existe una clave externa y autorizó agregar `ID_TECNICO` al final de `DIRECTORIO TECNICO`, después de todas las columnas existentes, sin sobrescribirlas ni interferir con ellas. Se acordaron códigos como `TEC-0001`, asignados una sola vez y nunca reutilizados. Antes de escribir hay que inspeccionar las columnas existentes, incluidas las que no tengan encabezado o estén ocultas, y comprobar que `ID_TECNICO` no haya sido agregado entretanto.

El usuario creó `ID_TECNICO` en `W1` y autorizó usar W. La inspección confirmó 88 técnicos en filas 2–89, sin valores ni fórmulas previos en sus celdas W y sin celdas combinadas o rangos protegidos en el directorio.

El 2026-09-09 se escribieron 88 valores de texto fijos, `TEC-0001` a `TEC-0088`, únicamente en `W2:W89`. Se releyó el directorio antes de escribir para detectar cambios y después se comparó el contenido completo contra el resultado esperado. Se verificaron 88 IDs únicos y la conservación de los valores/fórmulas de todas las demás celdas. No se modificaron formatos ni estructura del libro.

El usuario autorizó una excepción puntual para usar las credenciales existentes de la aplicación dentro del proceso para esta inicialización, sin mostrarlas ni copiarlas. Esa excepción no modifica las instrucciones generales de `AGENTS.md`. El comando de mantenimiento `apps/api/scripts/assign-technician-ids.mjs` inspecciona por defecto; la escritura exige `--apply` y la huella del contenido inspeccionado. Una repetición sobre un directorio completamente inicializado no escribe nada. Las altas posteriores requieren un asignador persistente que no recicle IDs de técnicos retirados; el comando rechaza mezclar IDs existentes con nuevas celdas vacías.

El UUID local sigue siendo la referencia de órdenes y usuarios. Las filas movidas conservan identidad; las retiradas se desactivan sin eliminar historia; las claves duplicadas abortan la

importación completa. Una fila sin clave o nombre no es seleccionable. No se deriva identidad del número de fila.

La integración usa `GOOGLE_TECHNICIANS_SPREADSHEET_ID`, `GOOGLE_TECHNICIANS_SHEET_NAME` y `GOOGLE_TECHNICIAN_REPORT_SHEET_NAME`, separadas del libro de clientes e inventario. Los ejemplos de prueba utilizan identificadores ficticios como `development-workbook` y `TECH-DEMO-001`.

Directorio: campos requeridos por la issue #7

Posiciones y encabezados verificados por API en `DIRECTORIO TECNICO`, fila 1:

Columna	Encabezado exacto
A	TECNICO
B	TALLER
C	RAZON SOCIAL
D	TELEFONO
E	CORREO
F	DIRECCIÓN
G	CIUDAD
H	ESTADO
I	FACTURA
J	DATOS FISCALES
K	DIRECCION FISCAL
L	COSTO SALIDA DEL TALLER
M	CAMBIO DE EQUIPO
N	INSTALACION
O	REVISION
P	DESINSTALACION
Q	CITA FALLIDA
R	SENSOR FUEL
S	NOTAS ADICIONALES
T	NOMBRE DE LA CUENTA BANCARIA
U	CUENTA BANCARIA

Columna	Encabezado exacto
V	BANCO
W	ID_TECNICO

S–V ya existían y se preservaron. El catálogo solicitado no requiere importar los datos bancarios de T–V. La nullabilidad no especificada en la issue se debe acordar, no inferir; los tipos funcionales previstos se detallan abajo.

Campo lógico	Tipo previsto	Nullabilidad acordada en la issue
Clave externa	Texto estable único	No
Técnico	Texto	No
Taller	Texto	Pendiente
Razón social	Texto	Pendiente
Teléfono	Texto	Pendiente
Correo	Texto	Sí
Dirección	Texto	Pendiente
Ciudad	Texto	Pendiente
Estado	Texto	Pendiente
Factura	Indicador/dato por confirmar	Pendiente
Datos fiscales	Texto/estructura por confirmar	Sí
Dirección fiscal	Texto	Sí
Costo salida del taller	Decimal	Pendiente
Cambio de equipo	Decimal	Pendiente
Instalación	Decimal	Pendiente
Revisión	Decimal	Pendiente
Desinstalación	Decimal	Pendiente
Cita fallida	Decimal	Sí
Sensor fuel	Decimal	Pendiente

Una celda monetaria vacía se representa como `null`; cero requiere un valor explícito. Un importe inválido impide usar esa tarifa para cobro. Moneda, escala, redondeo y separadores requieren aprobación. No se interpreta `$` como prueba de una moneda.

Base de cobro confirmada

El usuario confirmó que se toma la tarifa del técnico correspondiente al servicio y se multiplica por el total de equipos de la orden, no por el número de vehículos: `costo base = tarifa por equipo × cantidad total de equipos`. Por ejemplo, dos vehículos con tres equipos cada uno multiplican la tarifa por seis. El snapshot debe conservar tanto la tarifa por equipo como la cantidad aplicada y el costo base resultante. El usuario confirmó que una orden `FAILED_APPOINTMENT` solo es cobrable cuando la tarifa del técnico en `CITA FALLIDA` existe y es válida; si está vacía o es inválida, no se genera cobro. El tratamiento de equipos retirados/nuevos en cambios de equipo se definirá al acordar ese flujo. El usuario confirmó que, cuando correspondan por servicio a domicilio, los viáticos equivalen al costo de salida del taller y se suman una sola vez por orden, sin multiplicarlos por la cantidad de equipos.

La tarifa se congela al completar la orden. Antes de `COMPLETED`, los cambios permitidos de técnico, servicio o modalidad pueden recalcular el importe. Si una orden completada se reabre por corrección, se recalcula con la tarifa vigente del técnico y se conserva una versión/auditoría del ajuste.

La moneda acordada para tarifa, viáticos y total es `MXN` (pesos mexicanos). Los importes se redondean a 2 decimales (centavos) con redondeo normal.

Reporte: mapeo solicitado

Columna	Encabezado propuesto	Fuente / regla
A	sem	Semana ISO provisional (lunes–domingo) desde `scheduledAt` en `America/Hermosillo`; configurable
B	orden	Folio único de la orden
C	tecnico	Nombre snapshot del técnico
D	nombre	Captura manual; la aplicación no escribe ni modifica
E	rfc	Captura manual; la aplicación no escribe ni modifica
F	economico	Etiquetas de unidades separadas por salto de línea
G	serie	Series, en el mismo orden que F
H	imei/id	IMEI o ID externo, agrupados en el orden de las unidades
I	**Sin encabezado**	Nombre snapshot del cliente
J	fecha	Fecha programada (`scheduledAt`) en `America/Hermosillo`
K	servicio	Etiqueta del tipo de servicio
L	costo	Costo base del snapshot
M	viaticos	Viáticos aplicados del snapshot

Columna	Encabezado propuesto	Fuente / regla
N	total	Costo + viáticos, aritmética decimal
O	envio dhl	Administración manual: la aplicación jamás escribe esta celda
P	notas	Notas de la orden y motivo de cita fallida cuando corresponda

Solo se autorizan las columnas administradas explícitamente por la aplicación en el futuro escritor del reporte. `D` (`nombre`), `E` (`rfc`) y `O` (`envio dhl`) son manuales: no se pueden leer-modificar-escribir ni incluir en `batchUpdate`, aunque estén vacías. No se puede enviar una fila `A:P`. Esta regla corresponde al **reporte**: la columna O de `SALIDAS` tiene otra función (serie) y conserva su política de escritura actual.

El folio identifica el cobro; filas duplicadas abortan la escritura. La clave local de idempotencia incluye orden y versión. Reintentos y correcciones actualizan el mismo folio. Antes de escribir se validan encabezados e identidad remota.

Decisiones funcionales y estado de implementación

Decisión	Regla confirmada e implementada	Estado
Unidad de cobro	Tarifa del técnico por servicio × total de equipos (1 en cambio de equipo)	Confirmado e implementado localmente
Moneda, decimales y redondeo	Aritmética decimal en MXN, 2 decimales y redondeo normal	Confirmado e implementado localmente
Congelación	Al completar (`COMPLETED`); congelado en snapshot de la orden	Confirmado e implementado localmente
Viáticos	`isHomeService` activa costo de salida del taller una vez por orden	Confirmado e implementado localmente
Estados cobrables	`COMPLETED`; `FAILED_APPOINTMENT` solo con tarifa CITA FALLIDA válida	Confirmado e implementado localmente
Inclusión en reporte	Toda `FAILED_APPOINTMENT` aparece en outbox aunque no tenga tarifa	Confirmado e implementado localmente
Importe sin tarifa	`0.00` en costo y total cuando CITA FALLIDA no tiene tarifa válida	Confirmado e implementado localmente
Nombre/RFC	Columnas D (`nombre`) y E (`rfc`) son manuales: no se tocan ni escriben	Confirmado e implementado

Decisión	Regla confirmada e implementada	Estado
		localmente
Columna DHL	Columna O (`envio dhl`) es manual: nunca se escribe ni se sobrescribe	Confirmado e implementado localmente
Fecha, semana, zona horaria	`scheduledAt` ; semana ISO y zona `America/Hermosillo`	Confirmado e implementado localmente
Correcciones	Reabrir folio, recalcular con tarifa vigente, actualizar fila y version	Confirmado e implementado localmente
Permiso de corrección	Solo el rol `ADMIN` puede reabrir una orden completada	Confirmado e implementado localmente
Motivo de corrección	Comentario opcional al reabrir; no bloquea la operación	Confirmado e implementado localmente
Fallo de sincronización	Outbox pendiente reintentable; no bloquea cierre ni corrección	Confirmado e implementado localmente
Programación del reporte	Corte martes; envío miércoles 17:00 en `America/Hermosillo` ; configuración administrativa	Confirmado e implementado localmente
Reintento de sincronización	Manual inmediato por administrador o automático con backoff vía worker	Confirmado e implementado localmente
Fila inexistente	Inserción automática (rangos A:C, F:N, P); búsqueda por folio	Confirmado e implementado localmente
Estado final en SALIDAS	Al completar se reencola por equipo para reparar asignaciones omitidas o desactualizadas	Confirmado e implementado localmente
Instalación	Cliente y vehículo en J, L:O; estado `INSTALADO` en K	Flujo implementado localmente
Revisión	Conserva cliente, vehículo y estado `INSTALADO`	Flujo implementado localmente
Sensor fuel	Equipo de stock; al finalizar escribe cliente, vehículo y estado `INSTALADO`	Flujo implementado localmente
Desinstalación	Equipo retirado pasa a `STOCK TECNICO` (K) y se limpian J, L, M, N, O	Flujo implementado localmente

Decisión	Regla confirmada e implementada	Estado
Cambio de equipo	1 unidad cobrada; anterior -> `STOCK TECNICO` + limpia J y L:O; nuevo -> hereda vehículo	Flujo implementado localmente

Validación en desarrollo y staging

La lectura del directorio y la escritura de IDs en W se ejecutaron en staging. Los flujos de cálculo de snapshots de cobro, persistencia en Postgres, outbox, sincronización con preservación de columnas D, E y O, y control de correcciones están cubiertos con pruebas unitarias y de integración automáticas. Las pruebas automatizadas utilizan identificadores ficticios y mocks locales; nunca tocan producción ni registran información fiscal, contactos o payloads completos en logs ni métricas.

Matriz de trazabilidad del MVP

La columna de pruebas identifica el nivel mínimo que debe existir antes de cerrar la fase correspondiente.

> Esta matriz es el contrato de aceptación del MVP, no el catálogo vigente de rutas. Las rutas son relativas a `/api/v1` y los endpoints de ejecución/archivos evolucionaron para soportar Drive; consulta el catálogo HTTP actual. Estado de implementación revisado 2026-10-02: las capacidades principales están implementadas, pero staging AWS está eliminado y no hay despliegue de producción.

Regla o capacidad	Endpoints previstos	Evidencia de aceptación
Sesión y rotación segura	`POST /auth/login`, `/auth/refresh`, `/auth/logout`	Integración: login genérico, rotación, reutilización y revocación
Usuarios y técnicos administrativos	`/users`, `/technicians`	Integración: capacidades por rol y auditoría
Catálogo de clientes	`GET /external-clients`	Adaptador simulado: normalización, caché y refresco
Catálogo e identidad de equipos	`GET /external-equipment`, `POST /external-equipment/refresh`	Duplicados, filas movidas, `PENDIENTE`, `N/A` e incompletos
Borradores, unidades y equipos	`/orders`, `/orders/:id/units`, `/units/:id/equipment`	Integración: varias unidades/equipos, serie única y propiedad
Concurrencia optimista	`PATCH /orders/:id`	Dos ediciones con la misma versión; la segunda recibe `409`
Asignación y Sheets	`POST /orders/:id/assign`	Contrato: revalidación, columnas permitidas, conflicto e idempotencia
Outbox y reintento	`/orders/:id/sync`, `/sync`, `/sync/:id/retry`	Integración: fallo visible, backoff y ausencia de duplicados
Corte semanal del reporte técnico	`/sync/schedule`	Unidad: corte, siguiente semana, zona horaria y reprogramación
Trabajo del técnico	`/me/orders`, `/orders/:id/start`	Autorización entre dos técnicos y transiciones inválidas
Verificación y discrepancias	`/order-equipment/:id/verify`, `/orders/:id/discrepancies`	Discrepancia no bloquea cierre; usuario ajeno recibe denegación
Checklist por unidad	`/units/:id/checklist`	Cinco categorías; observación obligatoria en `BAD`
Evidencias privadas	`/units/:id/evidence`, `/evidence/:id/complete`	MIME, tamaño, objeto inexistente, propiedad y URL temporal

Regla o capacidad	Endpoints previstos	Evidencia de aceptación
Notas de ejecución	`PATCH /orders/:id/execution-notes`	Nota general por orden, máximo 5000 caracteres y auditoría
Firma de orden	`/orders/:id/signature`	Nombre obligatorio, PNG privado y autorización
Cierre normal	`POST /orders/:id/complete`	Integración: todos los requisitos e idempotencia
Cita fallida	`POST /orders/:id/fail-appointment`	Solo motivo obligatorio; no exige checklist, fotos ni firma
Resolución de cita fallida	`POST /orders/:id/cancel`, `POST /orders/:id/reschedule`	Cancelación preserva el cierre; reprogramación crea un borrador único
Reporte PDF de orden de servicio	`/orders/:id/report`, `/orders/:id/reports`	Contrato, autorización, versionado y regresión visual del PDF
Corrección y cancelación	`/orders/:id/request-correction`, `/orders/:id/cancel`	Matriz de estados, inmutabilidad y auditoría
Tarifas acordadas y liquidación	`PATCH /orders/:id/pricing`	Unidad e integración: cálculo decimal puro, auditoría y reporte Sheets multilínea
Acceso compartido de técnico	`/shared-technician-access`, `/me/order-access`	Cuenta TECHNICIAN designada, una orden habilitada, revisión optimista y auditoría
Administración de cuentas admin	`GET/POST/PATCH /users`	Capacidad `users:manage`, límites de auto-bloqueo/último admin y auditoría
Salud y formato HTTP	`/health/live`, `/health/ready`	Supertest y contrato OpenAPI con `requestId`

Local container stack

The root Compose file builds the API and web applications and starts PostgreSQL and MinIO. Development defaults are non-production values and can be overridden through a local `.env` file.

The MinIO initialization container creates the configured private bucket. Named volumes preserve PostgreSQL and object data between restarts; `docker compose down` leaves both volumes intact.

The pinned MinIO image is the final official container published by the archived community repository. It is restricted to local development and is not the selected staging provider: CDK configures Google Drive shared storage. AWS staging is currently deleted, and production has no deployment of record. S3 remains an optional code adapter, not a requirement for staging or production.

Database migrations

Generate migrations through Prisma from the repository root. Once a migration has been applied to a shared environment, add a new migration instead of editing the existing one.

Service types added on 2026-09-08

`20260908120000_add_order_service_types` adds `FUEL_SENSOR` and `EQUIPMENT_REPLACEMENT` without replacing the enum or updating historical orders. Apply it before deploying consumers that use the new values. PostgreSQL requires committing an enum addition before using its value in the same migration sequence (ALTER TYPE documentation).

The application supports both values in service configuration, orders, pricing and Google Sheets synchronization. `FUEL_SENSOR` selects available stock for installation; `EQUIPMENT_REPLACEMENT` links an installed device to its replacement. Their workflow and workbook rules are documented in `docs/technician-workbook-contract.md`. This migration note is historical and does not mean the values are still awaiting approval. `Order.technicianId` remains nullable for historical records. Regularization must use verified technician assignments and audit each change; no automatic backfill or `NOT NULL` constraint is introduced.

Especificación: almacenamiento de órdenes en Google Drive

Estado: implementación de aplicación completada e integrada; la IaC de staging configura Drive, pero AWS staging está eliminado y no hay despliegue actual. La validación en AWS y las políticas de respaldo/retención de producción siguen pendientes. Esta especificación conserva los requisitos y límites originales; no autoriza migrar ni borrar archivos históricos de S3.

Objetivo

Guardar las nuevas evidencias fotográficas, firmas y versiones PDF de órdenes de servicio en un Drive compartido de la organización. La aplicación conserva el control de acceso: los navegadores no reciben credenciales de Google ni enlaces públicos de Drive.

Esta especificación define el comportamiento a implementar. El contexto y el flujo actual se describen en el plan de integración.

Alcance y exclusiones

- Los objetos y registros actuales de S3 son datos de prueba descartables. **No se migran** ni se requiere lectura histórica dual S3/Drive.
- Esta feature no autoriza borrar objetos de S3 ni ejecutar `cleanup-orphaned-objects.ts`. El descarte físico requiere una operación posterior, explícita y verificada.
- La aplicación debe funcionar en desarrollo y pruebas sin acceso a un tenant real mediante un adaptador de almacenamiento de prueba.
- No se publican archivos de Drive ni se solicita inicio de sesión de Google a técnicos o administradores.
- La conservación o eliminación de firmas originales y versiones PDF no cambia en esta feature.

Estructura y referencias

```
Drive compartido de la aplicación/
  Nombre del cliente/
    Serie o VIN de la unidad/
      Folio de la orden/
        Evidencias de esa unidad
    Órdenes/
      Folio de la orden/
        firma.png
```

```
reporte-v1.pdf
reporte-v2.pdf
```

1. La carpeta de cliente se identifica en la aplicación por `Order.clientSheetId` y muestra `clientNameSnapshot` como nombre. Si cambia el nombre visible, se conserva su identidad de carpeta.
2. Dentro del cliente, la carpeta de unidad se identifica por la serie/VIN normalizada de `OrderUnit.serialNumber`. `OrderUnit.id` no se usa como identidad entre órdenes. El formato de normalización debe definirse y probarse para evitar que variaciones de escritura creen carpetas distintas de la misma unidad.
3. Las evidencias de una unidad se guardan bajo su serie/VIN y el `Order.folio` correspondiente.
4. La firma única de la orden y cada versión PDF se guardan una sola vez bajo `Órdenes/<folio>`, incluso si hay una sola unidad. El nombre de cada versión debe distinguirla sin sobrescribir otra.
5. La base de datos conserva los IDs de Drive necesarios para localizar carpetas y archivos. Las rutas y nombres visibles no son el mecanismo de autorización ni la única fuente de identidad.
6. La API crea las carpetas faltantes bajo el padre correcto y reutiliza las carpetas cuya identidad lógica ya está registrada en la base de datos. No adopta una carpeta preexistente únicamente porque coincida su nombre visible: ese nombre no prueba a qué cliente o unidad pertenece. La operación debe tolerar reintentos y cargas simultáneas sin asociar un archivo a una carpeta de otro cliente, unidad u orden.

Requisitos funcionales

RF-1. Carga de evidencias

- Una persona autorizada adjunta una fotografía a una unidad y categoría de checklist de una orden.
- La web envía el archivo a un endpoint autenticado de la API; deja de hacer `PUT` directo a S3/MinIO mediante URL firmada.
- La API valida acceso, tipo real y tamaño antes de subir a Drive; resuelve `Cliente/Serie-VIN/Folio`, guarda el ID de archivo y confirma la carga solo después de comprobar el éxito de Drive.
- Una falla o interrupción no deja evidencia marcada como completa sin archivo recuperable. Los reintentos no crean registros completos duplicados.
- Cada carga de evidencia usa una clave UUID de idempotencia. La API reserva el ID de Drive y la fila pendiente antes de transferir bytes; repetir la misma clave reconcilia el archivo reservado y no crea otra evidencia.

RF-2. Firma de la orden

- La web envía el PNG de firma a la API bajo una sesión autorizada para esa orden.
- La API la guarda en `Cliente/Órdenes/Folio`, conserva su ID de Drive y la vincula a la orden, sin aceptar una referencia a un archivo ajeno proporcionada por el cliente.

- Cada intento usa una clave UUID de idempotencia y un ID de Drive reservado por el servidor. Una firma sustituida conserva su referencia anterior para respetar la política actual de retención; esta feature no ejecuta limpieza física.
- La firma original sigue disponible para vista previa y regeneración de reportes conforme a la política vigente.

RF-3. Reportes PDF

- El generador lee las evidencias y la firma desde Drive, incorpora todas las unidades de la orden y guarda cada versión del PDF en `Cliente/Órdenes/Folio`.
- Cada versión mantiene su propio ID de archivo, estado y metadatos. El estado `READY` solo se establece cuando el PDF es recuperable.
- El ID de Drive de cada versión se reserva antes de generar o subir el PDF; si el proceso se interrumpe, el siguiente intento reutiliza esa identidad y verifica el contenido remoto antes de marcar `READY`.
- Una regeneración no sobrescribe ni pierde una versión anterior que deba conservarse.

RF-4. Lectura y descarga privadas

- Para fotografía, firma o PDF, la API verifica autorización sobre la orden y transmite el contenido desde Drive al cliente.
- La web usa endpoints de la aplicación, no URLs públicas ni tokens de Google; conserva las vistas previas y la descarga de PDF.
- Los errores de acceso o ausencia de archivo no revelan contenido a personas no autorizadas.

RF-5. Configuración y puesta en marcha

- La identidad de la integración tiene acceso acotado al Drive compartido de la aplicación; no se requiere delegación de todo el dominio por defecto.
- ID de Drive y mecanismo de autenticación se configuran por entorno. Ninguna credencial ni contenido sensible se almacena en código, en el spec o en registros.
- Sin configuración válida de Drive en el entorno de producción, la aplicación falla de forma explícita o impide activar nuevas escrituras; no guarda silenciosamente archivos reales en S3.
- Antes de activar Drive con datos reales se invalidan de forma controlada las referencias de prueba a S3 que ya no serán accesibles. Esto no implica borrar físicamente el bucket.

Criterios de aceptación

1. Una orden con dos unidades de distinto VIN crea/reutiliza un cliente, dos carpetas de unidad y una carpeta común de orden; sus fotos no se mezclan, y solo hay una firma y una copia de cada versión PDF.
2. Dos órdenes del mismo cliente y VIN reutilizan la carpeta de unidad, pero mantienen carpetas de folio distintas.

3. Nombres iguales entre clientes no mezclan carpetas; cambios de nombre visible del cliente no rompen referencias existentes.
4. Cargas repetidas o concurrentes con la misma clave no dejan carpetas, archivos o registros activos duplicados para la misma identidad lógica.
5. Fallas de Drive antes de finalizar una carga no producen evidencia `COMPLETED` ni reporte `READY` sin archivo; los reintentos pueden recuperarse.
6. Personas no autorizadas no pueden cargar, consultar ni descargar archivos; los archivos no quedan públicos.
7. Fotografías, firmas y todas las versiones PDF nuevas pueden verse o descargarse desde la aplicación, sin depender de URLs firmadas de S3.
8. Pruebas automáticas cubren resolución de carpetas, autorización, errores de Drive, tamaños y tipos de archivo, subida, lectura y regeneración de reportes.
9. El corte no exige migración histórica y no ejecuta la limpieza de S3.

Dependencias y decisiones externas

- TI debe confirmar un Drive compartido, su capacidad, permisos de la identidad de aplicación y políticas del tenant.
- El mecanismo exacto de autenticación desde AWS y el alcance OAuth mínimo se validan en un piloto; no se incluyen secretos reales en esta especificación.
- El descarte de registros de prueba y el borrado físico de S3 se planifican como operaciones separadas, con autorización y verificación específicas.

Trazabilidad

- Plan de integración: `docs/google-drive-integration-plan.md`.
- Issue de GitHub: #28 — Google Drive storage.